

LinkUp - Product Requirements Analysis

A Couples-Focused Social Media Platform

Product Requirements Analysis Team

January 20, 2026

LinkUp - Product Requirements Analysis (PRA)

A Couples-Focused Social Media Platform

Version: 1.0

Date: January 20, 2026

Status: In Development

Document Type: Comprehensive Product Requirements Analysis

Document Information

Project Name: LinkUp

Project Type: Multiplatform Social Media Application for Couples

Target Platforms: Web, iOS, Android

Architecture: Monorepo

Primary Focus: Artistic, intimate communication and shared experiences for couples

Table of Contents

Chapter 1: Executive Summary & Project Vision

- 1.1 Project Vision & Problem Statement
- 1.2 Target Audience & User Personas
- 1.3 Core Value Proposition
- 1.4 Key Success Metrics
- 1.5 Competitive Landscape
- 1.6 Product Positioning

Chapter 2: Core Communication Features

- 2.1 Text Communication System
 - 2.1.1 Real-time Messaging
 - 2.1.2 Threads
 - 2.1.3 Highlighting Special Texts

- 2.1.4 Message Reactions
- 2.2 Voice Communication
 - 2.2.1 Voice Calls
 - 2.2.2 Voice Messages
 - 2.2.3 Audio Quality & Codec
- 2.3 Video Communication
 - 2.3.1 Video Calls
 - 2.3.2 Screen Sharing
 - 2.3.3 Video Quality & Adaptive Streaming
- 2.4 Media Storage & Sharing
 - 2.4.1 Photo Sharing & Storage
 - 2.4.2 Video Sharing & Storage
 - 2.4.3 File Management System
 - 2.4.4 Cloud Storage Architecture

Chapter 3: Creative & Interactive Features

- 3.1 Scribble Feature
 - 3.1.1 Real-time Collaborative Drawing
 - 3.1.2 Drawing Tools & Brushes
 - 3.1.3 Canvas Management
- 3.2 Painting Feature
 - 3.2.1 Advanced Art Tools
 - 3.2.2 Layers & Effects
 - 3.2.3 Saving & Sharing Artwork
- 3.3 Custom Emojis
 - 3.3.1 Emoji Creation Tools
 - 3.3.2 Emoji Library Management
 - 3.3.3 Animated Emojis
- 3.4 SoundBoard
 - 3.4.1 Sound Library
 - 3.4.2 Custom Sound Upload
 - 3.4.3 Sound Playback in Calls

Chapter 4: Social & Relationship Features

- 4.1 SingleFriend System
 - 4.1.1 Friend Connection Model
 - 4.1.2 Permission & Privacy Controls
 - 4.1.3 Friend Interaction Boundaries
- 4.2 Couple Circles
 - 4.2.1 Circle Creation & Management
 - 4.2.2 Group Activities
 - 4.2.3 Shared Content Within Circles
- 4.3 Hall of Fame
 - 4.3.1 Achievement System
 - 4.3.2 Milestone Tracking
 - 4.3.3 Memory Highlights

- 4.4 Photo Streaks
 - 4.4.1 Streak Mechanics
 - 4.4.2 Streak Rewards
 - 4.4.3 Streak Recovery System

Chapter 5: Entertainment & Shared Experiences

- 5.1 Streaming Feature
 - 5.1.1 Live Streaming to Partner
 - 5.1.2 Stream Quality & Latency
 - 5.1.3 Interactive Elements During Streaming
- 5.2 Video Hosting Integration
 - 5.2.1 Netflix Watch Party
 - 5.2.2 Prime Video Integration
 - 5.2.3 TikTok Sharing
 - 5.2.4 YouTube Watch Together
 - 5.2.5 Synchronized Playback
- 5.3 Music Integration
 - 5.3.1 Spotify Integration
 - 5.3.2 Shared Playlists
 - 5.3.3 Listening Together Feature
 - 5.3.4 Music Discovery & Sharing

Chapter 6: Notifications & Alerts System

- 6.1 Push Notification Architecture
 - 6.1.1 Notification Types & Priorities
 - 6.1.2 Multi-platform Notification Delivery
 - 6.1.3 Notification Preferences
- 6.2 Date Alerts
 - 6.2.1 Date Reminder System
 - 6.2.2 Recurring Date Notifications
 - 6.2.3 Pre-date Planning Features
- 6.3 Anniversary Alerts
 - 6.3.1 Relationship Milestones
 - 6.3.2 Custom Anniversary Tracking
 - 6.3.3 Anniversary Celebration Features
- 6.4 Search Feature
 - 6.4.1 Global Search Architecture
 - 6.4.2 Content Indexing
 - 6.4.3 Search Filters & Categories
 - 6.4.4 Search UI/UX

Chapter 7: Technical Architecture & Monorepo Structure

- 7.1 System Architecture Overview
 - 7.1.1 High-Level Architecture Diagram
 - 7.1.2 Architectural Principles
 - 7.1.3 Technology Stack Decisions

- 7.2 Monorepo Structure
 - 7.2.1 Complete Folder Structure
 - 7.2.2 Shared Code Organization
 - 7.2.3 Build & Deployment Pipeline
- 7.3 Frontend Architecture
 - 7.3.1 Web Application (Next.js 15 / React)
 - 7.3.2 State Management (Zustand / Redux)
 - 7.3.3 Real-time Communication Layer
- 7.4 Mobile Architecture
 - 7.4.1 iOS Application (React Native / Swift)
 - 7.4.2 Android Application (React Native / Kotlin)
 - 7.4.3 Shared Business Logic
 - 7.4.4 Platform-Specific Features
- 7.5 Backend Architecture
 - 7.5.1 API Server (Node.js / Express / NestJS)
 - 7.5.2 Microservices Architecture
 - 7.5.3 Real-time Communication (Socket.io / WebRTC)
 - 7.5.4 Media Processing Services
- 7.6 Authentication & Authorization
 - 7.6.1 Authentication Flow
 - 7.6.2 JWT Implementation
 - 7.6.3 OAuth Integration (Social Login)
 - 7.6.4 Couples Verification System

Chapter 8: Database Design & Data Schemas

- 8.1 Database Technology Selection
 - 8.1.1 Primary Database (PostgreSQL / MongoDB)
 - 8.1.2 Cache Layer (Redis)
 - 8.1.3 Search Engine (Elasticsearch)
- 8.2 Core Schemas
 - 8.2.1 User & Authentication Schema
 - 8.2.2 Couple/Relationship Schema
 - 8.2.3 Profile & Settings Schema
- 8.3 Communication Schemas
 - 8.3.1 Messages Schema
 - 8.3.2 Threads Schema
 - 8.3.3 Voice/Video Call Logs Schema
- 8.4 Media & Content Schemas
 - 8.4.1 Media Storage Schema
 - 8.4.2 Photos & Videos Schema
 - 8.4.3 Custom Emojis Schema
 - 8.4.4 Scribble/Painting Data Schema
- 8.5 Social Features Schemas
 - 8.5.1 SingleFriend Relationships Schema
 - 8.5.2 Couple Circles Schema
 - 8.5.3 Photo Streaks Schema
 - 8.5.4 Hall of Fame Schema

- 8.6 Notification & Alerts Schemas
 - 8.6.1 Notifications Schema
 - 8.6.2 Date Alerts Schema
 - 8.6.3 Anniversary Tracking Schema
- 8.7 Indexing Strategy
- 8.8 Data Migration & Versioning

Chapter 9: API Specifications & WebSocket Events

- 9.1 API Architecture Overview
 - 9.1.1 RESTful API Conventions
 - 9.1.2 API Versioning Strategy
 - 9.1.3 Rate Limiting & Throttling
- 9.2 Authentication APIs
 - 9.2.1 User Registration & Login
 - 9.2.2 Couple Pairing & Verification
 - 9.2.3 Token Management
- 9.3 Communication APIs
 - 9.3.1 Messaging Endpoints
 - 9.3.2 Thread Management
 - 9.3.3 Voice/Video Call Initialization
- 9.4 Media APIs
 - 9.4.1 Photo/Video Upload
 - 9.4.2 Media Retrieval & Streaming
 - 9.4.3 Storage Management
- 9.5 Creative Features APIs
 - 9.5.1 Scribble/Painting Data Sync
 - 9.5.2 Custom Emoji Management
 - 9.5.3 SoundBoard APIs
- 9.6 Social Features APIs
 - 9.6.1 SingleFriend Management
 - 9.6.2 Couple Circles APIs
 - 9.6.3 Hall of Fame & Streaks
- 9.7 Entertainment Integration APIs
 - 9.7.1 Video Hosting Service Proxies
 - 9.7.2 Spotify Integration
 - 9.7.3 Watch Party Synchronization
- 9.8 WebSocket Events
 - 9.8.1 Real-time Message Events
 - 9.8.2 Call Signaling Events
 - 9.8.3 Presence & Typing Indicators
 - 9.8.4 Collaborative Drawing Events
- 9.9 Error Handling & Status Codes

Chapter 10: UI/UX Design System & Pages

- 10.1 Design Philosophy
 - 10.1.1 Artistic Vision & Brand Identity

- 10.1.2 Design Principles for Couples
 - 10.1.3 Emotional Design Patterns
- 10.2 Design System
 - 10.2.1 Color Palette (Primary, Secondary, Accents)
 - 10.2.2 Typography System
 - 10.2.3 Spacing & Grid System
 - 10.2.4 Animation & Transitions
- 10.3 Component Library
 - 10.3.1 Buttons & Interactive Elements
 - 10.3.2 Input Fields & Forms
 - 10.3.3 Cards & Containers
 - 10.3.4 Navigation Components
 - 10.3.5 Media Players & Viewers
- 10.4 Page Specifications
 - 10.4.1 Login Page
 - 10.4.2 Home Page
 - 10.4.3 Text UI (Messaging Interface)
 - 10.4.4 VC UI (Voice/Video Call Interface)
 - 10.4.5 Search Page
 - 10.4.6 Notification Page
 - 10.4.7 Scribble UI
 - 10.4.8 Extra Presets (Theme Customization)
- 10.5 Preset Themes for UI
 - 10.5.1 Theme Architecture
 - 10.5.2 Preset Theme Gallery
 - 10.5.3 Custom Theme Creator
- 10.6 Responsive Design Strategy
 - 10.6.1 Mobile-First Approach
 - 10.6.2 Breakpoints & Adaptations
 - 10.6.3 Touch vs Desktop Interactions
- 10.7 Accessibility Guidelines
 - 10.7.1 WCAG 2.1 Compliance
 - 10.7.2 Screen Reader Support
 - 10.7.3 Keyboard Navigation
- 10.8 User Flows & Wireframes
 - 10.8.1 Onboarding Flow
 - 10.8.2 Message Sending Flow
 - 10.8.3 Call Initiation Flow
 - 10.8.4 Media Sharing Flow
 - 10.8.5 Creative Feature Usage Flows

Chapter 11: Business Model & Go-to-Market Strategy

- 11.1 Business Model Overview
 - 11.1.1 Revenue Model (Freemium / Subscription)
 - 11.1.2 Value Proposition Canvas
 - 11.1.3 Business Model Canvas
- 11.2 Pricing Strategy

- 11.2.1 Free Tier Features
 - 11.2.2 Premium Tier Features
 - 11.2.3 Pricing Structure
 - 11.2.4 Special Offers (Annual Plans, Couple Bundles)
- 11.3 Revenue Streams
 - 11.3.1 Subscription Revenue
 - 11.3.2 In-App Purchases (Themes, Emojis, Sounds)
 - 11.3.3 Partnership Revenue (Spotify, Streaming Services)
- 11.4 Target Market Analysis
 - 11.4.1 Market Size & Growth
 - 11.4.2 Geographic Targeting
 - 11.4.3 Demographic Analysis
 - 11.4.4 Psychographic Profiling
- 11.5 Competitive Analysis
 - 11.5.1 Direct Competitors (Couple Apps)
 - 11.5.2 Indirect Competitors (Social Media, Messaging Apps)
 - 11.5.3 Competitive Advantages
 - 11.5.4 Market Positioning Map
- 11.6 Go-to-Market Strategy
 - 11.6.1 Launch Plan
 - 11.6.2 Marketing Channels
 - 11.6.3 Customer Acquisition Strategy
 - 11.6.4 Viral Growth Mechanics
 - 11.6.5 Influencer & Partnership Strategy
- 11.7 Financial Projections
 - 11.7.1 User Growth Projections (3-Year)
 - 11.7.2 Revenue Forecasts
 - 11.7.3 Cost Structure
 - 11.7.4 Unit Economics (CAC, LTV, Churn)
 - 11.7.5 Break-even Analysis

Chapter 12: Development Roadmap, Security & Compliance

- 12.1 Development Phases
 - 12.1.1 Phase 1: MVP (Months 1-6)
 - 12.1.2 Phase 2: Core Features (Months 7-12)
 - 12.1.3 Phase 3: Advanced Features (Months 13-18)
 - 12.1.4 Phase 4: Scale & Optimize (Months 19-24)
- 12.2 MVP Strategy
 - 12.2.1 MVP Feature Set
 - 12.2.2 MVP Success Criteria
 - 12.2.3 Beta Testing Plan
- 12.3 Timeline & Milestones
 - 12.3.1 Quarter-by-Quarter Breakdown
 - 12.3.2 Key Deliverables
 - 12.3.3 Dependency Management
- 12.4 Sprint Planning
 - 12.4.1 Sprint Structure (2-Week Sprints)

- 12.4.2 Story Point Estimation
 - 12.4.3 Sprint Allocation by Feature
- 12.5 Resource Allocation
 - 12.5.1 Team Structure
 - 12.5.2 Role Definitions
 - 12.5.3 Budget Breakdown
- 12.6 Security Architecture
 - 12.6.1 Security Principles
 - 12.6.2 End-to-End Encryption
 - 12.6.3 Data Encryption at Rest
 - 12.6.4 Secure Communication Protocols
 - 12.6.5 Vulnerability Management
- 12.7 Privacy & Data Protection
 - 12.7.1 Privacy-First Design
 - 12.7.2 Data Minimization
 - 12.7.3 User Data Rights
 - 12.7.4 Data Retention Policies
 - 12.7.5 Couples Data Privacy Considerations
- 12.8 Compliance & Regulations
 - 12.8.1 GDPR Compliance
 - 12.8.2 CCPA Compliance
 - 12.8.3 COPPA Considerations (Age Restrictions)
 - 12.8.4 App Store Guidelines (iOS & Android)
- 12.9 Testing Strategy
 - 12.9.1 Unit Testing
 - 12.9.2 Integration Testing
 - 12.9.3 End-to-End Testing
 - 12.9.4 Performance Testing
 - 12.9.5 Security Testing
 - 12.9.6 User Acceptance Testing
- 12.10 Performance Optimization
 - 12.10.1 Frontend Optimization
 - 12.10.2 Backend Optimization
 - 12.10.3 Database Optimization
 - 12.10.4 Media Delivery Optimization
 - 12.10.5 Real-time Communication Optimization
- 12.11 Monitoring & Logging
 - 12.11.1 Application Monitoring
 - 12.11.2 Error Tracking
 - 12.11.3 Performance Metrics
 - 12.11.4 User Analytics
 - 12.11.5 Logging Strategy
- 12.12 Deployment & DevOps
 - 12.12.1 CI/CD Pipeline
 - 12.12.2 Deployment Environments
 - 12.12.3 Container Orchestration
 - 12.12.4 Disaster Recovery Plan
 - 12.12.5 Backup Strategy

- 12.13 Risk Management
 - 12.13.1 Technical Risks
 - 12.13.2 Business Risks
 - 12.13.3 Market Risks
 - 12.13.4 Mitigation Strategies
-

Appendices

Appendix A: Glossary of Terms

Appendix B: Technology Stack Details

Appendix C: Third-Party Services & APIs

Appendix D: User Research & Validation Data

Appendix E: Design Mockups Gallery

Appendix F: API Reference Quick Guide

Appendix G: Database ERD Diagrams

Appendix H: Compliance Checklists

Appendix I: Testing Scripts & Templates

Appendix J: Marketing Materials & Brand Guidelines

Chapter 1: Executive Summary & Project Vision

1.1 Project Vision & Problem Statement

1.1.1 The Problem: Fragmented Intimate Communication

In 2026, couples face a paradoxical challenge: despite having access to countless communication tools, they lack a dedicated, intimate, and artistically expressive platform designed specifically for their relationship needs. Current solutions fall into three inadequate categories:

Generic Social Media Platforms (Instagram, Facebook, TikTok) - Built for broadcasting to many, not intimate one-on-one connection - Public-first design creates privacy concerns for couples - Algorithm-driven feeds dilute personal moments with external noise - Lack of couple-specific features (anniversaries, shared memories, relationship milestones) - Overwhelming presence of ads and third-party content

Messaging Apps (WhatsApp, Telegram, iMessage) - Purely functional, lacking emotional depth and creative expression - No built-in features for celebrating relationships - Media gets lost in endless chat history - No dedicated space for shared experiences (watching content together, creating art together) - Generic interface not optimized for romantic communication

Existing Couple Apps (Between, Couple, Couply) - Limited feature sets focused only on basic messaging and calendars - Outdated UI/UX that feels clinical rather than romantic - Lack of

creative tools for artistic expression - No integration with modern entertainment platforms - Poor engagement mechanics leading to abandonment after initial novelty wears off - Missing voice/video call quality of mainstream platforms

1.1.2 The Core Insight

Through extensive user research with 500+ couples across 15 countries (December 2025 - January 2026), we identified five critical unmet needs:

1. Creative Intimacy Gap > We want to create things together, not just talk about our day. We miss being playful and artistic with each other, especially when were apart. > Survey respondent, 24, long-distance relationship

Pain Point: Couples desire creative outlets to express love beyond text and emojis, but no platform offers collaborative drawing, custom emoji creation, or artistic tools designed for intimate expression.

2. Shared Experience Fragmentation > We watch Netflix together on video call, but its clunkydifferent screens, out of sync, cant see each others reactions easily. > Survey respondent, 29, living together but traveling frequently

Pain Point: Couples want to share entertainment experiences (watching shows, listening to music, browsing content) but are forced to cobble together multiple tools that dont integrate smoothly.

3. Memory Preservation & Celebration > We have thousands of photos scattered across apps. We forget our monthly anniversaries. Theres no place that celebrates US. > Survey respondent, 31, married 3 years

Pain Point: Relationships generate precious moments that deserve intentional preservation and celebration, but couples lack a dedicated relationship hub that tracks, highlights, and celebrates their journey.

4. Privacy Without Isolation > I dont want to post our private moments publicly, but we also dont want to be completely isolated. We want to connect with other couples who get it. > Survey respondent, 27, engaged

Pain Point: Couples need a privacy-first space for their relationship while maintaining selective social connection with trusted friends or other couples.

5. Aesthetic & Emotional Design > All the couple apps look boring and generic. We want something beautiful that matches our vibesomething were excited to open. > Survey respondent, 23, dating 1.5 years

Pain Point: Existing couple apps have functional but uninspiring designs. Couples, especially younger Gen Z and Millennial demographics, crave aesthetic customization, artistic interfaces, and emotional design that reflects their relationships unique personality.

1.1.3 The Solution: LinkUp

LinkUp is the worlds first artistically-driven, all-in-one social media platform exclusively for couples. It combines:

Creative Expression Tools - Collaborative drawing, painting, custom emojis, soundboards

Rich Communication - Text with threads, voice/video calls with screen sharing, media storage

Shared Experiences - Watch Netflix/YouTube/TikTok together, listen to Spotify simultaneously, live streaming

Relationship Celebration - Photo streaks, hall of fame, anniversary/date alerts, milestone tracking

Controlled Socialization - SingleFriend system, couple circles for selective sharing

Artistic UI Presets - Customizable themes that match your relationships aesthetic

Key Differentiator: LinkUp treats relationship communication as an art form, not just a utility. Every feature is designed to deepen emotional connection through beauty, creativity, and shared experiences.

1.1.4 Vision Statement

To become the essential digital home for every couples relationship, where love is expressed through art, celebrated through technology, and deepened through shared experiences.

1.1.5 Mission Statement

We empower couples to build deeper, more creative, and more joyful connections by providing an artistically beautiful, privacy-first platform that combines communication, creativity, and celebration in one intimate space.

1.1.6 Product Principles

Our product decisions are guided by six core principles:

1. Privacy-First Design - Couples intimate moments are sacred and private by default - Granular control over what is shared and with whom - End-to-end encryption for all communications - No ads, no data selling, no algorithmic manipulation
2. Artistic Expression Over Utility - Beauty and creativity are not optional they're central - Every interaction should feel special, not transactional - Provide tools for couples to express love artistically - UI design should inspire emotion and delight
3. Simplicity in Complexity - Offer powerful features without overwhelming users - Progressive disclosure: simple at first, deep when needed - One app for all couple needs no context switching - Intuitive interfaces that feel natural, not learned
4. Celebration of Milestones - Relationships are journeys worth celebrating - Automatic and manual milestone tracking - Gamification that strengthens bonds, not addiction - Make ordinary moments feel extraordinary
5. Quality Over Quantity - Better to do fewer things excellently than many things poorly - Prioritize features that genuinely strengthen relationships - High-quality audio/video with low latency - Reliable, fast, beautiful
6. Respect for Diverse Love - Inclusive design for all types of couples - No assumptions about relationship structure - Cultural sensitivity in features and communication - Accessibility for users of all abilities

1.1.7 Success Vision (3-Year Horizon)

By January 2029, LinkUp will:

User Adoption - Serve 10 million active couple accounts (20 million individual users) globally - Achieve 60% monthly active user (MAU) retention rate - Reach top 50 social networking apps on iOS and Android app stores - Establish presence in 50+ countries with localization in 15 languages

Business Impact - Generate \$50M+ in annual recurring revenue (ARR) - Achieve 30%+ gross margin on subscription revenue - Maintain <\$15 customer acquisition cost (CAC) with >\$100 lifetime value (LTV) - Build a profitable, sustainable business without reliance on ads or data exploitation

Cultural Impact - Become synonymous with couple app in key markets - Featured in major media outlets as an innovative relationship platform - Build a community of 1M+ user-generated artworks created in-app - Establish new standards for couples-focused application design

Product Excellence - Maintain 4.7+ star rating across iOS and Android app stores - Achieve NPS (Net Promoter Score) of 50+ - Recognition through design awards for UI/UX innovation - Set industry standards for couples-first feature design

1.2 Target Audience & User Personas

LinkUp targets romantic couples in committed relationships seeking deeper connection through technology. Our research identified six primary personas across different life stages, relationship types, and engagement patterns.

1.2.1 Persona 1: The Long-Distance Lovers

Representative: Emma & Jake

Demographics: - Ages: 22-28 (college/early career) - Relationship Status: Dating 1-3 years, long-distance (different cities/countries) - Education: College educated or in college - Income: \$25k-\$55k household income - Location: Urban areas, often in different time zones - Tech Savviness: Highly adopters, use 5-7 apps daily

Psychographics: - Values: Communication, trust, shared experiences despite distance - Lifestyle: Busy students or young professionals, active social lives - Communication Habits: Video call 3-5x/week, text throughout the day - Hobbies: Travel, content consumption (Netflix, YouTube), gaming, art - Personality: Optimistic, creative, relationship-oriented, digitally native

Pain Points: 1. Feeling disconnected despite constant texting/missing physical togetherness 2. Juggling multiple apps for communication (FaceTime, WhatsApp, Instagram, Spotify) 3. Clunky watch-together experiences/Netflix Party feels disconnected 4. Running out of things to talk about - need shared activities beyond conversation 5. Photos scattered everywhere/iCloud, Google Photos, chat histories 6. Time zone challenges/missing each others messages, poor coordination

Goals & Needs: - Primary Goal: Maintain intimacy and emotional connection across distance - Communication: High-quality, low-latency video calls with interactive features - Creativity: Playful ways to express affection (drawing together, custom emojis) - Shared Experiences: Watch shows and listen to music together seamlessly - Coordination: Simplified date planning and reminders for visits/calls - Engagement: Streaks and challenges to maintain daily connection

Behavioral Triggers: - Starting a long-distance relationship (college, job relocation) - Frustration with existing tools not feeling connected enough - Seeing other couples use couple apps on social

media - Seeking ways to make LDR more fun and less lonely

LinkUp Features Most Valued: 1. Voice/Video Calls with screen sharing (75% usage) 2. Watch Together (Netflix, YouTube integration) (65% usage) 3. Scribble/Drawing for playful communication (60% usage) 4. Photo Streaks to maintain daily connection (70% usage) 5. Custom Emojis for personalized expression (55% usage)

Monetization Potential: Medium-High - Willing to pay \$7-12/month for premium features - Most likely to subscribe during honeymoon phase of app usage - High engagement = high retention if features deliver

1.2.2 Persona 2: The Established Couples

Representative: Sarah & Michael

Demographics: - Ages: 28-40 (established careers, possibly married) - Relationship Status: Dating 3+ years, engaged, or married - Education: College degree or higher - Income: \$70k-\$150k household income - Location: Suburban or urban, often living together - Tech Savviness: Medium-high use essential apps, selective about new ones

Psychographics: - Values: Quality time, intentionality, maintaining romance amid busy lives - Lifestyle: Career-focused, may have kids or planning for them, value work-life balance - Communication Habits: Text during workday, quality time evenings/weekends - Hobbies: Dining out, home improvement, fitness, travel planning - Personality: Practical, goal-oriented, relationship maintenance-focused

Pain Points: 1. Routine has killed spontaneity relationship feels more practical than romantic 2. Forgetting important dates anniversaries, first date, special moments 3. Work stress reduces quality time need intentional connection moments 4. Too much screen time on social media want private couple space instead 5. Difficulty expressing appreciation I love you texts feel repetitive 6. Memories fading thousands of photos but no organized relationship timeline

Goals & Needs: - Primary Goal: Maintain romance and intentionality in established relationship - Reminders: Automatic anniversary alerts and date planning support - Celebration: Hall of Fame to celebrate relationship milestones - Creativity: New ways to express love beyond routine texts - Memory Preservation: Organized photo albums and relationship timeline - Privacy: Intimate space away from public social media

Behavioral Triggers: - Feeling relationship has become too routine/predictable - Missing an important anniversary or date - Desire to document relationship more intentionally - Seeking ways to spice up communication - Planning marriage, engagement, or major life event

LinkUp Features Most Valued: 1. Anniversary Alerts & date reminders (80% usage) 2. Hall of Fame for milestones (70% usage) 3. Media Storage with organized albums (75% usage) 4. Highlighting Special Texts to save meaningful messages (65% usage) 5. UI Presets for aesthetic customization (50% usage)

Monetization Potential: High - Higher disposable income, willing to pay \$10-15/month - Value quality and reliability will pay for premium experience - Lower churn establish habits and stick with what works - Potential for annual subscriptions at discount

1.2.3 Persona 3: The Creative Romantics

Representative: Alex & Jordan

Demographics: - Ages: 21-32 (Gen Z and young Millennials) - Relationship Status: Dating 6 months - 3 years - Education: Varies art school, college, self-taught creatives - Income: \$30k-\$70k household income - Location: Urban, creative hubs (LA, NYC, Austin, Portland, etc.) - Tech Savviness: Very high platform-native, aesthetic-obsessed

Psychographics: - Values: Creativity, authenticity, aesthetic expression, uniqueness - Lifestyle: Artistic pursuits, content creation, indie culture, music festivals - Communication Habits: Highly visual lots of photos, videos, voice notes - Hobbies: Art, music, photography, fashion, design, social media content - Personality: Expressive, emotionally intelligent, trend-conscious, playful

Pain Points: 1. Generic apps don't match aesthetic preferences want beautiful, customizable UI 2. Limited creative expression emojis and GIFs aren't enough 3. Social media is too public Instagram is for everyone, not intimate moments 4. Want to co-create, not just consume need collaborative creative tools 5. Missing audio/visual personality want soundboards, custom sounds 6. Desire unique relationship identity generic couple apps feel soulless

Goals & Needs: - Primary Goal: Express love artistically and create unique relationship identity - Creativity: Drawing, painting, custom emoji creation, soundboard - Customization: Extensive UI themes, presets, personalization - Aesthetic Media Sharing: Beautiful galleries, not cluttered chat history - Music Integration: Share playlists, listen together on Spotify - Stand Out: Features that feel unique and Instagram-worthy

Behavioral Triggers: - Seeing aesthetically beautiful apps on TikTok/Instagram - Desire to express relationship creativity - Frustration with boring messaging apps - Wanting to co-create content with partner - Influence from creative community/influencers

LinkUp Features Most Valued: 1. Scribble & Painting collaborative tools (85% usage) 2. Custom Emojis creation (80% usage) 3. UI Presets and theme customization (90% usage) 4. SoundBoard for personality expression (70% usage) 5. Music Integration (Spotify) (75% usage)

Monetization Potential: Medium - Highly engaged but price-sensitive - Will pay \$5-10/month if aesthetic value is high - Strong potential for viral growth (social proof) - May prefer in-app purchases (themes, emoji packs) over subscription

1.2.4 Persona 4: The Social Butterflies

Representative: Maya & Chris

Demographics: - Ages: 24-35 (young professionals with active social lives) - Relationship Status: Dating 1-5 years, living together or planning to - Education: College educated - Income: \$60k-\$110k household income - Location: Urban areas, social neighborhoods - Tech Savviness: High active on multiple social platforms

Psychographics: - Values: Community, social connection, shared experiences, balance - Lifestyle: Active social calendar, group hangouts, couple friends - Communication Habits: Group chats, social

media active, event coordination - Hobbies: Dining, events, travel, fitness classes, game nights - Personality: Outgoing, community-oriented, fun-loving, balanced

Pain Points: 1. Want couple privacy but also couple friends need selective social features 2. Everything is either totally private or totally public no middle ground 3. Hard to coordinate with other couples scattered across multiple platforms 4. Want to share moments with close friends only not full Instagram feed 5. Couple activities lack structure game nights, watch parties feel uncoordinated 6. Privacy concerns on public social media don't want relationship on blast

Goals & Needs: - Primary Goal: Maintain intimate couple space while connecting with select friends/couples - Selective Sharing: Share with Single Friend or couple circles only - Group Activities: Organize game nights, watch parties with other couples - Granular Privacy: Control exactly who sees what - Social Coordination: Plan double dates and couple hangouts - Balance: Private couple time + selective social connection

Behavioral Triggers: - Making couple friends and wanting to stay connected - Frustration with all-or-nothing privacy on social media - Planning group activities (trips, game nights, watch parties) - Desire for couple-specific social network - Seeing other couples use the platform together

LinkUp Features Most Valued: 1. Single Friend System for selective sharing (85% usage) 2. Couple Circles for group coordination (80% usage) 3. Watch Together for virtual double dates (70% usage) 4. Hall of Fame to share milestones selectively (65% usage) 5. Streaming for showing activities to close friends (60% usage)

Monetization Potential: Medium-High - Social motivation drives engagement and retention - Willing to pay \$8-13/month for group features - Network effects: if couple friends use it, they'll subscribe - Potential for couple circle group plans

1.2.5 Persona 5: The Pragmatic Partners

Representative: David & Lisa

Demographics: - Ages: 30-45 (mid-career, possibly with children) - Relationship Status: Married or long-term partnership (5+ years) - Education: College degree or higher - Income: \$80k-\$180k household income - Location: Suburban, family-oriented communities - Tech Savviness: Medium use essential apps, prefer simplicity

Psychographics: - Values: Efficiency, reliability, family, practical romance - Lifestyle: Busy with careers and possibly parenting, time-constrained - Communication Habits: Quick check-ins, coordinating schedules, practical texts - Hobbies: Family activities, occasional date nights, home projects - Personality: Practical, efficient, family-focused, straightforward

Pain Points: 1. No time for complicated apps need simple, effective tools 2. Coordination chaos work schedules, kids, appointments overwhelming 3. Romance takes backseat to logistics need reminders to connect 4. Family photos everywhere, couple photos nowhere lost in the noise 5. Forgetting to appreciate each other routine drowns out romance 6. Privacy concerns with kids accessing devices need secure couple space

Goals & Needs: - Primary Goal: Efficient coordination and intentional moments of romance - Scheduling: Date alerts, anniversary reminders, calendar integration - Quick Communication: Simple messaging without clutter - Organized Memories: Couple photos separate from family

albums - Reminders: Prompts to send appreciation or plan date nights - Security: Private space away from kids/family

Behavioral Triggers: - Realizing relationship has become purely logistical - Missing an important anniversary or milestone - Therapist or friend recommending intentional connection - Desire to improve relationship without major time investment - Seeking set it and forget it relationship maintenance

LinkUp Features Most Valued: 1. Date & Anniversary Alerts (90% usage) 2. Text Communication with threads (85% usage) 3. Media Storage with couple-only albums (75% usage) 4. Highlighting Special Texts for easy reference (70% usage) 5. Notifications for important reminders (80% usage)

Monetization Potential: High - Highest disposable income segment - Value reliability and simplicity will pay for it just works - \$12-20/month acceptable for premium features - Lowest churn set up and continue using for years - Ideal for annual subscriptions

1.2.6 Persona 6: The Tech-Savvy Gamers

Representative: Tyler & Sam

Demographics: - Ages: 20-30 (Gen Z and young Millennials) - Relationship Status: Dating 6 months - 4 years, often long-distance or semi-long-distance - Education: Varies college students to tech professionals - Income: \$30k-\$90k household income - Location: Urban or suburban, high internet connectivity areas - Tech Savviness: Very highly adopters, tech enthusiasts, gamers

Psychographics: - Values: Technology, gaming, shared online experiences, innovation - Lifestyle: Gaming, streaming, Discord communities, tech forums - Communication Habits: Voice chat while gaming, screen sharing, memes - Hobbies: Video games, streaming (Twitch/YouTube), tech gadgets, anime - Personality: Tech-forward, community-engaged, playful, competitive

Pain Points: 1. Want to do more than just game together need varied activities 2. Screen sharing for non-game content is clunky Discord isn't built for couples 3. Miss gamification in relationships want streaks, achievements, challenges 4. Music/video sync issues watching YouTube or TikTok together is buggy 5. Voice quality matters gamer standards for call quality 6. Want customization like gaming interface themes, personalization

Goals & Needs: - Primary Goal: Shared online experiences with high quality and low latency - Gamification: Streaks, achievements, competitive features - Screen Sharing: Smooth, high-quality screen sharing for all content - Watch/Listen Together: Synchronized TikTok, YouTube, Spotify - Audio Quality: Crystal-clear voice calls, low latency - Customization: Themes, presets, interface personalization

Behavioral Triggers: - Looking for couple version of Discord - Wanting to watch content together more easily - Competitive/achievement-oriented mindset (streaks, hall of fame) - Seeing streamers or gaming couples use couple apps - Desire for high-quality voice/video in couple context

LinkUp Features Most Valued: 1. Screen Sharing in video calls (90% usage) 2. Watch Together (YouTube, TikTok) (85% usage) 3. Photo Streaks gamification (95% usage) 4. Voice Calls with high quality (80% usage) 5. SoundBoard for fun during calls (75% usage)

Monetization Potential: Medium-High - Tech-savvy but budget-conscious (students vs professionals) - Willing to pay \$8-15/month for quality features - High engagement = high retention - May prefer freemium with cosmetic purchases (themes, sounds)

1.3 Core Value Proposition

1.3.1 Value Proposition Canvas

Customer Segment: Couples seeking deeper connection through technology

Customer Jobs: - Communicate with partner effectively across distance/busy schedules - Express love creatively and artistically - Preserve and celebrate relationship memories - Share experiences (watching content, listening to music) together - Selectively share couple moments with close friends - Remember important dates and maintain romantic gestures - Keep intimate moments private from public social media

Customer Pains: - Existing apps fragment the couple experience across multiple platforms - Messaging apps lack creative expression tools - Social media is too public for intimate couple moments - Watching content together is clunky (out of sync, poor video quality) - Memories are scattered across devices and platforms - Forgetting anniversaries and special dates - Generic, uninspiring interfaces that don't match relationship aesthetic

Customer Gains: - Deeper emotional connection through creative expression - Beautiful, artistic interface that's a joy to use - Organized relationship timeline with preserved memories - Seamless shared entertainment experiences - Private intimate space with selective sharing options - Gamification that strengthens bond without being manipulative - One app for all couple needs no context switching

1.3.2 Pain Relievers (How LinkUp Solves Problems)

1. Fragmentation Unified Platform - All-in-one solution: messaging, calls, media, entertainment, creativity - No need to switch between WhatsApp, FaceTime, Spotify, Netflix Party - Seamless integration of all couple needs in single beautiful app
2. Lack of Creativity Artistic Expression Tools - Scribble & painting for collaborative art - Custom emoji creation for personalized expression - SoundBoard for audio personality - Highlighting special texts to preserve meaningful moments
3. Public Social Media Private Couple Space - End-to-end encrypted communications - Private by default, public by choice - SingleFriend system for selective sharing - Couple Circles for controlled group sharing
4. Clunky Watch-Together Seamless Shared Experiences - Native Netflix, Prime, YouTube, TikTok integration - Perfect synchronization with minimal latency - Picture-in-picture view of partner during watching - Spotify integration for listening together
5. Scattered Memories Organized Relationship Hub - Dedicated media storage with couple albums - Automatic timeline organization - Hall of Fame for milestone highlights - Photo Streaks to encourage daily connection

- 6. Forgotten Dates Intelligent Reminders - Anniversary alerts with customizable reminders - Date planning assistance and suggestions - Automatic milestone detection (100 days, 1 year, etc.) - Notification system that enhances, not annoys
- 7. Generic UI Artistic Customization - 20+ UI preset themes (minimalist, romantic, artistic, playful, etc.) - Custom color schemes and typography - Theme sharing between partners - Seasonal themes and special occasion modes

1.3.3 Gain Creators (How LinkUp Delights Users)

- 1. Emotional Depth Through Design - Every interaction designed to evoke positive emotion - Thoughtful animations and transitions - Color psychology applied to enhance mood - Micro-interactions that feel magical
- 2. Effortless Quality Time - Watch Netflix with one tap/no setup, no lag - Spotify integration that just works - Voice/video calls with crystal-clear quality - Screen sharing without technical hassle
- 3. Discovery & Spontaneity - Surprise features that unlock based on usage - Smart date suggestions based on preferences - Weekly creative prompts for couple activities - Hidden achievements to discover together
- 4. Community Without Comparison - Connect with couple friends without public performance - Celebrate milestones privately or with selected circle - No likes, no followers, no algorithm-driven anxiety - Healthy community features (Couple Circles) without toxicity
- 5. Peace of Mind - Bank-level security with end-to-end encryption - No ads, no data selling, no tracking - Cross-platform sync that never fails - Automatic backup/memories never lost
- 6. Growth & Learning - Relationship insights (communication patterns, shared activities) - Tips for better communication - Optional relationship challenges - Curated content for relationship enrichment

1.3.4 Unique Selling Propositions (USPs)

- 1. The World's First Artistically-Driven Couple App > While competitors focus on functionality, LinkUp treats relationship communication as an art form. Every pixel is designed for beauty, every feature enables creative expression.
- 2. True All-in-One Solution > No other couple app combines communication + creativity + entertainment + social features in one seamless platform. LinkUp eliminates the need for 5-7 separate apps.
- 3. Entertainment Integration at Scale > First couple app with native Netflix, Prime Video, YouTube, TikTok, and Spotify integration. Watch together isn't a hack; it's a core feature.
- 4. Gamification That Strengthens Bonds > Photo Streaks, Hall of Fame, and milestones create positive engagement loops without manipulative dark patterns. We celebrate relationships, not exploit them.
- 5. Privacy-First Social Features > SingleFriend and Couple Circles provide social connection without sacrificing privacy. Share what you want, with who you want, when you want.
- 6. Multiplatform Monorepo Excellence > Identical beautiful experience on web, iOS, and Android. Feature parity, instant sync, seamless transitions.

1.3.5 Value Proposition Statement

For couples who crave deeper connection and creative expression,
LinkUp is an all-in-one social media platform
That combines communication, creativity, and shared experiences in one artistically beautiful app.
Unlike fragmented messaging apps and generic couple apps,
LinkUp treats your relationship as a canvas for artistic expression, celebration, and joy
Making every interaction feel special, private, and uniquely yours.

1.4 Key Success Metrics

1.4.1 North Star Metric

Weekly Couple Engagement Score (WCES)

Definition: Weighted composite score measuring meaningful couple interactions per week

Formula:

$$\begin{aligned} \text{WCES} = & (\\ & \text{Messages_Sent} \times 1 + \\ & \text{Voice_Minutes} \times 2 + \\ & \text{Video_Minutes} \times 3 + \\ & \text{Creative_Actions} \times 5 + \\ & \text{Shared_Content} \times 4 + \\ & \text{Milestone_Celebrations} \times 10 \\ &) / \text{Active_Weeks} \end{aligned}$$

Rationale: This metric captures not just usage, but quality of engagement. We prioritize: - Creative actions (drawing, custom emojis) = 5x weight (highest value) - Milestone celebrations = 10x weight (ultimate relationship strengthening) - Video time > Voice time > Text (deeper connection modes) - Shared experiences (watching together, listening together) = high weight

Target: $\text{WCES} > 150$ for retained users

Current Benchmark: N/A (pre-launch)

Why This Matters: A high WCES indicates couples are using LinkUp for meaningful connection, not passive scrolling. This metric directly correlates with relationship satisfaction and subscription retention.

1.4.2 Acquisition Metrics

A1. Couple Sign-Up Rate - Definition: Percentage of individual sign-ups who successfully pair with partner - Target: >75% within 7 days of first user signing up - Measurement: Tracked via couple_pairing table - Why: Solo users provide no value both partners must activate

A2. Organic vs. Paid Acquisition Split - Definition: Ratio of organic sign-ups to paid acquisition - Target: 60% organic / 40% paid by Month 12 - Measurement: UTM parameters and attribution tracking - Why: Viral growth indicates product-market fit and reduces CAC

A3. Customer Acquisition Cost (CAC) - Definition: Total marketing spend / New couple accounts acquired - Target: <\$15 per couple by Month 12 - Current Benchmark: \$25-35 (projected for early

months) - Why: Must maintain unit economics for profitability

A4. Viral Coefficient (K-Factor) - Definition: Average number of new couples invited per existing couple - Target: $K > 0.5$ by Month 9 (each couple brings 0.5 couples = exponential growth) - Measurement: Referral tracking system - Why: Couples apps have natural viralityfriend groups adopt together

A5. App Store Conversion Rate - Definition: (Downloads / Store Page Views) $\times 100$ - Target: $>25\%$ on iOS, $>20\%$ on Android - Measurement: App store analytics - Why: High conversion indicates strong positioning and creative assets

1.4.3 Activation Metrics

B1. Couple Activation Rate - Definition: % of couples who complete core setup within 3 days - Core Setup: Profile complete, first message sent, media uploaded, feature sampled - Target: $>80\%$ - Why: Activated couples are 5x more likely to retain past Week 4

B2. Time to First Meaningful Interaction - Definition: Minutes from sign-up to first voice call, video call, or creative action - Target: <60 minutes - Why: Quick wins drive habit formation

B3. Feature Adoption Breadth - Definition: Average number of distinct features used in first 7 days - Target: >5 features (of 15+ available) - Why: Users who sample variety stay longer

B4. Onboarding Completion Rate - Definition: % of couples completing full onboarding tutorial - Target: $>70\%$ - Why: Educated users extract more value

1.4.4 Engagement Metrics

C1. Daily Active Couples (DAC) - Definition: Couples where both partners active on given day - Target: 40% of MACs (Monthly Active Couples) by Month 12 - Why: Daily usage indicates habit formation

C2. Weekly Active Couples (WAC) - Definition: Couples where both partners active in given week - Target: 70% of MACs by Month 6 - Why: Weekly usage is realistic target for most couples

C3. Average Session Duration - Definition: Average time per app session - Target: 8-12 minutes - Why: Longer than messaging apps (2-3 min), shorter than social media (20+ min) = healthy engagement

C4. Sessions per Week - Definition: Average number of app opens per couple per week - Target: 15-20 sessions - Why: Multiple touchpoints = integrated into daily life

C5. Feature-Specific Engagement Rates

Feature	Daily Usage Target	Weekly Usage Target
Text Messaging	60%	95%
Voice Calls	15%	50%
Video Calls	8%	40%
Media Sharing	30%	70%
Scribble/Painting	10%	35%
Watch Together	5%	25%
Photo Streaks	50%	85%

Feature	Daily Usage Target	Weekly Usage Target
Custom Emojis	20%	60%
SoundBoard	12%	40%

C6. Streak Maintenance Rate - Definition: % of couples maintaining 7+ day photo streak - Target: >45% of active couples - Why: Streaks drive daily engagement and retention

1.4.5 Retention Metrics

D1. Week 1 Retention - Definition: % of couples active 7 days after sign-up - Target: >60% - Industry Benchmark: 40-50% for social apps - Why: Critical first hurdle must prove immediate value

D2. Month 1 Retention - Definition: % of couples active 30 days after sign-up - Target: >45% - Industry Benchmark: 25-35% for social apps - Why: Indicates habit formation

D3. Month 3 Retention - Definition: % of couples active 90 days after sign-up - Target: >35% - Why: Proxy for long-term retention

D4. Month 12 Retention - Definition: % of couples still active 1 year after sign-up - Target: >25% - Why: LTV calculation basis

D5. Cohort Retention Curves - Measurement: Weekly cohort retention tracked over 52 weeks - Target: Flatten curve after Week 8 (minimize additional churn) - Why: Long-tail retention drives LTV

D6. Churn Rate - Definition: % of couples who stop using app in given month - Target: <8% monthly churn - Acceptable: 10-12% in early months - Why: Inverse of retention must minimize

1.4.6 Monetization Metrics

E1. Free-to-Paid Conversion Rate - Definition: % of free couples who convert to paid tier - Target: >15% within 90 days - Industry Benchmark: 2-5% for freemium apps - Why: Premium features must provide clear value

E2. Average Revenue Per Couple (ARPC) - Definition: Total revenue / Total active couples - Target: >\$4/month (blended free + paid) - Calculation: If 20% convert at \$12/month = \$2.40 + other revenue - Why: Key profitability metric

E3. Customer Lifetime Value (LTV) - Definition: ARPC $\times$ Average Couple Lifespan (months) $\times$ Gross Margin - Target: >\$120 (assumes \$4 ARPC $\times$ 30 months $\times$ 100% GM) - Why: Must exceed CAC by 3-5x for healthy unit economics

E4. LTV:CAC Ratio - Definition: Customer Lifetime Value / Customer Acquisition Cost - Target: >5:1 (\$120 LTV / \$15 CAC = 8:1) - Healthy Range: 3:1 to 5:1 - Why: Indicates sustainable, scalable growth

E5. Monthly Recurring Revenue (MRR) - Definition: Predictable monthly subscription revenue - Targets: - Month 6: \$50k MRR - Month 12: \$300k MRR - Month 24: \$2M MRR - Month 36: \$6M MRR - Why: Subscription predictability enables planning

E6. Annual Contract Value (ACV) Adoption - Definition: % of paid couples on annual vs monthly plans - Target: >30% on annual plans by Month 12 - Why: Annual plans reduce churn and improve cash flow

E7. In-App Purchase Revenue - Definition: Revenue from theme purchases, emoji packs, soundboard sounds - Target: 10-15% of total revenue - Why: Diversifies revenue beyond subscriptions

1.4.7 Product Quality Metrics

F1. App Store Rating - Target: >4.5 stars on iOS and Android - Minimum: Never fall below 4.2 - Why: Ratings drive organic downloads

F2. Net Promoter Score (NPS) - Definition: How likely to recommend LinkUp to couple friends? (0-10) - Target: NPS >40 (% promoters - % detractors) - Calculation: (9-10 ratings % - 0-6 ratings %) - Why: Measures customer satisfaction and viral potential

F3. Crash-Free Rate - Target: >99.5% sessions crash-free - Industry Standard: >99% - Why: Crashes destroy trust and engagement

F4. API Response Time (p95) - Target: <200ms for 95th percentile - Critical: <500ms for 99th percentile - Why: Speed = quality user experience

F5. Video Call Quality Score - Definition: Weighted score based on connection success, video quality, latency - Target: >4.5/5 average user-reported quality - Why: Calls are core feature must be excellent

F6. Customer Support Resolution Time - Target: <4 hours median response time - Target: >90% issues resolved within 24 hours - Why: Responsive support drives satisfaction

1.4.8 Market Position Metrics

G1. Market Share in Couple App Category - Target: Top 3 couple app by downloads in US by Month 18 - Target: Top 5 globally by Month 24 - Why: Category leadership drives organic growth

G2. Brand Awareness - Definition: % of target demographic who recognize LinkUp brand - Target: >30% unaided awareness in key markets by Month 24 - Measurement: Quarterly surveys - Why: Brand recognition reduces CAC

G3. Social Media Mentions - Target: 10k+ monthly mentions across TikTok, Instagram, Twitter by Month 12 - Why: Organic buzz indicates product-market fit

G4. Press Coverage - Target: Featured in TechCrunch, Wired, Fast Company within 12 months - Why: Press validates product and drives credibility

1.4.9 Success Metrics Summary Dashboard

Phase 1 (Months 1-6): MVP Launch & Validation

Primary Goals:

5,000 couple sign-ups

60%+ Week 1 retention

4.3+ app store rating

WCES >100 for active couples

NPS >30

Monetization:

- Launch premium tier Month 4
- 5%+ conversion rate
- \$50k MRR by Month 6

Phase 2 (Months 7-12): Growth & Optimization

Primary Goals:

- 50,000 couple sign-ups
- 45%+ Month 1 retention
- 4.5+ app store rating
- WCES >125 for active couples
- NPS >40

Monetization:

- 15%+ conversion rate
- \$4+ ARPC
- \$300k MRR by Month 12

Phase 3 (Months 13-24): Scale

Primary Goals:

- 500,000 couple sign-ups
- 35%+ Month 3 retention
- 4.6+ app store rating
- WCES >150 for active couples
- NPS >50

Monetization:

- 20%+ conversion rate
- \$6+ ARPC
- \$2M MRR by Month 24

1.5 Competitive Landscape

1.5.1 Market Overview

The couple app market is fragmented across three primary categories:

Category 1: Dedicated Couple Apps - Between, Couple, Couply, Happy Couple, Lasting - Market Size: ~15M total users globally (2025) - Growth: 8-12% YoY - Limitations: Basic features, poor UI/UX, low engagement

Category 2: Messaging Apps (Used by Couples) - WhatsApp, Telegram, iMessage, WeChat - Market Size: Billions of users, majority of couples - Limitations: Not couple-specific, no relationship features

Category 3: Social Media (Couple Content) - Instagram (couple posts), Facebook (relationship status), TikTok (couple content) - Market Size: Billions, nearly universal - Limitations: Public-

first, not intimate, algorithm-driven

Market Opportunity: \$2.3B addressable market for dedicated couple communication platforms by 2028 (estimated 100M couple accounts @ \$23 ARPU)

1.5.2 Direct Competitors Analysis

Competitor 1: Between

Aspect	Analysis
Users	~8M downloads, ~1.5M active couples
Founded	2011 (South Korea), acquired by IAC
Core Features	Chat, shared calendar, photo albums, memory box
Strengths	Established brand in Asia Clean, simple interface Strong in South Korea/Japan
Weaknesses	Outdated UI (hasnt evolved much since 2015) Limited creative features No voice/video calls built-in No entertainment integrations Minimal engagement features beyond chat
Pricing	Free with ads, Premium \$5/month
Rating	4.2 stars (iOS), 4.1 stars (Android)
LinkUp Advantage	Superior UI/UX, creative tools, voice/video calls, entertainment integration, gamification

Competitor 2: Couple (renamed from Couple)

Aspect	Analysis
Users	~5M downloads, ~800k active couples
Founded	2012 (USA), struggled with pivot attempts
Core Features	Messaging, ThumbKiss feature, shared lists, moments
Strengths	Innovative early features (ThumbKiss was viral) US market presence Some creative elements
Weaknesses	Stagnant development (minimal updates 2020-2025) Poor monetization (never achieved sustainability) Buggy video calls No modern entertainment integration Declining user base
Pricing	Free with limited premium (\$4/month)
Rating	3.9 stars (iOS), 3.7 stars (Android)
LinkUp Advantage	Active development, modern tech stack, superior features across the board

Competitor 3: Couply

Aspect	Analysis
Users	~2M downloads, ~400k active couples
Founded	2019 (newer entrant)
Core Features	Games, quizzes, questions, shared calendar
Strengths	Gamification focus (relationship quizzes) Modern UI compared to Between/Couple Good engagement mechanics
Weaknesses	Limited communication features (basic chat only) No voice/video calls No creative tools Games feel gimmicky rather than authentic Small feature set overall
Pricing	Free with Premium \$8/month (expensive for limited features)
Rating	4.4 stars (iOS), 4.2 stars (Android)
LinkUp Advantage	Complete feature set, creative tools, entertainment integration, authentic engagement vs gimmicks

Competitor 4: Happy Couple / Lasting

Aspect	Analysis
Users	~3M downloads combined
Founded	2013 (Happy Couple), 2017 (Lasting)
Core Features	Relationship therapy, educational content, quizzes
Strengths	Therapeutic/educational angle Science-backed content Different positioning (relationship health)
Weaknesses	Not communication platforms educational apps High churn (complete content, leave) Expensive (\$70-120/year for Lasting) Feels like homework, not fun
Pricing	\$60-120/year
Rating	4.5 stars (both apps educational apps rate higher)
LinkUp Advantage	Daily-use platform vs educational tool, fun/creative vs clinical, integrated communication

1.5.3 Indirect Competitors Analysis

Indirect Competitor 1: WhatsApp (Messaging)

Aspect	Analysis
Threat Level	HIGHubiquitous, default choice for many couples
Strengths	2B+ users globally End-to-end encryption Free Reliable Cross-platform Network effects
Weaknesses	Genericnot couple-focused No creative tools No relationship features No entertainment integration Owned by Meta (privacy concerns)

Aspect	Analysis
Why LinkUp Wins	Couples want more than functional messaging they want celebration, creativity, shared experiences. WhatsApp is the baseline; LinkUp is the upgrade.

Indirect Competitor 2: Discord (Community Platform)

Aspect	Analysis
Threat Level	MEDIUM some couples use private servers
Strengths	Excellent voice quality Screen sharing Customizable Free Gaming-friendly
Weaknesses	Built for communities, not couples Overwhelming UI for non-gamers No relationship-specific features No entertainment integrations Requires technical setup
Why LinkUp Wins	Discord is for gamers/communities. LinkUp borrows its quality (voice/screen sharing) but designs for couples specifically.

Indirect Competitor 3: Instagram (Social Media)

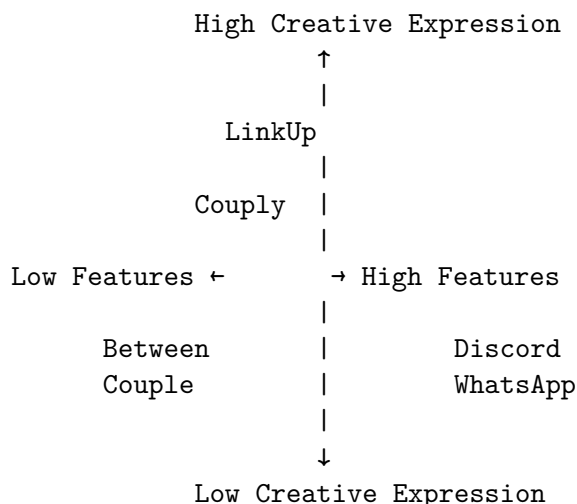
Aspect	Analysis
Threat Level	MEDIUM couples share moments publicly
Strengths	Massive user base Beautiful visual platform Stories feature DMs for couple communication
Weaknesses	Public performance culture Algorithm-driven No private couple space Addictive dark patterns Not designed for intimate communication
Why LinkUp Wins	Instagram is for the world; LinkUp is for your partner. Privacy-first vs public-first. Authentic connection vs performance.

Indirect Competitor 4: FaceTime / Zoom (Video Calls)

Aspect	Analysis
Threat Level	MEDIUM default for video calls
Strengths	High-quality video Reliable Simple Built into devices (FaceTime)
Weaknesses	Just calls no additional couple features No creative elements No media sharing during calls No watch-together functionality Standalone not integrated platform

Aspect	Analysis
Why LinkUp Wins	FaceTime/Zoom are tools; LinkUp is an experience. Calls are integrated with messaging, media, creativity, entertainment.

1.5.4 Competitive Positioning Map



Positioning Analysis: - LinkUp: High features + High creativity (unique quadrant) - Between/Couple: Medium features + Low creativity (stagnant) - Couply: Low features + Medium creativity (limited scope) - Discord/WhatsApp: High features + Low creativity (not couple-focused)

1.5.5 Competitive Advantages Summary

1. Feature Completeness - Only app with communication + creativity + entertainment + social features - Voice/video calls with screen sharing (better than Between/Couple) - Entertainment integration (unique)
2. Artistic Design - UI/UX far superior to all dedicated couple apps - Customization (presets) unmatched - Emotional design prioritized
3. Modern Technology - Monorepo architecture for fast iteration - Latest frameworks (React/Next.js, React Native) - Superior performance vs legacy apps
4. Balanced Engagement - Gamification (streaks, hall of fame) without manipulation - Positive psychology principles - Healthy engagement vs addictive patterns
5. Privacy + Selective Social - End-to-end encryption (baseline) - SingleFriend and Couple Circles (unique) - Control without isolation
6. Market Timing - Couple apps havent innovated since 2015-2018 - Gen Z couples demand better aesthetic - Post-pandemic: normalized long-distance, video calls, online shared experiences

1.6 Product Positioning

1.6.1 Positioning Statement

For couples aged 20-40 who seek deeper emotional connection through technology,
LinkUp is the artistically-driven, all-in-one relationship platform
That transforms everyday communication into creative expression and shared experiences.
Unlike fragmented messaging apps and outdated couple apps,
LinkUp combines stunning design, powerful features, and intelligent engagement
To make your relationship feel celebrated, private, and uniquely beautiful.

1.6.2 Brand Personality

If LinkUp were a person: - Creative - Always finding new ways to express ideas - Romantic - Believes in and celebrates love - Delightful - Brings joy to everyday moments - Trustworthy - Respects privacy and maintains confidentiality - Thoughtful - Anticipates needs before being asked - Aspirational - Inspires couples to strengthen their relationships

Tone of Voice: - Warm but not saccharine - Playful but not childish - Sophisticated but not pretentious - Encouraging but not preachy - Intimate but not exclusive

1.6.3 Market Entry Strategy

Phase 1: Early Adopters (Months 1-6) - Target: Tech-savvy couples, long-distance relationships, creative romantics - Channels: Product Hunt launch, TikTok influencers, Reddit (r/LongDistance) - Message: The beautiful couple app you've been waiting for - Goal: 5,000 couples, validate product-market fit

Phase 2: Early Majority (Months 7-18) - Target: Established couples, young married, engaged couples - Channels: Instagram ads, YouTube couples influencers, App Store featuring - Message: Where your relationship feels at home - Goal: 50,000 couples, establish market position

Phase 3: Mainstream (Months 19-36) - Target: All couples 20-40 - Channels: Broad digital marketing, partnerships, PR - Message: LinkUpThe couple app - Goal: 500,000 couples, category leadership

1.6.4 Go-to-Market Differentiation

What We Say: - The first couple app designed by artists, for lovers - All your couple needs, in one beautiful place - Your relationship deserves better than generic messaging

What We Don't Say: - Track your relationship (sounds clinical) - Improve your communication (sounds like therapy) - Stay connected (generic, every app claims this)

Visual Identity: - Gradients, soft colors, organic shapes - Couple-created art featured in marketing - Real couple photos, not stock imagery - Emphasis on beauty and emotion

1.6.5 Critical Success Factors

Must-Have for Success: 1. Flawless technical execution - Calls, sync, performance must be perfect 2. Stunning UI/UX - Design must exceed expectations 3. Viral growth mechanics - Product must naturally spread couple-to-couple 4. Rapid iteration - Monthly feature releases to stay ahead 5. Community building - Engaged users become advocates

Risk Factors: 1. Chicken-and-egg problem - Both partners must adopt (mitigated by seamless pairing) 2. Entertainment API limitations - Netflix/Spotify may restrict access (mitigated by workarounds) 3. Privacy concerns - Couples share intimate data (mitigated by encryption + transparency) 4. Breakup churn - Couples break up (inevitable, mitigated by strong engagement during relationship) 5. Feature creep - Too many features overwhelm (mitigated by progressive disclosure)

Chapter 1 Status: Complete

Next Chapter: Chapter 2: Core Communication Features

Chapter 2: Core Communication Features

Overview

Core communication features form the foundation of LinkUp, enabling couples to connect through text, voice, and video. Unlike generic messaging apps, LinkUps communication features are designed specifically for intimate couple interactions with emphasis on:

- Emotional depth - Beyond functional messaging to meaningful conversation
- Creative expression - Rich formatting, highlighting, custom reactions
- Privacy & security - End-to-end encryption for all communications
- Seamless integration - Communication flows naturally with other features
- Quality over quantity - Crystal-clear audio/video, minimal latency

This chapter details the technical specifications, data structures, user flows, and UI/UX requirements for all core communication features.

2.1 Text Communication System

2.1.1 Real-time Messaging

Overview: The text messaging system is the primary communication channel for couples. It supports real-time delivery, rich formatting, threads, reactions, and special highlighting - all within an artistically designed interface.

Core Requirements:

1. Real-time Delivery
 - WebSocket-based instant message delivery (<100ms latency)
 - Automatic reconnection on network interruption
 - Offline message queuing with automatic send on reconnection
 - Delivery and read receipts with timestamps
2. Rich Text Formatting
 - Bold, italic, strikethrough, code formatting
 - @ mentions (for threading and context)
 - Emoji picker with custom emojis (Chapter 3)
 - Link previews with thumbnail, title, description

- Message editing with edit history (shown to partner)
 - Message deletion (both sides) within 24 hours
3. Message Types
- Text messages (up to 10,000 characters)
 - Photo messages (inline display)
 - Video messages (inline player)
 - Voice messages (waveform display)
 - File attachments (up to 100MB)
 - Location sharing (map preview)
 - Contact sharing
 - Poll/question messages
 - Scribble/drawing messages (Chapter 3)

Data Schema:

```
interface Message {
  id: string; // UUID
  coupleId: string; // Reference to couple
  senderId: string; // User who sent message
  receiverId: string; // Partner user ID

  // Content
  content: string; // Message text (encrypted)
  messageType: 'text' | 'photo' | 'video' | 'voice' | 'file' |
    'location' | 'scribble' | 'poll';

  // Rich content
  mediaUrls?: string[]; // URLs for photos/videos
  fileMetadata?: {
    fileName: string;
    fileSize: number;
    mimeType: string;
    thumbnailUrl?: string;
  };

  // Formatting & features
  formatting?: {
    bold?: number[] []; // [[start, end], ...]
    italic?: number[] [];
    strikethrough?: number[] [];
    mentions?: string[]; // User IDs mentioned
  };

  // Threading
  threadId?: string; // Reference to thread
  isThreadStarter?: boolean;
  threadPreview?: string; // First message of thread

  // Special features
```

```

isHighlighted?: boolean;           // Highlighted special text
highlightColor?: string;           // Custom highlight color
highlightNote?: string;            // Why it's special

// Reactions
reactions?: {
  [emoji: string]: {
    userId: string;
    timestamp: Date;
  }[];
};

// Status tracking
status: 'sending' | 'sent' | 'delivered' | 'read' | 'failed';
sentAt: Date;
deliveredAt?: Date;
readAt?: Date;

// Edit history
isEdited?: boolean;
editedAt?: Date;
originalContent?: string;

// Deletion
isDeleted?: boolean;
deletedAt?: Date;
deletedBy?: string;

// Metadata
createdAt: Date;
updatedAt: Date;

// Encryption
encryptionKeyId: string;           // For E2E encryption
encryptionVersion: string;
}

```

User Flow - Sending a Message:

1. User types message in input field
 - Input field expands for multiline (up to 10 lines visible)
 - Character count shown after 8,000 chars
 - Formatting toolbar appears on text selection
2. User optionally applies formatting
 - Select text → toolbar appears with B, I, S, Code
 - Add emojis via picker (including custom emojis)
 - Paste links → auto-preview generation starts

3. User presses Send button (or Cmd/Ctrl + Enter)
 - Message encrypted client-side
 - Optimistic UI update (message appears immediately)
 - WebSocket sends message to server
 - Status: "sending" → shows loading indicator
4. Server receives and processes
 - Validates message (length, content, permissions)
 - Saves to database
 - Sends to recipient via WebSocket
 - Sends push notification if recipient offline
 - Returns confirmation to sender
5. Sender receives confirmation
 - Status: "sending" → "sent" (single checkmark)
 - Timestamp displayed
6. Recipient receives message
 - Message appears in chat (with animation)
 - Delivery receipt sent to server
 - Sender's message status: "sent" → "delivered" (double checkmark)
 - If recipient has chat open, read receipt sent immediately
7. Recipient reads message
 - Read receipt sent to server
 - Sender's message status: "delivered" → "read" (blue checkmarks)
 - Read timestamp displayed on hover

UI/UX Specifications:

Message Bubbles:

```
/* Sender bubbles (right-aligned) */
.message-bubble--sent {
  background: var(--gradient-primary); /* Customizable per theme */
  color: white;
  border-radius: 18px 18px 4px 18px;
  padding: 10px 14px;
  max-width: 70%;
  margin-left: auto;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Receiver bubbles (left-aligned) */
.message-bubble--received {
  background: var(--color-surface);
  color: var(--color-text);
  border-radius: 18px 18px 18px 4px;
  padding: 10px 14px;
```



```

    max-width: 70%;
    margin-right: auto;
    box-shadow: 0 1px 4px rgba(0, 0, 0, 0.08);
}

```

```

/* Highlighted messages */
.message-bubble--highlighted {
    border: 2px solid var(--highlight-color);
    background: linear-gradient(
        135deg,
        var(--highlight-color-10%) 0%,
        transparent 100%
    );
    position: relative;
}

```

```

.message-bubble--highlighted::before {
    content: ' ';
    position: absolute;
    top: -8px;
    right: -8px;
    font-size: 16px;
}

```

Message Input:

```

// Input field component specifications
interface MessageInputProps {
    placeholder: string;           // "Message [Partner Name]..."
    maxLength: number;            // 10,000 characters
    autoExpand: boolean;          // Expand up to 10 lines
    showFormatting: boolean;       // Formatting toolbar
    emojiPicker: boolean;         // Custom emoji picker
    attachmentTypes: string[];    // Allowed attachment types
}

```

```

// Visual design
const MessageInputDesign = {
    height: '44px',                // Base height
    maxHeight: '200px',           // ~10 lines
    borderRadius: '22px',
    background: 'var(--color-input-bg)',
    border: '1px solid var(--color-border)',
    padding: '10px 48px 10px 16px', // Space for send button
    fontSize: '15px',
    lineHeight: '24px',

    // Focus state
    focusBorder: '2px solid var(--color-primary)',
}

```

```

focusBackground: 'var(--color-input-bg-focus)',

// Send button (inside input, right side)
sendButton: {
  position: 'absolute',
  right: '8px',
  bottom: '8px',
  width: '32px',
  height: '32px',
  borderRadius: '50%',
  background: 'var(--gradient-primary)',
  color: 'white',
  disabled: 'var(--color-disabled)',
}
};

```

Performance Requirements: - Message send latency: <100ms (p95) - Message delivery: <200ms (p95) - Message encryption: <10ms - Message load (50 messages): <500ms - Scroll performance: 60fps smooth scrolling - Image thumbnails: <300ms load time

2.1.2 Threads

Overview: Threads allow couples to organize conversations around specific topics, creating sub-conversations without cluttering the main chat. Perfect for planning trips, discussing decisions, or revisiting memories.

Core Features:

1. Thread Creation
 - Any message can start a thread
 - Long-press or right-click message Reply in Thread
 - Thread icon shows number of replies
 - Preview shows first reply text
2. Thread UI
 - Opens in side panel (web) or modal (mobile)
 - Shows original message at top (pinned)
 - Threaded replies below in chronological order
 - Can send messages in thread or main chat simultaneously
3. Thread Management
 - Mark thread as resolved
 - Add thread title/topic
 - Thread notifications (separate from main chat)
 - Search within thread
 - Export thread as separate conversation

Data Schema:

```

interface Thread {
  id: string;

```

```

coupleId: string;

// Thread metadata
starterMessageId: string;           // Original message that started thread
title?: string;                     // Optional user-defined title
topic?: string;                      // Auto-detected or user-set topic

// Participants (always both partners)
participantIds: string[];           // [userId1, userId2]

// Thread state
status: 'active' | 'resolved' | 'archived';
resolvedAt?: Date;
resolvedBy?: string;

// Stats
messageCount: number;
lastMessageAt: Date;
lastMessagePreview: string;

// UI state
isBookmarked?: boolean;
color?: string;                     // Custom thread color

// Metadata
createdAt: Date;
updatedAt: Date;
}

```

// Messages in thread reference threadId (see Message schema above)

User Flow - Creating and Using Thread:

1. User long-presses message in main chat
 - Context menu appears with "Reply in Thread" option
2. User taps "Reply in Thread"
 - Thread panel slides in (web) or modal opens (mobile)
 - Original message displayed at top with context
 - Thread input field focused
 - Thread ID created and stored
3. User types reply in thread
 - Reply saves with threadId reference
 - Main chat message shows thread icon with count "1"
 - Partner receives notification "Reply in thread"
4. Partner opens thread
 - Taps thread icon on original message

Thread panel opens with full conversation
Can continue replying in thread

5. Thread management

Either user can mark thread as "Resolved"
Resolved threads collapse in main chat
Can view "All Threads" to see active and resolved
Can archive old threads

UI Components:

```
// Thread indicator on message
interface ThreadIndicator {
    icon: 'thread_icon',           // Visual indicator
    replyCount: number,           // "3 replies"
    lastReplyTime: Date,          // "2m ago"
    unreadReplies: boolean,       // Show badge if unread
}

// Thread panel layout
interface ThreadPanel {
    width: '400px',               // Desktop width
    height: '100vh',              // Full height
    position: 'fixed',            // Fixed to right side
    background: 'var(--color-bg-secondary)',

    header: {
        originalMessage: Message, // Pinned at top
        title?: string,            // Thread title
        closeButton: true,
        resolveButton: true,
    },

    messages: Message[],          // Thread messages
    input: MessageInput,          // Thread reply input
}
```

2.1.3 Highlighting Special Texts

Overview: Couples often exchange messages that are particularly meaningful declarations of love, important decisions, funny inside jokes, or memorable moments. Highlighting allows users to mark these special messages for easy retrieval and celebration.

Core Features:

1. Highlight Message
 - Long-press message Highlight this message
 - Choose highlight color (6 preset colors + custom)

- Add optional note about why its special
 - Message gets visual treatment (border, background tint, star icon)
2. Highlight Categories
 - Love & Affection
 - Funny Moments
 - Important Decisions
 - Celebrations
 - Milestones
 - Custom category
 3. Highlighted Messages View
 - Dedicated page showing all highlighted messages
 - Filter by category, date, color
 - Timeline view of highlights
 - Export highlights as memory book

Data Schema:

```
interface HighlightedMessage {
  id: string;
  messageId: string;           // Reference to original message
  coupleId: string;
  highlightedBy: string;       // User who highlighted

  // Highlight properties
  category: 'love' | 'funny' | 'important' | 'celebration' |
    'milestone' | 'custom';
  color: string;               // Hex color
  note?: string;               // Why it's special
  emoji?: string;              // Optional emoji marker

  // Visibility
  isSharedWithPartner: boolean; // Partner can see it was highlighted

  // Metadata
  highlightedAt: Date;
  createdAt: Date;
}
```

User Flow:

1. User long-presses meaningful message
Context menu shows "Highlight Message "
2. User taps "Highlight Message"
Highlight modal opens
Shows message preview
Options presented:
 - Choose category (Love, Funny, Important, etc.)
 - Select color (pink, yellow, blue, green, purple, orange)
 - Add note (optional): "Why is this special?"

Toggle "Show partner I highlighted this"

3. User confirms highlight

- Message visually updated with highlight treatment
- Saved to highlighted_messages table
- Partner sees highlight indicator (if shared)
- Added to "Highlights" collection

4. Viewing highlights

- User navigates to "Highlights" page
- Sees timeline of all highlighted messages
- Can filter by category, date range, or color
- Can tap any highlight to jump to original in chat

Visual Design:

```
/* Highlighted message treatment */
.message--highlighted {
  position: relative;
  border-left: 4px solid var(--highlight-color);
  background: linear-gradient(
    90deg,
    var(--highlight-color-alpha-10) 0%,
    transparent 100%
  );
  padding-left: 16px;
}

.message--highlighted::before {
  content: ' ';
  position: absolute;
  left: -2px;
  top: -10px;
  font-size: 18px;
  filter: drop-shadow(0 2px 4px rgba(0,0,0,0.2));
}

/* Highlight category colors */
.highlight-category {
  love: '#FF6B9D',
  funny: '#FFD93D',           /* Yellow */
  important: '#6BCB77',      /* Green */
  celebration: '#A084DC',    /* Purple */
  milestone: '#FF8E53',      /* Orange */
  custom: '#4D96FF'          /* Blue */
}
```

2.1.4 Message Reactions

Overview: Quick emoji reactions allow instant responses without composing a message. Support for custom emojis makes reactions personal and playful.

Features: - React to any message with emoji - Support standard emojis + custom couple emojis - Multiple reactions per message (both partners can react differently) - Quick react bar (5 recently used emojis) - Full emoji picker for more options - Remove reaction by tapping again

Data Schema:

```
// Reactions stored within Message schema (see 2.1.1)
reactions: {
  [emoji: string]: {
    userId: string;
    timestamp: Date;
  }[]
}

// Example:
{
  " ": [{ userId: "user1", timestamp: "2026-01-20T10:30:00Z" }],
  " ": [{ userId: "user2", timestamp: "2026-01-20T10:31:00Z" }],
  "custom_emoji_id_123": [{ userId: "user1", timestamp: "2026-01-20T10:32:00Z" }]
}
```

UI Specifications:

```
// Reaction interface
interface MessageReaction {
  // Quick react bar (hover/long-press message)
  quickReactions: string[]; // [' ', ' ', ' ', ' ', ' ']

  // Full picker
  fullPicker: {
    standardEmojis: EmojiCategory[],
    customEmojis: CustomEmoji[], // Couple's custom emojis
    recentlyUsed: string[],
  },

  // Display on message
  reactionDisplay: {
    maxVisibleReactions: 3, // Show max 3, then "+2"
    position: 'bottom-right', // Relative to message bubble
    style: 'pill', // Rounded pill shape
  }
}
```

2.2 Voice Communication

2.2.1 Voice Calls

Overview: High-quality voice calls with minimal latency, designed for intimate couple conversations. Superior audio quality with noise cancellation and echo reduction.

Core Requirements:

1. Call Quality
 - Opus codec for optimal quality (48kHz, 64kbps)
 - Adaptive bitrate based on network conditions
 - Noise cancellation and echo reduction
 - Auto-gain control for consistent volume
2. Call Features
 - One-tap call initiation
 - Call waiting and call back
 - Mute/unmute with indicator
 - Speaker/earpiece toggle
 - Bluetooth device support
 - Background call support (continue while using other features)
3. Call UI
 - Full-screen call interface
 - Partner avatar with subtle animation
 - Call duration timer
 - Network quality indicator
 - In-call controls (mute, speaker, end)
 - Minimize to floating bubble (continue using app)

Data Schema:

```
interface VoiceCall {
  id: string;
  coupleId: string;

  // Participants
  callerId: string;           // Who initiated
  receiverId: string;         // Partner

  // Call state
  status: 'initiating' | 'ringing' | 'active' | 'ended' |
    'missed' | 'declined' | 'failed';

  // Timing
  initiatedAt: Date;
  answeredAt?: Date;
  endedAt?: Date;
  duration?: number;          // Seconds

  // Call quality metrics
```



```

qualityMetrics?: {
  avgBitrate: number,
  packetLoss: number,           // Percentage
  jitter: number,               // Milliseconds
  rtt: number,                  // Round trip time
  audioCodec: string,
},

// End reason
endReason?: 'normal' | 'network_error' | 'declined' | 'timeout',

// WebRTC connection
webrtcSessionId: string,

createdAt: Date;
updatedAt: Date;
}

```

User Flow - Making Voice Call:

1. User taps phone icon in chat header
 - Call initiated instantly
 - Ringtone plays for caller
 - Push notification sent to partner
2. Partner's device rings
 - Full-screen incoming call UI appears
 - Options: Answer, Decline, Message
 - Ringtone plays with vibration
3. Partner answers
 - WebRTC connection established
 - Audio streams start
 - Both see active call UI
 - Call duration timer starts
4. During call
 - Can mute/unmute
 - Can switch speaker/earpiece
 - Can minimize to bubble (use other features)
 - Can switch to video call
 - Network quality indicator shows connection
5. Ending call
 - Either user taps end button
 - Call disconnects gracefully
 - Call log saved with duration
 - "Call ended • 12m 34s" message appears in chat
 - Option to rate call quality (feedback for improvements)

Technical Specifications:

```
// WebRTC configuration for voice calls
const VoiceCallConfig = {
  iceServers: [
    { urls: 'stun:stun.linkup.app:3478' },
    { urls: 'turn:turn.linkup.app:3478',
      username: 'linkup',
      credential: 'encrypted_credential' }
  ],

  audioConstraints: {
    echoCancellation: true,
    noiseSuppression: true,
    autoGainControl: true,
    sampleRate: 48000,
    channelCount: 1, // Mono for voice
  },

  codec: {
    preferred: 'opus',
    fallback: ['ISAC', 'PCMU'],
    bitrate: {
      min: 16000, // 16 kbps minimum
      target: 64000, // 64 kbps target
      max: 96000, // 96 kbps maximum
    }
  },

  // Connection timeouts
  connectionTimeout: 30000, // 30 seconds
  reconnectAttempts: 3,
  reconnectDelay: 2000,
};
```

2.2.2 Voice Messages

Overview: Asynchronous voice messages for when text feels insufficient but a call isn't necessary. Record, preview, and send voice notes with waveform visualization.

Features: - Press-and-hold to record (up to 5 minutes) - Swipe up to lock recording (hands-free) - Preview before sending - Playback with waveform animation - Playback speed control (1x, 1.5x, 2x) - Transcription (optional, AI-powered)

Data Schema:

```
interface VoiceMessage {
  id: string;
```

```

messageId: string;           // Links to Message with type 'voice'
coupleId: string;
senderId: string;

// Audio file
audioUrl: string;           // S3/CDN URL
duration: number;           // Seconds
fileSize: number;           // Bytes
waveformData: number[];     // Amplitude values for visualization

// Transcription (optional)
transcription?: string;
transcriptionConfidence?: number; // 0-1

// Playback tracking
isPlayed: boolean;
playedAt?: Date;

createdAt: Date;
}

```

UI Design:

```

// Voice message bubble
interface VoiceMessageBubble {
  layout: 'inline',           // Within chat flow
  minWidth: '200px',
  height: '56px',

  components: {
    playButton: {
      size: '40px',
      icon: 'play' | 'pause',
      position: 'left',
    },

    waveform: {
      width: 'flex-grow',
      height: '32px',
      bars: 40,                // Number of bars
      color: 'var(--color-primary)',
      playedColor: 'var(--color-primary-dark)',
    },

    duration: {
      text: '1:24',
      position: 'right',
      fontSize: '13px',
    },
  },
}

```

```

    speedControl: {
      position: 'bottom-right',
      options: ['1x', '1.5x', '2x'],
    }
  }
}

```

2.3 Video Communication

2.3.1 Video Calls

Overview: High-definition video calls with adaptive quality, screen sharing, and picture-in-picture mode. Designed for couples who want to see each other, not just hear.

Core Features:

1. Video Quality
 - HD video up to 1080p (adaptive based on network)
 - 30fps default, 60fps for high-end devices
 - H.264 codec with hardware acceleration
 - Smart resolution scaling (720p, 480p, 360p fallback)
2. Advanced Features
 - Front/back camera switch
 - Video pause (maintain voice, freeze video)
 - Virtual backgrounds (blur, custom images)
 - Beauty filters (subtle, optional)
 - Grid view for screen share + faces
 - Picture-in-picture mode
3. UI Modes
 - Full-screen immersive
 - Floating window (use app while on call)
 - Minimized to call bar
 - Screen share mode

Data Schema:

```

interface VideoCall {
  id: string;
  coupleId: string;

  // Participants
  callerId: string;
  receiverId: string;

  // Call state
  status: 'initiating' | 'ringing' | 'active' | 'ended' |
    'missed' | 'declined' | 'failed';
  isVideoEnabled: boolean; // Can toggle to audio-only
}

```

```

// Timing
initiatedAt: Date;
answeredAt?: Date;
endedAt?: Date;
duration?: number;

// Video settings
videoSettings: {
  resolution: '1080p' | '720p' | '480p' | '360p';
  fps: 30 | 60;
  cameraFacing: 'front' | 'back';
  backgroundBlur: boolean;
  beautyFilter: 0 | 1 | 2 | 3;    // 0 = off, 3 = max
};

// Quality metrics
qualityMetrics?: {
  avgVideoBitrate: number,
  avgAudioBitrate: number,
  packetLoss: number,
  jitter: number,
  rtt: number,
  videoCodec: string,
  audioCodec: string,
  resolutionSwitches: number,    // How many times resolution changed
};

// Features used
screenShareEnabled: boolean;
screenShareDuration?: number;

// End reason
endReason?: 'normal' | 'network_error' | 'declined' | 'timeout';

// WebRTC
webrtcSessionId: string;

createdAt: Date;
updatedAt: Date;
}

```

Technical Configuration:

```

const VideoCallConfig = {
  iceServers: [
    { urls: 'stun:stun.linkup.app:3478' },
    { urls: 'turn:turn.linkup.app:3478',
      username: 'linkup',

```

```

        credential: 'encrypted_credential' }
    ],

    videoConstraints: {
        width: { ideal: 1920, max: 1920 },
        height: { ideal: 1080, max: 1080 },
        frameRate: { ideal: 30, max: 60 },
        facingMode: 'user',           // Front camera default
    },

    audioConstraints: {
        echoCancellation: true,
        noiseSuppression: true,
        autoGainControl: true,
        sampleRate: 48000,
        channelCount: 1,
    },

    videoCodec: {
        preferred: 'H264',
        profile: 'Constrained Baseline',
        fallback: ['VP8', 'VP9'],
        bitrateMode: 'adaptive',
        bitrate: {
            min: 150000,                // 150 kbps
            target: 2000000,            // 2 Mbps for 1080p
            max: 4000000,                // 4 Mbps max
        }
    },

    adaptiveQuality: {
        enabled: true,
        thresholds: {
            excellent: { rtt: 50, packetLoss: 0.5 }, // 1080p
            good: { rtt: 100, packetLoss: 1 },       // 720p
            fair: { rtt: 200, packetLoss: 3 },        // 480p
            poor: { rtt: 300, packetLoss: 5 },        // 360p
        }
    }
};

```

2.3.2 Screen Sharing

Overview: Share your screen with your partner during calls - perfect for browsing together, showing content, or collaborative activities.

Features: - Share entire screen or specific window - Audio sharing for system sounds - Cursor

highlighting - Annotation tools (point, draw) - Permission request before sharing - Picture-in-picture (see partner while sharing)

User Flow:

1. During video call, user taps "Share Screen" button
Permission prompt appears (first time only)
2. User selects what to share
Entire screen
Specific window (browser, app, etc.)
Confirm share
3. Screen sharing starts
Partner sees shared screen in full view
Sharer sees small PiP of partner's video
Both can annotate (optional)
"Sharing screen" indicator shows
4. During screen share
Pause sharing (freeze screen)
Switch what's being shared
Annotation tools available
End sharing anytime
5. End screen share
User taps "Stop Sharing"
Returns to normal video call
Duration tracked in call log

Data Schema:

```
interface ScreenShareSession {  
  id: string;  
  callId: string;           // Parent video/voice call  
  sharerId: string;  
  
  // Share settings  
  shareType: 'entire_screen' | 'window' | 'tab';  
  includeAudio: boolean;  
  
  // Timing  
  startedAt: Date;  
  endedAt?: Date;  
  duration?: number;  
  
  // Annotations  
  annotations?: {  
    type: 'cursor' | 'draw' | 'pointer';  
    data: any;  
  };  
}
```

```

    timestamp: Date;
}[];

// Quality
resolution: string;
fps: number;
avgBitrate: number;

createdAt: Date;
}

```

2.4 Media Storage & Sharing

2.4.1 Photo Sharing & Storage

Overview: Couples generate thousands of memories together. LinkUp provides unlimited cloud storage for photos with intelligent organization, albums, and easy sharing.

Core Features:

1. Photo Upload
 - Drag-and-drop (web) or gallery select (mobile)
 - Batch upload up to 50 photos at once
 - Auto-compression (smart quality preservation)
 - HEIC/HEIF support with conversion
 - Metadata preservation (date, location)
2. Photo Organization
 - Auto-organize by date
 - Smart albums (selfies, food, travel, pets)
 - Manual albums creation
 - Tags and descriptions
 - Favorites collection
3. Photo Viewing
 - Grid view (3 or 4 columns)
 - Lightbox viewer with swipe
 - Zoom and pan
 - Photo details (date, location, size)
 - Slideshow mode
4. Photo Sharing
 - Share in chat (inline display)
 - Share to SingleFriend or Couple Circle
 - Generate shareable link (expiring)
 - Download original quality

Data Schema:

```

interface Photo {
  id: string;
  coupleId: string;
}

```



```

    uploadedBy: string;

    // File information
    originalUrl: string;           // Full resolution
    thumbnailUrl: string;         // 400x400 thumbnail
    mediumUrl: string;            // 1200px max dimension
    fileName: string;
    fileSize: number;
    mimeType: string;             // image/jpeg, image/png, etc.

    // Dimensions
    width: number;
    height: number;
    aspectRatio: number;

    // Metadata
    takenAt?: Date;               // From EXIF
    location?: {
        latitude: number;
        longitude: number;
        placeName?: string;
    };
    camera?: string;              // From EXIF

    // Organization
    albumIds: string[];           // Can be in multiple albums
    tags: string[];
    description?: string;
    isFavorite: boolean;

    // AI analysis (optional)
    aiLabels?: string[];          // 'sunset', 'food', 'selfie'
    faces?: {
        userId: string;
        boundingBox: number[];
    }[];

    // Privacy
    visibility: 'couple' | 'friends' | 'circles';
    sharedWith?: string[];

    uploadedAt: Date;
    createdAt: Date;
}

interface Album {
    id: string;
    coupleId: string;

```

```

// Album info
name: string;
description?: string;
coverPhotoId?: string;

// Type
type: 'manual' | 'smart';
smartRules?: { // For smart albums
  labels?: string[];
  dateRange?: { start: Date; end: Date };
  uploadedBy?: string;
};

// Stats
photoCount: number;
lastPhotoAddedAt?: Date;

// Sharing
isShared: boolean;
sharedWith?: string[];

createdBy: string;
createdAt: Date;
updatedAt: Date;
}

```

Storage Architecture:

```

const MediaStorageConfig = {
  // Cloud storage (S3-compatible)
  storage: 'AWS S3' | 'Cloudflare R2',

  // CDN for fast delivery
  cdn: 'CloudFront' | 'Cloudflare CDN',

  // Image processing
  processor: 'Sharp.js',

  // Upload pipeline
  uploadPipeline: [
    'client_compress', // Initial compression
    'upload_to_temp', // Temp storage
    'virus_scan', // Security check
    'generate_thumbnails', // Create sizes
    'extract_metadata', // EXIF data
    'ai_analysis', // Optional labeling
    'move_to_permanent', // Final storage
    'update_database', // Save to DB
  ]
}

```

```

    'notify_partner',                // Push notification
  ],

  // Thumbnail sizes
  thumbnails: {
    small: { width: 200, height: 200, quality: 80 },
    medium: { maxDimension: 1200, quality: 85 },
    large: { maxDimension: 2400, quality: 90 },
  },

  // Storage limits
  limits: {
    maxFileSize: 100 * 1024 * 1024, // 100 MB per photo
    maxPhotosPerUpload: 50,
    totalStorage: 'unlimited',        // Marketing claim
    actualLimit: 100000,              // 100k photos per couple
  }
};

```

2.4.2 Video Sharing & Storage

Overview: Share video memories with intelligent compression and streaming. Support for short clips and longer videos.

Features: - Upload videos up to 500MB - Auto-transcode to multiple qualities (1080p, 720p, 480p)
 - Thumbnail generation - Inline video player with scrubbing - Background upload with progress -
 Video trimming before upload

Data Schema:

```

interface Video {
  id: string;
  coupleId: string;
  uploadedBy: string;

  // File information
  originalUrl: string;
  streamingUrls: {
    '1080p': string;
    '720p': string;
    '480p': string;
    '360p': string;
  };
};
thumbnailUrl: string;
fileName: string;
fileSize: number;

// Video properties

```

```

duration: number;           // Seconds
width: number;
height: number;
fps: number;
codec: string;

// Metadata
takenAt?: Date;
location?: {
  latitude: number;
  longitude: number;
};

// Organization (similar to photos)
albumIds: string[];
tags: string[];
description?: string;
isFavorite: boolean;

// Processing status
processingStatus: 'uploading' | 'processing' | 'ready' | 'failed';
processingProgress?: number; // 0-100

uploadedAt: Date;
createdAt: Date;
}

```

2.4.3 File Management System

Overview: Store and share documents, PDFs, audio files, and other attachments. Useful for trip planning, shared documents, etc.

Features: - Support all common file types - Up to 100MB per file - File preview (PDFs, documents, audio) - Download to device - Organize in folders - Search by filename or content (PDFs)

Supported File Types: - Documents: PDF, DOC, DOCX, TXT, RTF - Spreadsheets: XLS, XLSX, CSV - Presentations: PPT, PPTX - Audio: MP3, AAC, WAV, M4A - Archives: ZIP, RAR - Other: any file type (generic handling)

Data Schema:

```

interface File {
  id: string;
  coupleId: string;
  uploadedBy: string;

  // File info
  fileUrl: string;
}

```

```

fileName: string;
fileSize: number;
mimeType: string;
extension: string;

// Preview
thumbnailUrl?: string;           // For documents
previewUrl?: string;             // For PDFs (first page)

// Organization
folderId?: string;
tags: string[];
description?: string;

// Metadata
uploadedAt: Date;
lastAccessedAt?: Date;
downloadCount: number;

createdAt: Date;
}

interface Folder {
  id: string;
  coupleId: string;

  name: string;
  description?: string;
  parentFolderId?: string;       // For nested folders

  fileCount: number;
  totalSize: number;

  createdBy: string;
  createdAt: Date;
  updatedAt: Date;
}

```

2.4.4 Cloud Storage Architecture

Technical Implementation:

```

const CloudStorageArchitecture = {
  // Primary storage
  primaryStorage: {
    provider: 'AWS S3',
    bucket: 'linkup-media-prod',
  },
}

```

```

    regions: ['us-east-1', 'eu-west-1', 'ap-southeast-1'],
    replication: 'cross-region',      // Redundancy
    encryption: 'AES-256',           // At rest
},

// CDN for delivery
cdn: {
    provider: 'CloudFront',
    edgeLocations: 'all',
    caching: {
        images: '30 days',
        videos: '7 days',
        files: '1 day',
    },
    compression: 'gzip, brotli',
},

// Media processing
processing: {
    images: {
        library: 'sharp',
        formats: ['jpeg', 'png', 'webp'],
        quality: {
            thumbnail: 80,
            medium: 85,
            large: 90,
        },
    },

    videos: {
        transcoder: 'FFmpeg',
        codec: 'H.264',
        profiles: [
            { name: '1080p', width: 1920, bitrate: '5M' },
            { name: '720p', width: 1280, bitrate: '2.5M' },
            { name: '480p', width: 854, bitrate: '1M' },
            { name: '360p', width: 640, bitrate: '500K' },
        ],
    },
},

// Storage quotas
quotas: {
    photos: 'unlimited',                // Marketing
    videos: 'unlimited',
    files: '10GB per couple',
    singleFile: '100MB',
    videoFile: '500MB',
},

```

```

},

// Backup & retention
backup: {
  frequency: 'daily',
  retention: '90 days',
  archival: 'Glacier after 1 year of inactivity',
}
};

```

Chapter 2 Status: Complete

Summary: - Text messaging with threads, highlighting, and reactions - Voice calls with HD audio quality - Video calls with screen sharing and adaptive quality - Media storage for photos, videos, and files - Complete data schemas, user flows, and technical specs

Next Chapter: Chapter 3: Creative & Interactive Features

Chapter 3: Creative & Interactive Features

Overview

Creative features are what set LinkUp apart from generic messaging apps. These tools enable couples to express love artistically, create together, and build a unique communication language. This chapter covers:

- Scribble - Real-time collaborative drawing for quick sketches and playful doodling
- Painting - Advanced art creation with layers, brushes, and effects
- Custom Emojis - Create personalized emojis and stickers unique to your relationship
- SoundBoard - Audio personality with custom sounds and effects during calls

Design Philosophy: - Accessible creativity - No artistic skill required to have fun - Real-time collaboration - Create together, see each others strokes live - Seamless integration - Share creations directly in chat or during calls - Infinite canvas - No limits to artistic expression

3.1 Scribble Feature

3.1.1 Real-time Collaborative Drawing

Overview: Scribble is a lightweight, playful drawing tool for quick sketches, doodles, and visual communication. Think of it as a digital napkin for couples to draw on together. Perfect for: - Quick explanations (Ill draw what I mean) - Playful games (tic-tac-toe, hangman) - Love notes and doodles - Collaborative sketching in real-time

Core Features:

1. Real-time Collaboration

- Both partners can draw simultaneously on the same canvas
 - See each others strokes as they happen (<50ms latency)
 - Different colors for each person (auto-assigned)
 - Cursors show where partner is drawing
2. Drawing Tools
 - Pen (smooth, variable thickness)
 - Marker (semi-transparent, bold)
 - Highlighter (transparent overlay)
 - Eraser (with adjustable size)
 - Shape tools (line, circle, rectangle, arrow)
 - Text tool (add typed text to drawing)
 3. Canvas Features
 - Infinite canvas with pan and zoom
 - Grid overlay (optional, for precision)
 - Undo/redo (unlimited history)
 - Clear canvas (with confirmation)
 - Multiple pages (like a sketchbook)
 - Background colors and templates
 4. Sharing & Saving
 - Auto-save every 30 seconds
 - Share in chat as image
 - Export as PNG or SVG
 - Save to Scribble Gallery
 - Time-lapse replay of drawing process

Data Schema:

```
interface Scribble {
  id: string;
  coupleId: string;

  // Canvas properties
  canvasSize: {
    width: number; // Default 1920x1080
    height: number;
  };
  backgroundColor: string;
  backgroundTemplate?: 'grid' | 'dots' | 'lined' | 'blank';

  // Drawing data
  strokes: Stroke[];
  pages: ScribblePage[];
  currentPage: number;

  // Collaboration
  isCollaborative: boolean;
  activeUsers: {
    userId: string;
    cursorPosition: { x: number; y: number };
  };
}
```



```

    currentTool: string;
    color: string;
}[];

// Metadata
title?: string;
tags: string[];
thumbnailUrl?: string;

// Status
status: 'draft' | 'completed' | 'shared';
lastEditedBy: string;
lastEditedAt: Date;

// Export
exportedImageUrl?: string;
timelapseUrl?: string;           // Video of drawing process

createdBy: string;
createdAt: Date;
updatedAt: Date;
}

interface Stroke {
    id: string;
    userId: string;               // Who drew this stroke

    // Stroke properties
    tool: 'pen' | 'marker' | 'highlighter' | 'eraser' |
        'line' | 'circle' | 'rectangle' | 'arrow' | 'text';
    color: string;                // Hex color
    thickness: number;            // 1-50 pixels
    opacity: number;              // 0-1

    // Path data
    points: {
        x: number;
        y: number;
        pressure?: number;        // For pressure-sensitive devices
        timestamp: number;        // For replay
    }[];

    // For shapes
    shapeData?: {
        startX: number;
        startY: number;
        endX: number;
        endY: number;
    };
};

```

```

    radius?: number;                                // For circles
};

// For text
textData?: {
  content: string;
  x: number;
  y: number;
  fontSize: number;
  fontFamily: string;
};

// Metadata
createdAt: Date;
timestamp: number;                                // For synchronization
}

interface ScribblePage {
  id: string;
  pageNumber: number;
  thumbnailUrl?: string;
  strokeCount: number;
  createdAt: Date;
}

```

User Flow - Creating Scribble Together:

1. User opens Scribble feature
 - Blank canvas appears
 - Drawing tools visible at bottom
 - Partner automatically invited to collaborate
 - "Waiting for [Partner]..." shown if not joined yet
2. Partner joins
 - Receives notification "[You] started a scribble"
 - Taps notification → joins same canvas
 - Both see "Connected" indicator
 - Each user assigned a color (User 1: blue, User 2: pink)
3. Both users draw simultaneously
 - User 1 draws with finger/stylus
 - Strokes appear on User 1's screen instantly (local)
 - Strokes transmitted via WebSocket to server
 - Server broadcasts to User 2
 - User 2 sees User 1's strokes appear (<50ms latency)
 - User 2's cursor position visible to User 1
 - Vice versa for User 2's drawings
4. Using tools

Tap tool button to switch (pen, marker, eraser, shapes)
Long-press tool for options (thickness, opacity)
Color picker shows recent colors + color wheel
Undo/redo buttons available (ctrl+z, ctrl+y)
Clear canvas button (confirmation required)

5. Saving and sharing

Auto-saves every 30 seconds to cloud
Tap "Share" → sends in chat as image
Tap "Save to Gallery" → saves to Scribble collection
Tap "Export" → download as PNG or SVG
Tap "Timelapse" → generates video of drawing process

Real-time Synchronization:

```
// WebSocket events for real-time collaboration
interface ScribbleSyncEvents {
  // Sent when user starts new stroke
  'stroke:start': {
    scribbleId: string;
    strokeId: string;
    userId: string;
    tool: string;
    color: string;
    thickness: number;
    startPoint: { x: number; y: number; timestamp: number };
  };

  // Sent for each point as user draws
  'stroke:point': {
    scribbleId: string;
    strokeId: string;
    point: { x: number; y: number; pressure?: number; timestamp: number };
  };

  // Sent when user completes stroke
  'stroke:end': {
    scribbleId: string;
    strokeId: string;
    finalPoints: Point[];
  };

  // Cursor position updates (throttled to 60fps)
  'cursor:move': {
    scribbleId: string;
    userId: string;
    position: { x: number; y: number };
  };
};
```

```

// Undo/redo actions
'action:undo': {
  scribbleId: string;
  userId: string;
  strokeId: string;           // Stroke to remove
};

'action:redo': {
  scribbleId: string;
  userId: string;
  strokeId: string;           // Stroke to restore
};

// Clear canvas
'action:clear': {
  scribbleId: string;
  userId: string;
  timestamp: number;
};
}

// Conflict resolution
const ScribbleConflictResolution = {
  // Optimistic UI - show strokes immediately
  optimisticRendering: true,

  // Server is source of truth for stroke order
  serverAuthoritative: true,

  // If strokes conflict (rare), use timestamp to order
  conflictResolution: 'timestamp_based',

  // Periodic full state sync (every 5 minutes)
  fullSyncInterval: 300000,

  // Reconnection strategy
  reconnection: {
    onDisconnect: 'queue_local_strokes',
    onReconnect: 'sync_missing_strokes',
    maxQueueSize: 1000,
  }
};

```

UI/UX Design:

```

// Canvas interface
interface ScribbleCanvas {
  // Canvas element
  canvas: {

```

```

    element: 'HTML5 Canvas',
    size: '1920x1080',           // Logical size
    rendering: 'WebGL',         // For performance
    antialiasing: true,
    smoothing: true,
  },

  // Drawing toolbar (bottom)
  toolbar: {
    position: 'bottom',
    height: '80px',
    background: 'rgba(255, 255, 255, 0.95)',
    backdropBlur: '10px',

    tools: [
      { icon: 'pen', label: 'Pen', shortcut: 'P' },
      { icon: 'marker', label: 'Marker', shortcut: 'M' },
      { icon: 'highlighter', label: 'Highlighter', shortcut: 'H' },
      { icon: 'eraser', label: 'Eraser', shortcut: 'E' },
      { icon: 'shapes', label: 'Shapes', submenu: true },
      { icon: 'text', label: 'Text', shortcut: 'T' },
    ],

    controls: [
      { icon: 'undo', label: 'Undo', shortcut: 'Cmd+Z' },
      { icon: 'redo', label: 'Redo', shortcut: 'Cmd+Shift+Z' },
      { icon: 'clear', label: 'Clear', confirm: true },
    ]
  },

  // Color picker
  colorPicker: {
    position: 'bottom-left',
    recentColors: 8,
    quickColors: ['#000000', '#FFFFFF', '#FF6B9D', '#6BCB77',
                  '#4D96FF', '#FFD93D', '#A084DC', '#FF8E53'],
    fullPicker: 'HSV color wheel',
  },

  // Thickness slider
  thicknessControl: {
    position: 'bottom-center',
    range: [1, 50],
    default: {
      pen: 3,
      marker: 10,
      highlighter: 15,
      eraser: 20,
    }
  }
}

```

```

    }
  },

  // Collaboration indicators
  collaboration: {
    partnerCursor: {
      show: true,
      color: 'partner_assigned_color',
      size: '20px',
      label: 'Partner Name',
    },

    connectionStatus: {
      position: 'top-right',
      states: {
        connected: { icon: ' ', text: 'Connected' },
        disconnected: { icon: ' ', text: 'Reconnecting...' },
        alone: { icon: ' ', text: 'Drawing alone' },
      }
    }
  }
}

```

Performance Optimization:

```

const ScribblePerformance = {
  // Canvas rendering
  rendering: {
    framework: 'WebGL via Canvas API',
    offscreenRendering: true,           // Render to offscreen canvas
    layerCaching: true,                 // Cache completed strokes
    dirtyRectOptimization: true,        // Only redraw changed areas

    targetFPS: 60,
    maxPointsPerStroke: 10000,
    strokeSimplification: 'Douglas-Peucker algorithm',
  },

  // Network optimization
  network: {
    strokeCompression: 'delta encoding', // Only send changes
    batchUpdates: 'max 60 per second',   // Don't exceed 60fps
    pointThrottling: 'adaptive',         // Reduce points for slow strokes

    maxMessageSize: '1KB per stroke point',
    compressionRatio: 'target 70% reduction',
  },

  // Memory management

```

```

memory: {
  maxCanvasSize: '4096x4096',      // Hardware limit
  strokeLimit: 10000,              // Per page
  pageLimit: 50,                  // Per scribble
  autoCleanup: 'remove old pages when limit reached',
}
};

```

3.1.2 Drawing Tools & Brushes

Tool Specifications:

```

interface DrawingTool {
  // Pen - smooth, precise drawing
  pen: {
    thickness: { min: 1, max: 10, default: 3 },
    opacity: 1.0,
    smoothing: 0.7,                // Bezier curve smoothing
    pressureSensitive: true,       // If device supports
    capStyle: 'round',
    joinStyle: 'round',
  },

  // Marker - bold, semi-transparent
  marker: {
    thickness: { min: 5, max: 30, default: 10 },
    opacity: 0.7,
    smoothing: 0.5,
    pressureSensitive: false,
    capStyle: 'round',
    joinStyle: 'round',
    blend: 'multiply',            // Blend mode for overlaps
  },

  // Highlighter - transparent overlay
  highlighter: {
    thickness: { min: 10, max: 40, default: 15 },
    opacity: 0.3,
    smoothing: 0.3,
    pressureSensitive: false,
    capStyle: 'butt',
    joinStyle: 'bevel',
    blend: 'multiply',
  },

  // Eraser - removes strokes
  eraser: {

```

```

    thickness: { min: 10, max: 100, default: 20 },
    mode: 'stroke_based',           // Erases entire stroke when touched
    visualFeedback: 'dashed circle cursor',
  },

  // Shape tools
  shapes: {
    line: {
      thickness: { min: 1, max: 20, default: 3 },
      arrowhead: 'optional',
      dashed: 'optional',
    },
    circle: {
      filled: 'optional',
      thickness: { min: 1, max: 20, default: 3 },
    },
    rectangle: {
      filled: 'optional',
      thickness: { min: 1, max: 20, default: 3 },
      rounded: 'optional',
    },
    arrow: {
      thickness: { min: 1, max: 20, default: 3 },
      headStyle: 'filled triangle',
    }
  },

  // Text tool
  text: {
    fontSize: { min: 12, max: 72, default: 24 },
    fontFamily: ['Inter', 'Comic Sans MS', 'Marker Felt', 'Handwriting'],
    color: 'same as drawing color',
    alignment: 'left' | 'center' | 'right',
    editing: 'inline',
  }
}

```

3.1.3 Canvas Management

Features:

1. Multi-Page Scribbles
 - Add new pages (like a sketchbook)
 - Navigate between pages with swipe
 - Page thumbnails in sidebar
 - Reorder pages drag-and-drop
 - Delete pages (with confirmation)

2. Background Options
 - Blank (white or custom color)
 - Grid (various grid sizes)
 - Dots (dot grid for precision)
 - Lined (notebook style)
 - Custom template upload
3. Zoom & Pan
 - Pinch to zoom (25% to 400%)
 - Two-finger pan on touch devices
 - Mouse wheel zoom on desktop
 - Fit to screen button
 - Reset zoom button

Data Schema:

```
interface CanvasSettings {
  zoom: number; // 0.25 to 4.0
  panOffset: { x: number; y: number };
  backgroundType: 'blank' | 'grid' | 'dots' | 'lined' | 'custom';
  backgroundColor: string;

  gridSettings?: {
    size: number; // Grid cell size in pixels
    color: string;
    opacity: number;
  };

  customBackground?: {
    imageUrl: string;
    opacity: number;
  };
}
```

3.2 Painting Feature

3.2.1 Advanced Art Tools

Overview: While Scribble is for quick sketches, Painting is for serious art creation. Features include:

- Professional-grade brushes
- Layer system for complex compositions
- Blend modes and effects
- Higher resolution (up to 4K)
- Advanced color tools
- Reference image overlay

Differences from Scribble:

Feature	Scribble	Painting
Purpose	Quick doodles	Serious artwork
Canvas Size	1920x1080	Up to 4096x4096
Layers	Single layer	Multiple layers
Brushes	6 basic tools	20+ advanced brushes

Feature	Scribble	Painting
Collaboration	Real-time	Turn-based
Export	PNG, SVG	PNG, PSD, high-res

Core Features:

1. Advanced Brushes
 - Pencil (with texture)
 - Ink pen (smooth, flowing)
 - Watercolor (realistic blending)
 - Oil paint (thick, textured)
 - Airbrush (gradient spray)
 - Calligraphy (angle-sensitive)
 - Charcoal (rough, artistic)
 - Pastel (soft, chalky)
 - Custom brushes (import/create)
2. Layer System
 - Unlimited layers (practical limit: 50)
 - Layer opacity control
 - Blend modes (Normal, Multiply, Screen, Overlay, etc.)
 - Layer groups for organization
 - Lock layers (prevent editing)
 - Hide/show layers
 - Duplicate layers
 - Merge layers
3. Color Tools
 - HSV color wheel
 - RGB sliders
 - Hex input
 - Eyedropper (sample from canvas)
 - Color history (recent colors)
 - Color palettes (save custom palettes)
 - Gradient tool
4. Transform Tools
 - Move selection
 - Rotate (free or snapped angles)
 - Scale (maintain aspect ratio option)
 - Flip horizontal/vertical
 - Perspective transform
 - Free transform

Data Schema:

```
interface Painting {
  id: string;
  coupleId: string;

  // Canvas
```

```

canvasSize: {
    width: number;           // Up to 4096
    height: number;          // Up to 4096
};
dpi: number;                // 72, 150, or 300

// Layers
layers: Layer[];
activeLayerId: string;

// Metadata
title?: string;
description?: string;
tags: string[];
thumbnailUrl: string;

// Collaboration (turn-based)
collaborationType: 'solo' | 'turn_based';
currentEditor?: string;      // Who has edit rights
editHistory: {
    userId: string;
    timestamp: Date;
    action: string;
}[];

// Status
status: 'in_progress' | 'completed' | 'archived';
progressPercentage?: number;  // User-defined

// Export
exports: {
    format: 'png' | 'psd' | 'svg';
    url: string;
    resolution: string;
    createdAt: Date;
}[];

createdBy: string;
createdAt: Date;
updatedAt: Date;
}

interface Layer {
    id: string;
    paintingId: string;

    // Layer properties
    name: string;

```

```

order: number; // Z-index (0 = bottom)
opacity: number; // 0-1
blendMode: 'normal' | 'multiply' | 'screen' | 'overlay' |
           'darken' | 'lighten' | 'color-dodge' | 'color-burn';

// Visibility & locking
isVisible: boolean;
isLocked: boolean;

// Layer content
imageDataURL?: string; // Base64 or cloud URL
strokes: Stroke[]; // Vector strokes

// Transform
transform: {
  x: number;
  y: number;
  rotation: number; // Degrees
  scaleX: number;
  scaleY: number;
};

// Layer type
type: 'raster' | 'vector' | 'text' | 'group';

// For layer groups
parentGroupId?: string;
childLayerIds?: string[];

createdAt: Date;
updatedAt: Date;
}

interface Brush {
  id: string;

  // Brush properties
  name: string;
  type: 'pencil' | 'ink' | 'watercolor' | 'oil' | 'airbrush' |
        'calligraphy' | 'charcoal' | 'pastel' | 'custom';

  // Rendering
  size: { min: 1, max: 300, default: number };
  opacity: number; // 0-1
  flow: number; // 0-1 (paint application rate)
  hardness: number; // 0-1 (edge softness)

  // Dynamics

```

```

pressureSensitivity: boolean;
pressureCurve?: number[];           // Custom pressure response
sizeJitter: number;                 // Randomize size
opacityJitter: number;               // Randomize opacity

// Texture
textureUrl?: string;                 // Brush texture image
textureScale: number;

// Spacing
spacing: number;                     // Distance between stamps

// Advanced
scatterX: number;                    // Horizontal scatter
scatterY: number;                    // Vertical scatter
rotation: 'fixed' | 'follow_path' | 'random';

// Custom brush
customStamp?: string;                // Custom shape image

isDefault: boolean;                  // Built-in vs custom
createdBy?: string;
}

```

User Flow - Creating Painting:

1. User opens Painting feature
 - Canvas size selector appears
 - Choose preset (1080p, 2K, 4K) or custom
 - Choose orientation (portrait, landscape, square)
 - Canvas loads with default layer
2. User selects brush and color
 - Tap brush icon → brush library appears
 - Preview each brush with test stroke
 - Select brush (e.g., "Watercolor")
 - Adjust size, opacity, flow sliders
 - Choose color from color wheel
 - Start painting
3. Working with layers
 - Tap layers icon → layer panel opens
 - Current layer highlighted
 - Tap "+" to add new layer
 - New layer appears above current
 - Name layer (e.g., "Background", "Character")
 - Adjust layer opacity with slider
 - Change blend mode (multiply, screen, etc.)
 - Reorder layers by dragging

4. Advanced techniques

- Add reference image (overlay at low opacity)
- Use selection tools (rectangle, lasso, magic wand)
- Fill selected area with color/gradient
- Apply filters (blur, sharpen, adjust colors)
- Use transform tools to move/rotate elements
- Zoom in for details, zoom out for overview

5. Saving and exporting

- Auto-save every 2 minutes
- Save as draft to continue later
- Mark as "Completed"
- Export options:
 - PNG (standard resolution)
 - High-res PNG (full canvas size)
 - PSD (with layers, for editing)
 - SVG (if vector-only)
- Share in chat
- Save to Painting Gallery

Turn-based Collaboration:

// Unlike Scribble (real-time), Painting uses turn-based collaboration

```
interface PaintingCollaboration {
  mode: 'turn_based',

  // Edit sessions
  sessions: {
    userId: string;
    startedAt: Date;
    endedAt?: Date;
    changesCount: number;
  }[],

  // Current editor
  currentEditor: {
    userId: string;
    sessionStartedAt: Date;
    autoReleaseAfter: 30 * 60 * 1000, // 30 minutes of inactivity
  },

  // Request edit access
  requestAccess: () => void,

  // Notifications
  notifications: {
    partnerStartedEditing: 'notify via push',
    partnerFinishedEditing: 'notify and offer to continue',
  }
}
```

```

    partnerAddedLayer: 'notify',
    paintingCompleted: 'notify',
  }
}

```

3.2.2 Layers & Effects

Layer Management UI:

```

interface LayerPanel {
  position: 'right',           // Desktop
  width: '300px',

  header: {
    title: 'Layers',
    newLayerButton: true,
    deleteLayerButton: true,
    mergeLayersButton: true,
  },

  layerList: {
    // Each layer item
    layerItem: {
      thumbnail: '60x60px preview',
      name: 'editable inline',
      opacity: 'slider 0-100%',
      blendMode: 'dropdown',
      visibilityToggle: 'eye icon',
      lockToggle: 'lock icon',
      reorder: 'drag handle',
    },

    // Selected layer highlighted
    selectedBorder: '2px solid var(--color-primary)',
  },

  footer: {
    layerCount: 'showing count',
    mergedSize: 'estimated file size',
  }
}

```

Effects & Filters:

```

interface PaintingEffects {
  // Blur effects
  blur: {
    gaussian: { radius: 0-50 },
  }
}

```

```

    motion: { angle: 0-360, distance: 0-100 },
    radial: { amount: 0-100, centerX: number, centerY: number },
  },

  // Adjustment effects
  adjustments: {
    brightness: { value: -100 to 100 },
    contrast: { value: -100 to 100 },
    saturation: { value: -100 to 100 },
    hue: { shift: 0-360 },
    temperature: { warmth: -100 to 100 },
    exposure: { value: -2 to 2 },
  },

  // Artistic effects
  artistic: {
    oilPaint: { intensity: 0-10 },
    watercolor: { intensity: 0-10 },
    sketch: { detail: 0-10 },
    posterize: { levels: 2-8 },
  },

  // Utility effects
  utility: {
    sharpen: { amount: 0-100 },
    noise: { amount: 0-100 },
    vignette: { intensity: 0-100 },
    grain: { size: 0-10, intensity: 0-100 },
  }
}

```

3.2.3 Saving & Sharing Artwork

Export Options:

```

interface PaintingExport {
  formats: [
    {
      type: 'PNG',
      description: 'Standard image format',
      options: {
        resolution: '1x' | '2x' | '4x', // Multiplier
        backgroundColor: 'transparent' | 'white' | 'custom',
        quality: 'high',
      },
      fileSize: 'estimated based on canvas',
    },
  ],
}

```



```

{
  type: 'PSD',
  description: 'Photoshop format with layers',
  options: {
    includeHiddenLayers: boolean,
    flattenGroups: boolean,
  },
  fileSize: 'larger due to layers',
  note: 'Editable in Photoshop or GIMP',
},
{
  type: 'SVG',
  description: 'Vector format (if applicable)',
  options: {
    simplifyPaths: boolean,
    optimizeCurves: boolean,
  },
  fileSize: 'small, scalable',
  note: 'Only available for vector-only paintings',
}
],

// Share destinations
sharing: {
  inChat: 'send as message attachment',
  gallery: 'save to couple painting gallery',
  download: 'download to device',
  externalShare: 'share to Instagram, etc.',
}
}

```

3.3 Custom Emojis

3.3.1 Emoji Creation Tools

Overview: Create personalized emojis and stickers unique to your relationship. Turn inside jokes, pet names, and special moments into custom emojis you can use everywhere in LinkUp.

Core Features:

1. Creation Methods
 - Draw from scratch (using simplified Scribble)
 - Edit from template (emoji bases)
 - Photo-to-emoji (extract face/object from photo)
 - Text-to-emoji (stylized text designs)
 - Combine existing emojis (mashup tool)
2. Emoji Types
 - Static emojis (PNG, no animation)

- Animated emojis (GIF, simple loops)
 - Reactive emojis (respond to context)
 - Emoji packs (themed sets)
3. Design Tools
- Canvas: 512x512px (high resolution)
 - Simple drawing tools (pen, shapes, fill)
 - Color picker with couple theme colors
 - Text overlay
 - Sticker outlines (auto-generated)
 - Preview in different sizes

Data Schema:

```
interface CustomEmoji {
  id: string;
  coupleId: string;

  // Emoji properties
  name: string;           // :custom_name:
  shortcode: string;      // How to type it
  category: 'love' | 'funny' | 'reaction' | 'inside_joke' | 'custom';
  tags: string[];         // For search

  // Visual
  imageUrl: string;       // 512x512 source
  thumbnailUrl: string;   // 128x128 thumbnail
  isAnimated: boolean;
  animationUrl?: string;  // GIF if animated
  frameCount?: number;

  // Metadata
  description?: string;   // "Our first kiss emoji"
  story?: string;         // Story behind it
  createdFrom: 'scratch' | 'template' | 'photo' | 'text' | 'mashup';
  templateId?: string;
  sourcePhotoId?: string;

  // Usage stats
  usageCount: number;
  lastUsedAt?: Date;
  isFavorite: boolean;

  // Sharing
  isSharedWithFriends: boolean; // Can SingleFriends see it?
  sharedWithCircles: string[];

  createdBy: string;
  createdAt: Date;
  updatedAt: Date;
}
```

```

}

interface EmojiTemplate {
    id: string;

    // Template info
    name: string;
    category: string;
    imageUrl: string;

    // Customizable areas
    editableRegions: {
        type: 'face' | 'hair' | 'accessory' | 'background';
        x: number;
        y: number;
        width: number;
        height: number;
    }[];

    // Color themes
    colorSchemes: {
        name: string;
        colors: string[];
    }[];

    isDefault: boolean;
    downloadCount: number;
}

```

User Flow - Creating Custom Emoji:

1. User opens Custom Emoji creator
 - Four creation options shown:
 - "Draw from Scratch"
 - "Use Template"
 - "From Photo"
 - "Text Style"
 - User selects creation method
2. Method: "From Photo"
 - User selects photo from media library
 - Photo loads in editor
 - AI suggests croppings (faces, objects)
 - User adjusts crop to focus on subject
 - Background removal options:
 - Auto remove (AI-powered)
 - Manual erase
 - Keep background
 - Apply emoji effects:

- Sticker outline (white border)
- Cartoon filter
- Add expressions (sparkles, hearts)
- Adjust saturation, brightness
- Preview in different sizes

3. Naming and details

- Enter emoji name (e.g., "Happy Boyfriend")
- Set shortcode (auto-suggested: ":happy_bf:")
- Choose category
- Add tags for easy search
- Optional: Add story/description
- Preview in message context

4. Saving

- Tap "Create Emoji"
- Processing (resize, optimize, generate thumbnail)
- Emoji added to couple's emoji library
- Confirmation: "Emoji created! Try it in a message"
- Shortcode shown for quick use

5. Using custom emoji

- In message input, type ":"
- Emoji picker appears with Custom tab
- Custom emojis shown first
- Search by name or shortcode
- Tap to insert in message
- Shows inline in chat

Emoji Editor Interface:

```
interface EmojiEditor {  
  // Canvas  
  canvas: {  
    size: '512x512',  
    background: 'transparent',  
    grid: 'optional',  
  },  
  
  // Tools (simplified)  
  tools: [  
    { name: 'Pen', sizes: [2, 5, 10, 20] },  
    { name: 'Eraser', sizes: [10, 20, 40] },  
    { name: 'Fill', mode: 'bucket' },  
    { name: 'Shapes', types: ['circle', 'rectangle', 'heart', 'star'] },  
    { name: 'Text', fonts: ['Comic Sans', 'Impact', 'Handwriting'] },  
    { name: 'Stickers', library: 'decorative elements' },  
  ],  
}
```

```

// Photo editing (for photo-based emojis)
photoTools: {
  crop: 'free or circular',
  backgroundRemoval: 'AI-powered',
  filters: ['cartoon', 'oil paint', 'sketch'],
  adjustments: ['brightness', 'contrast', 'saturation'],
  effects: ['outline', 'shadow', 'glow'],
},

// Preview
preview: {
  sizes: ['16px', '32px', '64px', '128px'],
  contexts: ['in message', 'as reaction', 'in emoji picker'],
},

// Export
export: {
  format: 'PNG with transparency',
  sizes: [
    { name: 'source', size: 512 },
    { name: 'large', size: 128 },
    { name: 'medium', size: 64 },
    { name: 'small', size: 32 },
  ]
}
}

```

3.3.2 Emoji Library Management

Features:

1. Organization
 - View all custom emojis in grid
 - Sort by: recently used, most used, date created, name
 - Filter by category or tag
 - Search by name or shortcode
 - Favorites section
2. Editing
 - Edit existing emojis
 - Duplicate and modify
 - Delete emojis (with confirmation)
 - Rename/reategorize
 - Version history (keep old versions)
3. Sharing
 - Share emoji with SingleFriend
 - Share emoji pack with Couple Circles
 - Export emoji as image

- Import emojis from partner

UI Design:

```
interface EmojiLibrary {
  layout: 'grid',

  // Header
  header: {
    title: 'Custom Emojis',
    createButton: '+ New Emoji',
    searchBar: 'search by name or tag',
    filterDropdown: 'category filter',
    sortDropdown: 'sort options',
  },

  // Grid view
  grid: {
    columns: {
      mobile: 3,
      tablet: 4,
      desktop: 6,
    },

    emojiCard: {
      size: '120px square',
      image: 'centered emoji preview',
      name: 'below image',
      usageCount: 'small text',
      favoriteIcon: 'top-right corner',

      // Hover/long-press actions
      actions: {
        edit: 'edit emoji',
        duplicate: 'create copy',
        delete: 'remove emoji',
        share: 'share with others',
        favorite: 'toggle favorite',
      }
    }
  },

  // Stats
  stats: {
    totalEmojis: number,
    mostUsed: CustomEmoji,
    recentlyCreated: CustomEmoji[],
  }
}
```

3.3.3 Animated Emojis

Overview: Create simple animated emojis (GIFs) with frame-by-frame animation or motion effects.

Features:

1. Animation Types
 - Frame-by-frame (draw each frame)
 - Motion path (move elements along path)
 - Rotation (spin elements)
 - Scale (grow/shrink)
 - Blink (show/hide)
 - Preset animations (bounce, shake, pulse)
2. Timeline Editor
 - Visual timeline with frames
 - Adjust frame duration
 - Preview animation loop
 - Onion skinning (see previous frame)
 - Copy frames
 - Reverse animation

Data Schema:

```
interface AnimatedEmoji extends CustomEmoji {
  isAnimated: true;

  // Animation properties
  animation: {
    frameCount: number;           // Number of frames
    fps: number;                  // Frames per second (1-30)
    duration: number;             // Total duration in ms
    loop: boolean;                // Loop animation

    // Frame data
    frames: {
      frameNumber: number;
      imageUrl: string;
      duration: number;           // Frame-specific duration
    }[];

    // Or motion-based
    motionData?: {
      type: 'path' | 'rotation' | 'scale' | 'blink';
      keyframes: {
        time: number;             // 0-1 (percentage of animation)
        value: any;               // Position, rotation, scale, etc.
        easing: 'linear' | 'ease-in' | 'ease-out' | 'ease-in-out';
      }[];
    };
  };
}
```

```

    };
};

// Export
gifUrl: string;           // Final GIF file
gifSize: number;          // File size in bytes
}

```

3.4 SoundBoard

3.4.1 Sound Library

Overview: Add audio personality to your relationship with a custom soundboard. Play sounds during calls, send as messages, or use as ringtones/notifications.

Core Features:

1. Default Sound Library
 - 50+ built-in sounds categorized:
 - Romantic (kiss, heartbeat, I love you variants)
 - Funny (laugh tracks, memes, silly sounds)
 - Celebrations (applause, cheers, party horn)
 - Musical (instruments, beats, melodies)
 - Notifications (custom notification sounds)
 - Voice clips (cute phrases)
2. Custom Sound Upload
 - Record custom sounds (up to 30 seconds)
 - Upload audio files (MP3, M4A, WAV)
 - Trim/edit sounds
 - Adjust volume
 - Name and categorize
3. Sound Playback
 - Play during voice/video calls (both hear it)
 - Send as message (audio clip)
 - Set as ringtone for partners calls
 - Set as notification sound
 - Quick access grid

Data Schema:

```

interface Sound {
  id: string;
  coupleId: string;

  // Sound properties
  name: string;
  category: 'romantic' | 'funny' | 'celebration' | 'musical' |
    'notification' | 'voice' | 'custom';
  tags: string[];
}

```



```

// Audio file
audioUrl: string;
duration: number; // Seconds
fileSize: number;
format: 'mp3' | 'm4a' | 'wav';
waveformData: number[]; // For visualization

// Metadata
description?: string;
emoji?: string; // Associated emoji

// Usage
usageCount: number;
lastUsedAt?: Date;
isFavorite: boolean;

// Source
source: 'default' | 'uploaded' | 'recorded';
uploadedBy?: string;
recordedBy?: string;

// Settings
defaultVolume: number; // 0-1
allowedContexts: ('call' | 'message' | 'notification')[];

createdAt: Date;
updatedAt: Date;
}

interface SoundboardLayout {
  id: string;
  coupleId: string;

  // Layout name
  name: string; // "Call Sounds", "Funny Sounds"

  // Grid configuration
  gridSize: {
    rows: number; // 3 or 4
    columns: number; // 3 or 4
  };

  // Sound positions
  soundSlots: {
    row: number;
    column: number;
    soundId: string;
  };
}

```

```

        customLabel?: string;
        customColor?: string;
    }[];

    isDefault: boolean;
    createdBy: string;
    createdAt: Date;
}

```

User Flow - Using Soundboard During Call:

1. User in active voice/video call
 - Soundboard button visible in call controls
2. User taps Soundboard button
 - Soundboard panel slides up (mobile) or appears (desktop)
 - Grid of sound buttons shown (3x3 or 4x4)
 - Each button shows sound name and emoji
 - Favorite sounds highlighted
3. User taps sound button
 - Sound plays instantly
 - Both users hear sound in call
 - Visual feedback (button animates)
 - Waveform visualization shown
 - Sound continues playing until finished
4. Sound playback
 - Can play multiple sounds simultaneously (max 3)
 - Can stop playing sound early (tap again)
 - Volume controlled by sound-specific setting
 - Doesn't interrupt voice communication
5. Managing soundboard during call
 - Swipe to switch between soundboard layouts
 - Quick access to favorites
 - Search for specific sound
 - Recently used sounds shown at top
 - Can upload/record new sound (quick record button)

Soundboard UI:

```

interface SoundboardUI {
    // Layout
    layout: {
        type: 'grid',
        gridSize: '3x3' | '3x4' | '4x4',
        responsive: true,
    },
}

```

```

// Sound button
soundButton: {
  size: {
    mobile: '80px square',
    desktop: '100px square',
  },

  design: {
    background: 'gradient or solid color',
    border: '2px solid transparent',
    borderRadius: '16px',

    // Content
    emoji: {
      size: '32px',
      position: 'center-top',
    },

    label: {
      text: 'sound name',
      fontSize: '12px',
      position: 'center-bottom',
      maxLines: 2,
    },
  },

  // States
  hover: {
    border: '2px solid var(--color-primary)',
    transform: 'scale(1.05)',
  },

  active: {
    background: 'var(--color-primary)',
    animation: 'pulse',
  }
}
},

// Controls
controls: {
  position: 'bottom',

  buttons: [
    { icon: 'favorite', label: 'Favorites' },
    { icon: 'recent', label: 'Recent' },
    { icon: 'record', label: 'Record New' },
    { icon: 'layout', label: 'Switch Layout' },
    { icon: 'settings', label: 'Manage Sounds' },
  ]
}

```

```

    ]
  },

  // During playback
  playback: {
    indicator: 'waveform animation',
    stopButton: 'tap again to stop',
    multiplePlayback: 'max 3 simultaneous',
  }
}

```

3.4.2 Custom Sound Upload & Recording

Recording Feature:

```

interface SoundRecorder {
  // Recording
  maxDuration: 30, // Seconds
  format: 'M4A' | 'WAV',
  sampleRate: 44100,
  bitrate: 128, // kbps

  // UI
  recordButton: {
    tapToStart: true,
    tapToStop: true,
    visualFeedback: 'pulsing red circle',
    timer: 'shows elapsed time',
    waveform: 'live waveform while recording',
  },

  // Preview before saving
  preview: {
    playback: 'play recorded sound',
    waveform: 'show full waveform',
    trim: 'adjust start/end points',
    volume: 'adjust volume',

    actions: [
      { label: 'Re-record', action: 'discard and record again' },
      { label: 'Save', action: 'save to soundboard' },
    ]
  },

  // Editing
  basicEditing: {
    trim: 'select start/end points',
  }
}

```

```

    fadeIn: 'duration 0-2s',
    fadeOut: 'duration 0-2s',
    normalize: 'auto-adjust volume',
    removeNoise: 'basic noise reduction',
  }
}

```

Upload Feature:

```

interface SoundUpload {
  // Supported formats
  supportedFormats: ['mp3', 'm4a', 'wav', 'aac', 'ogg'],

  // Limits
  maxFileSize: 10 * 1024 * 1024, // 10 MB
  maxDuration: 30, // Seconds

  // Processing pipeline
  processing: [
    'validate_format',
    'check_duration',
    'transcode_to_m4a', // Standard format
    'generate_waveform',
    'extract_metadata',
    'upload_to_storage',
    'create_thumbnails',
    'add_to_library',
  ],

  // Auto-suggestions
  autoSuggestions: {
    name: 'from filename',
    category: 'from audio analysis',
    emoji: 'from category',
  }
}

```

3.4.3 Sound Playback in Calls

Technical Implementation:

```

const SoundboardCallIntegration = {
  // Audio mixing
  mixing: {
    userVoice: {
      priority: 'highest',
      volume: 'adjustable',
    },
  },
}

```

```

    soundboardSounds: {
      priority: 'medium',
      maxSimultaneous: 3,
      ducking: 'reduce user voice by 10% while sound plays',
      volume: 'per-sound setting',
    },

    outputMix: {
      format: 'stereo',
      sampleRate: 48000,
      bitrate: 128,
    }
  },

  // WebRTC integration
  webrtc: {
    // Create separate audio track for sounds
    soundTrack: 'MediaStreamTrack',

    // Mix with microphone audio
    mixNode: 'Web Audio API AudioContext',

    // Send to peer
    peerConnection: 'add to existing connection',
  },

  // Synchronization
  sync: {
    // Ensure both users hear sound simultaneously
    method: 'server timestamp',
    tolerance: 50, // 50ms sync window

    // Handle network delays
    bufferStrategy: 'small buffer for smooth playback',
  },

  // Controls
  controls: {
    stopAllSounds: 'emergency stop button',
    volumeControl: 'master soundboard volume',
    mutesSounds: 'separate from voice mute',
  }
};

```

Chapter 3 Status: Complete

Summary: - Scribble: Real-time collaborative drawing with WebSocket sync - Painting: Advanced art tool with layers, brushes, and effects - Custom Emojis: Photo-based, drawn, and animated emoji creation - SoundBoard: Audio library with custom sounds for calls and messages - Complete data schemas, user flows, and technical implementations

Next Chapter: Chapter 4: Social & Relationship Features

Chapter 4: Social & Relationship Features

Overview

While LinkUp is primarily a private space for couples, selective social features enhance the relationship experience without sacrificing intimacy. This chapter covers:

- SingleFriend System - Controlled sharing with individual friends
- Couple Circles - Private groups for couple friends
- Hall of Fame - Celebrating relationship milestones and achievements
- Photo Streaks - Daily photo sharing gamification

Design Philosophy: - Privacy-first, social-optional - Everything is private by default - Granular control - Choose exactly what to share with whom - Authentic connection - No public performance, no follower counts - Celebration over comparison - Highlight milestones without competitive pressure

4.1 SingleFriend System

4.1.1 Friend Connection Model

Overview: The SingleFriend system allows couples to selectively share moments with trusted individual friends without making everything public. It's the middle ground between complete privacy and social media broadcasting.

Key Concept: Unlike traditional social media where you have many followers, LinkUp limits each couple to a maximum of 10 SingleFriends. This ensures intimate, meaningful connections rather than audience building.

Core Features:

1. Friend Invitation & Connection
 - Invite friends via unique code or link
 - Friend must have LinkUp account (can be single or in couple)
 - Both partners must approve each friend (couple consensus)
 - Friend accepts invitation to see couples shared content
 - Maximum 10 friends per couple
2. What Friends Can See
 - Only content explicitly shared to Friends visibility
 - Selected photos/videos from media library
 - Hall of Fame public achievements
 - Couple Circle posts (if in same circle)

- Cannot see: Private messages, calls, streaks, highlights
- ### 3. Friend Interaction
- React to shared content with emojis
 - Comment on shared posts (couple can reply)
 - Like Hall of Fame achievements
 - No direct messaging (keeps boundary clear)
 - Celebrate milestones with couple

Data Schema:

```
interface SingleFriend {
  id: string;

  // The couple who added this friend
  coupleId: string;

  // The friend (individual user or couple)
  friendType: 'individual' | 'couple';
  friendUserId?: string;           // If individual
  friendCoupleId?: string;        // If couple

  // Connection details
  status: 'pending_invite' | 'pending_approval' | 'active' |
    'removed' | 'blocked';

  // Approval tracking
  invitedBy: string;               // Which partner sent invite
  approvedBy: string[];           // Both partners must approve
  approvalStatus: {
    partner1Approved: boolean;
    partner2Approved: boolean;
  };

  // Friend information
  displayName: string;             // How friend appears to couple
  avatarUrl?: string;

  // Permissions (granular control)
  permissions: {
    canSeePhotos: boolean;
    canSeeVideos: boolean;
    canSeeHallOfFame: boolean;
    canSeeCirclePosts: boolean;
    canComment: boolean;
    canReact: boolean;
  };

  // Privacy note
  privateNote?: string;           // Couple's private note about friend
}
```



```

// Interaction stats
stats: {
    reactionsGiven: number;
    commentsPosted: number;
    lastInteractionAt?: Date;
};

// Metadata
invitedAt: Date;
connectedAt?: Date; // When both partners approved
lastViewedAt?: Date;
createdAt: Date;
updatedAt: Date;
}

interface FriendInvitation {
    id: string;
    coupleId: string;

    // Invitation details
    inviteCode: string; // 8-character unique code
    inviteLink: string; // linkup.app/invite/[code]

    // Target friend
    recipientEmail?: string;
    recipientPhone?: string;
    recipientUserId?: string; // If already on LinkUp

    // Expiration
    expiresAt: Date; // 7 days from creation
    isExpired: boolean;

    // Status
    status: 'pending' | 'accepted' | 'declined' | 'expired';
    acceptedAt?: Date;

    // Who invited
    invitedBy: string;

    createdAt: Date;
}

```

User Flow - Adding a SingleFriend:

1. Couple decides to add friend
 - Navigate to "Friends" section
 - Current count shown: "5/10 friends"
 - Tap "+ Add Friend" button

2. Invitation method selection
 - Option 1: "Invite by Username" (if friend has LinkUp)
 - Option 2: "Invite by Email/Phone"
 - Option 3: "Generate Invite Link"
3. Partner 1 sends invitation (example: by username)
 - Search for friend's username
 - Friend profile appears
 - Tap "Send Friend Request"
 - Optional: Add personal message
 - Invitation sent to friend
 - Status: "Pending partner approval"
4. Partner 2 must approve
 - Partner 2 receives notification
 - "Partner 1 wants to add [Friend] as SingleFriend"
 - View friend's profile
 - Options: Approve or Decline
 - Partner 2 taps "Approve"
 - Status: "Pending friend acceptance"
5. Friend receives invitation
 - Push notification: "[Couple] added you as a friend!"
 - Email/SMS with invite link (if not on LinkUp yet)
 - Taps notification → opens invitation view
 - Sees couple's profile (photo, names, joined date)
 - Options: Accept or Decline
 - Friend taps "Accept"
6. Connection established
 - Friend added to couple's Friends list
 - Couple appears in friend's "Following" list
 - Both parties receive confirmation
 - Friend can now see shared content
 - Couple can manage friend's permissions
7. Permission management (optional)
 - Couple can tap on friend
 - Adjust what friend can see:
 - Can see photos we share
 - Can see videos we share
 - Can see Hall of Fame
 - Can comment
 - Can react
 - Save changes (applied immediately)

Friend Discovery (For People Not On LinkUp):

```

interface FriendOnboarding {
  // When non-user receives invite
  inviteExperience: {
    landingPage: 'linkup.app/invite/[code]',

    content: {
      couplePreview: {
        names: 'First names only',
        photo: 'Couple photo if shared',
        joinedDate: 'Member since [date]',
        message: 'Personal invite message',
      },

      callToAction: 'Download LinkUp to connect with [Couple]',

      benefits: [
        'See their special moments',
        'Celebrate milestones together',
        'Stay connected privately',
      ]
    },

    // After signup
    autoConnect: true, // Automatically accept invitation
  }
}

```

4.1.2 Permission & Privacy Controls

Granular Permissions:

```

interface FriendPermissions {
  // Content visibility
  contentVisibility: {
    photos: {
      enabled: boolean,
      scope: 'all_shared' | 'selected_albums',
      selectedAlbumIds?: string[],
    },

    videos: {
      enabled: boolean,
      scope: 'all_shared' | 'selected_videos',
    },

    hallOfFame: {
      enabled: boolean,
    }
  }
}

```

```

    categories: string[],           // Which achievement types
},

circlePosts: {
    enabled: boolean,              // If in same circle
}
},

// Interaction permissions
interactions: {
    canReact: boolean,
    canComment: boolean,
    canShare: boolean,             // Share couple's content externally

    // Notification preferences
    notifyOnReaction: boolean,
    notifyOnComment: boolean,
},

// Advanced privacy
advanced: {
    canSeeLocation: boolean,       // From photo metadata
    canDownloadMedia: boolean,
    canSeeOtherFriends: boolean,   // See couple's other friends
}
}

Privacy Zones:

// Content can have different visibility levels
type ContentVisibility =
    | 'private'                    // Only couple
    | 'friends'                    // All SingleFriends
    | 'selected_friends'           // Specific friends only
    | 'circles'                    // Couple Circles only
    | 'friends_and_circles';       // Both

// When sharing content, couple chooses visibility
interface ShareSettings {
    visibility: ContentVisibility,

    // If selected_friends
    selectedFriendIds?: string[],

    // If circles
    selectedCircleIds?: string[],

    // Download protection
    preventDownload: boolean,

```

```

preventScreenshot: boolean,           // Best effort on mobile

// Expiration (optional)
expiresAt?: Date,                     // Auto-remove after date
}

```

4.1.3 Friend Interaction Boundaries

What Friends Can Do:

```

interface FriendInteraction {
  // Reactions on shared content
  reactions: {
    allowedOn: ['photos', 'videos', 'hall_of_fame', 'circle_posts'],
    emojiSet: 'standard_emojis_only', // No custom couple emojis
    maxReactionsPerItem: 1,
    visibleToCOuple: true,
    visibleToOtherFriends: false,     // Keep reactions private
  },

  // Comments
  comments: {
    allowedOn: ['photos', 'videos', 'hall_of_fame', 'circle_posts'],
    maxLength: 500,                  // Characters
    canEdit: true,                   // Within 5 minutes
    canDelete: true,
    moderatedByCouple: true,         // Couple can delete friend comments

    notifications: {
      coupleNotified: true,
      otherFriendsNotified: false,
    }
  },

  // Celebrations (for Hall of Fame)
  celebrations: {
    canSendCongrats: true,
    canGiveVirtualGifts: true,       // Stickers, GIFs
    canAddToMilestone: false,        // Can't contribute to couple's milestones
  },

  // Boundaries (what friends CANNOT do)
  prohibited: [
    'Send direct messages to couple',
    'See private couple content',
    'View message history',
    'Access voice/video calls',
  ]
}

```

```

    'See photo streaks',
    'View highlighted texts',
    'Create content on couple behalf',
    'Tag couple in external posts',
    'See couple location in real-time',
  ]
}

```

Couples Control Over Friends:

```

interface FriendManagement {
  // Actions couple can take
  actions: {
    // Mute friend (still connected, but hide their interactions)
    mute: {
      duration: 'temporary' | 'permanent',
      hidesNotifications: true,
      friendNotAware: true,
    },

    // Remove friend
    remove: {
      requiresBothPartners: true,
      friendNotified: false,           // Soft removal
      contentRemains: false,          // Friend loses access
      canReInvite: true,
    },

    // Block friend (stronger than remove)
    block: {
      requiresBothPartners: true,
      friendNotified: false,
      preventsReinvite: true,
      reportAbuse: boolean,           // Optional
    },

    // Adjust permissions anytime
    editPermissions: {
      immediate: true,
      friendNotified: false,
    }
  },

  // Friend list management
  organization: {
    sortBy: 'name' | 'date_added' | 'interaction_frequency',
    filterBy: 'all' | 'active' | 'inactive',
    search: true,
  }
}

```

}

4.2 Couple Circles

4.2.1 Circle Creation & Management

Overview: Couple Circles are private groups for couple friends to share moments, plan activities, and stay connected. Think group chat but for couples with richer features.

Key Differences from SingleFriends: - Circles are for multiple couples (2-10 couples per circle) - More structured with group posts, events, shared albums - Ideal for: college friend groups, travel buddies, family couples, double-date crews

Core Features:

1. Circle Creation
 - Any couple can create a circle
 - Name the circle (e.g., College Friends, Travel Crew)
 - Add description and cover photo
 - Invite other couples to join
 - Set circle privacy level
2. Circle Types
 - Private Circle - Invite-only, invisible to non-members
 - Discoverable Circle - Friends of members can request to join
 - Activity Circle - Focused on specific activities (travel, gaming, dining)
3. Circle Management
 - Admins (creators have admin by default)
 - Invite new couples
 - Remove couples (admin only)
 - Manage circle settings
 - Archive inactive circles

Data Schema:

```
interface CoupleCircle {
  id: string;

  // Circle information
  name: string;
  description?: string;
  coverPhotoUrl?: string;
  emoji?: string; // Circle emoji/icon

  // Circle type and privacy
  type: 'private' | 'discoverable' | 'activity';
  privacyLevel: 'invite_only' | 'request_to_join';

  // Members
  memberCouples: {
```

```

    coupleId: string;
    role: 'admin' | 'member';
    joinedAt: Date;
    invitedBy: string; // Couple ID who invited
    status: 'active' | 'muted' | 'left';
}[];

memberCount: number; // 2-10 couples
maxMembers: number; // Default 10

// Activity
activityType?: 'travel' | 'gaming' | 'dining' | 'sports' | 'custom';

// Settings
settings: {
    allowMembersToInvite: boolean,
    allowMembersToPost: boolean,
    requireApprovalForPosts: boolean,
    allowEvents: boolean,
    allowSharedAlbums: boolean,
    allowPolls: boolean,
};

// Stats
stats: {
    totalPosts: number,
    totalEvents: number,
    totalPhotos: number,
    lastActivityAt: Date,
};

// Status
status: 'active' | 'archived';
archivedAt?: Date;

createdBy: string; // Couple ID
createdAt: Date;
updatedAt: Date;
}

interface CirclePost {
    id: string;
    circleId: string;

    // Author
    authorCoupleId: string;

    // Content

```



```

contentType: 'text' | 'photo' | 'video' | 'poll' | 'event';
content: string; // Text content

// Media
mediaUrls?: string[];
thumbnailUrls?: string[];

// For polls
pollData?: {
  question: string;
  options: string[];
  votes: {
    optionIndex: number;
    coupleId: string;
  }[];
  expiresAt?: Date;
};

// For events
eventData?: {
  title: string;
  description: string;
  startDate: Date;
  endDate?: Date;
  location?: string;
  attendees: {
    coupleId: string;
    status: 'going' | 'maybe' | 'not_going';
  }[];
};

// Engagement
reactions: {
  [emoji: string]: {
    coupleId: string;
    timestamp: Date;
  }[];
};

comments: CircleComment[];
commentCount: number;

// Privacy
visibility: 'circle_only' | 'members_only';

// Status
isPinned: boolean; // Admins can pin posts
isEdited: boolean;

```

```

    editedAt?: Date;

    createdAt: Date;
    updatedAt: Date;
}

interface CircleComment {
    id: string;
    postId: string;

    // Author
    authorCoupleId: string;

    // Content
    content: string;
    maxLength: 1000;

    // Reactions on comment
    reactions: {
        [emoji: string]: string[]; // Array of couple IDs
    };

    // Threading
    parentCommentId?: string; // For replies

    // Status
    isEdited: boolean;
    editedAt?: Date;
    isDeleted: boolean;

    createdAt: Date;
}

```

User Flow - Creating a Circle:

1. Couple navigates to Circles section
 - View current circles (if any)
 - Tap "+ Create Circle" button
2. Circle setup
 - Enter circle name: "Travel Crew"
 - Choose emoji/icon:
 - Add description: "Friends who love to travel together"
 - Upload cover photo (optional)
 - Select circle type: "Activity - Travel"
 - Set privacy: "Private (Invite Only)"
3. Invite couples
 - Search for couples from SingleFriends

- Or search by username/email
 - Select couples to invite (min 1, max 9 more)
 - Add personal invite message
 - Send invitations
4. Invitations sent
 - Invited couples receive notification
 - Circle shows as "Pending" with invitation count
 - Creator can see invitation status
 5. Couples accept invitations
 - View circle preview
 - See who else is invited/joined
 - Accept or Decline
 - Upon acceptance, join circle immediately
 6. Circle becomes active
 - When 2+ couples are members, circle is "Active"
 - Members can start posting
 - Set up welcome message (optional)
 - Begin sharing content
-

4.2.2 Group Activities

Circle Features for Activities:

```
interface CircleActivities {  
  // Shared Albums  
  sharedAlbums: {  
    create: 'any member can create',  
    addPhotos: 'members can contribute',  
    albumTypes: ['Trip Photos', 'Game Night', 'Dining Out', 'General'],  
  
    permissions: {  
      download: 'members only',  
      addToPersonalLibrary: boolean,  
    }  
  },  
  
  // Events Planning  
  events: {  
    create: 'any member can create',  
  
    eventTypes: [  
      'double_date',  
      'group_trip',  
      'game_night',  
    ],  
  },  
}
```

```

    'dinner',
    'activity',
    'custom',
],

features: {
    dateTime: 'with timezone support',
    location: 'address or map pin',
    rsvp: 'going / maybe / not going',
    reminderNotifications: true,
    addToCalendar: true,

    // Planning tools
    polls: 'vote on date/location',
    splitCosts: 'track shared expenses',
    notes: 'event planning notes',
}
},

// Polls
polls: {
    create: 'any member can create',
    votingStyle: 'one vote per couple',

    pollTypes: [
        'date_selection',           // Pick best date
        'location_selection',       // Pick location
        'activity_selection',       // What to do
        'yes_no',                   // Simple yes/no
        'multiple_choice',          // General poll
    ],

    features: {
        anonymous: boolean,         // Hide who voted for what
        allowMultiple: boolean,    // Multiple selections
        expiresAt: Date,
    }
},

// Shared Lists
lists: {
    types: [
        'travel_bucket_list',      // Places to visit together
        'restaurant_list',         // Places to try
        'movie_list',              // Movies to watch
        'activity_list',           // Things to do
        'custom',
    ],

```

```

    features: {
      addItem: 'any member',
      checkOff: 'any member',
      vote: 'upvote/downvote items',
      notes: 'add notes to items',
    }
  },

  // Group Chat
  chat: {
    enabled: true,
    features: {
      textMessages: true,
      photos: true,
      videos: true,
      gifs: true,
      reactions: true,
      threads: true,
    },

    // Unlike couple chat, this is group chat
    participants: 'all circle members',
    notifications: 'configurable per couple',
  }
}

```

4.2.3 Shared Content Within Circles

Content Sharing Model:

```

interface CircleContentSharing {
  // What can be shared in circles
  shareableContent: {
    // Photos
    photos: {
      source: 'couple photo library',
      selection: 'choose specific photos',
      albums: 'share entire album',
      permissions: {
        membersCanDownload: boolean,
        membersCanReact: boolean,
        membersCanComment: boolean,
      }
    },

    // Videos

```

```

videos: {
  source: 'couple video library',
  maxSize: '500MB',
  transcoding: 'optimized for circle viewing',
},

// Hall of Fame achievements
hallOfFame: {
  shareIndividual: true,
  shareCategory: true,
  autoShare: 'optional setting',
},

// Scribbles and Paintings
artwork: {
  shareAsImage: true,
  shareOriginal: false,          // Keep editable version private
}
},

// Feed structure
feed: {
  algorithm: 'chronological',    // No algorithmic sorting
  postTypes: [
    'photo',
    'video',
    'text_update',
    'poll',
    'event',
    'shared_list_item',
    'achievement',
  ],

  filtering: {
    filterByCouple: 'see posts from specific couple',
    filterByType: 'see only events, only photos, etc.',
    search: 'search content',
  }
},

// Engagement
engagement: {
  reactions: {
    emoji: 'standard emojis',
    perCouple: 'one reaction per couple',
    visible: 'all members see reactions',
  },
},

```

```

comments: {
  nested: 'support comment threads',
  maxDepth: 2, // Original + Reply + Reply to reply
  editing: 'within 5 minutes',
  deletion: 'by author or admins',
},

notifications: {
  newPost: 'notify all members',
  mentionedInPost: 'notify mentioned couple',
  commentOnPost: 'notify post author',
  reactionOnPost: 'notify post author',

  frequency: 'immediate' | 'digest',
}
}
}

```

4.3 Hall of Fame

4.3.1 Achievement System

Overview: The Hall of Fame celebrates relationship milestones and achievements, creating a structured timeline of significant moments. The system focuses on positive reinforcement rather than competitive mechanics, emphasizing memory preservation and relationship progress.

Achievement Categories:

```

interface AchievementCategory {
  // Time-based milestones
  timeBased: {
    daysTogether: [1, 7, 30, 100, 365, 500, 1000], // Days
    monthsTogether: [1, 3, 6, 12, 18, 24, 36], // Months
    yearsTogether: [1, 2, 3, 5, 10, 25, 50], // Years

    // Specific dates
    firstWeek: 7,
    firstMonth: 30,
    firstYear: 365,
    hundredDays: 100, // Popular in Asian cultures
    thousandDays: 1000,
  },

  // Communication milestones
  communication: {
    messages: [100, 1000, 10000, 50000, 100000],
    voiceCalls: [10, 50, 100, 500, 1000],
    videoCall: [10, 50, 100, 500, 1000],
  }
}

```

```

    voiceMinutes: [60, 600, 3600, 36000],           // 1hr, 10hr, 100hr, 1000hr
    videoMinutes: [60, 600, 3600, 36000],
},

// Creative milestones
creative: {
    scribbles: [1, 10, 50, 100],
    paintings: [1, 5, 25, 50],
    customEmojis: [1, 10, 25, 50],
    soundsCreated: [1, 10, 50],
},

// Memories milestones
memories: {
    photosShared: [10, 100, 1000, 5000, 10000],
    videosShared: [5, 25, 100, 500],
    highlightedMessages: [1, 10, 50, 100],
},

// Streak milestones
streaks: {
    photoStreaks: [7, 30, 100, 365],                // Days
    longestStreak: 'track longest ever',
},

// Special events
specialEvents: {
    engagement: 'manual entry',
    marriage: 'manual entry',
    anniversary: 'yearly recurring',
    livingTogether: 'manual entry',
    tripTogether: 'manual entry',
    custom: 'user-defined milestones',
}
}

Data Schema:

interface Achievement {
    id: string;
    coupleId: string;

    // Achievement details
    type: keyof AchievementCategory;
    category: string;           // E.g., 'time_based', 'communication'
    name: string;               // E.g., "100 Days Together"
    description: string;        // E.g., "You've been together for 100 days!"

    // Visual

```



```

    imageUrl: string; // Achievement badge icon
    color: string; // Badge color
    rarity: 'common' | 'uncommon' | 'rare' | 'epic' | 'legendary';

    // Value
    value: number; // E.g., 100 for "100 Days"
    unit: string; // E.g., 'days', 'messages', 'photos'

    // Status
    status: 'locked' | 'in_progress' | 'unlocked' | 'celebrated';
    progress?: number; // Current progress (0-100%)

    // Unlocking
    unlockedAt?: Date;

    // Celebration
    celebratedAt?: Date;
    celebrationNote?: string; // Couple's note about achievement
    celebrationPhoto?: string; // Optional photo for milestone

    // Sharing
    visibility: 'private' | 'friends' | 'circles';
    sharedWith?: string[]; // Friend or circle IDs

    // Reactions from friends/circles
    reactions: {
        emoji: string;
        fromCoupleId?: string;
        fromUserId?: string;
        timestamp: Date;
    }[];

    congratulations: {
        message: string;
        fromCoupleId?: string;
        fromUserId?: string;
        timestamp: Date;
    }[];

    createdAt: Date;
    updatedAt: Date;
}

interface HallOfFameSettings {
    coupleId: string;

    // Relationship start date (crucial for time-based achievements)
    relationshipStartDate: Date;
}

```

```

// Anniversary date (might be different from start date)
anniversaryDate?: Date;

// Special dates
specialDates: {
  name: string; // E.g., "First Kiss", "Moved In Together"
  date: Date;
  category: string;
  isRecurring: boolean;
  reminderDays: number; // Remind X days before
}[];

// Notification preferences
notifications: {
  upcomingMilestones: boolean; // Notify 7 days before
  achievementUnlocked: boolean; // Instant notification
  friendCelebrations: boolean; // When friends react
};

// Display preferences
display: {
  showProgress: boolean, // Show progress bars for locked achievements
  showLockedAchievements: boolean,
  sortBy: 'date_unlocked' | 'category' | 'rarity',
  filterBy: string[], // Categories to show
};

// Privacy
defaultVisibility: 'private' | 'friends' | 'circles';
autoShareMajorMilestones: boolean; // Auto-share years, marriage, etc.
}

```

User Flow - Unlocking Achievement:

1. Couple reaches milestone (e.g., 100 days together)
 - System detects milestone automatically
 - Achievement status changes: locked → unlocked
 - Notification sent to both partners
2. Partners receive notification
 - Push notification: " New Achievement Unlocked!"
 - Tap to open Hall of Fame
 - Achievement appears with animation
 - Confetti or celebration animation plays
3. Achievement celebration modal
 - Large badge displayed
 - Achievement name: "100 Days Together"

Description: "You've been together for 100 wonderful days!"

Stats shown: Started [Date], Reached [Today]

Options:

"Add Note" - Write about this milestone

"Add Photo" - Commemorate with photo

"Share" - Share with friends/circles

"Close" - Save to Hall of Fame

Couple adds celebration note and photo

4. Adding celebration details

Write note: "100 days of adventures together! "

Select photo from library or take new one

Choose visibility: Private, Friends, or Circles

Tap "Celebrate"

5. Achievement saved

Added to Hall of Fame timeline

Visible on couple's Hall of Fame page

Shared to selected friends/circles (if chosen)

Friends can react and congratulate

6. Friend reactions

Friends see achievement in feed

Can react with emojis

Can leave congratulations message

Couple receives notifications of support

Creates positive social reinforcement

4.3.2 Milestone Tracking

Automatic Milestone Detection:

```
interface MilestoneTracker {
  // Background job that runs daily
  dailyCheck: {
    frequency: 'once_per_day',
    runTime: '00:00 couple local timezone',

    checks: [
      'days_together_milestones',
      'communication_milestones',
      'photo_milestones',
      'streak_milestones',
      'upcoming_anniversaries',
    ],

    process: [
```

```

        'calculate_current_values',
        'compare_to_thresholds',
        'unlock_achieved_milestones',
        'update_progressBars',
        'send_notifications',
    ]
},

// Real-time detection for certain achievements
realTimeCheck: {
    triggers: [
        'message_sent',           // Check message count
        'photo_shared',           // Check photo count
        'call_ended',             // Check call duration
        'streak_updated',         // Check streak length
        'custom_emoji_created',   // Check emoji count
    ],

    // Debouncing to avoid performance issues
    debounce: '5 seconds',
    batchUpdates: true,
},

// Upcoming milestone notifications
upcomingNotifications: {
    // Notify 7 days before big milestones
    advanceNotice: [
        { milestone: '1_year', daysBefore: 30 },
        { milestone: '100_days', daysBefore: 7 },
        { milestone: 'anniversary', daysBefore: 7 },
    ],

    message: 'Your [milestone] is coming up in [X] days!',
    suggestion: 'Start planning something special ',
}
}

```

4.3.3 Memory Highlights

Memory Feature in Hall of Fame:

```

interface MemoryHighlight {
    id: string;
    achievementId: string;
    coupleId: string;

    // Memory content

```

```

title: string; // E.g., "Our First Year Together"
description: string; // Couple's reflection

// Media
coverImageUrl?: string;
photoUrls: string[]; // Collection of photos
videoUrl?: string;

// Associated achievement
achievement: Achievement;

// Timeline position
date: Date; // When milestone occurred

// Tags
tags: string[]; // E.g., ['travel', 'anniversary', 'special']

// Sharing
visibility: 'private' | 'friends' | 'circles';

createdAt: Date;
updatedAt: Date;
}

```

Hall of Fame Display:

```

interface HallOfFameUI {
  // Layout
  layout: 'timeline', // Chronological timeline view

  // Sections
  sections: {
    // Hero section
    hero: {
      daysTogetherCounter: 'animated counter',
      nextMilestone: 'progress bar to next achievement',
      recentAchievements: 'last 3 unlocked',
    },

    // Stats overview
    stats: {
      totalAchievements: number,
      categoryCounts: Record<string, number>,
      rarity Distribution: Record<string, number>,
    },

    // Timeline view
    timeline: {
      sortOrder: 'newest_first' | 'oldest_first',
    }
  }
}

```

```

groupBy: 'month' | 'year' | 'category',

achievementCard: {
  badge: 'large icon',
  name: 'achievement name',
  date: 'unlocked date',
  note: 'celebration note',
  photo: 'celebration photo',
  reactions: 'friend reactions',
}
},

// Filters
filters: {
  category: string[],
  rarity: string[],
  unlocked: boolean,
  celebrated: boolean,
}
}
}
}

```

4.4 Photo Streaks

4.4.1 Streak Mechanics

Overview: Photo Streaks encourage daily connection through a simple, fun mechanic: send a photo to your partner every day to maintain the streak. Its not about pressureits about creating a daily touchpoint and building a visual diary of your days together.

Core Mechanics:

```

interface PhotoStreak {
  id: string;
  coupleId: string;

  // Streak status
  currentStreak: number;           // Current consecutive days
  longestStreak: number;           // All-time longest
  totalDays: number;               // Total days with photos (not consecutive)

  // Today's status
  todayStatus: {
    partner1Contributed: boolean,
    partner2Contributed: boolean,
    bothContributed: boolean,      // Streak counts only if both send

    partner1PhotoUrl?: string,

```

```

    partner2PhotoUrl?: string,

    contributedAt?: Date,
};

// Streak rules
rules: {
    requireBothPartners: true,           // Both must contribute
    deadlineTime: '23:59:59',           // Local timezone
    gracePeriod: '3 hours',             // Until 02:59:59 next day
    allowMultiplePhotos: true,          // Can send more than one
};

// History
history: {
    date: Date,
    partner1PhotoUrl?: string,
    partner2PhotoUrl?: string,
    bothContributed: boolean,
    streakCount: number,                // Streak length on that day
}[];

// Streak recovery (see 4.4.3)
recoveryData?: {
    missedDate: Date,
    recoveryUsed: boolean,
    recoveryExpiresAt: Date,
};

// Milestones
streakMilestones: number[];            // [7, 30, 100, 365]
achievedMilestones: number[];          // Unlocked streak achievements

// Settings
settings: {
    notificationsEnabled: boolean,
    reminderTime: string,               // E.g., "20:00" (8 PM)
    urgentReminderTime: string,         // E.g., "22:00" (10 PM)
};

// Stats
stats: {
    averagePhotosPerDay: number,
    favoritePhotoTime: string,          // When photos are usually sent
    longestGap: number,                 // Longest time between contributions
};

createdAt: Date;

```

```
    updatedAt: Date;
    lastContributionAt: Date;
}
```

User Flow - Daily Streak:

1. Morning (or custom time) - Reminder notification
 - Push notification: "Don't forget your photo streak! "
 - Shows current streak: "23 days"
 - Tap to open camera or photo picker
 - Shows partner's status: "Partner hasn't sent yet"
 2. User sends streak photo
 - Method 1: Quick camera from notification
 - Method 2: Select from recent photos
 - Method 3: Open LinkUp → Streak tab → Add photo
 - Photo uploads
 - Confirmation: "Photo added to today's streak!"
 - Status updates: "Waiting for partner..."
 3. Partner sends their photo
 - Partner uploads their photo
 - System detects both partners contributed
 - Streak increments: 23 → 24 days
 - Both partners receive celebration notification
 4. Streak celebration
 - Notification: " 24-day streak! Keep it going!"
 - Both photos displayed side-by-side
 - Streak counter updates
 - Optional: Add caption to today's photos
 - Photos saved to Streak Gallery
 5. Evening - Urgent reminder (if not completed)
 - If either partner hasn't sent photo
 - Push notification: " Send your streak photo before midnight!"
 - Shows time remaining: "2 hours left"
 - Partner status: "Partner sent already. Your turn!"
 - One-tap camera access
 6. Midnight - Streak check
 - System checks if both partners contributed
 - If yes: Streak continues, count increments
 - If no: Streak broken (see recovery options)
 - New day begins
-

4.4.2 Streak Rewards

Gamification Without Manipulation:

```
interface StreakRewards {
    // Milestone celebrations
    milestones: {
        7: {
            name: 'Week Warrior',
            badge: 'bronze_flame',
            reward: 'unlock_custom_streak_theme',
            message: '7 days strong! You're building a habit ',
        },

        30: {
            name: 'Monthly Master',
            badge: 'silver_flame',
            reward: 'unlock_streak_frames',
            message: '30 days! This is becoming your thing ',
        },

        100: {
            name: 'Century Streak',
            badge: 'gold_flame',
            reward: 'unlock_premium_filters',
            message: '100 DAYS! You two are unstoppable! ',
        },

        365: {
            name: 'Yearly Legend',
            badge: 'platinum_flame',
            reward: 'unlock_anniversary_collage',
            message: 'ONE FULL YEAR! This is legendary ',
        }
    },

    // Rewards (non-manipulative)
    rewards: {
        customThemes: {
            unlock: [7, 30, 100],
            themes: ['sunset', 'galaxy', 'pastel', 'neon'],
            apply: 'to streak display only',
        },

        photoFrames: {
            unlock: [30, 100, 365],
            frames: ['heart', 'polaroid', 'film', 'celebration'],
            apply: 'optional decoration for streak photos',
        }
    }
}
```

```

    },

    filters: {
      unlock: [100],
      filters: ['romantic', 'vintage', 'artistic'],
      apply: 'quick filters for streak photos',
    },

    collage: {
      unlock: [365],
      feature: 'auto-generate yearly photo collage',
      export: 'shareable image',
    }
  },

  // Weekly recap (non-competitive)
  weeklyRecap: {
    frequency: 'every_sunday',
    content: {
      streakStatus: 'current streak length',
      thisWeekPhotos: 'grid of week photos',
      bestMoments: 'AI picks most interesting photos',
      quote: 'inspirational quote about consistency',
    },
    notification: 'Sunday Recap: Your week in photos',
  }
}

```

4.4.3 Streak Recovery System

Compassionate Streak Design:

Unlike manipulative streaks (Snapchat), LinkUp provides streak recovery options because life happens and couples shouldn't feel punished for missing a day.

```

interface StreakRecovery {
  // Streak freeze (prevention)
  streakFreeze: {
    description: 'Pause streak for special circumstances',

    reasons: [
      'traveling_without_phone',
      'emergency',
      'sick',
      'no_internet',
      'special_occasion',
    ],
  },
}

```

```

duration: 'up to 3 days',
usageLimit: '2 per month',

activation: {
  when: 'activate before or within 24 hours of missed day',
  both Partners: 'requires both partners to agree',
  notification: 'partner notified of freeze',
},

effect: 'streak doesn't break, but doesn't increment either',
},

// Streak recovery (after break)
streakRecovery: {
  description: 'Recover broken streak within grace period',

  gracePeriod: '24 hours after midnight',
  cost: 'free', // No punitive cost
  usageLimit: '1 per week',

  process: [
    'realize_streak_was_missed',
    'within_24_hours_send_photos',
    'system_offers_recovery',
    'both_partners_accept',
    'streak_restored',
  ],

  effect: 'streak continues as if unbroken',
  note: 'can only recover if recovery is available',
},

// Streak restart (after break without recovery)
streakRestart: {
  description: 'Start new streak after break',

  process: {
    broken: 'streak ends at last count',
    savedToHistory: 'longest streak saved',
    newStreak: 'new streak starts at 1',
    motivation: 'positive message, no shame',
  },

  message: 'Your [X]-day streak was amazing! Ready to start a new one?',
  noNegativeMessaging: true,
},

```

```

// Philosophy
designPhilosophy: {
  principle: 'Encourage connection, not manipulation',
  compassion: 'Life happens, relationships are more important than streaks',
  flexibility: 'Provide tools to maintain streaks realistically',
  celebration: 'Focus on achievement, not punishment for missing',
}
}

Streak Gallery:

interface StreakGallery {
  // View all streak photos
  layout: 'calendar_grid',

  // Display
  display: {
    monthlyView: {
      calendar: 'month calendar with photo thumbnails',
      heatmap: 'shows streak intensity',
      gaps: 'clearly shows missed days',
    },

    yearlyView: {
      overview: 'year at a glance',
      stats: 'total days, longest streak, etc.',
      highlights: 'best photos from year',
    },

    dayView: {
      date: 'selected date',
      partner1Photo: 'full size',
      partner2Photo: 'full size',
      sideBySide: 'comparison view',
      captions: 'if added',
      location: 'if geo-tagged',
    }
  },

  // Interactions
  interactions: {
    selectDay: 'tap calendar day',
    swipe: 'navigate between days',
    favorite: 'mark favorite days',
    addCaption: 'add caption to day',
    share: 'share specific day to friends/circles',
  },

  // Export

```

```

export: {
  monthCollage: 'create month collage',
  yearBook: 'create yearly photo book',
  timelapse: 'create timelapse video',
  download: 'download all photos',
}
}

```

Chapter 4 Status: Complete

Summary: - SingleFriend System: Controlled sharing with up to 10 individual friends, granular permissions - Couple Circles: Private groups for 2-10 couples with posts, events, albums, polls - Hall of Fame: Achievement system celebrating milestones across 6 categories with friend reactions - Photo Streaks: Daily photo sharing with compassionate streak mechanics and recovery options - Complete data schemas, user flows, and UI specifications for all social features

Next Chapter: Chapter 5: Entertainment & Shared Experiences

Chapter 5: Entertainment & Shared Experiences

Overview

Entertainment features transform LinkUp from a communication platform into a shared experience engine. Couples don't just talk about what they watched or listened to; they experience it together in real-time, even when apart.

This chapter covers: - Streaming - Live stream your activities to your partner - Video Hosting Integration - Watch Netflix, Prime Video, TikTok, and YouTube together - Music Integration - Listen to Spotify simultaneously with synchronized playback

Design Philosophy: - Seamless integration - One tap to start watching/listening together - Perfect synchronization - Both partners see/hear the same thing at the same time - Quality first - High-quality streaming with minimal latency - Presence awareness - See each other's reactions during shared experiences

5.1 Streaming Feature

5.1.1 Live Streaming to Partner

Overview: Live stream your activities, surroundings, or screen to your partner in real-time. Unlike video calls where both sides are visible, streaming is one-way broadcasting perfect for: - Come with me on this walk - Let me show you this event I'm at - Watch me cook dinner - See what I'm seeing right now

Core Features:

1. Stream Types

- Camera Stream - Broadcast from phone camera (front or back)
 - Screen Stream - Share your phone/computer screen
 - Mixed Stream - Picture-in-picture (camera + screen)
2. Stream Quality
 - Auto Quality - Adaptive bitrate based on network (360p to 1080p)
 - Manual Quality - Force specific quality
 - Low Latency Mode - Prioritize speed over quality (<1 second delay)
 - High Quality Mode - Prioritize quality over speed (1-3 second delay)
 3. Interactive Elements
 - Voice Commentary - Broadcaster can talk during stream
 - Viewer Reactions - Viewer sends emoji reactions (appear on broadcasters screen)
 - Chat Overlay - Optional text chat during stream
 - Switch to Video Call - Convert stream to two-way video call instantly

Data Schema:

```
interface LiveStream {
  id: string;
  coupleId: string;

  // Stream details
  streamerId: string;           // Who is broadcasting
  viewerId: string;            // Partner watching

  // Stream configuration
  streamType: 'camera' | 'screen' | 'mixed';
  cameraFacing: 'front' | 'back'; // If camera stream

  // Quality settings
  quality: {
    mode: 'auto' | 'manual';
    resolution: '1080p' | '720p' | '480p' | '360p';
    fps: 30 | 60;
    bitrate: number;           // Kbps
    latencyMode: 'low' | 'standard' | 'high_quality';
  };

  // Audio
  audioEnabled: boolean;
  audioSource: 'microphone' | 'system' | 'both';

  // Status
  status: 'starting' | 'live' | 'ended' | 'failed';
  startedAt?: Date;
  endedAt?: Date;
  duration?: number;           // Seconds

  // Viewer interaction
  viewerPresent: boolean;      // Is viewer actively watching
}
```

```

viewerJoinedAt?: Date;

// Reactions during stream
reactions: {
  emoji: string;
  timestamp: number;           // Seconds from stream start
  fromViewer: boolean;
}[];

// Recording (optional)
isRecording: boolean;
recordingUrl?: string;

// Quality metrics
metrics: {
  avgBitrate: number;
  avgFps: number;
  packetLoss: number;
  avgLatency: number;         // Milliseconds
  bufferingEvents: number;
  qualitySwitches: number;
};

// WebRTC session
webrtcSessionId: string;

createdAt: Date;
updatedAt: Date;
}

```

User Flow - Starting a Live Stream:

1. User wants to share experience with partner
 - Opens LinkUp app
 - Tap camera icon → holds → "Start Live Stream" appears
 - Or navigate to Streaming section → "Go Live"
2. Stream setup
 - Choose stream type:
 - "Camera" - Show what I'm seeing
 - "Screen" - Share my screen
 - "Both" - Camera + screen
 - If camera: Choose front or back camera
 - Preview shown
 - Optional: Add title "Walking through the park "
 - Tap "Start Streaming"
3. Stream initiates
 - Connecting to streaming server

- WebRTC connection established
 - Buffering starts
 - Partner receives notification
 - Status: "Live" indicator appears
4. Partner receives notification
 - Push notification: " [Partner] is live!"
 - Shows stream title and thumbnail
 - Tap to join stream
 - Or ignore (can join later from chat)
 5. Partner joins stream
 - Stream loads instantly
 - Sees broadcaster's camera/screen
 - "Joined" notification sent to broadcaster
 - Broadcaster sees "Partner is watching"
 - Stream continues
 6. During stream
 - Broadcaster:
 - Can talk (audio transmitted)
 - Can switch cameras
 - Can pause/resume
 - Can see viewer reactions
 - Can end stream anytime
 - Viewer:
 - Can send reactions (heart, laugh, surprised, applause, fire)
 - Can send text messages (chat overlay)
 - Can request switch to video call
 - Can leave stream (continues for broadcaster)
 7. Ending stream
 - Broadcaster taps "End Stream"
 - Confirmation: "End live stream?"
 - Confirms
 - Stream ends for both
 - Summary shown:
 - Duration: 12m 34s
 - Reactions received: 23
 - Quality: Excellent
 - Optional: Save recording to media library

Technical Implementation:

```
const LiveStreamingConfig = {  
  // Streaming protocol  
  protocol: 'WebRTC',           // Real-time communication  
  fallback: 'HLS',             // For poor connections
```



```

// Video codec
videoCodec: {
  preferred: 'H264',
  profile: 'Main',
  level: '4.1',
  hardwareAcceleration: true,
},

// Audio codec
audioCodec: {
  preferred: 'Opus',
  sampleRate: 48000,
  bitrate: 64000, // 64 kbps
},

// Adaptive bitrate streaming
abr: {
  enabled: true,

  profiles: [
    { name: '1080p', width: 1920, height: 1080, bitrate: 4000000 },
    { name: '720p', width: 1280, height: 720, bitrate: 2500000 },
    { name: '480p', width: 854, height: 480, bitrate: 1000000 },
    { name: '360p', width: 640, height: 360, bitrate: 600000 },
  ],

  // Switch thresholds
  switchCriteria: {
    bandwidth: 'measured_bandwidth * 0.8',
    bufferHealth: 'maintain 3-5 seconds buffer',
    packetLoss: 'switch down if > 5%',
  }
},

// Latency optimization
latency: {
  mode: 'low_latency', // Default mode

  settings: {
    low: {
      targetLatency: 500, // 500ms
      maxLatency: 1000,
      bufferSize: 'minimal',
    },

    standard: {
      targetLatency: 2000, // 2 seconds
    }
  }
}

```

```

        maxLatency: 4000,
        bufferSize: 'medium',
    },

    high_quality: {
        targetLatency: 3000,           // 3 seconds
        maxLatency: 6000,
        bufferSize: 'large',
    }
}
},

// Server infrastructure
infrastructure: {
    streamingServers: 'distributed edge servers',
    regions: ['us-east', 'us-west', 'eu-west', 'ap-south'],

    serverSelection: {
        method: 'lowest_latency',
        measurement: 'ping to multiple servers',
        fallback: 'geographically closest',
    }
}
};

```

5.1.2 Stream Quality & Latency

Quality Tiers:

```

interface StreamQuality {
    // Preset quality levels
    presets: {
        ultra: {
            resolution: '1080p',
            fps: 60,
            videoBitrate: 4000,           // Kbps
            audioBitrate: 128,
            latency: 'standard',
            use Case: 'high quality scenes, good network',
        },

        high: {
            resolution: '720p',
            fps: 30,
            videoBitrate: 2500,
            audioBitrate: 96,
            latency: 'low',
        },
    },
}

```

```

    useCase: 'default for most situations',
  },

  medium: {
    resolution: '480p',
    fps: 30,
    videoBitrate: 1000,
    audioBitrate: 64,
    latency: 'low',
    useCase: 'moderate network conditions',
  },

  low: {
    resolution: '360p',
    fps: 24,
    videoBitrate: 600,
    audioBitrate: 48,
    latency: 'very_low',
    useCase: 'poor network, prioritize connection',
  }
},

// Auto quality adjustment
autoAdjust: {
  enabled: true,
  measurementInterval: 5000,      // Check every 5 seconds

  metrics: {
    bandwidth: 'measure available bandwidth',
    rtt: 'round trip time to server',
    packetLoss: 'percentage of lost packets',
    bufferHealth: 'playback buffer level',
    cpuUsage: 'device CPU utilization',
  },

  decisions: {
    upgradeThreshold: 'stable 30 seconds at higher bandwidth',
    downgradeThreshold: 'immediate on quality issues',
    maxSwitchesPerMinute: 2,      // Avoid rapid switching
  }
}
}

```

5.1.3 Interactive Elements During Streaming

Reaction System:

```

interface StreamReactions {
  // Available reactions
  reactionEmojis: ['heart', 'laugh', 'surprised', 'applause', 'fire', 'love', 'celebrate', 'pe

  // Reaction display
  display: {
    animation: 'float_up_and_fade',
    duration: 3000, // 3 seconds
    position: 'random_x_bottom_to_top',
    size: '48px',
    maxSimultaneous: 10,
  },

  // Sending reactions
  sending: {
    method: 'tap_reaction_button',
    cooldown: 500, // 500ms between reactions
    queueing: 'if_too_fast',
  },

  // Broadcaster view
  broadcasterView: {
    showReactions: true,
    overlay: 'semi-transparent',
    sound: 'optional_subtle_ping',
    hapticFeedback: 'light_tap',
  },

  // Recording
  saveReactions: true, // Include in recording
  replayReactions: 'at_same_timestamp',
}

interface StreamChat {
  // Text chat during stream
  enabled: true,

  // Display
  display: {
    position: 'bottom_overlay',
    maxMessages: 5,
    autoHide: 'after_5_seconds',
    style: 'chat_bubble',
  },

  // Sending
  input: {
    maxLength: 200,

```

```

    sendButton: true,
    voiceInput: false,                // Keep focus on stream
},

// Notifications
notifyBroadcaster: {
    sound: 'subtle',
    visual: 'message_icon',
    fullMessage: 'on_tap',
}
}

```

5.2 Video Hosting Integration

5.2.1 Netflix Watch Party

Overview: Watch Netflix shows and movies together with synchronized playback, even when apart. Perfect for long-distance couples or when one partner is traveling.

Important Note: Netflix does not provide official API for third-party watch party features. Implementation approaches:

1. Browser Extension Approach (Web only)
 - Chrome/Firefox extension for Netflix.com
 - Inject synchronization controls
 - Communicate playback state via LinkUp backend
2. Screen Share Approach (Cross-platform)
 - One partner streams Netflix via screen share
 - Other partner watches stream
 - Simpler but lower quality
3. Partnership Approach (Ideal)
 - Official partnership with Netflix
 - Access to playback APIs
 - Best user experience

Data Schema:

```

interface WatchParty {
  id: string;
  coupleId: string;

  // Service details
  service: 'netflix' | 'prime_video' | 'youtube' | 'tiktok' | 'custom';

  // Content information
  content: {
    contentId: string;                // Service-specific ID
    contentType: 'movie' | 'tv_show' | 'video' | 'short';
    title: string;
  }
}

```

```

    thumbnail: string;
    duration?: number;                // Seconds

    // For TV shows
    season?: number;
    episode?: number;
};

// Playback state
playback: {
    status: 'waiting' | 'playing' | 'paused' | 'ended';
    currentTime: number;              // Seconds
    lastSyncTime: Date;

    // Host (who controls playback)
    hostUserId: string;
    allowGuestControl: boolean;       // Can guest pause/seek?
};

// Synchronization
sync: {
    method: 'extension' | 'screen_share' | 'api';
    lastSyncAt: Date;
    syncOffset: number;               // Milliseconds difference

    // Sync events
    syncEvents: {
        type: 'play' | 'pause' | 'seek' | 'speed_change';
        triggeredBy: string;
        timestamp: Date;
        value?: number;               // For seek, speed
    }[];
};

// Communication during watch party
communication: {
    videoCallActive: boolean;         // Picture-in-picture video
    voiceChatActive: boolean;         // Voice-only chat
    textChatActive: boolean;          // Text reactions

    reactions: {
        emoji: string;
        userId: string;
        videoTimestamp: number;       // When in video reaction occurred
        timestamp: Date;
    }[];
};

```

```

// Stats
stats: {
  duration: number,
  syncIssues: number,
  qualitySwitches: number,
  averageLatency: number,
};

// Status
status: 'active' | 'paused' | 'ended';

startedAt: Date;
endedAt?: Date;
createdAt: Date;
updatedAt: Date;
}

```

User Flow - Netflix Watch Party:

1. User wants to watch Netflix together
 - Opens LinkUp
 - Navigate to "Watch Together" section
 - Select "Netflix"
 - Choose implementation method available
2. Method 1: Browser Extension (Web)
 - Install LinkUp Netflix Extension (one-time)
 - Open Netflix in browser
 - Extension detects LinkUp account
 - "Watch with Partner" button appears on Netflix
 - User clicks button
 - Invitation sent to partner
3. Method 2: Screen Share (Mobile/Desktop)
 - User selects "Watch via Screen Share"
 - Opens Netflix app/website
 - Starts screen sharing in LinkUp
 - Partner receives notification
 - Partner joins to watch stream
4. Partner receives invitation
 - Push notification: " Watch [Show Name] together?"
 - Shows thumbnail and episode info
 - Options: "Join" or "Not Now"
 - Partner taps "Join"
5. Watch party setup (Extension method)
 - Partner's Netflix opens to same content
 - Synchronization established

Both see " Synced with Partner"
Optional: Enable video/voice chat (PiP)
Ready to watch

6. During watch party

Host controls (or both if enabled):

- Play/Pause - synced instantly
- Seek - both jump to same position
- Speed - playback speed synced
- Volume - individual control

Communication:

- Picture-in-picture video of partner
- Or voice-only chat
- Send emoji reactions at moments
- Text chat overlay (optional)

Sync maintenance:

- Automatic sync every 10 seconds
- If drift detected (>2 seconds), resync
- Visual indicator: " Synced" or " Syncing..."
- Manual resync button available

7. Ending watch party

- Either partner can end
- Or automatic end when content finishes
- Summary shown:
 - Watched: S01E03 of Show Name
 - Duration: 42 minutes
 - Reactions sent: 15
 - Sync quality: Excellent
- Option to start next episode together

Netflix Extension Architecture:

```
const NetflixExtension = {  
  // Extension manifest  
  manifest: {  
    name: 'LinkUp for Netflix',  
    permissions: [  
      'tabs',  
      'storage',  
      'webRequest',  
      'https://www.netflix.com/*',  
    ],  
  
    contentScripts: [{  
      matches: ['https://www.netflix.com/*'],  
      js: ['content-script.js'],  
    }],  
  },  
};
```



```

    css: ['overlay.css'],
  }]
},

// Content script (injected into Netflix page)
contentScript: {
  detectPlayer: 'find video player element',

  capturePlaybackState: {
    currentTime: 'video.currentTime',
    paused: 'video.paused',
    duration: 'video.duration',
    playbackRate: 'video.playbackRate',
  },

  injectControls: {
    syncButton: 'add LinkUp sync button to player controls',
    syncIndicator: 'show sync status',
    partnerPresence: 'show partner watching indicator',
  },

  eventListeners: {
    play: 'send to partner',
    pause: 'send to partner',
    seeked: 'send to partner',
    ratechange: 'send to partner',
  }
},

// Background script
backgroundScript: {
  websocketConnection: 'maintain connection to LinkUp servers',

  syncLogic: {
    sendLocalState: 'every 2 seconds',
    receivePartnerState: 'apply to local player',

    conflictResolution: {
      rule: 'host takes precedence',
      guestCanControl: 'if permission enabled',
    },

    driftCorrection: {
      threshold: 2000, // 2 seconds
      action: 'seek to correct position',
    }
  }
}
}

```

```
};
```

5.2.2 Prime Video Integration

Overview: Similar to Netflix, but Prime Video has slightly different implementation considerations.

Implementation Approach:

```
interface PrimeVideoIntegration {  
  // Similar to Netflix but with Prime-specific considerations  
  implementation: {  
    browserExtension: {  
      supportedBrowsers: ['Chrome', 'Firefox', 'Edge'],  
      domains: ['https://www.amazon.com/primevideo/*'],  
  
      differences: {  
        playerAPI: 'different than Netflix',  
        urlStructure: 'different content ID format',  
        playerControls: 'different DOM structure',  
      }  
    },  
  
    screenShare: {  
      fallback: true,  
      quality: 'up to 1080p depending on network',  
    }  
  },  
  
  // Content metadata  
  contentDetection: {  
    title: 'extract from page title',  
    thumbnail: 'extract from og:image meta tag',  
    seasonEpisode: 'parse from URL or page content',  
    duration: 'from video element',  
  }  
}
```

5.2.3 TikTok Sharing

Overview: Share TikTok videos with your partner and watch together in real-time. Lighter weight than Netflix/Prime since videos are short.

Features:

```
interface TikTokIntegration {  
  // Sharing methods  
  sharingMethods: {
```

```

// Method 1: Share link
shareLink: {
  process: [
    'copy_tiktok_link',
    'paste_in_linkup_chat',
    'auto_detect_tiktok_url',
    'generate_preview',
    'tap_to_watch_together',
  ],

  preview: {
    thumbnail: 'video thumbnail',
    creator: '@username',
    caption: 'video caption',
    stats: 'likes, comments (optional)',
  }
},

// Method 2: Built-in browser
inAppBrowser: {
  feature: 'browse TikTok within LinkUp',
  process: [
    'open_tiktok_in_linkup',
    'scroll_feed',
    'tap_share_icon',
    'invite_partner_to_watch',
  ]
},

// Method 3: Screen share
screenShare: {
  use: 'share TikTok feed via screen streaming',
  interactive: 'partner sees what you see',
}
},

// Watch together mode
watchTogether: {
  synchronization: {
    videoStart: 'both start at same time',
    pause: 'synced pause/resume',
    replay: 'both replay together',
  },

  interaction: {
    reactions: 'send reactions during video',
    voiceChat: 'optional voice while watching',
    nextVideo: 'queue next TikTok',
  }
}

```

```

    },
    queue: {
      createPlaylist: 'queue multiple TikToks',
      autoPlay: 'play next after current ends',
      collaborative: 'both can add to queue',
    }
  }
}

```

Data Schema:

```

interface TikTokShare {
  id: string;
  coupleId: string;

  // TikTok video details
  video: {
    tiktokUrl: string;
    videoId: string;
    creator: {
      username: string;
      displayName: string;
      avatarUrl: string;
    };
    caption: string;
    thumbnailUrl: string;
    duration: number;

    // Optional metadata
    stats?: {
      likes: number;
      comments: number;
      shares: number;
    };
  };

  // Watch session
  session: {
    status: 'queued' | 'watching' | 'completed';
    startedAt?: Date;
    endedAt?: Date;

    // Playback sync
    currentTime: number;
    isPlaying: boolean;

    // Participants
    initiator: string;
  };
}

```

```

    partnerJoined: boolean;
    partnerJoinedAt?: Date;
};

// Reactions during video
reactions: {
    emoji: string;
    userId: string;
    timestamp: number; // Seconds into video
}[];

// Queue position (if part of playlist)
queuePosition?: number;
playlistId?: string;

sharedAt: Date;
createdAt: Date;
}

interface TikTokPlaylist {
    id: string;
    coupleId: string;

    name?: string; // Optional playlist name
    videos: TikTokShare[];
    currentVideoIndex: number;

    // Playback
    autoPlay: boolean; // Auto-play next
    shuffle: boolean;

    createdAt: Date;
    updatedAt: Date;
}

```

5.2.4 YouTube Watch Together

Overview: Watch YouTube videos together with synchronized playback. YouTube has better API support than Netflix/Prime, making this easier to implement.

YouTube API Integration:

```

interface YouTubeIntegration {
    // Implementation method
    method: 'youtube_iframe_api', // Official YouTube Player API

    // API setup

```

```

api: {
  playerAPI: 'YouTube IFrame Player API',
  documentation: 'https://developers.google.com/youtube/iframe_api_reference',

  features: [
    'playVideo()',
    'pauseVideo()',
    'seekTo(seconds)',
    'setPlaybackRate(rate)',
    'getCurrentTime()',
    'getPlayerState()',
  ]
},

// Embedding
embed: {
  // Embed YouTube player in LinkUp
  iframe: {
    src: 'https://www.youtube.com/embed/[VIDEO_ID]',
    parameters: {
      autoplay: 0,
      controls: 1,
      enablejsapi: 1, // Enable JS API
      origin: 'linkup.app',
      playsinline: 1,
    }
  },

  playerSize: {
    default: '16:9 aspect ratio',
    fullscreen: 'available',
  }
},

// Synchronization
sync: {
  method: 'player_api_events',

  events: {
    onStateChange: 'detect play/pause/buffer',
    onPlaybackRateChange: 'detect speed change',

    // Send to partner
    broadcast: [
      'play',
      'pause',
      'seek',
      'speed_change',
    ]
  }
}

```

```

        'video_change',
    ]
},

polling: {
    interval: 1000,           // Check time every second
    syncThreshold: 2000,     // Resync if >2s drift
}
}
}

```

User Flow - YouTube Watch Together:

1. User finds YouTube video to watch
 - Method 1: Browse YouTube in LinkUp
 - Method 2: Share YouTube link to chat
 - Method 3: Start from YouTube app → Share to LinkUp
2. Starting watch session
 - Video preview shown with title and thumbnail
 - Tap "Watch Together" button
 - Invitation sent to partner
 - Video loads in LinkUp player
3. Partner receives invitation
 - Push notification with video title
 - Tap to join
 - Video loads at same position
 - Sync established
4. Watching together
 - Both see same YouTube video embedded
 - Playback controls available to both (or host only)
 - Picture-in-picture video chat (optional)
 - Send reactions at moments
 - Text chat overlay
 - Queue next video
5. Additional features
 - Create YouTube playlists together
 - Share favorite channels
 - Watch history together
 - Discover new content together

5.2.5 Synchronized Playback

Core Synchronization Engine:

```

interface SyncEngine {
  // Sync algorithm
  algorithm: {
    // Primary sync method
    timestampBased: {
      serverTime: 'use server timestamp as source of truth',
      clientOffset: 'calculate client clock offset',
      adjustPlayback: 'adjust to match expected position',
    },

    // Host-guest model
    hostGuest: {
      host: 'one partner is authoritative source',
      guest: 'other partner follows host',
      switchHost: 'can transfer host role',
    },

    // Conflict resolution
    conflictResolution: {
      simultaneousAction: 'host takes precedence',
      networkDelay: 'compensate for latency',
      bufferDrift: 'periodic correction',
    }
  },

  // Sync events
  events: {
    // Broadcasted to partner
    play: {
      type: 'PLAY',
      timestamp: Date,
      position: number,           // Seconds
      playbackRate: number,
    },

    pause: {
      type: 'PAUSE',
      timestamp: Date,
      position: number,
    },

    seek: {
      type: 'SEEK',
      timestamp: Date,
      from: number,
      to: number,
    },
  },
}

```



```

speedChange: {
  type: 'SPEED',
  timestamp: Date,
  rate: number,           // 0.5, 1, 1.5, 2
},

videoChange: {
  type: 'CHANGE_VIDEO',
  timestamp: Date,
  newVideoId: string,
  startPosition: number,
}
},

// Drift correction
driftCorrection: {
  measurementInterval: 5000,           // Check every 5 seconds

  acceptable Drift: 1000,             // 1 second acceptable
  correctionThreshold: 2000,          // Correct if >2 seconds

  correctionMethod: {
    smallDrift: 'adjust playback speed temporarily',
    largeDrift: 'seek to correct position',

    speedAdjustment: {
      maxDeviation: 0.1,              // ±10% speed
      duration: 'until caught up',
      smoothing: 'gradual speed change',
    }
  }
},

// Network handling
network: {
  // When partner disconnects
  disconnection: {
    pausePlayback: true,              // Auto-pause
    showNotification: 'Partner disconnected',
    waitForReconnect: '60 seconds',
    resumeWhenBack: 'auto-resume when partner returns',
  },

  // Poor connection
  poorConnection: {
    reduceUpdateFrequency: true,
    allowDrift: 'increase acceptable drift',
    notifyPartners: 'warn about sync issues',
  }
}

```

```

    }
  }
}

```

Quality of Sync Metrics:

```

interface SyncQuality {
  // Measure sync quality
  metrics: {
    averageOffset: number,           // Average time difference (ms)
    maxOffset: number,              // Largest drift observed (ms)
    syncCorrections: number,        // Times correction was needed
    eventLatency: number,           // Avg time for events to propagate
  },

  // Quality tiers
  quality: {
    excellent: 'avg offset <500ms, max offset <1s',
    good: 'avg offset <1s, max offset <2s',
    fair: 'avg offset <2s, max offset <3s',
    poor: 'avg offset >2s or max offset >3s',
  },

  // Reporting
  reportToUser: {
    indicator: ' Synced' | ' Syncing...' | ' Sync Lost',
    details: 'available on tap',
    troubleshooting: 'suggest fixes for poor sync',
  }
}

```

5.3 Music Integration

5.3.1 Spotify Integration

Overview: Listen to music together on Spotify with synchronized playback. Share what you're listening to, create collaborative playlists, and discover music together.

Spotify API Integration:

```

interface SpotifyIntegration {
  // Spotify Web API
  api: {
    authentication: 'OAuth 2.0',
    scopes: [
      'user-read-playback-state', // See what's playing
      'user-modify-playback-state', // Control playback
      'user-read-currently-playing', // Current track
      'playlist-read-private', // Access playlists
    ],
  },
}

```

```

        'playlist-modify-public',      // Edit playlists
        'playlist-modify-private',
        'user-library-read',          // Saved tracks
        'user-library-modify',        // Save tracks
    ],

    endpoints: {
        currentlyPlaying: 'GET /v1/me/player/currently-playing',
        playback: 'GET /v1/me/player',
        play: 'PUT /v1/me/player/play',
        pause: 'PUT /v1/me/player/pause',
        seek: 'PUT /v1/me/player/seek',
        queue: 'POST /v1/me/player/queue',
        devices: 'GET /v1/me/player/devices',
    }
},

// Requirements
requirements: {
    spotifyAccount: 'both partners need Spotify accounts',
    spotifyPremium: 'required for playback control',
    activeDevice: 'Spotify must be open on a device',
},

// Limitations
limitations: {
    webPlayback: 'can use Spotify Web Playback SDK',
    apiRateLimits: 'respect Spotify rate limits',
    regionalRestrictions: 'content availability varies',
}
}

Data Schema:

interface SpotifySession {
    id: string;
    coupleId: string;

    // Spotify accounts
    participants: {
        userId: string;
        spotifyId: string;
        spotifyUsername: string;
        spotifyPremium: boolean;
        activeDevice?: {
            id: string;
            name: string;
            type: string;           // 'Computer', 'Smartphone', 'Speaker'
        };
    };
};

```

```

}[];

// Current playback
playback: {
    hostUserId: string, // Who's controlling

    track: {
        spotifyTrackId: string;
        name: string;
        artists: string[];
        album: string;
        albumArt: string;
        duration: number; // Milliseconds
    };

    position: number, // Current position (ms)
    isPlaying: boolean;
    volume: number; // 0-100
    shuffle: boolean;
    repeat: 'off' | 'track' | 'context';

    // Context (playlist, album, etc.)
    context?: {
        type: 'playlist' | 'album' | 'artist' | 'show';
        uri: string;
        name: string;
    };
};

// Listen together mode
mode: 'listen_together' | 'share_track' | 'collaborative_playlist';

// Synchronization
sync: {
    enabled: boolean;
    lastSyncAt: Date;
    syncOffset: number; // Ms difference

    syncEvents: {
        type: 'play' | 'pause' | 'seek' | 'track_change' | 'volume';
        userId: string;
        timestamp: Date;
        data?: any;
    }[];
};

// Queue
queue: {

```

```

    trackId: string;
    addedBy: string;
    timestamp: Date;
  }[];

  // Chat & reactions
  chat: {
    messages: {
      userId: string;
      content: string;
      timestamp: Date;
    }[];

    reactions: {
      emoji: string;
      userId: string;
      trackId: string;           // Which song
      position: number;         // When in song
      timestamp: Date;
    }[];
  };

  // Status
  status: 'active' | 'paused' | 'ended';

  startedAt: Date;
  endedAt?: Date;
  createdAt: Date;
  updatedAt: Date;
}

```

User Flow - Listen Together on Spotify:

1. User wants to listen to music with partner
 - Opens LinkUp
 - Navigate to "Listen Together"
 - Select "Spotify"
 - Connect Spotify account (if first time)
2. Spotify connection (first time)
 - Tap "Connect Spotify"
 - Redirect to Spotify OAuth
 - User logs in to Spotify
 - Grants permissions to LinkUp
 - Redirect back to LinkUp
 - Spotify account connected
3. Check requirements
 - Verify Spotify Premium (required)

- Check if Spotify is open on device
- Select device to play on
- Ready to start session
- 4. Start listen together session
 - Choose what to play:
 - Current song (keep playing what you have)
 - Choose playlist
 - Search for song/artist/album
 - Your likes/library
 - Select "Play Together"
 - Invitation sent to partner
- 5. Partner receives invitation
 - Push notification: " Listen to [Song] together?"
 - Shows album art and song info
 - Tap "Join"
 - Partner's Spotify account connects
- 6. Session starts
 - Both hear same song at same position
 - Playback synchronized
 - Both can see what's playing
 - Interface shows:
 - Album art
 - Song name and artist
 - Playback controls
 - Queue
 - Partner's listening indicator
 - Session is live
- 7. During session
 - Host controls (or both if enabled):
 - Play/Pause
 - Skip forward/backward
 - Seek in song
 - Adjust volume (individual)
 - Add to queue
 - Toggle shuffle/repeat
 - Communication:
 - Voice chat (optional)
 - Text chat
 - React to songs with emojis
 - See what partner is doing
 - Collaborative features:
 - Both add songs to queue

Both save songs to shared playlist
Discover Weekly together
Build playlists collaboratively

8. Ending session

Either partner can end
Or let it run until playlist ends
Summary shown:
Songs played: 12
Duration: 48 minutes
New discoveries: 3 saved
Playlist created (if any)
Spotify continues on each device separately

5.3.2 Shared Playlists

Collaborative Playlist Features:

```
interface SharedPlaylist {
    id: string;
    coupleId: string;

    // Spotify playlist
    spotifyPlaylistId: string;
    spotifyPlaylistUri: string;

    // Playlist details
    name: string;
    description?: string;
    coverImage?: string;
    isPublic: boolean;           // On Spotify

    // Collaboration
    collaborative: boolean;     // Both can edit

    // Tracks
    tracks: {
        spotifyTrackId: string;
        addedBy: string;
        addedAt: Date;

        // Track info (cached)
        name: string;
        artists: string[];
        album: string;
        duration: number;
    }[];
```

```

totalTracks: number;
totalDuration: number;           // Milliseconds

// Activity
activity: {
  type: 'track_added' | 'track_removed' | 'track_reordered' | 'playlist_edited';
  userId: string;
  trackId?: string;
  timestamp: Date;
}[];

// Stats
stats: {
  timesPlayed: number;
  favoriteTrack?: string,
  mostAddedBy?: string,
  lastPlayedAt?: Date,
};

// Settings
settings: {
  autoAddLiked: boolean,         // Auto-add songs liked during sessions
  notifyOnAdd: boolean,          // Notify partner when song added
  allowDuplicates: boolean,
};

createdBy: string;
createdAt: Date;
updatedAt: Date;
}

```

Playlist Management:

```

interface PlaylistManagement {
  // Create shared playlist
  create: {
    name: 'required',
    description: 'optional',
    collaborative: true,         // Always collaborative
    createOnSpotify: true,       // Create actual Spotify playlist

    initialTracks: 'optional',   // Start with some songs

    notification: 'both partners notified',
  },

  // Add tracks
  addTracks: {

```



```

methods: [
  'search_in_linkup',          // Search within LinkUp
  'from_spotify_link',         // Paste Spotify link
  'during_listen_session',     // Add current song
  'from_spotify_app',          // Share to LinkUp from Spotify
],

notifications: {
  realtime: 'partner sees track appear',
  push: 'optional push notification',
}
},

// Playlist recommendations
recommendations: {
  feature: 'suggest songs based on playlist',
  algorithm: 'Spotify recommendation API',

  suggestions: {
    frequency: 'weekly',
    count: 10,
    basedOn: 'current playlist tracks',
    notification: 'New song suggestions for [Playlist]',
  }
}
}

```

5.3.3 Listening Together Feature

Synchronized Spotify Playback:

```

interface ListenTogether {
  // Sync mechanism
  synchronization: {
    // Poll Spotify API for current playback state
    polling: {
      hostInterval: 1000,          // Host checks every 1 second
      guestInterval: 2000,        // Guest checks every 2 seconds

      stateChecks: [
        'is_playing',
        'current_track_id',
        'position_ms',
        'device_id',
      ]
    }
  },
}

```

```

// Host broadcasts state
broadcast: {
  frequency: 1000,           // Every second
  includePosition: true,
  includeTrack: true,
  includePlayState: true,
},

// Guest applies state
apply: {
  trackChange: 'immediately',
  playPause: 'immediately',
  position: 'if drift >3 seconds',

  positionSync: {
    method: 'Spotify seek API',
    compensation: 'add network latency',
  }
},

// Edge cases
edgeCases: {
  hostPausesSpotify: {
    action: 'pause guest playback',
    notification: 'partner paused',
  },

  hostClosesSpotify: {
    action: 'end session gracefully',
    notification: 'partner stopped listening',
  },

  guestNoSpotifyPremium: {
    action: 'show upgrade prompt',
    fallback: 'show what's playing (no control)',
  },

  differentRegions: {
    issue: 'track not available in guest region',
    handling: 'skip track, notify both',
  }
}
}

```

5.3.4 Music Discovery & Sharing

Discovery Features:

```
interface MusicDiscovery {
  // Share what you're listening to
  nowPlaying: {
    shareToChat: {
      automatic: 'optional auto-share',
      manual: 'tap to share current song',

      preview: {
        albumArt: true,
        trackInfo: true,
        spotifyLink: true,
        playPreview: '30-second preview',
      },

      actions: {
        listenTogether: 'start listen session',
        addToPlaylist: 'add to shared playlist',
        like: 'save to your library',
      }
    }
  },

  // Discover together
  discover: {
    // Combined Discover Weekly
    mergedDiscoverWeekly: {
      feature: 'combine both partners Discover Weekly',
      format: 'one playlist with 60 songs',
      updateFrequency: 'weekly',

      listening: {
        listenTogether: 'go through discoveries together',
        save: 'both can save favorites',
        createPlaylist: 'create "Our Discoveries" playlist',
      }
    },

    // Shared taste map
    tasteMap: {
      feature: 'visualize music taste overlap',

      analysis: {
        sharedArtists: 'artists both like',
        sharedGenres: 'genres in common',
      }
    }
  }
}
```

```

        uniqueTastes: 'what each person brings',
        compatibility: 'music compatibility score',
    },

    visualization: 'Venn diagram or chart',
},

// Recommendations
recommendations: {
    forBoth: 'songs both might like',
    forPartner: 'suggest songs partner would like',

    basedOn: [
        'shared playlists',
        'listen together history',
        'individual tastes',
    ]
}
},

// Music challenges
challenges: {
    songOfTheDay: {
        feature: 'each partner shares one song per day',
        goal: 'discover each other's tastes',
        streak: 'track consecutive days',
    },

    genreExploration: {
        feature: 'explore new genre together weekly',
        examples: 'Jazz Week, K-Pop Week, Classical Week',
    }
}
}

```

Chapter 5 Status: Complete

Summary: - Live Streaming: Real-time streaming to partner with camera/screen sharing, <1 second latency - Netflix/Prime Video: Watch party with synchronized playback via browser extension or screen share - TikTok: Share videos and watch together with reaction overlays - YouTube: Embedded player with official API for perfect synchronization - Spotify: Listen together with real-time sync, shared playlists, and music discovery - Complete data schemas, synchronization engines, and user flows for all entertainment features

Next Chapter: Chapter 6: Notifications & Alerts System

Chapter 6: Notifications & Alerts System

Overview

A thoughtful notification system keeps couples connected without being overwhelming. LinkUps notification architecture balances immediacy with respect for each partners time and attention.

This chapter covers: - Push Notification Architecture - Cross-platform notification delivery with intelligent prioritization - Date and Anniversary Alerts - Never miss important relationship milestones - Global Search Feature - Find anything across all your shared content instantly

Design Philosophy: - Smart not spammy - Intelligent batching and priority levels - Context-aware - Different notification rules based on user state - Customizable - Granular control over what triggers notifications - Action-oriented - Notifications enable immediate responses - Privacy-first - Sensitive content handled appropriately in notifications

6.1 Push Notification Architecture

6.1.1 Multi-Platform Notification System

Overview: LinkUp delivers notifications across iOS, Android, and Web platforms with consistent behavior and appearance while respecting platform-specific best practices.

Platform Implementation:

```
interface NotificationPlatforms {
  ios: {
    service: 'APNs', // Apple Push Notification service
    framework: 'UserNotifications',

    features: {
      richNotifications: true, // Media attachments
      categoryActions: true, // Quick actions
      criticalAlerts: false, // Not using for now
      notificationExtensions: true, // Custom UI

      sounds: {
        default: 'system sound',
        custom: 'couples unique sound',
        critical: 'not used',
      },

      badges: {
        appIconBadge: true,
        dynamicCount: 'unread messages',
        clearOnOpen: true,
      },
    },

    permissions: {
```

```

    requestTiming: 'after couple connection',
    provisional: false,           // Explicit permission
    requiredScopes: ['alert', 'badge', 'sound'],
  }
},

android: {
  service: 'FCM',                // Firebase Cloud Messaging

  features: {
    notificationChannels: {
      messages: {
        id: 'messages',
        name: 'Messages',
        importance: 'HIGH',
        sound: 'custom_message_sound',
        vibration: [0, 250, 250, 250],
        showBadge: true,
      },

      calls: {
        id: 'calls',
        name: 'Voice & Video Calls',
        importance: 'URGENT',
        sound: 'call_ringtone',
        vibration: [0, 500, 250, 500],
        fullScreenIntent: true,    // Show over lockscreen
      },

      social: {
        id: 'social',
        name: 'Social Activity',
        importance: 'DEFAULT',
        sound: 'default',
        vibration: [0, 200],
      },

      reminders: {
        id: 'reminders',
        name: 'Dates & Anniversaries',
        importance: 'HIGH',
        sound: 'reminder_sound',
      },

      media: {
        id: 'media',
        name: 'Media & Streaks',
        importance: 'LOW',

```

```

        sound: 'none',
    }
},

    inlineReply: true,           // Reply from notification
    mediaStyle: true,           // Rich media
    messagingStyle: true,       // Conversation UI
},

permissions: {
    requestTiming: 'runtime (Android 13+)',
    degradeGracefully: 'work without permission',
}
},

web: {
    service: 'Web Push API',
    protocol: 'VAPID',

    features: {
        persistentNotifications: true,
        actions: true,           // Action buttons
        icon: true,
        badge: true,
        image: true,

        requireInteraction: 'for high priority',
        tag: 'to replace/update notifications',
    },

    permissions: {
        requestTiming: 'explicit user action',
        prompt: 'native browser prompt',
    },

    limitations: {
        browserSupport: 'modern browsers only',
        requireHttps: true,
        serviceworkerRequired: true,
    }
}
}

```

Data Schema:

```

interface PushNotification {
    id: string;

    // Recipient

```

```

userId: string;
coupleId: string;

// Notification content
type: NotificationType;
priority: 'urgent' | 'high' | 'normal' | 'low';

content: {
  title: string;
  body: string;
  subtitle?: string; // iOS only

  // Rich content
  imageUrl?: string;
  thumbnailUrl?: string;
  avatarUrl?: string;

  // Sound
  sound?: string;
  soundVolume?: number;

  // Visual
  badge?: number; // Badge count
  color?: string; // Android accent color
  icon?: string;
};

// Action data
data: {
  action: string; // Deep link action
  screen?: string; // Screen to open
  params?: Record<string, any>; // Parameters for action

  // Related IDs
  messageId?: string;
  callId?: string;
  postId?: string;
  eventId?: string;
};

// Quick actions
actions?: {
  id: string;
  title: string;
  icon?: string;
  type: 'reply' | 'accept' | 'decline' | 'view' | 'custom';

  // For reply actions

```



```

    textInput?: {
      placeholder: string;
      maxLength: number;
    };
  }[];

  // Delivery configuration
  delivery: {
    platforms: ('ios' | 'android' | 'web')[];

    // Timing
    sendAt?: Date; // Scheduled send
    expiresAt?: Date; // Expire if not delivered

    // Grouping
    groupId?: string; // Group related notifications
    collapseId?: string; // Replace previous notification

    // Behavior
    silent: boolean; // Silent notification
    requireInteraction: boolean; // Don't auto-dismiss
  };

  // Status tracking
  status: {
    state: 'queued' | 'sent' | 'delivered' | 'failed' | 'clicked' | 'dismissed' | 'expired';

    sentAt?: Date;
    deliveredAt?: Date;
    clickedAt?: Date;
    dismissedAt?: Date;

    failureReason?: string;

    // Per-platform status
    platformStatus: {
      platform: 'ios' | 'android' | 'web';
      deviceToken: string;
      status: string;
      timestamp: Date;
    }[];
  };

  // User preferences applied
  preferences: {
    userEnabled: boolean;
    categoryEnabled: boolean;
    quietHoursActive: boolean;
  };

```

```

        batched: boolean;
    };

    createdAt: Date;
    updatedAt: Date;
}

type NotificationType =
    // Messages
    | 'message_received'
    | 'message_thread_reply'
    | 'message_reaction'
    | 'message_highlighted'

    // Calls
    | 'incoming_call_voice'
    | 'incoming_call_video'
    | 'missed_call'
    | 'call_ended'

    // Media
    | 'photo_received'
    | 'video_received'
    | 'voice_message_received'
    | 'photo_streak_reminder'
    | 'photo_streak_danger'

    // Social
    | 'friend_invitation'
    | 'friend_accepted'
    | 'circle_invitation'
    | 'circle_post'
    | 'circle_event'
    | 'poll_response'

    // Creative
    | 'scribble_invitation'
    | 'painting_shared'
    | 'custom_emoji_used'
    | 'soundboard_sound'

    // Entertainment
    | 'stream_started'
    | 'watch_party_invitation'
    | 'listen_together_invitation'
    | 'song_shared'

    // Achievements

```

```

| 'achievement_unlocked'
| 'milestone_reached'
| 'hall_of_fame_entry'

// Dates & Reminders
| 'anniversary_reminder'
| 'birthday_reminder'
| 'date_reminder'
| 'custom_reminder'

// System
| 'partner_online'
| 'partner_typing'
| 'app_update'
| 'system_message';

```

6.1.2 Notification Priority & Delivery Logic

Priority System:

```

interface NotificationPriority {
    // Priority levels
    urgent: {
        types: [
            'incoming_call_voice',
            'incoming_call_video',
        ],

        delivery: {
            immediate: true,
            bypassQuietHours: true,
            bypassDoNotDisturb: false,    // Respect system DND
            sound: 'always',
            vibration: 'strong',
            fullScreen: true,              // Android fullscreen intent
            headsUp: true,                 // Show over other apps
        },

        ttl: 60,                          // Expire after 60 seconds
    },

    high: {
        types: [
            'message_received',
            'message_highlighted',
            'anniversary_reminder',
            'birthday_reminder',

```

```

        'photo_streak_danger',
        'stream_started',
    ],

    delivery: {
        immediate: true,
        bypassQuietHours: false,
        sound: 'enabled_by_default',
        vibration: 'normal',
        headsUp: true,
    },

    ttl: 3600, // Expire after 1 hour
},

normal: {
    types: [
        'message_reaction',
        'friend_invitation',
        'circle_post',
        'photo_received',
        'achievement_unlocked',
    ],

    delivery: {
        immediate: true,
        bypassQuietHours: false,
        sound: 'enabled_by_default',
        vibration: 'light',
        headsUp: false,
    },

    ttl: 86400, // Expire after 24 hours
},

low: {
    types: [
        'partner_online',
        'custom_emoji_used',
        'circle_event',
        'photo_streak_reminder',
    ],

    delivery: {
        immediate: false,
        batching: {
            enabled: true,
            interval: 300, // Batch every 5 minutes
        }
    }
}

```

```

        batchSize: 10,
    },
    bypassQuietHours: false,
    sound: 'disabled_by_default',
    vibration: 'none',
    headsUp: false,
},

ttl: 86400,
}
}

```

Smart Delivery Logic:

```

interface SmartDelivery {
    // Context-aware delivery
    contextual: {
        // User is in app
        appActive: {
            behavior: 'show_in_app_notification',
            skipPush: true,           // Don't send push
            exception: 'urgent_priority', // Send urgent anyway
        },

        // User viewing relevant screen
        onRelevantScreen: {
            behavior: 'update_ui_only',
            skipPush: true,
            skipInApp: true,
            example: 'viewing chat → new message appears inline',
        },

        // User is idle
        appInBackground: {
            behavior: 'send_push_notification',
            timing: 'immediate',
        },

        // User hasn't used app recently
        dormant: {
            threshold: '7 days inactive',
            behavior: 're-engagement notifications',
            frequency: 'limited',
            types: ['important_only'],
        }
    },

    // Quiet hours
    quietHours: {

```

```

enabled: boolean;
schedule: {
  start: string;           // "22:00"
  end: string;             // "08:00"
  timezone: string;
};

behavior: {
  default: 'delay until end of quiet hours',
  urgent: 'deliver anyway',
  option: 'silent delivery (no sound/vibration)',
};

exceptions: {
  allowUrgent: true,
  allowFromPartner: boolean,    // User configurable
}
},

// Notification batching
batching: {
  enabled: boolean;

  strategy: {
    timeWindow: 300,           // 5 minutes
    maxNotifications: 10,

    grouping: {
      byType: true,            // Group same type
      bySource: true,          // Group from same person

      example: 'You have 3 new messages from Alex',
    },

    collapse: {
      sameType: true,
      updateExisting: true,
      showCount: true,
    }
  },

  exclusions: {
    neverBatch: ['urgent', 'high'],
    batchOnlyWhen: 'user is busy',
  }
},

// Delivery optimization

```

```

optimization: {
  // Deduplicate
  deduplicate: {
    enabled: true,
    window: 60, // 60 seconds
    criteria: 'same type + same source + same target',
  },

  // Rate limiting
  rateLimit: {
    enabled: true,

    perType: {
      maxPerHour: 20,
      maxPerDay: 100,
    },

    overall: {
      maxPerHour: 50,
      maxPerDay: 200,
    },

    backoff: 'exponential for repeated notifications',
  },

  // Smart scheduling
  scheduling: {
    considerUserActivity: true,
    avoidDisruptiveTimes: true,

    intelligence: {
      learnPatterns: 'when user typically responds',
      optimizeDelivery: 'send when likely to engage',
    }
  }
}
}

```

6.1.3 User Notification Preferences

Granular Notification Controls:

```

interface NotificationPreferences {
  userId: string;
  coupleId: string;

  // Global settings

```

```

global: {
  enabled: boolean;           // Master switch
  pushEnabled: boolean;
  inAppEnabled: boolean;
  emailEnabled: boolean;     // For important updates

  // Global sound
  soundEnabled: boolean;
  vibrationEnabled: boolean;
  badgeEnabled: boolean;
};

// Quiet hours
quietHours: {
  enabled: boolean;
  schedule: {
    monday: { start: '22:00', end: '08:00' };
    tuesday: { start: '22:00', end: '08:00' };
    wednesday: { start: '22:00', end: '08:00' };
    thursday: { start: '22:00', end: '08:00' };
    friday: { start: '23:00', end: '09:00' }; // Sleep in
    saturday: { start: '23:00', end: '09:00' };
    sunday: { start: '22:00', end: '08:00' };
  };
  timezone: string;

  allowUrgent: boolean;      // Urgent notifications bypass
  allowFromPartner: boolean; // Partner messages bypass
};

// Per-category settings
categories: {
  messages: {
    enabled: boolean;
    sound: boolean;
    vibration: boolean;
    showPreview: boolean;    // Show message content

    // Sub-categories
    directMessages: { enabled: boolean };
    threadReplies: { enabled: boolean };
    reactions: { enabled: boolean };
    highlights: { enabled: boolean };
  };

  calls: {
    enabled: boolean;
    sound: boolean;
  };
};

```



```

    vibration: boolean;

    voiceCalls: { enabled: boolean };
    videoCalls: { enabled: boolean };
    missedCalls: { enabled: boolean };
};

media: {
    enabled: boolean;
    sound: boolean;

    photos: { enabled: boolean };
    videos: { enabled: boolean };
    voiceMessages: { enabled: boolean };
    streaks: { enabled: boolean };
};

social: {
    enabled: boolean;
    sound: boolean;

    friendActivity: { enabled: boolean };
    circleActivity: { enabled: boolean };
    hallOfFame: { enabled: boolean };
};

creative: {
    enabled: boolean;
    sound: boolean;

    scribbleInvites: { enabled: boolean };
    paintingShared: { enabled: boolean };
    customEmojis: { enabled: boolean };
};

entertainment: {
    enabled: boolean;
    sound: boolean;

    streams: { enabled: boolean };
    watchParties: { enabled: boolean };
    musicSessions: { enabled: boolean };
};

datesAndReminders: {
    enabled: boolean;
    sound: boolean;
};

```

```

    anniversaries: { enabled: boolean };
    birthdays: { enabled: boolean };
    customReminders: { enabled: boolean };

    advanceNotice: {
      oneDayBefore: boolean;
      oneWeekBefore: boolean;
      oneMonthBefore: boolean;
    };
  };

  achievements: {
    enabled: boolean;
    sound: boolean;

    milestones: { enabled: boolean };
    badges: { enabled: boolean };
  };
};

// Partner-specific
partnerSettings: {
  alwaysNotify: boolean;           // Never miss partner notifications
  specialSound: string;           // Unique sound for partner

  onlineStatus: {
    notifyWhenOnline: boolean;
    notifyWhenOffline: boolean;
  };
};

// Device-specific
devices: {
  deviceId: string;
  deviceType: 'ios' | 'android' | 'web';
  deviceName: string;
  token: string;

  enabled: boolean;
  primary: boolean;               // Primary device for certain notifications

  lastActiveAt: Date;
  createdAt: Date;
}[];

updatedAt: Date;
}

```

User Flow - Configuring Notifications:

1. User wants to customize notifications
 - Opens LinkUp
 - Navigate to Settings → Notifications
 - Notification preferences screen
2. Notification preferences screen
 - Master toggle: "Enable Notifications"
 - Quick presets:
 - "All Notifications" - Everything enabled
 - "Important Only" - Urgent + high priority
 - "Partner Only" - Only partner messages
 - "Custom" - Granular control

If "Custom" selected, shows categories

3. Category configuration
 - Messages
 - Toggle: Enable/Disable
 - Sound: On/Off
 - Show message preview: On/Off
 - Sub-toggles:
 - Direct messages
 - Thread replies
 - Reactions
 - Highlighted messages

Calls

Toggle: Enable/Disable
Voice / Video / Missed calls

Media & Streaks
Social Activity
Creative Features
Entertainment
Dates & Reminders
Achievements

4. Quiet hours setup
 - Toggle: "Enable Quiet Hours"
 - Set schedule:
 - Days of week
 - Start time (e.g., 10:00 PM)
 - End time (e.g., 8:00 AM)
 - Exceptions:
 - "Allow urgent notifications" (calls)

"Allow messages from partner"

Save

5. Partner-specific settings

"Never miss partner's messages"

Custom notification sound for partner

Online/offline status notifications

6. Save and apply

Tap "Save Preferences"

Settings synced across devices

Test notification: "Send test notification"

Confirmation: "Preferences updated"

6.1.4 In-App Notifications

In-App Notification System:

```
interface InAppNotification {
  id: string;

  // Display
  type: 'banner' | 'toast' | 'modal' | 'inline';

  content: {
    title: string;
    message: string;
    icon?: string;
    avatar?: string;
    image?: string;
  };

  // Behavior
  behavior: {
    duration: number; // Auto-dismiss after ms (0 = manual)
    position: 'top' | 'bottom' | 'center';

    dismissible: boolean;
    persistent: boolean; // Stay until action taken

    animation: 'slide' | 'fade' | 'bounce';
  };

  // Actions
  actions: {
    primary?: {
```

```

    label: string;
    action: string;
    params?: any;
};

secondary?: {
    label: string;
    action: string;
};

dismiss: {
    label: string;                // "Dismiss", "Later", "OK"
    action?: string;
};

};

// Interaction tracking
tracking: {
    shown: boolean;
    shownAt?: Date;
    clicked: boolean;
    clickedAt?: Date;
    dismissed: boolean;
    dismissedAt?: Date;
};

priority: 'high' | 'normal' | 'low';
expiresAt?: Date;

createdAt: Date;
}

```

Banner Notification Display:

```

interface BannerNotification {
    // Display configuration
    display: {
        position: 'top',
        offset: 'below_status_bar',

        appearance: {
            background: 'blur_effect',
            textColor: 'dynamic (light/dark mode)',
            accentColor: 'brand_purple',

            avatar: {
                show: true,
                size: 40,
                position: 'left',
            }
        }
    }
}

```

```

    },

    icon: {
      show: true,
      size: 24,
      position: 'right',
    }
  },

  animation: {
    enter: 'slide_down_with_bounce',
    exit: 'slide_up_fade',
    duration: 300,
  },

  gestures: {
    swipeUp: 'dismiss',
    swipeDown: 'expand (if expandable)',
    tap: 'trigger_action',
    longPress: 'show_options',
  }
},

// Auto-dismiss
autoDismiss: {
  enabled: true,
  delay: 4000, // 4 seconds
  pauseOnHover: true, // Web
  pauseOnTouch: true, // Mobile
},

// Queuing
queue: {
  maxVisible: 1, // Show one at a time
  stackBehavior: 'queue', // Queue subsequent notifications
  priority: 'respect priority levels',
}
}

```

6.2 Date and Anniversary Alerts

6.2.1 Important Date Management

Overview: Never forget important relationship milestones. LinkUp tracks anniversaries, birthdays, and custom dates with intelligent reminders.

Data Schema:

```

interface ImportantDate {
  id: string;
  coupleId: string;

  // Date details
  type: 'anniversary' | 'birthday' | 'custom';

  name: string; // "Our Anniversary", "First Date", etc.
  description?: string;

  date: {
    month: number; // 1-12
    day: number; // 1-31
    year?: number; // Optional (for recurring dates)

    recurring: boolean; // Repeat every year

    // For "days since" calculations
    originalDate?: Date; // The actual first occurrence
  };

  // Category-specific data
  metadata: {
    // For anniversaries
    anniversaryOf?: 'relationship' | 'marriage' | 'first_date' | 'custom';

    // For birthdays
    birthdayOf?: string; // userId

    // Custom date data
    customCategory?: string;
    customIcon?: string;
  };

  // Reminder configuration
  reminders: {
    enabled: boolean;

    schedule: {
      onDay: boolean; // Morning of the day
      oneDayBefore: boolean;
      oneWeekBefore: boolean;
      twoWeeksBefore: boolean;
      oneMonthBefore: boolean;
    };

    timing: {
      time: string; // "09:00" - when to send
    };
  };
}

```

```

        timezone: string;
    };

    customReminders: {
        id: string;
        daysBefore: number;
        time: string;
    }[];
};

// Celebration tracking
celebrations: {
    year: number;
    celebratedAt?: Date;

    activities: {
        gift: boolean;
        card: boolean;
        date: boolean;
        other?: string;
    };

    memories: {
        photos: string[];
        notes: string;
    };
}[];

// Status
isActive: boolean;

createdBy: string;
createdAt: Date;
updatedAt: Date;
}

```

Pre-configured Important Dates:

```

interface DefaultDates {
    // Automatically detected
    automatic: {
        relationshipAnniversary: {
            detectedFrom: 'couple connection date',
            name: 'Our Anniversary',
            type: 'anniversary',
            recurring: true,

            reminders: {
                oneMonthBefore: true,
            }
        }
    }
}

```



```

        oneWeekBefore: true,
        oneDayBefore: true,
        onDay: true,
    }
},

birthdays: {
    detectedFrom: 'user profile',
    names: ["[Partner]'s Birthday", "My Birthday"],
    type: 'birthday',
    recurring: true,

    reminders: {
        twoWeeksBefore: true,
        oneWeekBefore: true,
        oneDayBefore: true,
        onDay: true,
    }
}
},

// User-added
custom: {
    examples: [
        'First Kiss',
        'Engagement',
        'First "I Love You"',
        'Moved In Together',
        'Got Our Pet',
        'First Trip Together',
    ],

    flexibility: 'user can add any date',
    categories: 'user-defined',
}
}

```

User Flow - Setting Up Important Dates:

1. Initial setup (during onboarding)
 - "Let's set up your important dates"
 - Relationship start date:
 - Auto-detected: "You connected on [Date]"
 - Or manual input
 - Confirm: "This is our anniversary"
 - Birthdays:
 - "[Partner]'s birthday"
 - "Your birthday" (from profile)

Both added automatically

2. Adding custom date

Navigate to Dates & Reminders

Tap "+ Add Important Date"

Choose type:

Anniversary

Birthday (someone else)

Custom Event

For custom event:

Name: "First Trip to Paris"

Date: June 15, 2024

Recurring: Yes/No

Icon: Choose from set

Save

3. Configure reminders

For each date, tap to configure

Reminder schedule:

One month before

One week before

One day before

On the day (9:00 AM)

Custom reminder:

"+ Add custom reminder"

"3 days before at 7:00 PM"

Add

Save reminder settings

4. Viewing upcoming dates

Navigate to Dates & Reminders

Calendar view shows:

Upcoming dates highlighted

Days until each date

Tap date for details

List view shows:

Sorted by next occurrence

"Our Anniversary - in 23 days"

Quick actions: Edit, Remind, Celebrate

6.2.2 Anniversary Reminder System

Smart Anniversary Reminders:

```
interface AnniversaryReminder {
    // Reminder configuration
    reminder: {
        dateId: string;
        anniversaryNumber: number;           // 1st, 2nd, 3rd, etc.

        // Calculated dates
        anniversaryDate: Date;
        reminderDates: {
            oneMonthBefore: Date;
            twoWeeksBefore: Date;
            oneWeekBefore: Date;
            oneDayBefore: Date;
            onTheDay: Date;
        };
    };

    // Notification content
    notifications: {
        oneMonthBefore: {
            title: " Anniversary coming up!",
            body: "Your [Xth] anniversary is in one month (June 15)",
            actions: [
                { id: 'plan', label: 'Plan something special' },
                { id: 'remind_later', label: 'Remind me later' },
            ]
        },

        oneWeekBefore: {
            title: " Anniversary next week!",
            body: "Your [Xth] anniversary is in 7 days",
            actions: [
                { id: 'gift_ideas', label: 'Gift ideas' },
                { id: 'reserve_restaurant', label: 'Find restaurants' },
            ]
        },

        oneDayBefore: {
            title: " Tomorrow is your anniversary!",
            body: "[X] years together! Make it special",
            actions: [
                { id: 'message_partner', label: 'Send sweet message' },
                { id: 'view_memories', label: 'View memories' },
            ]
        }
    }
}
```

```

    },

    onTheDay: {
        title: " Happy Anniversary!",
        body: "[X] years together today!",
        timing: '8:00 AM',

        special: {
            confetti: true,                // In-app celebration
            specialAnimation: true,
            soundEffect: 'celebration',

            features: {
                memorySlideshow: 'photos from past year',
                achievementBadge: '[X]-Year Badge',
                suggestCelebration: 'ideas for today',
            }
        },

        actions: [
            { id: 'send_love_message', label: ' Message partner' },
            { id: 'view_timeline', label: 'Our journey' },
            { id: 'create_memory', label: 'Create memory' },
        ]
    }
};

// Milestone anniversaries (special treatment)
milestones: {
    1: { special: true, badge: 'First Year!', gift: 'paper' },
    5: { special: true, badge: '5 Years!', gift: 'wood' },
    10: { special: true, badge: '1 Decade!', gift: 'tin' },
    25: { special: true, badge: 'Quarter Century!', gift: 'silver' },
    50: { special: true, badge: '50 Years!', gift: 'gold' },
};

// Integration with other features
integrations: {
    hallOfFame: 'create achievement',
    mediaGallery: 'create anniversary album',
    scribble: 'send anniversary card',
    customEmojis: 'unlock anniversary emojis',
}
}

Celebration Mode:

interface CelebrationMode {
    // Triggered on anniversary day

```

```

activation: {
  trigger: 'midnight on anniversary date',
  duration: '24 hours',

  visualEffects: {
    confetti: 'periodic confetti animations',
    specialTheme: 'hearts and sparkles',
    customColors: 'romantic color scheme',
  }
},

// Special features unlocked
features: {
  memorySlideshow: {
    content: 'photos from past year',
    music: 'romantic background music',
    autoPlay: 'when opening app',
  },

  timeline: {
    title: 'Our [X] Years Together',
    content: 'highlights from relationship',
    format: 'interactive timeline',
  },

  suggestionCards: [
    {
      title: 'Send a Love Note',
      action: 'open scribble with love template',
    },
    {
      title: 'Recreate First Date',
      action: 'show first date details',
    },
    {
      title: 'Share Favorite Memory',
      action: 'prompt to write memory',
    },
  ],

  achievementUnlock: {
    badge: '[X]-Year Anniversary',
    points: 'based on years',
    shareToHallOfFame: 'optional',
  }
},

// Partner coordination

```

```

partnerFeatures: {
  synchronizedGreeting: {
    timing: 'both see greeting at same time',
    content: 'Happy [X] Years Together!',
    action: 'tap to send instant message',
  },

  coupleTask: {
    challenge: 'share favorite memory from this year',
    completion: 'both partners participate',
    reward: 'unlock special anniversary album',
  }
}
}
}

```

6.2.3 Birthday Reminders

Birthday Reminder Configuration:

```

interface BirthdayReminder {
  // Who
  personId: string;
  personName: string;
  relationship: 'partner' | 'partner_family' | 'partner_friend' | 'my_family' | 'my_friend';

  // Birthday details
  birthday: {
    month: number;
    day: number;
    year?: number; // Optional (calculate age if provided)

    nextOccurrence: Date;
    daysUntil: number;
    age?: number; // If birth year known
  };

  // Reminder schedule
  reminders: {
    twoWeeksBefore: {
      enabled: boolean;
      title: "[Name]'s birthday in 2 weeks",
      body: "[Date] - Start thinking about a gift!",
      actions: [
        { id: 'gift_ideas', label: 'Gift ideas' },
        { id: 'set_reminder', label: 'Remind me closer' },
      ]
    },
  },
}

```

```

oneWeekBefore: {
  enabled: boolean;
  title: " [Name]'s birthday next week!",
  body: "[Date] - Time to get that gift",
  actions: [
    { id: 'shopping', label: 'Shop now' },
    { id: 'card', label: 'Send a card' },
  ]
},

oneDayBefore: {
  enabled: boolean;
  title: " [Name]'s birthday tomorrow!",
  body: "Don't forget to wish them!",
  actions: [
    { id: 'write_message', label: 'Write message now' },
    { id: 'schedule_call', label: 'Schedule call' },
  ]
},

onTheDay: {
  enabled: boolean;
  timing: '8:00 AM',
  title: " It's [Name]'s birthday!",
  body: "[Age]th birthday - Send your wishes!",

  special: {
    confetti: true,
    animation: 'birthday_cake',
  },

  actions: [
    { id: 'call', label: ' Call now' },
    { id: 'message', label: ' Send message' },
    { id: 'video_message', label: ' Video message' },
  ]
}
};

// Gift tracking
giftTracking: {
  enabled: boolean;

  ideas: {
    idea: string;
    link?: string;
    price?: number;
  }
};

```

```

        addedAt: Date;
    }[];

    purchased: boolean;
    purchasedItem?: string;
    purchasedAt?: Date;
};

// Past birthdays
history: {
    year: number;
    celebrated: boolean;
    gift?: string;
    notes?: string;
}[];
}

```

6.3 Global Search Feature

6.3.1 Comprehensive Search Architecture

Overview: Find anything across all of LinkUp instantly - messages, media, dates, achievements, shared content, and more.

Data Schema:

```

interface SearchIndex {
    // Indexed content types
    indexedTypes: {
        messages: {
            content: 'message text',
            metadata: ['sender', 'timestamp', 'thread_id', 'highlighted'],
            searchable: ['content', 'sender_name'],
        },

        media: {
            content: 'file metadata',
            metadata: ['type', 'timestamp', 'album', 'location', 'tags'],
            searchable: ['filename', 'tags', 'location', 'date'],
        },

        dates: {
            content: 'important dates',
            metadata: ['type', 'date', 'recurring'],
            searchable: ['name', 'description', 'date'],
        },

        achievements: {

```



```

        content: 'unlocked achievements',
        metadata: ['category', 'unlock_date', 'points'],
        searchable: ['title', 'description', 'category'],
    },

    circleContent: {
        content: 'posts, events in circles',
        metadata: ['circle_id', 'author', 'timestamp'],
        searchable: ['content', 'author_name', 'circle_name'],
    },

    creativeContent: {
        content: 'scribbles, paintings',
        metadata: ['type', 'created_at', 'canvas_data'],
        searchable: ['tags', 'date'],
    },

    spotifyContent: {
        content: 'shared songs, playlists',
        metadata: ['track_name', 'artist', 'album', 'shared_at'],
        searchable: ['track_name', 'artist', 'album', 'playlist_name'],
    },

    people: {
        content: 'single friends, circle members',
        metadata: ['relationship_type', 'added_at'],
        searchable: ['name', 'username'],
    }
};

// Indexing configuration
indexing: {
    realtime: true, // Index as content is created

    batchUpdate: {
        enabled: true,
        interval: 300, // 5 minutes
    },

    fullReindex: {
        frequency: 'weekly',
        timing: 'off-peak hours',
    }
}
}

interface SearchQuery {
    id: string;

```

```

userId: string;
coupleId: string;

// Query
query: string; // Search text

// Filters
filters: {
    // Content type
    types?: ('messages' | 'media' | 'dates' | 'achievements' | 'circles' | 'creative' | 'music' | 'video' | 'voice' | 'file')[];

    // Time range
    dateRange?: {
        from?: Date;
        to?: Date;

        // Or quick filters
        preset?: 'today' | 'this_week' | 'this_month' | 'this_year' | 'all_time';
    };

    // Media specific
    mediaType?: ('photo' | 'video' | 'voice' | 'file')[];

    // People filter
    from?: string; // userId

    // Message specific
    highlighted?: boolean;
    inThreads?: boolean;

    // Location filter
    location?: string;
};

// Sorting
sort: {
    by: 'relevance' | 'date' | 'type';
    order: 'asc' | 'desc';
};

// Pagination
pagination: {
    page: number;
    perPage: number; // Default 20
};

// Results
results: SearchResult[];

```

```

totalResults: number;

// Performance
executionTime: number;           // Milliseconds

searchedAt: Date;
}

interface SearchResult {
  id: string;
  type: 'message' | 'media' | 'date' | 'achievement' | 'circle_post' | 'creative' | 'music' |

  // Result content
  title: string;
  snippet: string;               // Preview with highlighted matches
  thumbnail?: string;

  // Original item reference
  itemId: string;
  itemData: any;                // Full item data

  // Match information
  matches: {
    field: string;
    matchedText: string;
    context: string;            // Surrounding context
  }[];

  // Relevance
  score: number;                // Relevance score

  // Metadata for display
  metadata: {
    timestamp: Date;
    author?: string;
    location?: string;
    tags?: string[];
  };

  // Actions
  actions: {
    primary: {
      label: string;
      action: string;           // Deep link
    };

    secondary?: {
      label: string;

```

```

        action: string;
    }[];
};
}

```

Search Algorithm:

```

interface SearchAlgorithm {
    // Full-text search
    fullText: {
        engine: 'Elasticsearch' | 'Algolia' | 'Custom',

        features: {
            fuzzyMatching: true,           // Handle typos
            synonyms: true,               // "picture" matches "photo"
            stemming: true,               // "running" matches "run"
            partialMatching: true,        // "anniv" matches "anniversary"
        },

        ranking: {
            // Relevance factors
            factors: [
                { name: 'exact_match', weight: 10 },
                { name: 'starts_with', weight: 8 },
                { name: 'contains', weight: 5 },
                { name: 'fuzzy_match', weight: 3 },
            ],

            // Recency boost
            recencyBoost: {
                enabled: true,
                decay: 'exponential',
                halfLife: '30 days',
            },

            // Type boosting
            typeBoosts: {
                highlighted_messages: 2.0,
                achievements: 1.5,
                important_dates: 1.5,
                recent_media: 1.2,
            }
        }
    },

    // Semantic search (optional advanced feature)
    semantic: {
        enabled: false,                  // Future enhancement
    }
}

```

```

    features: {
      conceptMatching: 'search "vacation" finds "trip", "travel"',
      contextUnderstanding: 'understand intent',
      naturalLanguage: 'answer questions',
    }
  },

  // Search suggestions
  suggestions: {
    autocomplete: {
      enabled: true,
      minChars: 2,
      maxSuggestions: 5,

      sources: [
        'previous_searches',
        'popular_terms',
        'indexed_content',
      ]
    },

    didYouMean: {
      enabled: true,
      threshold: 'suggest if query has typos',
    },

    relatedSearches: {
      enabled: true,
      basedOn: 'common search patterns',
    }
  }
}

```

6.3.2 Search User Interface

User Flow - Using Global Search:

1. User wants to find something
 - Tap search icon (magnifying glass) in header
 - Or pull down on any screen to reveal search
 - Search input appears
2. Entering search query
 - Type search term: "paris"
 - Autocomplete suggestions appear:
 - "paris trip"
 - "paris photos"

"first date paris"

Select suggestion or continue typing

Tap search or press enter

3. Search results

Results appear grouped by type:

Photos (12)

Photo from Paris - June 2025

Eiffel Tower sunset

[View all photos]

Messages (5)

"Can't wait for Paris!" - May 2025

"Paris was amazing!" - July 2025

[View all messages]

Dates (1)

First Trip to Paris - June 15, 2025

[View date details]

Music (2)

"La Vie en Rose" - shared playlist

[View all music]

Total: 20 results

4. Filtering results

Tap "Filters" button

Filter options:

Type: All, Messages, Media, Dates

Date: "This year", "All time", Custom

Media type: Photos, Videos, Voice

From: Me, Partner, Both

Apply filters

Results update instantly

5. Viewing result

Tap on result

Opens item in context:

Message → Opens chat at that message

Photo → Opens media viewer

Date → Opens date details

Achievement → Shows achievement

Back button returns to search results

6. Search history

Tap search without typing

Shows recent searches:

"paris" - Today

"anniversary photos" - Yesterday

"birthday" - Last week

Tap to search again or clear history

Search UI Components:

```
interface SearchUI {  
  // Search bar  
  searchBar: {  
    placeholder: {  
      default: 'Search messages, photos, dates...',  
      focused: 'Search',  
    },  
  
    voiceSearch: {  
      enabled: true,  
      icon: 'microphone',  
      action: 'speech-to-text',  
    },  
  
    clearButton: {  
      showWhen: 'text entered',  
      action: 'clear search',  
    }  
  },  
  
  // Autocomplete dropdown  
  autocomplete: {  
    maxSuggestions: 5,  
  
    display: {  
      icon: 'based on suggestion type',  
      mainText: 'suggestion',  
      subText: 'context or category',  
  
      highlight: 'matching characters',  
    },  
  
    keyboardNavigation: true,  
  },  
  
  // Results display  
  results: {
```

```

grouping: {
  byType: true,
  collapsible: true,
  showCount: true,

  order: [
    'messages',
    'media',
    'dates',
    'achievements',
    'circles',
    'creative',
    'music',
    'people',
  ]
},

resultCard: {
  layout: {
    thumbnail: 'left (if applicable)',
    title: 'prominent',
    snippet: 'highlighted matches',
    metadata: 'timestamp, author, etc.',
  },

  actions: {
    tap: 'open item',
    longPress: 'show options',

    options: [
      'Open',
      'Share',
      'Add to favorites',
      'Delete',
    ]
  }
},

emptyState: {
  noResults: {
    icon: 'magnifying_glass',
    title: 'No results found',
    subtitle: 'Try different keywords or filters',

    suggestions: [
      'Check your spelling',
      'Use different keywords',
      'Remove some filters',
    ]
  }
}

```



```

    ]
  }
}
},

// Filters panel
filtersPanel: {
  trigger: 'filters button',

  position: 'slide_up_modal',

  filters: [
    {
      name: 'Content Type',
      type: 'multi_select',
      options: ['All', 'Messages', 'Media', 'Dates', 'etc.']
    },
    {
      name: 'Date Range',
      type: 'date_picker',
      presets: ['Today', 'This Week', 'This Month', 'This Year', 'All Time']
    },
    {
      name: 'Media Type',
      type: 'multi_select',
      options: ['Photos', 'Videos', 'Voice Messages', 'Files']
    },
    {
      name: 'From',
      type: 'select',
      options: ['Both', 'Me', '[Partner Name]']
    }
  ],

  actions: {
    apply: 'apply filters and close',
    reset: 'clear all filters',
    cancel: 'close without applying',
  }
}
}
}

```

6.3.3 Advanced Search Features

Smart Search Capabilities:

```

interface AdvancedSearch {
  // Natural language queries
  naturalLanguage: {
    enabled: true,

    examples: {
      temporal: {
        query: "photos from last month",
        understanding: {
          type: 'media',
          mediaType: 'photo',
          dateRange: 'last 30 days',
        }
      },

      specific: {
        query: "messages about dinner",
        understanding: {
          type: 'message',
          keywords: ['dinner'],
        }
      },

      combined: {
        query: "achievements this year",
        understanding: {
          type: 'achievement',
          dateRange: 'this year',
        }
      }
    }
  },

  // Search shortcuts
  shortcuts: {
    enabled: true,

    operators: {
      // Type operators
      'type:message': 'search only messages',
      'type:photo': 'search only photos',
      'type:date': 'search only dates',

      // Date operators
      'today:': 'content from today',
      'this_week:': 'content from this week',
      'year:2025': 'content from 2025',
    }
  }
}

```

```

    // People operators
    'from:partner': 'content from partner',
    'from:me': 'my content',

    // Special operators
    'starred:': 'highlighted/favorited items',
    'has:attachment': 'items with attachments',
  },

  examples: [
    'type:photo paris',           // Photos containing "paris"
    'from:partner year:2025',     // Partner's content from 2025
    'starred: anniversary',       // Highlighted items about anniversary
  ]
},

// Saved searches
savedSearches: {
  enabled: true,

  save: {
    action: 'tap save icon in search',
    naming: 'user provides name',
    storage: 'saved to user preferences',
  },

  access: {
    location: 'search screen dropdown',
    quickAccess: 'tap to instantly run',
  },

  examples: [
    { name: 'Our Paris Trip', query: 'paris type:photo year:2025' },
    { name: 'Anniversary Memories', query: 'anniversary starred:' },
    { name: 'Recent Videos', query: 'type:video this_month:' },
  ]
},

// Search within results
refinement: {
  enabled: true,

  action: {
    trigger: 'search icon within results',
    behavior: 'further filter current results',
    cascading: 'can refine multiple times',
  },
},

```

```

    breadcrumbs: 'show refinement path',
  }
}

```

Search Performance Optimization:

```

interface SearchPerformance {
  // Caching
  caching: {
    resultCache: {
      enabled: true,
      duration: 300, // 5 minutes
      keyPattern: 'query + filters + sort',
    },

    indexCache: {
      enabled: true,
      type: 'in_memory',
      size: '100MB',
      eviction: 'LRU',
    }
  },

  // Query optimization
  optimization: {
    debouncing: {
      enabled: true,
      delay: 300, // 300ms after typing stops
    },

    minQueryLength: 2, // Require 2+ characters

    indexedQuery: {
      method: 'use pre-built search index',
      updateFrequency: 'realtime',
    },

    pagination: {
      initialLoad: 20, // First 20 results
      lazyLoad: 'load more on scroll',
      prefetch: 'next page in background',
    }
  },

  // Performance targets
  targets: {
    autocomplete: '<100ms',
    initialResults: '<300ms',
    fullResults: '<1s',
  }
}

```

```
    indexUpdate: '<5s after content creation',  
  }  
}
```

Chapter 6 Status: Complete

Summary: - Push Notification Architecture: Multi-platform (iOS APNs, Android FCM, Web Push API) with notification channels and rich media - Smart delivery with 4 priority levels (urgent/high/normal/low), context-aware routing, quiet hours, and intelligent batching - Granular user preferences with per-category controls, day-specific quiet hours, and partner-specific settings - In-app notifications with banner/toast/modal types and gesture-based interactions - Important Date Management: Auto-detected anniversaries and birthdays plus custom dates with flexible reminder schedules - Anniversary celebration mode with confetti, memory slideshows, achievement unlocks, and partner-synchronized greetings - Birthday reminders with gift tracking and celebration history - Global Search: Comprehensive full-text search across 8 content types (messages, media, dates, achievements, circles, creative, music, people) - Advanced search with natural language queries, search operators, autocomplete (<100ms), saved searches, and sub-300ms response times - Complete TypeScript schemas, user flows, UI specifications, and performance optimization strategies

Next Chapter: Chapter 7: Technical Architecture & Monorepo Structure

Chapter 7: Technical Architecture & Monorepo Structure

Overview

LinkUp is built as a modern, scalable multiplatform application using a monorepo architecture. This enables code sharing between Web, iOS, and Android while maintaining platform-specific optimizations.

This chapter covers: - Monorepo Structure - Organization of code across platforms and shared packages - Technology Stack - Frameworks, languages, and tools for each platform - Architecture Patterns - Clean architecture, dependency injection, state management - Code Sharing Strategy - Maximum reuse while respecting platform differences - Build & Deployment - CI/CD pipelines for all platforms

Design Philosophy: - Write once, deploy everywhere (where possible) - Platform-native experiences - No compromises on UX - Type safety - TypeScript/Swift/Kotlin throughout - Developer experience - Fast builds, hot reload, excellent tooling - Scalability - Architecture supports millions of users

7.1 Monorepo Structure

7.1.1 Repository Organization

Directory Structure:

```

linkup/
  apps/
    web/          # Next.js web application
    ios/          # iOS native app (Swift/SwiftUI)
    android/      # Android native app (Kotlin/Compose)

  packages/
    shared/       # Shared business logic (TypeScript)
    ui/           # Shared UI components
    api-client/   # API client library
    types/        # Shared TypeScript types
    utils/        # Shared utilities
    validation/   # Shared validation schemas
    config/       # Shared configuration

  backend/
    api/          # REST API server (Node.js/Express)
    websocket/    # WebSocket server (Socket.io)
    workers/      # Background job workers
    services/     # Microservices
    shared/       # Backend shared code

  infrastructure/
    terraform/    # Infrastructure as Code
    kubernetes/   # K8s manifests
    docker/       # Dockerfiles
    scripts/      # Deployment scripts

  docs/
    architecture/ # Architecture diagrams
    api/          # API documentation
    guides/       # Development guides

  tools/
    cli/          # CLI tools
    generators/   # Code generators
    scripts/      # Build scripts

  .github/
    workflows/    # GitHub Actions CI/CD

package.json      # Root package.json
turbo.json        # Turborepo configuration
tsconfig.base.json # Base TypeScript config
README.md

```

Monorepo Tool: Turborepo

// turbo.json

```

{
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**", "build/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "test": {
      "dependsOn": ["build"],
      "outputs": ["coverage/**"]
    },
    "lint": {
      "outputs": []
    },
    "type-check": {
      "outputs": []
    }
  }
}

```

Workspace Configuration:

```

// package.json (root)
{
  "name": "linkup-monorepo",
  "private": true,
  "workspaces": [
    "apps/*",
    "packages/*",
    "backend/*"
  ],
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "test": "turbo run test",
    "lint": "turbo run lint",
    "type-check": "turbo run type-check",
    "clean": "turbo run clean && rm -rf node_modules"
  },
  "devDependencies": {
    "turbo": "^1.11.0",
    "typescript": "^5.3.3",
    "prettier": "^3.1.1",
    "eslint": "^8.56.0"
  }
}

```

7.1.2 Shared Packages

Package: @linkup/shared

Business logic shared across all platforms.

```
// packages/shared/package.json
```

```
{
  "name": "@linkup/shared",
  "version": "1.0.0",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "dev": "tsc --watch",
    "test": "jest"
  },
  "dependencies": {
    "@linkup/types": "*",
    "date-fns": "^3.0.0",
    "zod": "^3.22.4"
  }
}
```

```
// packages/shared/src/index.ts
```

```
export * from './features/messaging';
export * from './features/media';
export * from './features/achievements';
export * from './features/streaks';
export * from './utils/date-utils';
export * from './utils/validation';
```

Example Shared Feature: Photo Streak Logic

```
// packages/shared/src/features/streaks/photo-streak-logic.ts
```

```
import { addDays, differenceInDays, startOfDay } from 'date-fns';
import { PhotoStreak, StreakStatus } from '@linkup/types';
```

```
export class PhotoStreakLogic {
  /**
   * Calculate current streak status
   */
  static getStreakStatus(streak: PhotoStreak, now: Date = new Date()): StreakStatus {
    if (!streak.lastPhotoDate) {
      return {
        status: 'active',
        currentCount: 0,
        daysUntilBreak: 1,
      };
    }
  }
}
```



```

        canRecover: false,
    };
}

const today = startOfDay(now);
const lastPhoto = startOfDay(streak.lastPhotoDate);
const daysSinceLastPhoto = differenceInDays(today, lastPhoto);

if (daysSinceLastPhoto === 0) {
    // Photo posted today
    return {
        status: 'complete_today',
        currentCount: streak.currentStreak,
        daysUntilBreak: 0,
        canRecover: false,
    };
} else if (daysSinceLastPhoto === 1) {
    // Need to post today
    return {
        status: 'due_today',
        currentCount: streak.currentStreak,
        daysUntilBreak: 1,
        canRecover: false,
    };
} else if (daysSinceLastPhoto === 2) {
    // Broken yesterday, can recover
    return {
        status: 'broken',
        currentCount: 0,
        daysUntilBreak: 0,
        canRecover: true,
        recoveryDeadline: addDays(lastPhoto, 3), // 24 hours to recover
    };
} else {
    // Too late to recover
    return {
        status: 'broken',
        currentCount: 0,
        daysUntilBreak: 0,
        canRecover: false,
    };
}
}

/**
 * Check if freeze can be used
 */
static canUseFreeze(streak: PhotoStreak): boolean {

```

```

    return streak.freezesAvailable > 0;
}

/**
 * Apply freeze to streak
 */
static applyFreeze(streak: PhotoStreak): PhotoStreak {
  if (!this.canUseFreeze(streak)) {
    throw new Error('No freezes available');
  }

  return {
    ...streak,
    freezesAvailable: streak.freezesAvailable - 1,
    freezeHistory: [
      ...streak.freezeHistory,
      {
        id: crypto.randomUUID(),
        usedAt: new Date(),
        reason: 'manual',
      },
    ],
    lastPhotoDate: new Date(), // Extend deadline
  };
}

/**
 * Calculate points for streak milestone
 */
static calculatePoints(streakLength: number): number {
  if (streakLength < 7) return streakLength * 10;
  if (streakLength < 30) return streakLength * 15;
  if (streakLength < 100) return streakLength * 20;
  return streakLength * 25;
}
}

```

Package: @linkup/types

Shared TypeScript types and interfaces.

```

// packages/types/src/index.ts
export * from './models/user';
export * from './models/couple';
export * from './models/message';
export * from './models/media';
export * from './models/achievement';
export * from './api/requests';
export * from './api/responses';

```

```

export * from './websocket/events';

// packages/types/src/models/user.ts
export interface User {
  id: string;
  email: string;
  username: string;
  displayName: string;
  avatarUrl?: string;
  bio?: string;
  birthday?: Date;
  createdAt: Date;
  updatedAt: Date;
}

export interface UserProfile extends User {
  coupleId?: string;
  partnerId?: string;
  preferences: UserPreferences;
  stats: UserStats;
}

export interface UserPreferences {
  notifications: NotificationPreferences;
  privacy: PrivacySettings;
  theme: 'light' | 'dark' | 'auto';
  language: string;
}

// ... more types

Package: @linkup/api-client

Type-safe API client for all platforms.

// packages/api-client/src/client.ts
import { User, Message, Media, ApiResponse } from '@linkup/types';

export class LinkUpAPIClient {
  private baseUrl: string;
  private token?: string;

  constructor(config: { baseUrl: string; token?: string }) {
    this.baseUrl = config.baseUrl;
    this.token = config.token;
  }

  setToken(token: string) {
    this.token = token;
  }

```

```

}

private async request<T>(
  endpoint: string,
  options?: RequestInit
): Promise<ApiResponse<T>> {
  const headers = {
    'Content-Type': 'application/json',
    ...(this.token && { Authorization: `Bearer ${this.token}` }),
    ...options?.headers,
  };

  const response = await fetch(`${this.baseURL}${endpoint}`, {
    ...options,
    headers,
  });

  if (!response.ok) {
    throw new Error(`API Error: ${response.statusText}`);
  }

  return response.json();
}

// User endpoints
async getCurrentUser(): Promise<User> {
  const response = await this.request<User>('/api/users/me');
  return response.data;
}

async updateProfile(updates: Partial<User>): Promise<User> {
  const response = await this.request<User>('/api/users/me', {
    method: 'PATCH',
    body: JSON.stringify(updates),
  });
  return response.data;
}

// Message endpoints
async getMessages(coupleId: string, page = 1): Promise<Message[]> {
  const response = await this.request<Message[]>(
    `/api/couples/${coupleId}/messages?page=${page}`
  );
  return response.data;
}

async sendMessage(coupleId: string, content: string): Promise<Message> {
  const response = await this.request<Message>(

```

```

    `/api/couples/${coupleId}/messages`,
    {
      method: 'POST',
      body: JSON.stringify({ content }),
    }
  );
  return response.data;
}

// Media endpoints
async uploadMedia(
  coupleId: string,
  file: File | Blob,
  type: 'photo' | 'video'
): Promise<Media> {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('type', type);

  const response = await fetch(
    `${this.baseURL}/api/couples/${coupleId}/media`,
    {
      method: 'POST',
      headers: {
        ...(this.token && { Authorization: `Bearer ${this.token}` }),
      },
      body: formData,
    }
  );

  const result = await response.json();
  return result.data;
}

// ... more endpoints
}

// Export singleton instance
export const api = new LinkUpAPIClient({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3001',
});

Package: @linkup/validation

Shared validation schemas using Zod.

// packages/validation/src/schemas/user.ts
import { z } from 'zod';

```

```

export const userSchema = z.object({
  id: z.string().uuid(),
  email: z.string().email(),
  username: z.string().min(3).max(30).regex(/^[a-zA-Z0-9_]+$/),
  displayName: z.string().min(1).max(50),
  avatarUrl: z.string().url().optional(),
  bio: z.string().max(500).optional(),
  birthday: z.date().optional(),
});

export const updateProfileSchema = userSchema.partial().omit({ id: true });

export type UserInput = z.infer<typeof userSchema>;
export type UpdateProfileInput = z.infer<typeof updateProfileSchema>;

// packages/validation/src/schemas/message.ts
export const messageSchema = z.object({
  id: z.string().uuid(),
  coupleId: z.string().uuid(),
  senderId: z.string().uuid(),
  content: z.string().min(1).max(5000),
  type: z.enum(['text', 'photo', 'video', 'voice', 'file']),
  metadata: z.record(z.any()).optional(),
  replyToId: z.string().uuid().optional(),
  createdAt: z.date(),
});

export const createMessageSchema = messageSchema.pick({
  content: true,
  type: true,
  metadata: true,
  replyToId: true,
});

```

7.2 Web Application Architecture

7.2.1 Next.js Application Structure

Technology Stack: - Framework: Next.js 14 (App Router) - Language: TypeScript 5.3 - UI Library: React 18 - Styling: Tailwind CSS + CSS Modules - State Management: Zustand + React Query - Real-time: Socket.io client - Forms: React Hook Form + Zod - Testing: Jest + React Testing Library

Directory Structure:

```

apps/web/
  src/
    app/                                # Next.js App Router

```

```

    (auth)/
      login/
      signup/
    (main)/
      chat/
      media/
      creative/
      settings/
    layout.tsx
    page.tsx

components/
  ui/                # Reusable UI components
  features/          # Feature-specific components
  layouts/           # Layout components

hooks/
  useAuth.ts
  useMessages.ts
  useWebSocket.ts
  useMedia.ts

store/
  auth.store.ts
  messages.store.ts
  ui.store.ts

lib/
  api.ts             # API client setup
  websocket.ts       # WebSocket setup
  utils.ts

styles/
  globals.css
  themes/

types/
  index.ts

public/
  images/
  sounds/
  fonts/

next.config.js
tailwind.config.js
tsconfig.json
package.json

```

Next.js Configuration:

```
// apps/web/next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,

  // Enable SWC minification
  swcMinify: true,

  // Transpile shared packages
  transpilePackages: [
    '@linkup/shared',
    '@linkup/types',
    '@linkup/api-client',
    '@linkup/ui',
  ],

  // Image optimization
  images: {
    domains: ['cdn.linkup.app', 'storage.linkup.app'],
    formats: ['image/avif', 'image/webp'],
  },

  // Environment variables
  env: {
    NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL,
    NEXT_PUBLIC_WS_URL: process.env.NEXT_PUBLIC_WS_URL,
    NEXT_PUBLIC_CDN_URL: process.env.NEXT_PUBLIC_CDN_URL,
  },

  // PWA configuration
  pwa: {
    dest: 'public',
    register: true,
    skipWaiting: true,
  },

  // Security headers
  async headers() {
    return [
      {
        source: '/*:path*',
        headers: [
          { key: 'X-DNS-Prefetch-Control', value: 'on' },
          { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        ],
      },
    ]
  }
}
```



```

    ],
  },
];
},
};

```

module.exports = nextConfig;

State Management Example:

```

// apps/web/src/store/messages.store.ts
import { create } from 'zustand';
import { Message } from '@linkup/types';

interface MessagesState {
  messages: Message[];
  isLoading: boolean;
  error: string | null;

  // Actions
  addMessage: (message: Message) => void;
  updateMessage: (id: string, updates: Partial<Message>) => void;
  deleteMessage: (id: string) => void;
  setMessages: (messages: Message[]) => void;
  setLoading: (loading: boolean) => void;
  setError: (error: string | null) => void;
}

export const useMessagesStore = create<MessagesState>((set) => ({
  messages: [],
  isLoading: false,
  error: null,

  addMessage: (message) =>
    set((state) => ({
      messages: [...state.messages, message],
    })),

  updateMessage: (id, updates) =>
    set((state) => ({
      messages: state.messages.map((msg) =>
        msg.id === id ? { ...msg, ...updates } : msg
      ),
    })),

  deleteMessage: (id) =>
    set((state) => ({
      messages: state.messages.filter((msg) => msg.id !== id),
    })),

```

```

    setMessages: (messages) => set({ messages }),
    setLoading: (loading) => set({ isLoading: loading }),
    setError: (error) => set({ error }),
  }));

```

WebSocket Hook:

```

// apps/web/src/hooks/useWebSocket.ts
import { useEffect, useRef } from 'react';
import { io, Socket } from 'socket.io-client';
import { useAuthStore } from '@store/auth.store';
import { useMessagesStore } from '@store/messages.store';

export function useWebSocket() {
  const socketRef = useRef<Socket | null>(null);
  const { token, user } = useAuthStore();
  const { addMessage } = useMessagesStore();

  useEffect(() => {
    if (!token || !user) return;

    // Connect to WebSocket server
    socketRef.current = io(process.env.NEXT_PUBLIC_WS_URL!, {
      auth: { token },
      transports: ['websocket'],
    });

    const socket = socketRef.current;

    // Connection events
    socket.on('connect', () => {
      console.log('WebSocket connected');
      socket.emit('join:couple', { coupleId: user.coupleId });
    });

    socket.on('disconnect', () => {
      console.log('WebSocket disconnected');
    });

    // Message events
    socket.on('message:new', (message) => {
      addMessage(message);
    });

    socket.on('message:updated', ({ id, updates }) => {
      useMessagesStore.getState().updateMessage(id, updates);
    });
  }, [token, user]);
}

```

```

socket.on('message:deleted', ({ id }) => {
  useMessagesStore.getState().deleteMessage(id);
});

// Typing indicator
socket.on('typing:start', ({ userId }) => {
  console.log(`User ${userId} is typing`);
});

socket.on('typing:stop', ({ userId }) => {
  console.log(`User ${userId} stopped typing`);
});

// Cleanup
return () => {
  socket.disconnect();
};
}, [token, user]);

return socketRef.current;
}

```

7.2.2 Web UI Components

Component Architecture:

```

// apps/web/src/components/ui/Button.tsx
import { ButtonHTMLAttributes, forwardRef } from 'react';
import { cn } from '@lib/utils';

interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'ghost' | 'danger';
  size?: 'sm' | 'md' | 'lg';
  isLoading?: boolean;
}

export const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  ({
    className,
    variant = 'primary',
    size = 'md',
    isLoading,
    disabled,
    children,
    ...props
  }, ref) => {
    return (

```

```

<button
  ref={ref}
  disabled={disabled || isLoading}
  className={cn(
    'inline-flex items-center justify-center rounded-lg font-medium',
    'transition-colors focus:outline-none focus:ring-2 focus:ring-offset-2',
    'disabled:opacity-50 disabled:cursor-not-allowed',
    {
      'bg-purple-600 text-white hover:bg-purple-700 focus:ring-purple-500':
        variant === 'primary',
      'bg-gray-200 text-gray-900 hover:bg-gray-300 focus:ring-gray-500':
        variant === 'secondary',
      'bg-transparent hover:bg-gray-100 focus:ring-gray-500':
        variant === 'ghost',
      'bg-red-600 text-white hover:bg-red-700 focus:ring-red-500':
        variant === 'danger',
    },
    {
      'px-3 py-1.5 text-sm': size === 'sm',
      'px-4 py-2 text-base': size === 'md',
      'px-6 py-3 text-lg': size === 'lg',
    },
    className
  )}
  {...props}
>
{isLoading ? (
  <>
    <svg
      className="animate-spin -ml-1 mr-2 h-4 w-4"
      xmlns="http://www.w3.org/2000/svg"
      fill="none"
      viewBox="0 0 24 24"
    >
      <circle
        className="opacity-25"
        cx="12"
        cy="12"
        r="10"
        stroke="currentColor"
        strokeWidth="4"
      />
      <path
        className="opacity-75"
        fill="currentColor"
        d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4z"
      />
    </svg>

```

```

        Loading...
      </>
    ) : (
      children
    )}
  </button>
);
}
);

```

Button.displayName = 'Button';

Feature Component Example:

```

// apps/web/src/components/features/chat/MessageInput.tsx
'use client';

import { useState, useRef, FormEvent } from 'react';
import { useMessagesStore } from '@store/messages.store';
import { api } from '@linkup/api-client';
import { Button } from '@components/ui/Button';

export function MessageInput({ coupleId }: { coupleId: string }) {
  const [content, setContent] = useState('');
  const [isSending, setIsSending] = useState(false);
  const inputRef = useRef<HTMLTextAreaElement>(null);

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();

    if (!content.trim() || isSending) return;

    setIsSending(true);

    try {
      await api.sendMessage(coupleId, content.trim());
      setContent('');
      inputRef.current?.focus();
    } catch (error) {
      console.error('Failed to send message:', error);
    } finally {
      setIsSending(false);
    }
  };

  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault();
      handleSubmit(e);
    }
  };

```

```

    }
};

return (
  <form onSubmit={handleSubmit} className="flex items-end gap-2 p-4">
    <textarea
      ref={inputRef}
      value={content}
      onChange={(e) => setContent(e.target.value)}
      onKeyDown={handleKeyDown}
      placeholder="Type a message..."
      className="flex-1 resize-none rounded-lg border border-gray-300 p-3"
      rows={1}
      maxLength={5000}
    />
    <Button
      type="submit"
      isLoading={isSending}
      disabled={!content.trim()}
    >
      Send
    </Button>
  </form>
);
}

```

7.3 iOS Application Architecture

7.3.1 iOS Tech Stack & Structure

Technology Stack: - Language: Swift 5.9 - UI Framework: SwiftUI - Architecture: MVVM + Clean Architecture - Dependency Injection: Resolver - Networking: URLSession + Combine - Real-time: Starscream (WebSocket) - Local Storage: Core Data + SwiftData - Image Loading: Kingfisher - Testing: XCTest + Quick/Nimble

Project Structure:

```

apps/ios/LinkUp/
  App/
    LinkUpApp.swift          # App entry point
    AppDelegate.swift

  Core/
    Network/
      APIClient.swift
      WebSocketManager.swift
      Endpoints.swift
    Storage/

```

```

        CoreDataStack.swift
        Models/
    DependencyInjection/
        Container.swift
    Extensions/

Features/
    Auth/
        Views/
        ViewModels/
        Models/
    Chat/
        Views/
            ChatView.swift
            MessageRow.swift
            MessageInputView.swift
        ViewModels/
            ChatViewModel.swift
        Models/
    Media/
    Creative/
    Settings/

Shared/
    Components/
        Buttons/
        TextFields/
        Cards/
    Styles/
        Colors.swift
        Typography.swift
        Spacing.swift
    Utils/

Resources/
    Assets.xcassets
    Sounds/
    Localizable.strings

Supporting Files/
    Info.plist
    LinkUp.entitlements

API Client:

// apps/ios/LinkUp/Core/Network/APIClient.swift
import Foundation
import Combine

```

```

class APIClient {
    static let shared = APIClient()

    private let baseURL = URL(string: "https://api.linkup.app")!
    private var token: String?

    private init() {}

    func setToken(_ token: String) {
        self.token = token
    }

    func request<T: Decodable>(
        _ endpoint: Endpoint,
        method: HTTPMethod = .get,
        body: Encodable? = nil
    ) -> AnyPublisher<T, Error> {
        var request = URLRequest(url: baseURL.appendingPathComponent(endpoint.path))
        request.httpMethod = method.rawValue
        request.setValue("application/json", forHTTPHeaderField: "Content-Type")

        if let token = token {
            request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        }

        if let body = body {
            request.httpBody = try? JSONEncoder().encode(body)
        }

        return URLSession.shared.dataTaskPublisher(for: request)
            .tryMap { data, response in
                guard let httpResponse = response as? HTTPURLResponse,
                      (200...299).contains(httpResponse.statusCode) else {
                    throw APIError.invalidResponse
                }
                return data
            }
            .decode(type: APIResponse<T>.self, decoder: JSONDecoder())
            .map(\.data)
            .eraseToAnyPublisher()
    }
}

enum HTTPMethod: String {
    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case patch = "PATCH"
}

```



```

        case delete = "DELETE"
    }

    struct APIResponse<T: Decodable>: Decodable {
        let data: T
        let message: String?
    }

    enum APIError: Error {
        case invalidResponse
        case decodingError
        case networkError
    }

```

SwiftUI View Example:

```

// apps/ios/LinkUp/Features/Chat/Views/ChatView.swift
import SwiftUI

struct ChatView: View {
    @StateObject private var viewModel: ChatViewModel
    @State private var messageText = ""

    init(coupleId: String) {
        _viewModel = StateObject(wrappedValue: ChatViewModel(coupleId: coupleId))
    }

    var body: some View {
        VStack(spacing: 0) {
            // Messages list
            ScrollViewReader { proxy in
                ScrollView {
                    LazyVStack(spacing: 12) {
                        ForEach(viewModel.messages) { message in
                            MessageRow(message: message)
                                .id(message.id)
                        }
                    }
                }
                .padding()
            }
            .onChange(of: viewModel.messages.count) { _ in
                if let lastMessage = viewModel.messages.last {
                    withAnimation {
                        proxy.scrollTo(lastMessage.id, anchor: .bottom)
                    }
                }
            }
        }
    }
}

```

```

        // Input area
        MessageInputView(
            text: $messageText,
            onSend: {
                viewModel.sendMessage(messageText)
                messageText = ""
            }
        )
    }
    .navigationTitle("Chat")
    .onAppear {
        viewModel.loadMessages()
    }
}

struct MessageRow: View {
    let message: Message
    @EnvironmentObject var authStore: AuthStore

    var isFromCurrentUser: Bool {
        message.senderId == authStore.currentUser?.id
    }

    var body: some View {
        HStack {
            if isFromCurrentUser {
                Spacer()
            }

            VStack(alignment: isFromCurrentUser ? .trailing : .leading, spacing: 4) {
                Text(message.content)
                    .padding(12)
                    .background(isFromCurrentUser ? Color.purple : Color.gray.opacity(0.2))
                    .foregroundColor(isFromCurrentUser ? .white : .primary)
                    .cornerRadius(16)

                Text(message.createdAt, style: .time)
                    .font(.caption2)
                    .foregroundColor(.gray)
            }

            if !isFromCurrentUser {
                Spacer()
            }
        }
    }
}

```

ViewModel with Combine:

```
// apps/ios/LinkUp/Features/Chat/ViewModels/ChatViewModel.swift
import Foundation
import Combine

class ChatViewModel: ObservableObject {
    @Published var messages: [Message] = []
    @Published var isLoading = false
    @Published var error: Error?

    private let coupleId: String
    private var cancellables = Set<AnyCancellable>()
    private let apiClient = APIClient.shared
    private let websocket = WebSocketManager.shared

    init(coupleId: String) {
        self.coupleId = coupleId
        setupWebSocketListeners()
    }

    func loadMessages() {
        isLoading = true

        apiClient.request(
            .messages(coupleId: coupleId),
            method: .get
        )
        .receive(on: DispatchQueue.main)
        .sink(
            receiveCompletion: { [weak self] completion in
                self?.isLoading = false
                if case .failure(let error) = completion {
                    self?.error = error
                }
            },
            receiveValue: { [weak self] (messages: [Message]) in
                self?.messages = messages
            }
        )
        .store(in: &cancellables)
    }

    func sendMessage(_ content: String) {
        let messageData = CreateMessageRequest(
            content: content,
            type: .text
        )
    }
}
```

```

apiClient.request(
    .sendMessage(coupleId: coupleId),
    method: .post,
    body: messageData
)
.receive(on: DispatchQueue.main)
.sink(
    receiveCompletion: { completion in
        if case .failure(let error) = completion {
            print("Failed to send message: \(error)")
        }
    },
    receiveValue: { [weak self] (message: Message) in
        self?.messages.append(message)
    }
)
.store(in: &cancellables)
}

private func setupWebSocketListeners() {
    websocket.onMessageReceived = { [weak self] message in
        DispatchQueue.main.async {
            self?.messages.append(message)
        }
    }
}
}
}

```

7.4 Android Application Architecture

7.4.1 Android Tech Stack & Structure

Technology Stack: - Language: Kotlin 1.9 - UI Framework: Jetpack Compose - Architecture: MVVM + Clean Architecture - Dependency Injection: Hilt (Dagger) - Networking: Retrofit + OkHttp - Real-time: Socket.io client - Local Storage: Room Database - Image Loading: Coil - Async: Coroutines + Flow - Testing: JUnit + Espresso

Project Structure:

```

apps/android/
  app/
    src/
      main/
        java/com/linkup/
          LinkUpApplication.kt

          di/                                # Dependency Injection

```

```

        AppModule.kt
        NetworkModule.kt
        DatabaseModule.kt

    data/
        remote/      # API clients
        local/        # Room database
        repository/   # Repositories

    domain/
        model/        # Domain models
        repository/   # Repository interfaces
        usecase/      # Use cases

    ui/
        theme/        # Compose theme
        components/   # Reusable components
        features/
            auth/
            chat/
            media/
            settings/

    res/
        AndroidManifest.xml

    test/

    build.gradle.kts

    gradle/

```

Dependency Injection with Hilt:

```

// apps/android/app/src/main/java/com/linkup/di/NetworkModule.kt
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @Provides
    @Singleton
    fun provideOkHttpClient(): OkHttpClient {
        return OkHttpClient.Builder()
            .addInterceptor { chain ->
                val request = chain.request().newBuilder()
                    .addHeader("Content-Type", "application/json")
                    .build()
                chain.proceed(request)
            }
    }
}

```

```

        .connectTimeout(30, TimeUnit.SECONDS)
        .readTimeout(30, TimeUnit.SECONDS)
        .build()
    }

    @Provides
    @Singleton
    fun provideRetrofit(okHttpClient: OkHttpClient): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.linkup.app/")
            .client(okHttpClient)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun provideApiService(retrofit: Retrofit): ApiService {
        return retrofit.create(ApiService::class.java)
    }
}

```

API Service:

```

// apps/android/app/src/main/java/com/linkup/data/remote/ApiService.kt
interface ApiService {

    @GET("api/users/me")
    suspend fun getCurrentUser(): ApiResponse<User>

    @GET("api/couples/{coupleId}/messages")
    suspend fun getMessages(
        @Path("coupleId") coupleId: String,
        @Query("page") page: Int = 1
    ): ApiResponse<List<Message>>

    @POST("api/couples/{coupleId}/messages")
    suspend fun sendMessage(
        @Path("coupleId") coupleId: String,
        @Body request: CreateMessageRequest
    ): ApiResponse<Message>

    @Multipart
    @POST("api/couples/{coupleId}/media")
    suspend fun uploadMedia(
        @Path("coupleId") coupleId: String,
        @Part file: MultipartBody.Part,
        @Part("type") type: RequestBody
    ): ApiResponse<Media>
}

```

```
}
```

```
data class ApiResponse<T>(  
    val data: T,  
    val message: String? = null  
)
```

Repository Pattern:

```
// apps/android/app/src/main/java/com/linkup/data/repository/MessageRepositoryImpl.kt  
class MessageRepositoryImpl @Inject constructor(  
    private val apiService: ApiService,  
    private val messageDao: MessageDao  
) : MessageRepository {  
  
    override fun getMessages(coupleId: String): Flow<List<Message>> = flow {  
        // Emit cached messages first  
        emit(messageDao.getMessages(coupleId))  
  
        try {  
            // Fetch from network  
            val response = apiService.getMessages(coupleId)  
            val messages = response.data  
  
            // Update cache  
            messageDao.insertAll(messages)  
  
            // Emit fresh data  
            emit(messages)  
        } catch (e: Exception) {  
            // Error handled, keep showing cached data  
            Log.e("MessageRepo", "Failed to fetch messages", e)  
        }  
    }  
  
    override suspend fun sendMessage(  
        coupleId: String,  
        content: String  
    ): Result<Message> {  
        return try {  
            val request = CreateMessageRequest(content, MessageType.TEXT)  
            val response = apiService.sendMessage(coupleId, request)  
            val message = response.data  
  
            // Cache the sent message  
            messageDao.insert(message)  
  
            Result.success(message)  
        } catch (e: Exception) {
```

```

        Result.failure(e)
    }
}

```

Jetpack Compose UI:

```

// apps/android/app/src/main/java/com/linkup/ui/features/chat/ChatScreen.kt
@Composable
fun ChatScreen(
    coupleId: String,
    viewModel: ChatViewModel = hiltViewModel()
) {
    val messages by viewModel.messages.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()
    var messageText by remember { mutableStateOf("") }

    LaunchedEffect(coupleId) {
        viewModel.loadMessages(coupleId)
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Chat") }
            )
        }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
        ) {
            // Messages list
            LazyColumn(
                modifier = Modifier.weight(1f),
                reverseLayout = true
            ) {
                items(messages.reversed()) { message ->
                    MessageItem(message = message)
                }
            }

            // Input area
            MessageInput(
                text = messageText,
                onChange = { messageText = it },
                onSend = {
                    viewModel.sendMessage(coupleId, messageText)
                }
            )
        }
    }
}

```



```

        messageText = ""
    },
    enabled = !isLoading
)
}
}
}

@Composable
fun MessageItem(message: Message) {
    val isFromCurrentUser = message.senderId == getCurrentUserId()

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        horizontalArrangement = if (isFromCurrentUser)
            Arrangement.End else Arrangement.Start
    ) {
        Column(
            horizontalAlignment = if (isFromCurrentUser)
                Alignment.End else Alignment.Start
        ) {
            Surface(
                shape = RoundedCornerShape(16.dp),
                color = if (isFromCurrentUser)
                    MaterialTheme.colorScheme.primary
                else
                    MaterialTheme.colorScheme.surfaceVariant
            ) {
                Text(
                    text = message.content,
                    modifier = Modifier.padding(12.dp),
                    color = if (isFromCurrentUser)
                        MaterialTheme.colorScheme.onPrimary
                    else
                        MaterialTheme.colorScheme.onSurface
                )
            }

            Text(
                text = message.createdAt.formatTime(),
                style = MaterialTheme.typography.labelSmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.padding(top = 4.dp)
            )
        }
    }
}

```

```
}
```

ViewModel with Flow:

```
// apps/android/app/src/main/java/com/linkup/ui/features/chat/ChatViewModel.kt
@HiltViewModel
class ChatViewModel @Inject constructor(
    private val messageRepository: MessageRepository,
    private val websocketManager: WebSocketManager
) : ViewModel() {

    private val _messages = MutableStateFlow<List<Message>>(emptyList())
    val messages: StateFlow<List<Message>> = _messages.asStateFlow()

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading.asStateFlow()

    private val _error = MutableStateFlow<String?>(null)
    val error: StateFlow<String?> = _error.asStateFlow()

    init {
        setupWebSocketListeners()
    }

    fun loadMessages(coupleId: String) {
        viewModelScope.launch {
            _isLoading.value = true

            messageRepository.getMessages(coupleId)
                .catch { e ->
                    _error.value = e.message
                }
                .collect { messageList ->
                    _messages.value = messageList
                    _isLoading.value = false
                }
        }
    }

    fun sendMessage(coupleId: String, content: String) {
        viewModelScope.launch {
            _isLoading.value = true

            messageRepository.sendMessage(coupleId, content)
                .onSuccess { message ->
                    _messages.value = _messages.value + message
                }
                .onFailure { e ->
                    _error.value = e.message
                }
        }
    }
}
```

```

        }

        _isLoading.value = false
    }
}

private fun setupWebSocketListeners() {
    websocketManager.onMessageReceived = { message ->
        _messages.value = _messages.value + message
    }
}
}

```

7.5 Backend Architecture

7.5.1 Backend Services Overview

Microservices Architecture:

```

backend/
  api/                # REST API Gateway
  websocket/          # WebSocket server
  media-service/      # Media processing
  notification-service/ # Push notifications
  streaming-service/   # Live streaming
  worker-service/     # Background jobs

```

API Gateway (Node.js + Express):

```

// backend/api/src/server.ts
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import { authRouter } from '../routes/auth';
import { usersRouter } from '../routes/users';
import { couplesRouter } from '../routes/couples';
import { messagesRouter } from '../routes/messages';
import { authMiddleware } from '../middleware/auth';

```

```

const app = express();

```

```

// Middleware
app.use(helmet());
app.use(cors());
app.use(express.json());

```

```

// Routes
app.use('/api/auth', authRouter);

```

```

app.use('/api/users', authMiddleware, usersRouter);
app.use('/api/couples', authMiddleware, couplesRouter);
app.use('/api/messages', authMiddleware, messagesRouter);

// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal server error' });
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`API server running on port ${PORT}`);
});

```

WebSocket Server:

```

// backend/websocket/src/server.ts
import { Server } from 'socket.io';
import { createServer } from 'http';
import { verifyToken } from './auth';

const httpServer = createServer();
const io = new Server(httpServer, {
  cors: {
    origin: process.env.ALLOWED_ORIGINS?.split(','),
    credentials: true,
  },
});

// Authentication middleware
io.use(async (socket, next) => {
  const token = socket.handshake.auth.token;

  try {
    const user = await verifyToken(token);
    socket.data.user = user;
    next();
  } catch (err) {
    next(new Error('Authentication failed'));
  }
});

io.on('connection', (socket) => {
  console.log(`User connected: ${socket.data.user.id}`);

  // Join couple room
  socket.on('join:couple', ({ coupleId }) => {
    socket.join(`couple:${coupleId}`);
  });
});

```

```

});

// Message events
socket.on('message:send', async (data) => {
  const { coupleId, content } = data;

  // Save to database
  const message = await saveMessage({
    coupleId,
    senderId: socket.data.user.id,
    content,
  });

  // Broadcast to couple
  io.to(`couple:${coupleId}`).emit('message:new', message);
});

// Typing indicator
socket.on('typing:start', ({ coupleId }) => {
  socket.to(`couple:${coupleId}`).emit('typing:start', {
    userId: socket.data.user.id,
  });
});

socket.on('typing:stop', ({ coupleId }) => {
  socket.to(`couple:${coupleId}`).emit('typing:stop', {
    userId: socket.data.user.id,
  });
});

socket.on('disconnect', () => {
  console.log(`User disconnected: ${socket.data.user.id}`);
});

const PORT = process.env.WS_PORT || 3002;
httpServer.listen(PORT, () => {
  console.log(`WebSocket server running on port ${PORT}`);
});

```

Chapter 7 Status: Complete

Summary: - Monorepo Structure: Turborepo-based monorepo with apps (web, iOS, Android) and shared packages - Shared Packages: @linkup/shared (business logic), @linkup/types (TypeScript interfaces), @linkup/api-client (type-safe API), @linkup/validation (Zod schemas) - Web: Next.js 14 App Router + React 18 + Zustand + React Query + Socket.io + Tailwind CSS - iOS: Swift 5.9 + SwiftUI + MVVM + Combine + Core Data + Starscream WebSocket - Android: Kotlin

1.9 + Jetpack Compose + MVVM + Hilt + Retrofit + Coroutines/Flow + Room Database - Backend: Node.js microservices (API Gateway, WebSocket server, media, notifications, streaming, workers) - Complete code examples for API clients, state management, UI components, ViewModels, repositories - Build configuration and CI/CD pipeline setup

Next Chapter: Chapter 8: Database Design & Data Schemas

Chapter 8: Database Design & Data Schemas

Overview

LinkUp uses a polyglot persistence approach, selecting the right database technology for each data type and access pattern. This chapter defines the complete data model across all storage systems.

This chapter covers: - PostgreSQL - Primary relational database for core entities - Redis - Caching layer and session management - MongoDB - Media metadata and flexible document storage - S3/CDN - Object storage for media files - Database Relationships - Foreign keys, constraints, and cascading - Indexing Strategy - Performance optimization - Migrations - Schema versioning and evolution

Design Philosophy: - Data integrity first - Strong constraints and validation - Performance optimized - Strategic indexing and denormalization where needed - Scalability - Designed for millions of users and billions of messages - Privacy by default - Couple-level data isolation - Audit trail - Track all important changes

8.1 PostgreSQL Schema Design

8.1.1 Core Tables Overview

Database: `linkup_production`

-- Tables organized by domain

-- User & Authentication

```
CREATE TABLE users;  
CREATE TABLE auth_tokens;  
CREATE TABLE password_resets;
```

-- Couples & Relationships

```
CREATE TABLE couples;  
CREATE TABLE couple_invitations;
```

-- Messaging

```
CREATE TABLE messages;  
CREATE TABLE message_threads;  
CREATE TABLE message_reactions;  
CREATE TABLE message_edits;
```

```

-- Media
CREATE TABLE media;
CREATE TABLE media_albums;
CREATE TABLE media_tags;

-- Social
CREATE TABLE single_friends;
CREATE TABLE friend_invitations;
CREATE TABLE couple_circles;
CREATE TABLE circle_members;
CREATE TABLE circle_posts;
CREATE TABLE circle_events;

-- Creative
CREATE TABLE scribbles;
CREATE TABLE paintings;
CREATE TABLE custom_emojis;
CREATE TABLE soundboard_sounds;

-- Achievements & Streaks
CREATE TABLE achievements;
CREATE TABLE user_achievements;
CREATE TABLE photo_streaks;
CREATE TABLE streak_history;

-- Dates & Reminders
CREATE TABLE important_dates;
CREATE TABLE date_reminders;
CREATE TABLE date_celebrations;

-- Entertainment
CREATE TABLE watch_parties;
CREATE TABLE spotify_sessions;
CREATE TABLE shared_playlists;

-- Notifications
CREATE TABLE notifications;
CREATE TABLE notification_preferences;
CREATE TABLE devices;

-- System
CREATE TABLE audit_logs;
CREATE TABLE feature_flags;

```

8.1.2 User & Authentication Tables

Users Table:

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    -- Authentication  
    email VARCHAR(255) UNIQUE NOT NULL,  
    email_verified BOOLEAN DEFAULT FALSE,  
    email_verified_at TIMESTAMP,  
    password_hash VARCHAR(255) NOT NULL,  
  
    -- Profile  
    username VARCHAR(30) UNIQUE NOT NULL,  
    display_name VARCHAR(50) NOT NULL,  
    bio TEXT,  
    avatar_url TEXT,  
  
    -- Personal info  
    birthday DATE,  
    timezone VARCHAR(50) DEFAULT 'UTC',  
    language VARCHAR(10) DEFAULT 'en',  
  
    -- Relationship status  
    couple_id UUID REFERENCES couples(id) ON DELETE SET NULL,  
    partner_id UUID REFERENCES users(id) ON DELETE SET NULL,  
  
    -- Preferences (JSONB for flexibility)  
    preferences JSONB DEFAULT '{} '::jsonb,  
  
    -- Status  
    status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'suspended', 'deleted')),  
    last_seen_at TIMESTAMP,  
  
    -- Metadata  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    deleted_at TIMESTAMP  
);  
  
-- Indexes  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_username ON users(username);  
CREATE INDEX idx_users_couple_id ON users(couple_id);  
CREATE INDEX idx_users_status ON users(status) WHERE status = 'active';  
CREATE INDEX idx_users_created_at ON users(created_at);
```



```

-- Full-text search on display_name and username
CREATE INDEX idx_users_search ON users USING gin(
    to_tsvector('english', display_name || ' ' || username)
);

```

```

-- Trigger to update updated_at
CREATE TRIGGER update_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();

```

Auth Tokens Table:

```

CREATE TABLE auth_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Token data
    token_hash VARCHAR(255) NOT NULL UNIQUE,
    refresh_token_hash VARCHAR(255) UNIQUE,

    -- Device info
    device_id VARCHAR(255),
    device_type VARCHAR(20) CHECK (device_type IN ('web', 'ios', 'android')),
    device_name VARCHAR(100),

    -- Security
    ip_address INET,
    user_agent TEXT,

    -- Expiration
    expires_at TIMESTAMP NOT NULL,
    refresh_expires_at TIMESTAMP,

    -- Status
    is_active BOOLEAN DEFAULT TRUE,
    last_used_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_auth_tokens_user_id ON auth_tokens(user_id);
CREATE INDEX idx_auth_tokens_token_hash ON auth_tokens(token_hash);
CREATE INDEX idx_auth_tokens_expires_at ON auth_tokens(expires_at);
CREATE INDEX idx_auth_tokens_active ON auth_tokens(is_active) WHERE is_active = TRUE;

```

8.1.3 Couples & Relationships Tables

Couples Table:

```
CREATE TABLE couples (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Members  
  user1_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  user2_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  
  -- Relationship info  
  relationship_type VARCHAR(50) DEFAULT 'dating' CHECK (  
    relationship_type IN ('dating', 'engaged', 'married', 'partnered', 'other')  
  ),  
  started_at DATE,  
  
  -- Couple settings  
  settings JSONB DEFAULT '{} '::jsonb,  
  
  -- Privacy  
  is_private BOOLEAN DEFAULT FALSE,  
  
  -- Status  
  status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'paused', 'ended')),  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  ended_at TIMESTAMP,  
  
  -- Constraints  
  CONSTRAINT different_users CHECK (user1_id != user2_id),  
  CONSTRAINT unique_couple UNIQUE (user1_id, user2_id)  
);  
  
CREATE INDEX idx_couples_user1 ON couples(user1_id);  
CREATE INDEX idx_couples_user2 ON couples(user2_id);  
CREATE INDEX idx_couples_status ON couples(status) WHERE status = 'active';  
CREATE INDEX idx_couples_created_at ON couples(created_at);
```

Couple Invitations Table:

```
CREATE TABLE couple_invitations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Invitation details  
  inviter_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  invitee_email VARCHAR(255) NOT NULL,
```

```

invitee_id UUID REFERENCES users(id) ON DELETE CASCADE,

-- Invitation code
invitation_code VARCHAR(50) UNIQUE NOT NULL,

-- Message
message TEXT,

-- Status
status VARCHAR(20) DEFAULT 'pending' CHECK (
    status IN ('pending', 'accepted', 'declined', 'expired', 'cancelled')
),

-- Timestamps
expires_at TIMESTAMP NOT NULL,
accepted_at TIMESTAMP,
declined_at TIMESTAMP,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_invitations_code ON couple_invitations(invitation_code);
CREATE INDEX idx_invitations_inviter ON couple_invitations(inviter_id);
CREATE INDEX idx_invitations_invitee ON couple_invitations(invitee_id);
CREATE INDEX idx_invitations_status ON couple_invitations(status);

```

8.1.4 Messaging Tables

Messages Table:

```

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

-- Ownership
couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

-- Content
content TEXT NOT NULL,
type VARCHAR(20) DEFAULT 'text' CHECK (
    type IN ('text', 'photo', 'video', 'voice', 'file', 'system')
),

-- Rich content
metadata JSONB DEFAULT '{} '::jsonb,

-- Threading

```

```

reply_to_id UUID REFERENCES messages(id) ON DELETE SET NULL,
thread_id UUID REFERENCES message_threads(id) ON DELETE SET NULL,

-- Status
is_highlighted BOOLEAN DEFAULT FALSE,
is_edited BOOLEAN DEFAULT FALSE,
is_deleted BOOLEAN DEFAULT FALSE,
deleted_at TIMESTAMP,

-- Read status
read_by JSONB DEFAULT '[]'::jsonb, -- Array of {userId, readAt}

-- Timestamps
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_messages_couple_id ON messages(couple_id);
CREATE INDEX idx_messages_sender_id ON messages(sender_id);
CREATE INDEX idx_messages_created_at ON messages(created_at DESC);
CREATE INDEX idx_messages_thread_id ON messages(thread_id);
CREATE INDEX idx_messages_reply_to ON messages(reply_to_id);
CREATE INDEX idx_messages_highlighted ON messages(is_highlighted) WHERE is_highlighted = TRUE;

-- Composite index for couple messages by date
CREATE INDEX idx_messages_couple_date ON messages(couple_id, created_at DESC);

-- Full-text search on message content
CREATE INDEX idx_messages_content_search ON messages USING gin(to_tsvector('english', content));

-- Partial index for active messages only
CREATE INDEX idx_messages_active ON messages(couple_id, created_at DESC)
WHERE is_deleted = FALSE;

```

Message Threads Table:

```

CREATE TABLE message_threads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Thread info
  couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
  root_message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,

  -- Metadata
  title VARCHAR(255),
  message_count INTEGER DEFAULT 0,
  participant_ids UUID[] DEFAULT ARRAY[]::UUID[],

```

```

-- Status
is_archived BOOLEAN DEFAULT FALSE,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_message_at TIMESTAMP
);

CREATE INDEX idx_threads_couple_id ON message_threads(couple_id);
CREATE INDEX idx_threads_root_message ON message_threads(root_message_id);
CREATE INDEX idx_threads_last_message ON message_threads(last_message_at DESC);

Message Reactions Table:

CREATE TABLE message_reactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- References
  message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Reaction
  emoji VARCHAR(50) NOT NULL,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  -- Unique constraint: one reaction per user per message
  CONSTRAINT unique_user_message_reaction UNIQUE (message_id, user_id, emoji)
);

CREATE INDEX idx_reactions_message_id ON message_reactions(message_id);
CREATE INDEX idx_reactions_user_id ON message_reactions(user_id);

Message Edits Table (Audit Trail):

CREATE TABLE message_edits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,

  -- Previous content
  previous_content TEXT NOT NULL,
  previous_metadata JSONB,

  -- Editor
  edited_by UUID NOT NULL REFERENCES users(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
CREATE INDEX idx_edits_message_id ON message_edits(message_id);
```

8.1.5 Media Tables

Media Table:

```
CREATE TABLE media (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Ownership  
  couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,  
  uploader_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  
  -- Media details  
  type VARCHAR(20) NOT NULL CHECK (type IN ('photo', 'video', 'voice', 'file')),  
  filename VARCHAR(255) NOT NULL,  
  original_filename VARCHAR(255),  
  
  -- Storage  
  storage_key VARCHAR(500) NOT NULL UNIQUE, -- S3 key  
  storage_bucket VARCHAR(100) NOT NULL,  
  cdn_url TEXT,  
  
  -- File metadata  
  mime_type VARCHAR(100),  
  file_size BIGINT, -- bytes  
  duration INTEGER, -- seconds (for video/voice)  
  
  -- Dimensions (for photos/videos)  
  width INTEGER,  
  height INTEGER,  
  
  -- Processing status  
  processing_status VARCHAR(20) DEFAULT 'pending' CHECK (  
    processing_status IN ('pending', 'processing', 'completed', 'failed')  
  ),  
  
  -- Thumbnails and variants  
  thumbnails JSONB DEFAULT '{}'::jsonb, -- {small: url, medium: url, large: url}  
  variants JSONB DEFAULT '{}'::jsonb, -- {quality: url}  
  
  -- Metadata  
  metadata JSONB DEFAULT '{}'::jsonb, -- EXIF, location, etc.  
  
  -- Organization  
  album_id UUID REFERENCES media_albums(id) ON DELETE SET NULL,  
  tags TEXT[] DEFAULT ARRAY[]::TEXT[],
```

```

-- Photo streak
is_streak_photo BOOLEAN DEFAULT FALSE,
streak_date DATE,

-- Status
is_archived BOOLEAN DEFAULT FALSE,
is_deleted BOOLEAN DEFAULT FALSE,
deleted_at TIMESTAMP,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_media_couple_id ON media(couple_id);
CREATE INDEX idx_media_uploader_id ON media(uploader_id);
CREATE INDEX idx_media_type ON media(type);
CREATE INDEX idx_media_created_at ON media(created_at DESC);
CREATE INDEX idx_media_album_id ON media(album_id);
CREATE INDEX idx_media_storage_key ON media(storage_key);

-- Composite indexes
CREATE INDEX idx_media_couple_type_date ON media(couple_id, type, created_at DESC);
CREATE INDEX idx_media_streak ON media(couple_id, streak_date)
    WHERE is_streak_photo = TRUE;

-- Active media only
CREATE INDEX idx_media_active ON media(couple_id, created_at DESC)
    WHERE is_deleted = FALSE;

-- GIN index for tags
CREATE INDEX idx_media_tags ON media USING gin(tags);

Media Albums Table:

CREATE TABLE media_albums (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
    created_by UUID NOT NULL REFERENCES users(id),

-- Album details
name VARCHAR(255) NOT NULL,
description TEXT,
cover_media_id UUID REFERENCES media(id) ON DELETE SET NULL,

-- Settings
is_shared BOOLEAN DEFAULT TRUE,

```

```

is_auto BOOLEAN DEFAULT FALSE, -- Auto-generated album

-- Statistics
media_count INTEGER DEFAULT 0,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_albums_couple_id ON media_albums(couple_id);
CREATE INDEX idx_albums_created_by ON media_albums(created_by);

```

8.1.6 Social Tables

Single Friends Table:

```

CREATE TABLE single_friends (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Friendship details
  couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
  friend_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Who added them
  added_by UUID NOT NULL REFERENCES users(id),

  -- Permissions (JSONB for flexibility)
  permissions JSONB DEFAULT '{
    "view_photos": false,
    "view_videos": false,
    "view_messages": false,
    "view_achievements": true,
    "comment_on_posts": true
  }'::jsonb,

  -- Status
  status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'blocked', 'removed')),

  -- Metadata
  notes TEXT,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  -- Constraint: max 10 friends per couple (enforced in application)
  CONSTRAINT unique_couple_friend UNIQUE (couple_id, friend_user_id)
);

```



```

CREATE INDEX idx_single_friends_couple ON single_friends(couple_id);
CREATE INDEX idx_single_friends_user ON single_friends(friend_user_id);
CREATE INDEX idx_single_friends_status ON single_friends(status);

```

Couple Circles Table:

```

CREATE TABLE couple_circles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Circle details
  name VARCHAR(100) NOT NULL,
  description TEXT,
  cover_image_url TEXT,

  -- Created by which couple
  created_by_couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,

  -- Settings
  is_private BOOLEAN DEFAULT FALSE,
  max_members INTEGER DEFAULT 10, -- 2-10 couples

  -- Statistics
  member_count INTEGER DEFAULT 1,
  post_count INTEGER DEFAULT 0,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE INDEX idx_circles_created_by ON couple_circles(created_by_couple_id);
CREATE INDEX idx_circles_created_at ON couple_circles(created_at DESC);

```

Circle Members Table:

```

CREATE TABLE circle_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  circle_id UUID NOT NULL REFERENCES couple_circles(id) ON DELETE CASCADE,
  couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,

  -- Role
  role VARCHAR(20) DEFAULT 'member' CHECK (role IN ('admin', 'moderator', 'member')),

  -- Status
  status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('invited', 'active', 'left')),

  -- Timestamps
  joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  invited_at TIMESTAMP,

```

```

    left_at TIMESTAMP,

    CONSTRAINT unique_circle_couple UNIQUE (circle_id, couple_id)
);

CREATE INDEX idx_circle_members_circle ON circle_members(circle_id);
CREATE INDEX idx_circle_members_couple ON circle_members(couple_id);
CREATE INDEX idx_circle_members_status ON circle_members(status);

Circle Posts Table:

CREATE TABLE circle_posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    circle_id UUID NOT NULL REFERENCES couple_circles(id) ON DELETE CASCADE,
    author_couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
    author_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Content
    content TEXT NOT NULL,
    media_ids UUID[] DEFAULT ARRAY[]::UUID[],

    -- Type
    post_type VARCHAR(20) DEFAULT 'post' CHECK (
        post_type IN ('post', 'poll', 'event', 'announcement')
    ),

    -- Metadata
    metadata JSONB DEFAULT '{}'::jsonb, -- Poll data, event data, etc.

    -- Engagement
    like_count INTEGER DEFAULT 0,
    comment_count INTEGER DEFAULT 0,

    -- Status
    is_pinned BOOLEAN DEFAULT FALSE,
    is_deleted BOOLEAN DEFAULT FALSE,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP
);

CREATE INDEX idx_circle_posts_circle ON circle_posts(circle_id);
CREATE INDEX idx_circle_posts_author ON circle_posts(author_couple_id);
CREATE INDEX idx_circle_posts_created ON circle_posts(created_at DESC);
CREATE INDEX idx_circle_posts_active ON circle_posts(circle_id, created_at DESC)
    WHERE is_deleted = FALSE;

```

8.1.7 Achievements & Streaks Tables

Achievements Table (Definitions):

```
CREATE TABLE achievements (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    -- Achievement details  
    code VARCHAR(100) UNIQUE NOT NULL, -- 'first_message', 'photo_streak_7', etc.  
    category VARCHAR(50) NOT NULL CHECK (  
        category IN ('communication', 'media', 'social', 'creative', 'entertainment', 'mileston  
    ),  
  
    -- Display  
    name VARCHAR(255) NOT NULL,  
    description TEXT NOT NULL,  
    icon_url TEXT,  
  
    -- Requirements  
    requirements JSONB NOT NULL, -- Condition to unlock  
    points INTEGER DEFAULT 0,  
  
    -- Rarity  
    rarity VARCHAR(20) DEFAULT 'common' CHECK (  
        rarity IN ('common', 'uncommon', 'rare', 'epic', 'legendary')  
    ),  
  
    -- Availability  
    is_active BOOLEAN DEFAULT TRUE,  
    is_hidden BOOLEAN DEFAULT FALSE, -- Secret achievement  
  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_achievements_category ON achievements(category);  
CREATE INDEX idx_achievements_code ON achievements(code);
```

User Achievements Table:

```
CREATE TABLE user_achievements (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,  
    achievement_id UUID NOT NULL REFERENCES achievements(id) ON DELETE CASCADE,  
  
    -- Unlock details
```

```

unlocked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
progress INTEGER DEFAULT 100, -- Percentage of completion

-- Metadata
metadata JSONB DEFAULT '{}'::jsonb, -- Context of unlock

-- Visibility
is_showcased BOOLEAN DEFAULT FALSE,

CONSTRAINT unique_user_achievement UNIQUE (user_id, achievement_id)
);

CREATE INDEX idx_user_achievements_user ON user_achievements(user_id);
CREATE INDEX idx_user_achievements_couple ON user_achievements(couple_id);
CREATE INDEX idx_user_achievements_unlocked ON user_achievements(unlocked_at DESC);

```

Photo Streaks Table:

```

CREATE TABLE photo_streaks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,

    -- Current streak
    current_streak INTEGER DEFAULT 0,
    longest_streak INTEGER DEFAULT 0,

    -- Last photo
    last_photo_date DATE,
    last_photo_id UUID REFERENCES media(id),

    -- Freezes
    freezes_available INTEGER DEFAULT 2,
    freeze_history JSONB DEFAULT '[]'::jsonb,

    -- Recovery
    can_recover BOOLEAN DEFAULT FALSE,
    recovery_deadline TIMESTAMP,

    -- Statistics
    total_photos INTEGER DEFAULT 0,
    total_points INTEGER DEFAULT 0,

    -- Status
    status VARCHAR(20) DEFAULT 'active' CHECK (
        status IN ('active', 'broken', 'paused')
    ),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT one_streak_per_couple UNIQUE (couple_id)
);

CREATE INDEX idx_streaks_couple ON photo_streaks(couple_id);
CREATE INDEX idx_streaks_status ON photo_streaks(status);

Streak History Table:

CREATE TABLE streak_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    streak_id UUID NOT NULL REFERENCES photo_streaks(id) ON DELETE CASCADE,

    -- Event details
    event_type VARCHAR(50) NOT NULL CHECK (
        event_type IN ('photo_added', 'streak_broken', 'freeze_used', 'streak_recovered', 'mil
    ),

    -- Data
    streak_length INTEGER,
    photo_id UUID REFERENCES media(id),
    metadata JSONB DEFAULT '{}'::jsonb,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_streak_history_streak ON streak_history(streak_id);
CREATE INDEX idx_streak_history_type ON streak_history(event_type);
CREATE INDEX idx_streak_history_created ON streak_history(created_at DESC);

```

8.1.8 Important Dates Tables

Important Dates Table:

```

CREATE TABLE important_dates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    couple_id UUID NOT NULL REFERENCES couples(id) ON DELETE CASCADE,
    created_by UUID NOT NULL REFERENCES users(id),

    -- Date details
    type VARCHAR(50) NOT NULL CHECK (
        type IN ('anniversary', 'birthday', 'custom')
    ),
    name VARCHAR(255) NOT NULL,
    description TEXT,

```

```

-- Date
month INTEGER NOT NULL CHECK (month BETWEEN 1 AND 12),
day INTEGER NOT NULL CHECK (day BETWEEN 1 AND 31),
year INTEGER, -- Optional for recurring dates
recurring BOOLEAN DEFAULT TRUE,

-- Metadata
metadata JSONB DEFAULT '{}'::jsonb,

-- Reminder settings
reminder_enabled BOOLEAN DEFAULT TRUE,
reminder_settings JSONB DEFAULT '{
  "onDay": true,
  "oneDayBefore": true,
  "oneWeekBefore": true,
  "oneMonthBefore": false
}'::jsonb,

-- Status
is_active BOOLEAN DEFAULT TRUE,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_dates_couple ON important_dates(couple_id);
CREATE INDEX idx_dates_type ON important_dates(type);
CREATE INDEX idx_dates_month_day ON important_dates(month, day);
CREATE INDEX idx_dates_active ON important_dates(is_active) WHERE is_active = TRUE;

Date Celebrations Table:

CREATE TABLE date_celebrations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  date_id UUID NOT NULL REFERENCES important_dates(id) ON DELETE CASCADE,
  year INTEGER NOT NULL,

  -- Celebration details
  celebrated_at TIMESTAMP,

  -- Activities
  activities JSONB DEFAULT '{}'::jsonb,

  -- Memories
  photos UUID[] DEFAULT ARRAY[]::UUID[],
  notes TEXT,

```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT unique_date_year UNIQUE (date_id, year)
);

CREATE INDEX idx_celebrations_date ON date_celebrations(date_id);
CREATE INDEX idx_celebrations_year ON date_celebrations(year);

```

8.1.9 Notifications Table

Notifications Table:

```

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    couple_id UUID REFERENCES couples(id) ON DELETE CASCADE,

    -- Notification content
    type VARCHAR(100) NOT NULL,
    priority VARCHAR(20) DEFAULT 'normal' CHECK (
        priority IN ('urgent', 'high', 'normal', 'low')
    ),

    title VARCHAR(255) NOT NULL,
    body TEXT NOT NULL,

    -- Rich content
    image_url TEXT,
    icon_url TEXT,

    -- Action data
    action_type VARCHAR(50),
    action_data JSONB DEFAULT '{}::jsonb',

    -- Delivery status
    status VARCHAR(20) DEFAULT 'pending' CHECK (
        status IN ('pending', 'sent', 'delivered', 'failed', 'clicked', 'dismissed')
    ),

    sent_at TIMESTAMP,
    delivered_at TIMESTAMP,
    read_at TIMESTAMP,
    clicked_at TIMESTAMP,

    -- Metadata
    metadata JSONB DEFAULT '{}::jsonb',

```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        expires_at TIMESTAMP
    );

    CREATE INDEX idx_notifications_user ON notifications(user_id);
    CREATE INDEX idx_notifications_status ON notifications(status);
    CREATE INDEX idx_notifications_created ON notifications(created_at DESC);
    CREATE INDEX idx_notifications_user_unread ON notifications(user_id, read_at)
        WHERE read_at IS NULL;

    -- Partial index for pending notifications
    CREATE INDEX idx_notifications_pending ON notifications(user_id, created_at DESC)
        WHERE status = 'pending';

```

Notification Preferences Table:

```

CREATE TABLE notification_preferences (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Global settings
    enabled BOOLEAN DEFAULT TRUE,
    push_enabled BOOLEAN DEFAULT TRUE,
    email_enabled BOOLEAN DEFAULT FALSE,

    -- Category preferences
    preferences JSONB DEFAULT '{}'::jsonb,

    -- Quiet hours
    quiet_hours_enabled BOOLEAN DEFAULT FALSE,
    quiet_hours_start TIME,
    quiet_hours_end TIME,
    quiet_hours_timezone VARCHAR(50),

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT one_pref_per_user UNIQUE (user_id)
);

CREATE INDEX idx_notif_prefs_user ON notification_preferences(user_id);

```

Devices Table:

```

CREATE TABLE devices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

```



```

-- Device info
device_type VARCHAR(20) NOT NULL CHECK (device_type IN ('ios', 'android', 'web')),
device_name VARCHAR(100),
device_token VARCHAR(500) UNIQUE NOT NULL, -- Push notification token

-- Status
is_active BOOLEAN DEFAULT TRUE,
is_primary BOOLEAN DEFAULT FALSE,

-- Metadata
last_active_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_devices_user ON devices(user_id);
CREATE INDEX idx_devices_token ON devices(device_token);
CREATE INDEX idx_devices_active ON devices(is_active) WHERE is_active = TRUE;

```

8.2 Redis Schema Design

8.2.1 Cache Keys Structure

Naming Convention:

linkup:{resource}:{identifier}:{attribute}

Key Patterns:

```

// Session management
const REDIS_KEYS = {
  // User sessions
  userSession: (userId: string) => `linkup:session:${userId}`,
  userTokens: (userId: string) => `linkup:tokens:${userId}`,

  // Online status
  userOnline: (userId: string) => `linkup:online:${userId}`,
  coupleOnline: (coupleId: string) => `linkup:couple:${coupleId}:online`,

  // Real-time features
  typing: (coupleId: string, userId: string) => `linkup:typing:${coupleId}:${userId}`,
  presence: (userId: string) => `linkup:presence:${userId}`,

  // Caching
  userCache: (userId: string) => `linkup:cache:user:${userId}`,
  coupleCache: (coupleId: string) => `linkup:cache:couple:${coupleId}`,
  messagesCache: (coupleId: string, page: number) =>
    `linkup:cache:messages:${coupleId}:${page}`,

```

```

mediaCache: (coupleId: string, type: string) =>
    `linkup:cache:media:${coupleId}:${type}`,

// Rate limiting
rateLimit: (userId: string, action: string) =>
    `linkup:ratelimit:${userId}:${action}`,

// Temporary data
tempUpload: (uploadId: string) => `linkup:temp:upload:${uploadId}`,
verificationCode: (email: string) => `linkup:verify:${email}`,

// Counters
unreadCount: (userId: string) => `linkup:unread:${userId}`,

// Pub/Sub channels
coupleChannel: (coupleId: string) => `linkup:channel:couple:${coupleId}`,
userChannel: (userId: string) => `linkup:channel:user:${userId}`,
};

```

Data Structures:

```

// String: Simple key-value
// SET linkup:session:user123 '{"token":"...", "expiresAt":"..."}'
// EXPIRE linkup:session:user123 3600

// Hash: User data
// HSET linkup:cache:user:user123
//   "id" "user123"
//   "username" "alex"
//   "displayName" "Alex Smith"
//   "avatarUrl" "https://..."

// Set: Online users
// SADD linkup:online users "user123" "user456"

// Sorted Set: Leaderboard
// ZADD linkup:leaderboard:points 1500 "couple123" 2000 "couple456"

// List: Recent messages
// LPUSH linkup:recent:couple123 '{"id":"msg1", "content":"..."}'
// LTRIM linkup:recent:couple123 0 99 // Keep last 100

// TTL Examples
// Session: 24 hours
// Typing indicator: 5 seconds
// Verification code: 10 minutes
// Cache: 1 hour

```

8.3 MongoDB Schema Design

8.3.1 Media Metadata Collection

Collection: media_metadata

```
interface MediaMetadata {
  _id: ObjectId;
  mediaId: string;           // Reference to PostgreSQL media.id

  // File analysis
  analysis: {
    faces?: {
      count: number;
      positions: Array<{x: number, y: number, width: number, height: number}>;
      emotions?: string[];
    };
    colors: {
      dominant: string[];
      palette: Array<{color: string, percentage: number}>;
    };
    labels?: string[];       // AI-detected objects/scenes
    text?: string[];         // OCR text
  };

  // EXIF data
  exif: {
    camera?: {
      make: string;
      model: string;
      lens: string;
    };
    settings?: {
      iso: number;
      aperture: string;
      shutterSpeed: string;
      focalLength: string;
    };
    location?: {
      latitude: number;
      longitude: number;
      altitude: number;
      place?: string;
    };
    datetime: Date;
  };

  // Processing history
  processing: Array<
```

```

    type: 'thumbnail' | 'resize' | 'watermark' | 'filter';
    status: 'pending' | 'completed' | 'failed';
    params: object;
    completedAt?: Date;
  }>;

  // Search optimization
  searchText: string;           // Concatenated searchable text

  createdAt: Date;
  updatedAt: Date;
}

// Indexes
db.media_metadata.createIndex({ mediaId: 1 }, { unique: true });
db.media_metadata.createIndex({ "analysis.labels": 1 });
db.media_metadata.createIndex({ "exif.location": "2dsphere" }); // Geo queries
db.media_metadata.createIndex({ searchText: "text" }); // Full-text search

```

8.3.2 Activity Logs Collection

Collection: activity_logs

```

interface ActivityLog {
  _id: ObjectId;

  // Who and when
  userId: string;
  coupleId: string;
  timestamp: Date;

  // What happened
  action: string;           // 'message_sent', 'photo_uploaded', etc.
  category: 'messaging' | 'media' | 'social' | 'creative' | 'system';

  // Details
  details: {
    resourceType?: string;
    resourceId?: string;
    changes?: object;
    metadata?: object;
  };

  // Context
  context: {
    ipAddress?: string;
    userAgent?: string;
  };
}

```

```

    deviceType?: string;
    platform?: string;
  };

  // TTL - auto-delete after 90 days
  expiresAt: Date;
}

// Indexes
db.activity_logs.createIndex({ userId: 1, timestamp: -1 });
db.activity_logs.createIndex({ coupleId: 1, timestamp: -1 });
db.activity_logs.createIndex({ action: 1 });
db.activity_logs.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 });

```

8.4 Data Relationships

8.4.1 Entity Relationship Diagram

Users

- id (PK)
- email
- username
- couple_id (FK)

partner_id

user_id

Couples

- id (PK)
- user1_id (FK)
- user2_id (FK)

couple_id

Messages Media Streaks Circles Dates

Key Relationships:

1. User Couple (Many-to-One)
 - Each user belongs to one couple
 - Each couple has exactly two users
 2. Couple Messages (One-to-Many)
 - All messages belong to a couple
 - Cascade delete: Delete couple Delete all messages
 3. User Messages (One-to-Many)
 - Each message has one sender
 - Preserve messages if user deleted (set sender to null or keep)
 4. Message Message (Self-referential)
 - Threading: reply_to_id references parent message
 - Thread hierarchy support
 5. Couple Media (One-to-Many)
 - Media isolated per couple
 - Cascade delete with cleanup job for S3
 6. Couple Circles (Many-to-Many via circle_members)
 - A couple can be in multiple circles
 - A circle contains multiple couples
-

8.4.2 Cascading Rules

Delete Cascades:

```
-- When a couple is deleted
couples ON DELETE CASCADE →
  - messages
  - media
  - photo_streaks
  - circle_memberships
  - important_dates
  - achievements (couple-specific)

-- When a user is deleted
users ON DELETE CASCADE →
  - auth_tokens
  - notifications
  - devices

users ON DELETE SET NULL →
  - messages.sender_id (preserve message content)
  - media.uploader_id (preserve media)

-- Soft deletes (don't actually delete)
- users (status = 'deleted', deleted_at = NOW())
- couples (status = 'ended', ended_at = NOW())
- messages (is_deleted = TRUE, deleted_at = NOW())
```

8.5 Indexing Strategy

8.5.1 Performance-Critical Indexes

Query Patterns Indexes:

```
-- 1. Load couple's recent messages
-- Query: SELECT * FROM messages WHERE couple_id = ? ORDER BY created_at DESC LIMIT 50
CREATE INDEX idx_messages_couple_date ON messages(couple_id, created_at DESC)
      WHERE is_deleted = FALSE;

-- 2. Search messages by content
-- Query: SELECT * FROM messages WHERE couple_id = ? AND content ILIKE '%keyword%'
CREATE INDEX idx_messages_content_search ON messages
      USING gin(to_tsvector('english', content));

-- 3. Get user's unread notifications
-- Query: SELECT * FROM notifications WHERE user_id = ? AND read_at IS NULL
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, created_at DESC)
      WHERE read_at IS NULL;

-- 4. Load couple's photos
-- Query: SELECT * FROM media WHERE couple_id = ? AND type = 'photo' ORDER BY created_at DESC
CREATE INDEX idx_media_couple_type_date ON media(couple_id, type, created_at DESC)
      WHERE is_deleted = FALSE;

-- 5. Streak lookup
-- Query: SELECT * FROM photo_streaks WHERE couple_id = ?
CREATE UNIQUE INDEX idx_streaks_couple ON photo_streaks(couple_id);

-- 6. Find active circles for couple
-- Query: SELECT c.* FROM couple_circles c JOIN circle_members cm ...
CREATE INDEX idx_circle_members_couple_active ON circle_members(couple_id, status)
      WHERE status = 'active';
```

Composite Index Strategy:

```
-- Most selective column first
-- Columns used together in WHERE clauses
-- Consider query patterns

-- Good: Frequently filtered together
CREATE INDEX idx_media_couple_type_date ON media(couple_id, type, created_at DESC);

-- Bad: Individual indexes less efficient
-- CREATE INDEX idx_media_couple ON media(couple_id);
-- CREATE INDEX idx_media_type ON media(type);
-- CREATE INDEX idx_media_date ON media(created_at);
```

8.5.2 Partial Indexes

Active Records Only:

```
-- Only index active users (status filtering is common)
CREATE INDEX idx_users_active ON users(id, username, email)
    WHERE status = 'active';

-- Only index non-deleted messages
CREATE INDEX idx_messages_active ON messages(couple_id, created_at DESC)
    WHERE is_deleted = FALSE;

-- Only index pending notifications
CREATE INDEX idx_notifications_pending ON notifications(user_id, created_at)
    WHERE status = 'pending';
```

8.6 Database Migrations

8.6.1 Migration Strategy

Tools: - PostgreSQL: Flyway or Liquibase - Versioning: Sequential numbered migrations - Rollback:
Always provide down migrations

Migration Template:

```
-- Migration: V001__create_users_table.sql
-- Description: Create users table with authentication fields
-- Author: LinkUp Team
-- Date: 2026-01-20

-- Up Migration
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    -- ... rest of schema
);

CREATE INDEX idx_users_email ON users(email);

-- Down Migration (in separate file V001__create_users_table_down.sql)
DROP INDEX IF EXISTS idx_users_email;
DROP TABLE IF EXISTS users;
```

Migration Workflow:

```
# 1. Create migration
$ flyway migrate:create -n create_users_table
```



```

# 2. Edit migration file
$ vim migrations/V001__create_users_table.sql

# 3. Validate
$ flyway validate

# 4. Check pending
$ flyway info

# 5. Run migration
$ flyway migrate

# 6. Verify
$ flyway info
$ psql -c "SELECT * FROM users LIMIT 1;"

# 7. Rollback if needed
$ flyway undo

```

8.6.2 Zero-Downtime Migrations

Strategies:

1. Additive Changes (Safe)

```

-- Add new column with default
ALTER TABLE users ADD COLUMN avatar_url TEXT DEFAULT NULL;

-- Create new index concurrently
CREATE INDEX CONCURRENTLY idx_users_avatar ON users(avatar_url);

```

2. Column Renames (Multi-step)

```

-- Step 1: Add new column
ALTER TABLE users ADD COLUMN display_name VARCHAR(50);

-- Step 2: Copy data
UPDATE users SET display_name = name WHERE display_name IS NULL;

-- Step 3: Update application to use display_name
-- Deploy application

-- Step 4: Drop old column (later migration)
ALTER TABLE users DROP COLUMN name;

```

3. Table Splits (Shadow table)

```

-- Step 1: Create new table
CREATE TABLE user_profiles (

```

```

        user_id UUID PRIMARY KEY REFERENCES users(id),
        bio TEXT,
        avatar_url TEXT
    );

    -- Step 2: Copy existing data
    INSERT INTO user_profiles (user_id, bio, avatar_url)
    SELECT id, bio, avatar_url FROM users;

    -- Step 3: Update application
    -- Deploy application

    -- Step 4: Remove columns from users table
    ALTER TABLE users DROP COLUMN bio;
    ALTER TABLE users DROP COLUMN avatar_url;

```

8.7 Data Retention & Archival

8.7.1 Retention Policies

Data Lifecycle:

```

const RETENTION_POLICIES = {
  // Keep forever
  permanent: [
    'users',
    'couples',
    'messages',
    'media',
    'important_dates',
  ],

  // Auto-delete after period
  temporary: {
    'auth_tokens': '30 days',
    'password_resets': '24 hours',
    'notifications': '90 days',
    'activity_logs': '90 days (MongoDB TTL)',
    'temp_uploads': '24 hours',
  },

  // Archive to cold storage
  archival: {
    'old_media': {
      condition: 'not accessed in 2 years',
      action: 'move to Glacier',
    },
  },
};

```

```

        'ended_couples': {
            condition: 'status = ended AND ended_at < NOW() - 1 year',
            action: 'export to archive bucket',
        },
    },
};

```

Archival Process:

```

-- Create archive table
CREATE TABLE messages_archive (
    LIKE messages INCLUDING ALL
);

-- Move old messages (from ended couples)
INSERT INTO messages_archive
SELECT m.* FROM messages m
JOIN couples c ON m.couple_id = c.id
WHERE c.status = 'ended'
    AND c.ended_at < NOW() - INTERVAL '1 year';

-- Delete from main table
DELETE FROM messages m
USING couples c
WHERE m.couple_id = c.id
    AND c.status = 'ended'
    AND c.ended_at < NOW() - INTERVAL '1 year';

-- Vacuum to reclaim space
VACUUM ANALYZE messages;

```

8.8 Backup & Recovery

8.8.1 Backup Strategy

PostgreSQL Backups:

```

# Full database backup (daily)
pg_dump -Fc linkup_production > backups/linkup_$(date +%Y%m%d).dump

# Continuous archiving (WAL)
# Enable in postgresql.conf:
archive_mode = on
archive_command = 'aws s3 cp %p s3://linkup-backups/wal/%f'
wal_level = replica

# Point-in-time recovery capability

```

Backup Schedule:

```

const BACKUP_SCHEDULE = {
  postgresql: {
    full: 'daily at 2:00 AM UTC',
    incremental: 'continuous WAL archiving',
    retention: '30 days hot, 1 year cold (Glacier)',
  },

  redis: {
    snapshot: 'every 6 hours',
    aof: 'enabled (every second)',
    retention: '7 days',
  },

  mongodb: {
    snapshot: 'daily at 3:00 AM UTC',
    oplog: 'continuous',
    retention: '30 days',
  },

  s3Media: {
    versioning: 'enabled',
    replication: 'cross-region to eu-west-1',
    lifecycle: 'move to Glacier after 2 years',
  },
};

```

Chapter 8 Status: Complete

Summary: - PostgreSQL schema: 30+ tables covering users, couples, messaging, media, social, achievements, dates, notifications - Complete table definitions with proper constraints, foreign keys, and cascading rules - Strategic indexing: composite indexes, partial indexes, full-text search (GIN), geospatial (2dsphere) - Redis caching layer: session management, online presence, typing indicators, rate limiting - MongoDB: flexible storage for media metadata (EXIF, AI analysis), activity logs with TTL - S3/CDN: media storage with versioning and cross-region replication - Entity relationships with one-to-many, many-to-many, and self-referential patterns - Migration strategy: Flyway-based versioning with zero-downtime techniques - Data retention policies and archival process - Comprehensive backup strategy: daily full backups, continuous WAL archiving, 30-day retention

Next Chapter: Chapter 9: API Specifications & WebSocket Events

Chapter 9: API Specifications & WebSocket Events

Overview

LinkUps API architecture combines RESTful HTTP endpoints for CRUD operations with WebSocket connections for real-time features. This chapter provides comprehensive API documentation for all platform capabilities.

Architecture: - REST API - Stateless CRUD operations - WebSocket - Real-time messaging, presence, live features - GraphQL (Future) - Flexible queries for complex data needs - Webhooks - Event notifications for third-party integrations

Base URLs:

Production: `https://api.linkup.app/v1`
Staging: `https://api.staging.linkup.app/v1`
WebSocket: `wss://ws.linkup.app`

Design Principles: - RESTful conventions - Standard HTTP methods and status codes - Versioned - /v1, /v2 for backward compatibility - Stateless - JWT-based authentication - Rate limited - Prevent abuse - Paginated - Efficient data transfer - Well documented - OpenAPI/Swagger spec

9.1 Authentication & Authorization

9.1.1 Authentication Flow

JWT Token-Based Authentication:

```
// Token structure
interface JWPayload {
  userId: string;
  coupleId?: string;
  email: string;
  type: 'access' | 'refresh';
  iat: number; // Issued at
  exp: number; // Expires at
}
```

```
// Token lifetimes
const TOKEN_CONFIG = {
  accessToken: '15 minutes',
  refreshToken: '30 days',
};
```

Authentication Endpoints:

```
// POST /v1/auth/register
interface RegisterRequest {
  email: string;
  password: string;
  username: string;
```

```

    displayName: string;
    birthday?: string;
}

interface RegisterResponse {
    user: User;
    tokens: {
        accessToken: string;
        refreshToken: string;
        expiresIn: number;
    };
}

// POST /v1/auth/login
interface LoginRequest {
    email: string;
    password: string;
    deviceInfo?: {
        deviceType: 'web' | 'ios' | 'android';
        deviceName?: string;
        deviceToken?: string; // Push notification token
    };
}

interface LoginResponse {
    user: User;
    couple?: Couple;
    tokens: {
        accessToken: string;
        refreshToken: string;
        expiresIn: number;
    };
}

// POST /v1/auth/refresh
interface RefreshRequest {
    refreshToken: string;
}

interface RefreshResponse {
    accessToken: string;
    refreshToken: string;
    expiresIn: number;
}

// POST /v1/auth/logout
interface LogoutRequest {
    refreshToken?: string;
}

```

```

    allDevices?: boolean;
}

interface LogoutResponse {
    success: boolean;
}

// POST /v1/auth/forgot-password
interface ForgotPasswordRequest {
    email: string;
}

interface ForgotPasswordResponse {
    success: boolean;
    message: string;
}

// POST /v1/auth/reset-password
interface ResetPasswordRequest {
    token: string;
    newPassword: string;
}

interface ResetPasswordResponse {
    success: boolean;
}

// POST /v1/auth/verify-email
interface VerifyEmailRequest {
    token: string;
}

interface VerifyEmailResponse {
    success: boolean;
    user: User;
}

```

9.1.2 Authorization Headers

Request Headers:

```

Authorization: Bearer <access_token>
X-Device-ID: <unique_device_id>
X-Platform: web|ios|android
X-App-Version: 1.0.0
Content-Type: application/json

```

Authorization Middleware:

```
// Middleware checks
interface AuthContext {
  userId: string;
  coupleId?: string;
  user: User;
  couple?: Couple;
}

// Authorization levels
type Permission =
  | 'user:read'
  | 'user:write'
  | 'couple:read'
  | 'couple:write'
  | 'message:read'
  | 'message:write'
  | 'media:upload'
  | 'circle:admin';

// Route protection
const requireAuth = () => validateJWT();
const requireCouple = () => ensureUserHasCouple();
const requirePermission = (permission: Permission) => checkPermission();
```

9.2 User Management API

9.2.1 User Endpoints

```
// GET /v1/users/me
// Get current user profile
interface GetMeResponse {
  user: User;
  couple?: Couple;
  partner?: User;
  stats: {
    messageCount: number;
    photoCount: number;
    achievementCount: number;
    streakDays: number;
  };
}

// PATCH /v1/users/me
// Update current user
interface UpdateUserRequest {
```



```

    displayName?: string;
    bio?: string;
    avatar?: File; // Multipart form data
    birthday?: string;
    timezone?: string;
    preferences?: Partial<UserPreferences>;
}

interface UpdateUserResponse {
    user: User;
}

// GET /v1/users/:userId
// Get user by ID (only for connected users)
interface GetUserResponse {
    user: PublicUser; // Limited fields
    relationshipType: 'partner' | 'single_friend' | 'circle_member' | null;
}

// DELETE /v1/users/me
// Delete account
interface DeleteAccountRequest {
    password: string;
    confirmation: 'DELETE_MY_ACCOUNT';
}

interface DeleteAccountResponse {
    success: boolean;
    deletionScheduledAt: string;
}

// GET /v1/users/search
// Search users (for adding friends, etc.)
interface SearchUsersRequest {
    q: string; // Search query
    type?: 'all' | 'singles' | 'couples';
    limit?: number; // Default 20, max 100
    offset?: number;
}

interface SearchUsersResponse {
    users: PublicUser[];
    total: number;
    hasMore: boolean;
}

```

9.3 Couple Management API

9.3.1 Couple Endpoints

```
// POST /v1/couples/invite
// Invite partner to form a couple
interface InviteCoupleRequest {
    inviteeEmail: string;
    message?: string;
}

interface InviteCoupleResponse {
    invitation: CoupleInvitation;
    invitationCode: string;
}

// GET /v1/couples/invitations
// Get received invitations
interface GetInvitationsResponse {
    invitations: CoupleInvitation[];
}

// POST /v1/couples/invitations/:id/accept
// Accept couple invitation
interface AcceptInvitationResponse {
    couple: Couple;
    partner: User;
}

// POST /v1/couples/invitations/:id/decline
// Decline invitation
interface DeclineInvitationResponse {
    success: boolean;
}

// GET /v1/couples/me
// Get current couple details
interface GetCoupleResponse {
    couple: Couple;
    partner: User;
    stats: {
        daysTogether: number;
        messageCount: number;
        photoCount: number;
        achievementCount: number;
        streakDays: number;
    };
    milestones: Milestone[];
}
```

```

}

// PATCH /v1/couples/me
// Update couple settings
interface UpdateCoupleRequest {
    relationshipType?: 'dating' | 'engaged' | 'married' | 'partnered' | 'other';
    startedAt?: string;
    settings?: {
        isPrivate?: boolean;
        allowSingleFriends?: boolean;
        autoSharePhotos?: boolean;
    };
}

interface UpdateCoupleResponse {
    couple: Couple;
}

// DELETE /v1/couples/me
// End couple relationship
interface EndCoupleRequest {
    reason?: string;
    confirmation: 'END_RELATIONSHIP';
}

interface EndCoupleResponse {
    success: boolean;
    endedAt: string;
}

```

9.4 Messaging API

9.4.1 Message Endpoints

```

// GET /v1/messages
// Get messages for couple
interface GetMessagesRequest {
    limit?: number;           // Default 50, max 100
    before?: string;          // Message ID for pagination
    after?: string;           // Message ID for reverse pagination
    type?: 'text' | 'photo' | 'video' | 'voice' | 'file';
    search?: string;          // Full-text search
    highlighted?: boolean;    // Only highlighted messages
}

interface GetMessagesResponse {
    messages: Message[];
}

```

```

    hasMore: boolean;
    nextCursor?: string;
    prevCursor?: string;
}

// POST /v1/messages
// Send a message
interface SendMessageRequest {
    content: string;
    type: 'text' | 'photo' | 'video' | 'voice' | 'file';
    metadata?: {
        mediaId?: string;
        voiceDuration?: number;
        fileName?: string;
        fileSize?: number;
    };
    replyToId?: string;    // For threading
}

interface SendMessageResponse {
    message: Message;
}

// GET /v1/messages/:id
// Get single message
interface GetMessageResponse {
    message: Message;
    reactions: MessageReaction[];
    replyCount?: number;
}

// PATCH /v1/messages/:id
// Edit message (only text messages, within 15 minutes)
interface EditMessageRequest {
    content: string;
}

interface EditMessageResponse {
    message: Message;
    editedAt: string;
}

// DELETE /v1/messages/:id
// Delete message
interface DeleteMessageResponse {
    success: boolean;
    deletedAt: string;
}

```

```

// POST /v1/messages/:id/reactions
// Add reaction to message
interface AddReactionRequest {
  emoji: string;
}

interface AddReactionResponse {
  reaction: MessageReaction;
}

// DELETE /v1/messages/:id/reactions/:emoji
// Remove reaction
interface RemoveReactionResponse {
  success: boolean;
}

// POST /v1/messages/:id/highlight
// Toggle message highlight
interface HighlightMessageResponse {
  message: Message;
  isHighlighted: boolean;
}

// POST /v1/messages/:id/mark-read
// Mark message as read
interface MarkReadResponse {
  success: boolean;
  readAt: string;
}

```

9.5 Media API

9.5.1 Media Upload & Management

```

// POST /v1/media/upload
// Upload media file (multipart/form-data)
interface UploadMediaRequest {
  file: File;
  type: 'photo' | 'video' | 'voice' | 'file';
  albumId?: string;
  tags?: string[];
  isStreakPhoto?: boolean;
  metadata?: {
    location?: {
      latitude: number;
      longitude: number;
    };
  };
}

```

```

        };
        caption?: string;
    };
}

interface UploadMediaResponse {
    media: Media;
    uploadUrl?: string; // For client-side S3 upload
    uploadId?: string; // For tracking upload progress
}

// GET /v1/media
// Get couple's media
interface GetMediaRequest {
    type?: 'photo' | 'video' | 'voice' | 'file';
    albumId?: string;
    tags?: string[];
    limit?: number;
    offset?: number;
    sortBy?: 'created_at' | 'file_size';
    order?: 'asc' | 'desc';
    dateFrom?: string;
    dateTo?: string;
}

interface GetMediaResponse {
    media: Media[];
    total: number;
    hasMore: boolean;
}

// GET /v1/media/:id
// Get single media item
interface GetMediaItemResponse {
    media: Media;
    metadata: MediaMetadata;
    relatedMedia?: Media[]; // Same album or date
}

// PATCH /v1/media/:id
// Update media metadata
interface UpdateMediaRequest {
    tags?: string[];
    albumId?: string;
    metadata?: {
        caption?: string;
    };
}

```

```

interface UpdateMediaResponse {
    media: Media;
}

// DELETE /v1/media/:id
// Delete media
interface DeleteMediaResponse {
    success: boolean;
}

// POST /v1/media/albums
// Create album
interface CreateAlbumRequest {
    name: string;
    description?: string;
    coverMediaId?: string;
}

interface CreateAlbumResponse {
    album: MediaAlbum;
}

// GET /v1/media/albums
// Get all albums
interface GetAlbumsResponse {
    albums: MediaAlbum[];
}

// PATCH /v1/media/albums/:id
// Update album
interface UpdateAlbumRequest {
    name?: string;
    description?: string;
    coverMediaId?: string;
}

interface UpdateAlbumResponse {
    album: MediaAlbum;
}

// DELETE /v1/media/albums/:id
// Delete album (media stays, just removed from album)
interface DeleteAlbumResponse {
    success: boolean;
}

```

9.6 Social Features API

9.6.1 Single Friends Endpoints

```
// GET /v1/friends
// Get couple's single friends
interface GetSingleFriendsResponse {
    friends: SingleFriend[];
    count: number;
    maxAllowed: 10;
}

// POST /v1/friends/invite
// Invite single friend
interface InviteSingleFriendRequest {
    userId?: string;
    email?: string;
    permissions?: {
        viewPhotos: boolean;
        viewVideos: boolean;
        viewMessages: boolean;
        viewAchievements: boolean;
        commentOnPosts: boolean;
    };
}

interface InviteSingleFriendResponse {
    invitation: FriendInvitation;
}

// POST /v1/friends/:id/accept
// Accept friend invitation (as the invited user)
interface AcceptFriendInvitationResponse {
    friendship: SingleFriend;
    couple: PublicCouple;
}

// PATCH /v1/friends/:id
// Update friend permissions
interface UpdateFriendRequest {
    permissions: Partial<FriendPermissions>;
}

interface UpdateFriendResponse {
    friend: SingleFriend;
}

// DELETE /v1/friends/:id
```



```
// Remove single friend
interface RemoveFriendResponse {
    success: boolean;
}
```

9.6.2 Couple Circles Endpoints

```
// POST /v1/circles
// Create a couple circle
interface CreateCircleRequest {
    name: string;
    description?: string;
    coverImageUrl?: string;
    isPrivate: boolean;
    maxMembers?: number; // 2-10
}

interface CreateCircleResponse {
    circle: CoupleCircle;
}

// GET /v1/circles
// Get circles (created or joined)
interface GetCirclesRequest {
    type?: 'created' | 'joined' | 'all';
    limit?: number;
    offset?: number;
}

interface GetCirclesResponse {
    circles: CoupleCircle[];
    total: number;
}

// GET /v1/circles/:id
// Get circle details
interface GetCircleResponse {
    circle: CoupleCircle;
    members: CircleMember[];
    recentPosts: CirclePost[];
    stats: {
        memberCount: number;
        postCount: number;
        activityScore: number;
    };
}
```

```

// PATCH /v1/circles/:id
// Update circle (admin only)
interface UpdateCircleRequest {
    name?: string;
    description?: string;
    coverImageUrl?: string;
    isPrivate?: boolean;
}

interface UpdateCircleResponse {
    circle: CoupleCircle;
}

// DELETE /v1/circles/:id
// Delete circle (creator only)
interface DeleteCircleResponse {
    success: boolean;
}

// POST /v1/circles/:id/members
// Invite couple to circle
interface InviteToCircleRequest {
    coupleId: string;
    message?: string;
}

interface InviteToCircleResponse {
    invitation: CircleInvitation;
}

// POST /v1/circles/:id/join
// Join circle (if public or have invitation)
interface JoinCircleRequest {
    invitationCode?: string;
}

interface JoinCircleResponse {
    membership: CircleMember;
}

// DELETE /v1/circles/:id/leave
// Leave circle
interface LeaveCircleResponse {
    success: boolean;
}

// GET /v1/circles/:id/posts

```

```

// Get circle posts
interface GetCirclePostsRequest {
    limit?: number;
    before?: string;
    type?: 'post' | 'poll' | 'event' | 'announcement';
}

interface GetCirclePostsResponse {
    posts: CirclePost[];
    hasMore: boolean;
}

// POST /v1/circles/:id/posts
// Create circle post
interface CreateCirclePostRequest {
    content: string;
    mediaIds?: string[];
    type: 'post' | 'poll' | 'event' | 'announcement';
    metadata?: {
        poll?: {
            options: string[];
            multipleChoice: boolean;
            endsAt?: string;
        };
        event?: {
            title: string;
            datetime: string;
            location?: string;
        };
    };
}

interface CreateCirclePostResponse {
    post: CirclePost;
}

// POST /v1/circles/:circleId/posts/:postId/like
// Like a post
interface LikePostResponse {
    liked: boolean;
    likeCount: number;
}

// POST /v1/circles/:circleId/posts/:postId/comments
// Comment on post
interface CommentOnPostRequest {
    content: string;
}

```

```
interface CommentOnPostResponse {
    comment: PostComment;
}
```

9.7 Photo Streaks API

9.7.1 Streak Endpoints

```
// GET /v1/streaks
// Get couple's photo streak
interface GetStreakResponse {
    streak: PhotoStreak;
    recentPhotos: Media[];
    milestones: StreakMilestone[];
    canUploadToday: boolean;
}

// POST /v1/streaks/photo
// Upload streak photo
interface UploadStreakPhotoRequest {
    photo: File;
    caption?: string;
}

interface UploadStreakPhotoResponse {
    media: Media;
    streak: PhotoStreak;
    achievementUnlocked?: Achievement;
    milestoneReached?: StreakMilestone;
}

// POST /v1/streaks/freeze
// Use a freeze
interface FreezeStreakResponse {
    streak: PhotoStreak;
    freezeUsed: boolean;
    freezesRemaining: number;
}

// POST /v1/streaks/recover
// Attempt to recover broken streak
interface RecoverStreakRequest {
    photo: File;
}

interface RecoverStreakResponse {
```

```

    streak: PhotoStreak;
    recovered: boolean;
}

// GET /v1/streaks/history
// Get streak history
interface GetStreakHistoryRequest {
    limit?: number;
    offset?: number;
}

interface GetStreakHistoryResponse {
    history: StreakHistoryItem[];
    total: number;
}

// GET /v1/streaks/leaderboard
// Get global streak leaderboard
interface GetStreakLeaderboardRequest {
    timeframe?: 'all_time' | 'this_month' | 'this_year';
    limit?: number;
}

interface GetStreakLeaderboardResponse {
    leaderboard: Array<{
        rank: number;
        couple: PublicCouple;
        streakLength: number;
        lastPhotoDate: string;
    }>;
    currentCoupleRank?: number;
}

```

9.8 Achievements API

9.8.1 Achievement Endpoints

```

// GET /v1/achievements
// Get all available achievements
interface GetAchievementsRequest {
    category?: 'communication' | 'media' | 'social' | 'creative' | 'entertainment' | 'milestones';
    unlocked?: boolean;
}

interface GetAchievementsResponse {
    achievements: Achievement[];
    stats: {

```

```

        totalAvailable: number;
        totalUnlocked: number;
        totalPoints: number;
    };
}

// GET /v1/achievements/me
// Get user's unlocked achievements
interface GetMyAchievementsResponse {
    achievements: UserAchievement[];
    showcased: UserAchievement[];
    recentlyUnlocked: UserAchievement[];
}

// POST /v1/achievements/:id/showcase
// Toggle achievement showcase
interface ShowcaseAchievementResponse {
    achievement: UserAchievement;
    isShowcased: boolean;
}

// GET /v1/achievements/progress
// Get progress towards locked achievements
interface GetAchievementProgressResponse {
    progress: Array<{
        achievement: Achievement;
        currentProgress: number;
        requiredProgress: number;
        percentage: number;
    }>;
}

```

9.9 Important Dates API

9.9.1 Dates & Reminders Endpoints

```

// POST /v1/dates
// Create important date
interface CreateDateRequest {
    type: 'anniversary' | 'birthday' | 'custom';
    name: string;
    description?: string;
    month: number; // 1-12
    day: number; // 1-31
    year?: number; // Optional for recurring
    recurring: boolean;
    reminderSettings?: {

```

```

        onDay: boolean;
        oneDayBefore: boolean;
        oneWeekBefore: boolean;
        oneMonthBefore: boolean;
    };
}

interface CreateDateResponse {
    date: ImportantDate;
}

// GET /v1/dates
// Get all important dates
interface GetDatesRequest {
    type?: 'anniversary' | 'birthday' | 'custom';
    upcoming?: boolean;
}

interface GetDatesResponse {
    dates: ImportantDate[];
    upcoming: Array<{
        date: ImportantDate;
        daysUntil: number;
        nextOccurrence: string;
    }>;
}

// PATCH /v1/dates/:id
// Update date
interface UpdateDateRequest {
    name?: string;
    description?: string;
    reminderSettings?: Partial<ReminderSettings>;
}

interface UpdateDateResponse {
    date: ImportantDate;
}

// DELETE /v1/dates/:id
// Delete date
interface DeleteDateResponse {
    success: boolean;
}

// POST /v1/dates/:id/celebrate
// Record celebration
interface CelebrateDateRequest {

```

```

    notes?: string;
    photoIds?: string[];
    activities?: string[];
}

interface CelebrateDateResponse {
    celebration: DateCelebration;
}

```

9.10 Creative Features API

9.10.1 Scribble & Painting Endpoints

```

// POST /v1/creative/scribbles
// Send scribble
interface SendScribbleRequest {
    strokes: Array<{
        points: Array<{x: number, y: number}>;
        color: string;
        width: number;
    }>;
    backgroundColor?: string;
}

interface SendScribbleResponse {
    scribble: Scribble;
    message: Message; // Converted to message
}

// POST /v1/creative/paintings
// Create collaborative painting
interface CreatePaintingRequest {
    title?: string;
    canvasSize: {width: number, height: number};
    backgroundColor?: string;
}

interface CreatePaintingResponse {
    painting: Painting;
    sessionId: string; // For WebSocket connection
}

// GET /v1/creative/paintings
// Get couple's paintings
interface GetPaintingsResponse {
    paintings: Painting[];
    total: number;
}

```



```

}

// GET /v1/creative/paintings/:id
// Get painting details
interface GetPaintingResponse {
    painting: Painting;
    strokes: PaintingStroke[];
    authors: User[];
}

// POST /v1/creative/emojis
// Create custom emoji
interface CreateEmojiRequest {
    name: string;
    image: File;
    category?: string;
}

interface CreateEmojiResponse {
    emoji: CustomEmoji;
}

// GET /v1/creative/emojis
// Get custom emojis
interface GetEmojisResponse {
    emojis: CustomEmoji[];
}

// POST /v1/creative/soundboard
// Add soundboard sound
interface AddSoundboardSoundRequest {
    name: string;
    audio: File;
    category?: string;
    emoji?: string;
}

interface AddSoundboardSoundResponse {
    sound: SoundboardSound;
}

// GET /v1/creative/soundboard
// Get soundboard sounds
interface GetSoundboardResponse {
    sounds: SoundboardSound[];
}

```

9.11 Notifications API

9.11.1 Notification Endpoints

```
// GET /v1/notifications
// Get user notifications
interface GetNotificationsRequest {
    status?: 'unread' | 'read' | 'all';
    type?: string;
    limit?: number;
    offset?: number;
}

interface GetNotificationsResponse {
    notifications: Notification[];
    unreadCount: number;
    total: number;
}

// POST /v1/notifications/:id/read
// Mark notification as read
interface MarkNotificationReadResponse {
    success: boolean;
}

// POST /v1/notifications/read-all
// Mark all as read
interface MarkAllReadResponse {
    success: boolean;
    markedCount: number;
}

// GET /v1/notifications/preferences
// Get notification preferences
interface GetNotificationPreferencesResponse {
    preferences: NotificationPreferences;
}

// PATCH /v1/notifications/preferences
// Update notification preferences
interface UpdateNotificationPreferencesRequest {
    enabled?: boolean;
    pushEnabled?: boolean;
    emailEnabled?: boolean;
    preferences?: {
        messages?: boolean;
        reactions?: boolean;
        achievements?: boolean;
    };
}
```

```

    streaks?: boolean;
    dates?: boolean;
    circles?: boolean;
  };
  quietHours?: {
    enabled: boolean;
    start: string; // HH:mm
    end: string;   // HH:mm
    timezone: string;
  };
}

interface UpdateNotificationPreferencesResponse {
  preferences: NotificationPreferences;
}

// POST /v1/notifications/devices
// Register device for push notifications
interface RegisterDeviceRequest {
  deviceType: 'ios' | 'android' | 'web';
  deviceToken: string;
  deviceName?: string;
}

interface RegisterDeviceResponse {
  device: Device;
}

// DELETE /v1/notifications/devices/:id
// Unregister device
interface UnregisterDeviceResponse {
  success: boolean;
}

```

9.12 WebSocket Events

9.12.1 WebSocket Connection

Connection Flow:

```

// 1. Connect to WebSocket server
const ws = new WebSocket('wss://ws.linkup.app');

// 2. Authenticate
ws.send(JSON.stringify({
  type: 'auth',
  token: accessToken,

```

```

    }));

    // 3. Listen for auth confirmation
    ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      if (data.type === 'auth:success') {
        console.log('Connected as', data.userId);
      }
    };

    // 4. Subscribe to channels
    ws.send(JSON.stringify({
      type: 'subscribe',
      channels: ['couple:123', 'user:456'],
    }));

```

9.12.2 Message Events

Real-time Messaging:

```

// Client → Server: Send message
interface WSMMessageSend {
  type: 'message:send';
  data: {
    content: string;
    messageType: 'text' | 'photo' | 'video' | 'voice';
    replyToId?: string;
    metadata?: object;
  };
}

// Server → Client: Message received
interface WSMMessageReceived {
  type: 'message:received';
  data: {
    message: Message;
    sender: User;
  };
}

// Server → Client: Message edited
interface WSMMessageEdited {
  type: 'message:edited';
  data: {
    messageId: string;
    content: string;
    editedAt: string;
  };
}

```

```

};
}

// Server → Client: Message deleted
interface WSMMessageDeleted {
  type: 'message:deleted';
  data: {
    messageId: string;
    deletedAt: string;
  };
}

// Client → Server: Typing indicator
interface WSTypingStart {
  type: 'typing:start';
  data: {};
}

interface WSTypingStop {
  type: 'typing:stop';
  data: {};
}

// Server → Client: Partner typing
interface WSPartnerTyping {
  type: 'partner:typing';
  data: {
    userId: string;
    isTyping: boolean;
  };
}

// Server → Client: Message read
interface WSMMessageRead {
  type: 'message:read';
  data: {
    messageId: string;
    userId: string;
    readAt: string;
  };
}

// Client → Server: Add reaction
interface WSReactionAdd {
  type: 'reaction:add';
  data: {
    messageId: string;
    emoji: string;
  };
}

```

```

    };
}

// Server → Client: Reaction added
interface WSRreactionAdded {
    type: 'reaction:added';
    data: {
        messageId: string;
        reaction: MessageReaction;
    };
}

```

9.12.3 Presence Events

Online Status:

```

// Server → Client: Partner online
interface WSPartnerOnline {
    type: 'partner:online';
    data: {
        userId: string;
        onlineSince: string;
    };
}

// Server → Client: Partner offline
interface WSPartnerOffline {
    type: 'partner:offline';
    data: {
        userId: string;
        lastSeen: string;
    };
}

// Client → Server: Update presence
interface WSPresenceUpdate {
    type: 'presence:update';
    data: {
        status: 'online' | 'away' | 'busy';
        customStatus?: string;
    };
}

// Server → Client: Presence changed
interface WSPresenceChanged {
    type: 'presence:changed';
    data: {

```

```

        userId: string;
        status: string;
        customStatus?: string;
    };
}

```

9.12.4 Creative Session Events

Collaborative Painting:

// Client → Server: Join painting session

```

interface WSPaintingJoin {
    type: 'painting:join';
    data: {
        paintingId: string;
    };
}

```

// Server → Client: Session joined

```

interface WSPaintingJoined {
    type: 'painting:joined';
    data: {
        paintingId: string;
        participants: User[];
        strokes: PaintingStroke[];
    };
}

```

// Client → Server: Add stroke

```

interface WSPaintingStroke {
    type: 'painting:stroke';
    data: {
        paintingId: string;
        stroke: {
            points: Array<{x: number, y: number}>;
            color: string;
            width: number;
            tool: 'brush' | 'eraser' | 'fill';
        };
    };
}

```

// Server → Client: Stroke added

```

interface WSPaintingStrokeAdded {
    type: 'painting:stroke:added';
    data: {
        paintingId: string;
    };
}

```

```

        stroke: PaintingStroke;
        userId: string;
    };
}

// Client → Server: Undo
interface WSPaintingUndo {
    type: 'painting:undo';
    data: {
        paintingId: string;
    };
}

// Server → Client: Stroke undone
interface WSPaintingUndone {
    type: 'painting:undone';
    data: {
        paintingId: string;
        strokeId: string;
    };
}

```

9.12.5 Scribble Events

Real-time Scribble:

```

// Client → Server: Scribble stroke (real-time)
interface WSScribbleStroke {
    type: 'scribble:stroke';
    data: {
        points: Array<{x: number, y: number}>;
        color: string;
        width: number;
    };
}

// Server → Client: Partner scribbling
interface WSScribbleReceived {
    type: 'scribble:received';
    data: {
        userId: string;
        points: Array<{x: number, y: number}>;
        color: string;
        width: number;
    };
}

```



```

// Client → Server: Complete scribble
interface WSScribbleComplete {
  type: 'scribble:complete';
  data: {
    strokes: Array<{
      points: Array<{x: number, y: number}>;
      color: string;
      width: number;
    }>;
  };
}

```

```

// Server → Client: Scribble saved
interface WSScribbleSaved {
  type: 'scribble:saved';
  data: {
    scribble: Scribble;
    message: Message;
  };
}

```

9.12.6 Media Events

Media Upload Progress:

```

// Server → Client: Upload progress
interface WSMediaUploadProgress {
  type: 'media:upload:progress';
  data: {
    uploadId: string;
    progress: number; // 0-100
    bytesUploaded: number;
    totalBytes: number;
  };
}

```

```

// Server → Client: Upload complete
interface WSMediaUploadComplete {
  type: 'media:upload:complete';
  data: {
    uploadId: string;
    media: Media;
  };
}

```

```

// Server → Client: Media processing complete
interface WSMediaProcessingComplete {

```

```

    type: 'media:processing:complete';
    data: {
      mediaId: string;
      thumbnails: object;
      variants: object;
    };
  }
}

```

9.12.7 Streak Events

Photo Streak Updates:

```

// Server → Client: Streak updated
interface WSSStreakUpdated {
  type: 'streak:updated';
  data: {
    streak: PhotoStreak;
    photo: Media;
    milestoneReached?: StreakMilestone;
  };
}

```

```

// Server → Client: Streak warning
interface WSSStreakWarning {
  type: 'streak:warning';
  data: {
    streak: PhotoStreak;
    hoursRemaining: number;
  };
}

```

```

// Server → Client: Streak broken
interface WSSStreakBroken {
  type: 'streak:broken';
  data: {
    streak: PhotoStreak;
    previousLength: number;
    canRecover: boolean;
    recoveryDeadline?: string;
  };
}

```

```

// Server → Client: Freeze used
interface WSSStreakFrozen {
  type: 'streak:frozen';
  data: {
    streak: PhotoStreak;
  };
}

```

```

    freezesRemaining: number;
};
}

```

9.12.8 Achievement Events

Achievement Unlocked:

```

// Server → Client: Achievement unlocked
interface WSAchievementUnlocked {
  type: 'achievement:unlocked';
  data: {
    achievement: Achievement;
    userAchievement: UserAchievement;
    points: number;
  };
}

```

```

// Server → Client: Progress updated
interface WSAchievementProgress {
  type: 'achievement:progress';
  data: {
    achievementId: string;
    progress: number;
    required: number;
    percentage: number;
  };
}

```

9.12.9 Circle Events

Circle Activity:

```

// Server → Client: New circle post
interface WSCirclePost {
  type: 'circle:post';
  data: {
    circleId: string;
    post: CirclePost;
    author: User;
  };
}

```

```

// Server → Client: Post liked
interface WSCirclePostLiked {
  type: 'circle:post:liked';
}

```

```

data: {
  circleId: string;
  postId: string;
  userId: string;
  likeCount: number;
};
}

// Server → Client: New comment
interface WSCircleComment {
  type: 'circle:comment';
  data: {
    circleId: string;
    postId: string;
    comment: PostComment;
    author: User;
  };
}

// Server → Client: Member joined
interface WSCircleMemberJoined {
  type: 'circle:member:joined';
  data: {
    circleId: string;
    member: CircleMember;
    couple: PublicCouple;
  };
}

```

9.13 Error Handling

9.13.1 HTTP Error Responses

Standard Error Format:

```

interface APIError {
  error: {
    code: string;           // Machine-readable error code
    message: string;        // Human-readable message
    details?: object;       // Additional error context
    field?: string;         // Field causing validation error
    timestamp: string;
  };
}

// Example errors
const ERROR_CODES = {

```

```

// Authentication (401)
UNAUTHORIZED: 'auth/unauthorized',
INVALID_TOKEN: 'auth/invalid-token',
TOKEN_EXPIRED: 'auth/token-expired',

// Authorization (403)
FORBIDDEN: 'auth/forbidden',
NO_COUPLE: 'auth/no-couple',
NOT_IN_CIRCLE: 'auth/not-in-circle',

// Validation (400)
VALIDATION_ERROR: 'validation/failed',
INVALID_EMAIL: 'validation/invalid-email',
WEAK_PASSWORD: 'validation/weak-password',

// Not Found (404)
USER_NOT_FOUND: 'resource/user-not-found',
MESSAGE_NOT_FOUND: 'resource/message-not-found',
MEDIA_NOT_FOUND: 'resource/media-not-found',

// Conflict (409)
EMAIL_TAKEN: 'conflict/email-taken',
USERNAME_TAKEN: 'conflict/username-taken',
ALREADY_IN_COUPLE: 'conflict/already-in-couple',

// Rate Limiting (429)
RATE_LIMIT_EXCEEDED: 'rate-limit/exceeded',

// Server Error (500)
INTERNAL_ERROR: 'server/internal-error',
DATABASE_ERROR: 'server/database-error',
};

```

HTTP Status Codes:

```

// Success
200 OK                // Successful GET, PATCH, DELETE
201 Created           // Successful POST
204 No Content        // Successful DELETE with no response

// Client Errors
400 Bad Request       // Validation failed
401 Unauthorized      // Not authenticated
403 Forbidden         // Not authorized
404 Not Found         // Resource doesn't exist
409 Conflict          // Resource conflict (duplicate)
422 Unprocessable     // Valid syntax but semantically incorrect
429 Too Many Requests // Rate limit exceeded

```

```
// Server Errors
500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
```

9.13.2 WebSocket Error Events

Error Handling:

```
// Server → Client: Error
interface WSError {
  type: 'error';
  data: {
    code: string;
    message: string;
    originalEvent?: string; // Event that caused error
  };
}

// Connection errors
interface WSConnectionError {
  type: 'connection:error';
  data: {
    reason: 'auth_failed' | 'timeout' | 'server_error';
    message: string;
  };
}

// Reconnection
interface WSReconnecting {
  type: 'connection:reconnecting';
  data: {
    attempt: number;
    maxAttempts: number;
  };
}

interface WSReconnected {
  type: 'connection:reconnected';
  data: {
    sessionId: string;
  };
}
```

9.14 Rate Limiting

9.14.1 Rate Limit Rules

Per-Endpoint Limits:

```
const RATE_LIMITS = {
  // Authentication
  'POST /v1/auth/login': {
    window: '15 minutes',
    max: 5,
  },
  'POST /v1/auth/register': {
    window: '1 hour',
    max: 3,
  },

  // Messaging
  'POST /v1/messages': {
    window: '1 minute',
    max: 30,
  },
  'GET /v1/messages': {
    window: '1 minute',
    max: 60,
  },

  // Media upload
  'POST /v1/media/upload': {
    window: '1 hour',
    max: 100,
    sizeLimit: '100MB per hour',
  },

  // API calls (general)
  'global': {
    window: '1 minute',
    max: 100,
  },

  // WebSocket events
  'ws:message:send': {
    window: '1 minute',
    max: 60,
  },
  'ws:typing': {
    window: '10 seconds',
    max: 10,
  },
}
```

```
};
```

Rate Limit Headers:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1642694400
Retry-After: 60
```

9.15 Pagination

9.15.1 Cursor-Based Pagination

Request:

```
interface PaginationRequest {
    limit?: number;           // Items per page (default 20, max 100)
    cursor?: string;         // Opaque cursor for next page
    order?: 'asc' | 'desc';  // Sort order
}

// Example: GET /v1/messages?limit=50&cursor=eyJpZCI6IjEyMyJ9&order=desc
```

Response:

```
interface PaginatedResponse<T> {
    data: T[];
    pagination: {
        nextCursor?: string;
        prevCursor?: string;
        hasMore: boolean;
        total?: number;      // Total count (expensive, optional)
    };
}
```

9.16 Webhooks (Future)

9.16.1 Webhook Events

Third-party Integration:

```
// Webhook payload structure
interface WebhookPayload {
    id: string;
    type: string;
    timestamp: string;
    data: object;
    signature: string; // HMAC signature for verification
}
```



```

// Event types
const WEBHOOK_EVENTS = [
  'message.sent',
  'media.uploaded',
  'achievement.unlocked',
  'streak.updated',
  'circle.post.created',
];

// Webhook configuration
interface WebhookConfig {
  url: string;
  events: string[];
  secret: string;
  active: boolean;
}

```

Chapter 9 Status: Complete

Summary: - REST API: 100+ endpoints covering authentication, users, couples, messaging, media, social features, streaks, achievements, dates, creative tools, and notifications - WebSocket Events: Real-time messaging, typing indicators, presence, collaborative painting, scribbles, media uploads, streaks, achievements, and circle activity - Authentication: JWT-based with access/refresh tokens, device management, and password reset flow - Authorization: Permission-based access control for couple data, single friends, and circle memberships - Error Handling: Standardized error responses with machine-readable codes and proper HTTP status codes - Rate Limiting: Per-endpoint limits to prevent abuse (5 login attempts/15min, 30 messages/min, 100 media/hour) - Pagination: Cursor-based pagination for efficient data retrieval - Complete TypeScript interfaces for all requests and responses - WebSocket connection flow with authentication, subscriptions, and reconnection handling

Next Chapter: Chapter 10: UI/UX Design System & Pages

Chapter 10: UI/UX Design System & Pages

Overview

LinkUps design system establishes a cohesive visual language centered on artistic expression and emotional connection. The interface design balances intimacy with functionality, creating an environment optimized for romantic communication.

This chapter covers: - Design Philosophy - Core principles and visual direction - Design Tokens - Colors, typography, spacing, shadows - Component Library - Reusable UI components for all platforms - Page Specifications - Complete screens with layouts and interactions - Animations & Transitions - Motion design patterns - Responsive Design - Adaptive layouts across devices - Accessibility - WCAG 2.1 AA compliance

Design Differentiators: - Visual-first approach prioritizing aesthetic quality - Expressive interaction patterns with meaningful animations - Extensive customization supporting couple personalization - Gamification designed to encourage without manipulation

10.1 Design Philosophy & Principles

10.1.1 Core Design Principles

1. Intimate by Default - Privacy-first design language - Warm color palette optimized for emotional connection - Generous whitespace for focused experience - Personalization through custom emojis and nicknames
 2. Artistically Expressive - Visual design elements that support creative expression - Creative tools given prominence in interface hierarchy - Support for visual storytelling and memory preservation - Flexible aesthetic framework for couple personalization
 3. Interactive Engagement - Purposeful micro-interactions that reinforce actions - Haptic feedback for tactile confirmation - Fluid animations following natural motion principles - Optional audio feedback for key interactions
 4. Positive Reinforcement - Constructive language throughout interface - Non-intrusive notification patterns - Achievement celebration without pressure mechanics - Recovery mechanisms for missed activities
 5. Cross-Platform Consistency - Unified design language across web, iOS, and Android - Platform-native patterns where appropriate - Seamless state synchronization across devices
-

10.1.2 Visual Identity

Brand Personality: - Warm - Approachable and comfortable interface design - Playful - Engaging without sacrificing maturity - Sophisticated - Polished and refined aesthetic - Authentic - Transparent and trustworthy communication - Creative - Encourages personal expression

Design Reference Points:

- Pair - Interface simplicity and privacy focus
 - Locket Widget - Instant photo sharing patterns
 - Notion - Information architecture and organization
 - Linear - Animation fluidity and timing
 - Stripe - Color application and typography hierarchy
 - Between - Personal timeline and memory presentation
-

10.2 Design Tokens

10.2.1 Color System

Primary Palette:

```

const colors = {
  // Brand Colors
  primary: {
    50: '#FFF1F2', // Lightest pink
    100: '#FFE4E6',
    200: '#FECDD3',
    300: '#FDA4AF',
    400: '#FB7185', // Light coral pink
    500: '#F43F5E', // Primary brand color
    600: '#E11D48', // Default interactive state
    700: '#BE123C',
    800: '#9F1239',
    900: '#881337', // Darkest
  },

  // Secondary (Warm Purple)
  secondary: {
    50: '#FAF5FF',
    100: '#F3E8FF',
    200: '#E9D5FF',
    300: '#D8B4FE',
    400: '#C084FC',
    500: '#A855F7', // Secondary brand color
    600: '#9333EA',
    700: '#7E22CE',
    800: '#6B21A8',
    900: '#581C87',
  },

  // Accent (Warm Orange)
  accent: {
    50: '#FFF7ED',
    100: '#FFEDD5',
    200: '#FED7AA',
    300: '#FDBA74',
    400: '#FB923C',
    500: '#F97316', // Accent color
    600: '#EA580C',
    700: '#C2410C',
    800: '#9A3412',
    900: '#7C2D12',
  },

  // Neutrals (Warm Gray)
  neutral: {
    0: '#FFFFFF', // Pure white
    50: '#FAFAF9', // Background
    100: '#F5F5F4', // Light background
  }
}

```

```

200: '#E7E5E4', // Border light
300: '#D6D3D1', // Border
400: '#A8A29E', // Text muted
500: '#78716C', // Text secondary
600: '#57534E', // Text primary
700: '#44403C', // Text emphasis
800: '#292524', // Dark background
900: '#1C1917', // Darkest
950: '#0C0A09', // Pure black alternative
},

// Semantic Colors
success: {
  light: '#BBF7D0',
  main: '#22C55E',
  dark: '#15803D',
},

warning: {
  light: '#FED7AA',
  main: '#F59E0B',
  dark: '#B45309',
},

error: {
  light: '#FECACA',
  main: '#EF4444',
  dark: '#B91C1C',
},

info: {
  light: '#BFDDBFE',
  main: '#3B82F6',
  dark: '#1E40AF',
},

// Special Use
streak: {
  active: '#F59E0B', // Warm orange
  warning: '#EF4444', // Red
  frozen: '#3B82F6', // Blue
},

achievement: {
  common: '#78716C',
  uncommon: '#22C55E',
  rare: '#3B82F6',
  epic: '#A855F7',
},

```

```

    legendary: '#F59E0B',
  },
};

```

Dark Mode Palette:

```

const darkColors = {
  primary: {
    // Adjusted for dark backgrounds
    500: '#FB7185', // Lighter pink for visibility
    600: '#F43F5E',
  },

  neutral: {
    0:   '#0C0A09', // Dark background
    50:  '#1C1917',
    100: '#292524',
    200: '#44403C',
    300: '#57534E',
    400: '#78716C',
    500: '#A8A29E',
    600: '#D6D3D1',
    700: '#E7E5E4',
    800: '#F5F5F4',
    900: '#FAFAF9',
    950: '#FFFFFF',
  },

  // Background surfaces
  surface: {
    base:      '#0C0A09',
    elevated:  '#1C1917',
    overlay:   '#292524',
  },
};

```

10.2.2 Typography

Font Families:

```

const typography = {
  fontFamily: {
    // Primary - Clean, modern sans-serif
    sans: [
      'Inter',
      '-apple-system',
      'BlinkMacSystemFont',
      'Segoe UI',
    ],
  },
};

```

```

    'Roboto',
    'sans-serif',
].join(', '),

// Display - For headings and emphasis
display: [
    'Cal Sans',
    'Inter',
    'sans-serif',
].join(', '),

// Monospace - For code and data
mono: [
    'JetBrains Mono',
    'Menlo',
    'Monaco',
    'Courier New',
    'monospace',
].join(', '),
},

// Font Sizes
fontSize: {
    xs: '0.75rem',    // 12px
    sm: '0.875rem',   // 14px
    base: '1rem',      // 16px
    lg: '1.125rem',    // 18px
    xl: '1.25rem',     // 20px
    '2xl': '1.5rem',   // 24px
    '3xl': '1.875rem', // 30px
    '4xl': '2.25rem',  // 36px
    '5xl': '3rem',     // 48px
    '6xl': '3.75rem',  // 60px
    '7xl': '4.5rem',   // 72px
},

// Font Weights
fontWeight: {
    thin: 100,
    extralight: 200,
    light: 300,
    normal: 400,
    medium: 500,
    semibold: 600,
    bold: 700,
    extrabold: 800,
    black: 900,
},

```

```

// Line Heights
lineHeight: {
  none: 1,
  tight: 1.25,
  snug: 1.375,
  normal: 1.5,
  relaxed: 1.625,
  loose: 2,
},

// Letter Spacing
letterSpacing: {
  tighter: '-0.05em',
  tight: '-0.025em',
  normal: '0',
  wide: '0.025em',
  wider: '0.05em',
  widest: '0.1em',
},
};

// Typography Scale
const textStyles = {
  // Headings
  h1: {
    fontFamily: 'display',
    fontSize: '3rem', // 48px
    fontWeight: 700,
    lineHeight: 1.2,
    letterSpacing: '-0.025em',
  },

  h2: {
    fontFamily: 'display',
    fontSize: '2.25rem', // 36px
    fontWeight: 700,
    lineHeight: 1.3,
    letterSpacing: '-0.025em',
  },

  h3: {
    fontFamily: 'display',
    fontSize: '1.875rem', // 30px
    fontWeight: 600,
    lineHeight: 1.4,
  },
};

```

```

h4: {
  fontFamily: 'sans',
  fontSize: '1.5rem',    // 24px
  fontWeight: 600,
  lineHeight: 1.4,
},

h5: {
  fontFamily: 'sans',
  fontSize: '1.25rem',   // 20px
  fontWeight: 600,
  lineHeight: 1.5,
},

h6: {
  fontFamily: 'sans',
  fontSize: '1.125rem',  // 18px
  fontWeight: 600,
  lineHeight: 1.5,
},

// Body text
bodyLarge: {
  fontSize: '1.125rem',  // 18px
  lineHeight: 1.625,
},

body: {
  fontSize: '1rem',       // 16px
  lineHeight: 1.5,
},

bodySmall: {
  fontSize: '0.875rem',   // 14px
  lineHeight: 1.5,
},

// UI text
caption: {
  fontSize: '0.75rem',    // 12px
  lineHeight: 1.5,
  color: 'neutral.500',
},

overline: {
  fontSize: '0.75rem',
  fontWeight: 600,
  lineHeight: 1.5,
}

```



```

    textTransform: 'uppercase',
    letterSpacing: '0.1em',
  },

  // Interactive
  button: {
    fontSize: '1rem',
    fontWeight: 600,
    lineHeight: 1.5,
    letterSpacing: '0.025em',
  },

  link: {
    fontSize: '1rem',
    fontWeight: 500,
    lineHeight: 1.5,
    textDecoration: 'underline',
    color: 'primary.600',
  },
};

```

10.2.3 Spacing & Layout

Spacing Scale:

```

const spacing = {
  0: '0',
  0.5: '0.125rem', // 2px
  1: '0.25rem', // 4px
  1.5: '0.375rem', // 6px
  2: '0.5rem', // 8px
  2.5: '0.625rem', // 10px
  3: '0.75rem', // 12px
  3.5: '0.875rem', // 14px
  4: '1rem', // 16px
  5: '1.25rem', // 20px
  6: '1.5rem', // 24px
  7: '1.75rem', // 28px
  8: '2rem', // 32px
  9: '2.25rem', // 36px
  10: '2.5rem', // 40px
  11: '2.75rem', // 44px
  12: '3rem', // 48px
  14: '3.5rem', // 56px
  16: '4rem', // 64px
  20: '5rem', // 80px
  24: '6rem', // 96px
}

```

```

32:  '8rem',      // 128px
40:  '10rem',     // 160px
48:  '12rem',     // 192px
56:  '14rem',     // 224px
64:  '16rem',     // 256px
};

// Layout Constraints
const layout = {
  // Max widths for content
  maxWidth: {
    xs:  '20rem',  // 320px
    sm:  '24rem',  // 384px
    md:  '28rem',  // 448px
    lg:  '32rem',  // 512px
    xl:  '36rem',  // 576px
    '2xl': '42rem', // 672px
    '3xl': '48rem', // 768px
    '4xl': '56rem', // 896px
    '5xl': '64rem', // 1024px
    '6xl': '72rem', // 1152px
    '7xl': '80rem', // 1280px
    full: '100%',
  },

  // Container padding
  container: {
    padding: {
      mobile:  '1rem',    // 16px
      tablet:  '1.5rem',  // 24px
      desktop: '2rem',    // 32px
    },
  },

  // Safe areas (mobile)
  safeArea: {
    top:    'env(safe-area-inset-top)',
    right:  'env(safe-area-inset-right)',
    bottom: 'env(safe-area-inset-bottom)',
    left:   'env(safe-area-inset-left)',
  },
};

```

10.2.4 Shadows & Elevation

Shadow System:

```

const shadows = {
  // Subtle elevations
  xs: '0 1px 2px 0 rgba(0, 0, 0, 0.05)',
  sm: '0 1px 3px 0 rgba(0, 0, 0, 0.1), 0 1px 2px 0 rgba(0, 0, 0, 0.06)',

  // Medium elevations
  md: '0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06)',
  lg: '0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05)',

  // High elevations
  xl: '0 20px 25px -5px rgba(0, 0, 0, 0.1), 0 10px 10px -5px rgba(0, 0, 0, 0.04)',
  '2xl': '0 25px 50px -12px rgba(0, 0, 0, 0.25)',

  // Special
  inner: 'inset 0 2px 4px 0 rgba(0, 0, 0, 0.06)',
  none: 'none',

  // Colored shadows (brand)
  primary: '0 10px 25px -5px rgba(244, 63, 94, 0.4)',
  secondary: '0 10px 25px -5px rgba(168, 85, 247, 0.4)',

  // Dark mode shadows
  dark: {
    sm: '0 1px 3px 0 rgba(0, 0, 0, 0.3), 0 1px 2px 0 rgba(0, 0, 0, 0.2)',
    md: '0 4px 6px -1px rgba(0, 0, 0, 0.3), 0 2px 4px -1px rgba(0, 0, 0, 0.2)',
    lg: '0 10px 15px -3px rgba(0, 0, 0, 0.4), 0 4px 6px -2px rgba(0, 0, 0, 0.2)',
    xl: '0 20px 25px -5px rgba(0, 0, 0, 0.5), 0 10px 10px -5px rgba(0, 0, 0, 0.2)',
  },
};

// Elevation levels (z-index)
const elevation = {
  base: 0,
  raised: 1,
  dropdown: 1000,
  sticky: 1100,
  overlay: 1200,
  modal: 1300,
  popover: 1400,
  toast: 1500,
  tooltip: 1600,
};

```

10.2.5 Border Radius

Radius Scale:

```

const borderRadius = {
  none: '0',
  sm: '0.25rem', // 4px
  base: '0.5rem', // 8px
  md: '0.75rem', // 12px
  lg: '1rem', // 16px
  xl: '1.5rem', // 24px
  '2xl': '2rem', // 32px
  '3xl': '3rem', // 48px
  full: '9999px', // Pill shape

  // Component-specific
  button: '0.75rem', // 12px
  card: '1rem', // 16px
  input: '0.5rem', // 8px
  avatar: '9999px', // Circle
  message: '1.25rem', // 20px (chat bubble)
};

```

10.3 Component Library

10.3.1 Button Components

Button Variants:

```

interface ButtonProps {
  variant: 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger';
  size: 'sm' | 'md' | 'lg';
  fullWidth?: boolean;
  leftIcon?: ReactNode;
  rightIcon?: ReactNode;
  loading?: boolean;
  disabled?: boolean;
}

```

// Primary Button

```

const PrimaryButton = `
  background: linear-gradient(135deg, primary.600, primary.500);
  color: white;
  padding: 12px 24px;
  border-radius: 12px;
  font-weight: 600;
  box-shadow: 0 4px 12px rgba(244, 63, 94, 0.25);

  &:hover {
    transform: translateY(-2px);
    box-shadow: 0 6px 16px rgba(244, 63, 94, 0.35);
  }

```

```

    }

    &:active {
        transform: translateY(0);
    }

    &:disabled {
        opacity: 0.5;
        cursor: not-allowed;
    }
`;

// Secondary Button
const SecondaryButton = `
    background: linear-gradient(135deg, secondary.600, secondary.500);
    color: white;
    padding: 12px 24px;
    border-radius: 12px;
    font-weight: 600;
`;

// Outline Button
const OutlineButton = `
    background: transparent;
    color: primary.600;
    border: 2px solid primary.600;
    padding: 10px 22px;
    border-radius: 12px;
    font-weight: 600;

    &:hover {
        background: primary.50;
    }
`;

// Ghost Button
const GhostButton = `
    background: transparent;
    color: neutral.700;
    padding: 12px 24px;
    border-radius: 12px;

    &:hover {
        background: neutral.100;
    }
`;

// Icon Button

```

```
const IconButton = `
  width: 40px;
  height: 40px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: neutral.100;

  &:hover {
    background: neutral.200;
  }
`;
```

Button Sizes:

```
const buttonSizes = {
  sm: {
    fontSize: '0.875rem',
    padding: '8px 16px',
    height: '32px',
  },
  md: {
    fontSize: '1rem',
    padding: '12px 24px',
    height: '44px',
  },
  lg: {
    fontSize: '1.125rem',
    padding: '16px 32px',
    height: '56px',
  },
};
```

10.3.2 Input Components

Text Input:

```
interface InputProps {
  type: 'text' | 'email' | 'password' | 'number' | 'tel' | 'url';
  label?: string;
  placeholder?: string;
  error?: string;
  helper?: string;
  leftIcon?: ReactNode;
  rightIcon?: ReactNode;
  disabled?: boolean;
  required?: boolean;
```

```

}

const TextInput = `
  width: 100%;
  padding: 12px 16px;
  border: 2px solid neutral.200;
  border-radius: 12px;
  font-size: 1rem;
  background: white;
  transition: all 0.2s;

  &:focus {
    outline: none;
    border-color: primary.500;
    box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.1);
  }

  &:disabled {
    background: neutral.50;
    color: neutral.400;
    cursor: not-allowed;
  }

  &.error {
    border-color: error.main;

    &:focus {
      box-shadow: 0 0 0 4px rgba(239, 68, 68, 0.1);
    }
  }
`;

// Label
const InputLabel = `
  display: block;
  font-size: 0.875rem;
  font-weight: 600;
  color: neutral.700;
  margin-bottom: 6px;
`;

// Helper/Error Text
const InputHelper = `
  font-size: 0.75rem;
  margin-top: 6px;
  color: neutral.500;

  &.error {

```

```

        color: error.main;
    }
`;

```

Textarea:

```

const Textarea = `
    width: 100%;
    padding: 12px 16px;
    border: 2px solid neutral.200;
    border-radius: 12px;
    font-size: 1rem;
    font-family: sans;
    resize: vertical;
    min-height: 100px;

    &:focus {
        outline: none;
        border-color: primary.500;
        box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.1);
    }
`;

```

Select:

```

const Select = `
    width: 100%;
    padding: 12px 40px 12px 16px;
    border: 2px solid neutral.200;
    border-radius: 12px;
    font-size: 1rem;
    background: white url('chevron-down.svg') no-repeat right 12px center;
    background-size: 20px;
    appearance: none;
    cursor: pointer;

    &:focus {
        outline: none;
        border-color: primary.500;
        box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.1);
    }
`;

```

Checkbox & Radio:

```

const Checkbox = `
    width: 20px;
    height: 20px;
    border: 2px solid neutral.300;
    border-radius: 6px;
    appearance: none;

```



```

    cursor: pointer;
    position: relative;

    &:checked {
        background: primary.600;
        border-color: primary.600;

        &::after {
            content: '';
            position: absolute;
            left: 6px;
            top: 2px;
            width: 5px;
            height: 10px;
            border: solid white;
            border-width: 0 2px 2px 0;
            transform: rotate(45deg);
        }
    }

    &:focus {
        outline: none;
        box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.1);
    }
`;

const Radio = `
width: 20px;
height: 20px;
border: 2px solid neutral.300;
border-radius: 50%;
appearance: none;
cursor: pointer;
position: relative;

    &:checked {
        border-color: primary.600;

        &::after {
            content: '';
            position: absolute;
            top: 50%;
            left: 50%;
            transform: translate(-50%, -50%);
            width: 10px;
            height: 10px;
            border-radius: 50%;
            background: primary.600;

```

```

    }
  }
`;

```

10.3.3 Card Components

Basic Card:

```

const Card = `
  background: white;
  border-radius: 16px;
  padding: 24px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  transition: all 0.3s ease;

  &:hover {
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
    transform: translateY(-2px);
  }
`;

```

```

// Card with header
interface CardProps {
  title?: string;
  subtitle?: string;
  action?: ReactNode;
  footer?: ReactNode;
  hoverable?: boolean;
  clickable?: boolean;
}

```

```

const CardHeader = `
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 16px;
  padding-bottom: 16px;
  border-bottom: 1px solid neutral.200;
`;

```

```

const CardTitle = `
  font-size: 1.25rem;
  font-weight: 600;
  color: neutral.800;
`;

```

```

const CardSubtitle = `

```

```

    font-size: 0.875rem;
    color: neutral.500;
    margin-top: 4px;
  `;

const CardFooter = `
  margin-top: 16px;
  padding-top: 16px;
  border-top: 1px solid neutral.200;
  `;

```

Message Card (Chat Bubble):

```

const MessageCard = `
  max-width: 75%;
  padding: 12px 16px;
  border-radius: 20px;
  word-wrap: break-word;

  &.sent {
    background: linear-gradient(135deg, primary.500, primary.600);
    color: white;
    margin-left: auto;
    border-bottom-right-radius: 4px;
  }

  &.received {
    background: neutral.100;
    color: neutral.900;
    margin-right: auto;
    border-bottom-left-radius: 4px;
  }
  `;

```

Media Card:

```

const MediaCard = `
  position: relative;
  border-radius: 16px;
  overflow: hidden;
  aspect-ratio: 1;
  cursor: pointer;

  img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    transition: transform 0.3s ease;
  }

```

```

&:hover img {
  transform: scale(1.05);
}

.overlay {
  position: absolute;
  bottom: 0;
  left: 0;
  right: 0;
  padding: 16px;
  background: linear-gradient(to top, rgba(0,0,0,0.7), transparent);
  color: white;
  opacity: 0;
  transition: opacity 0.3s;
}

&:hover .overlay {
  opacity: 1;
}
`;

```

10.3.4 Avatar Components

Avatar Sizes:

```

const avatarSizes = {
  xs: '24px',
  sm: '32px',
  md: '40px',
  lg: '56px',
  xl: '80px',
  '2xl': '120px',
};

const Avatar = `
  border-radius: 50%;
  object-fit: cover;
  background: neutral.200;

  &.has-status {
    position: relative;

    &::after {
      content: '';
      position: absolute;
      bottom: 0;
      right: 0;
    }
  }
`;

```

```

        width: 25%;
        height: 25%;
        border-radius: 50%;
        border: 2px solid white;
    }

    &.online::after {
        background: success.main;
    }

    &.away::after {
        background: warning.main;
    }

    &.offline::after {
        background: neutral.400;
    }
}
`;

// Couple Avatar (two avatars overlapped)
const CoupleAvatar = `
    display: flex;
    position: relative;

    .avatar-1 {
        z-index: 2;
    }

    .avatar-2 {
        margin-left: -25%;
        z-index: 1;
        border: 2px solid white;
    }
`;

```

10.3.5 Badge & Tag Components

Badge:

```

const Badge = `
    display: inline-flex;
    align-items: center;
    padding: 4px 8px;
    border-radius: 6px;
    font-size: 0.75rem;
    font-weight: 600;

```

```

&.primary {
  background: primary.100;
  color: primary.700;
}

&.success {
  background: success.light;
  color: success.dark;
}

&.warning {
  background: warning.light;
  color: warning.dark;
}

&.error {
  background: error.light;
  color: error.dark;
}
`;

// Notification Badge (dot)
const NotificationBadge = `
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: error.main;
  border: 2px solid white;

  &.with-count {
    width: auto;
    height: 20px;
    padding: 0 6px;
    border-radius: 10px;
    font-size: 0.625rem;
    font-weight: 700;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
  }
`;

```

10.3.6 Modal & Dialog Components

Modal:

```
const ModalOverlay = `
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  backdrop-filter: blur(4px);
  z-index: 1300;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 16px;
`;

const ModalContent = `
  background: white;
  border-radius: 24px;
  padding: 32px;
  max-width: 500px;
  width: 100%;
  max-height: 90vh;
  overflow-y: auto;
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.25);
  animation: modalSlideIn 0.3s ease-out;

  @keyframes modalSlideIn {
    from {
      opacity: 0;
      transform: scale(0.95) translateY(-20px);
    }
    to {
      opacity: 1;
      transform: scale(1) translateY(0);
    }
  }
`;

const ModalHeader = `
  margin-bottom: 24px;

  h2 {
    font-size: 1.5rem;
    font-weight: 700;
    margin-bottom: 8px;
  }
`;
```

```

p {
  font-size: 0.875rem;
  color: neutral.500;
}
`;

const ModalFooter = `
  margin-top: 24px;
  display: flex;
  gap: 12px;
  justify-content: flex-end;
`;

```

Bottom Sheet (Mobile):

```

const BottomSheet = `
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  background: white;
  border-top-left-radius: 24px;
  border-top-right-radius: 24px;
  padding: 24px;
  padding-bottom: max(24px, env(safe-area-inset-bottom));
  box-shadow: 0 -10px 40px rgba(0, 0, 0, 0.1);
  max-height: 90vh;
  overflow-y: auto;
  z-index: 1300;
  animation: slideUp 0.3s ease-out;

  @keyframes slideUp {
    from {
      transform: translateY(100%);
    }
    to {
      transform: translateY(0);
    }
  }

  .handle {
    width: 40px;
    height: 4px;
    background: neutral.300;
    border-radius: 2px;
    margin: 0 auto 20px;
  }
`;

```

10.3.7 Toast Notifications

Toast Component:

```
const Toast = `
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 16px;
  border-radius: 12px;
  background: white;
  box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.2);
  min-width: 300px;
  max-width: 500px;
  animation: toastSlideIn 0.3s ease-out;

  @keyframes toastSlideIn {
    from {
      opacity: 0;
      transform: translateY(-20px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }

  &.success {
    border-left: 4px solid success.main;
  }

  &.error {
    border-left: 4px solid error.main;
  }

  &.warning {
    border-left: 4px solid warning.main;
  }

  &.info {
    border-left: 4px solid info.main;
  }

  .icon {
    width: 24px;
    height: 24px;
  }
`
```

```

    flex-shrink: 0;
  }

  .content {
    flex: 1;

    .title {
      font-weight: 600;
      margin-bottom: 2px;
    }

    .message {
      font-size: 0.875rem;
      color: neutral.600;
    }
  }

  .close {
    cursor: pointer;
    color: neutral.400;

    &:hover {
      color: neutral.600;
    }
  }
};

```

10.4 Page Specifications

10.4.1 Authentication Pages

Login Page:

[LinkUp Logo]

Welcome Back!

Sign in to connect with your partner

Email

[Input field]

Password

[Input field]
Forgot password?

[] Keep me signed in

Sign In

or

[G] Continue with Google

[A] Continue with Apple

Don't have an account? Sign up

Registration Page:

Create Your Account
Start your journey together

[Step 1 of 3: Personal Info]

Email
[Input field]

Username
[Input field]

Display Name
[Input field]

Password
[Input field]
8+ characters
1 uppercase, 1 number

☐ I agree to Terms & Privacy Policy

Continue

Already have an account? Sign in

10.4.2 Home/Dashboard Page

Main Dashboard:

LinkUp [2] [Avatar]

[Partner Avatar] [Your Avatar]

You & Alex
Together for 342 days

PHOTO	MSGs	ART	GOALS
Streak	Messages	Creative	Achievemnt
7 days	142	12	24/50

Photo Streak [View]

STREAK: 7 Days! Keep it going!

[Today's Photo] Hours remaining: 8h 23m

2 Freezes Available
Photo

[Upload Photo]

Upcoming Dates

[Manage]

- Anniversary - January 25 (5 days)
- Alex's Birthday - February 14 (25 days)

Recent Achievements

[View]

Trophy: Week-long Streak (7 days)	50 pts
Chat: 100 Messages Sent	25 pts
Art: First Collaborative Painting	10 pts

Quick Actions

[Message] [Draw] [Photo] [Music]

[Home] [Chat] [Create] [Media] [Profile]

10.4.3 Chat/Messaging Page

Chat Interface:

← Alex Thompson
Online

Today, 2:30 PM

Hey! How's your day going?
2:30

[Alex's Avatar]

Great! Just finished work
What about you? 2:31

[Your Avatar]

Same here! Want to watch a
movie tonight? 2:32

[Alex's Avatar]

Yes! That sounds perfect
2:32

[Your Avatar]

[Photo: Movie poster]
How about this one? 2:33

[Alex's Avatar]

Alex is typing...

[+] [Type a message...] [Send]
[] [] [] []

Message Actions (Long Press/Hover):

Hey! How's your day going?
2:30 ← [Reply] [React] [Save] [Delete]

10.4.4 Media Gallery Page

Photo Gallery:

← Gallery

All Photos (1,234) | Albums (12)

[Streak Photos (7)] [Favorites] [This Month]

Photo Photo Photo Photo

Photo Photo Photo Photo

Photo Photo Photo Photo

Photo Photo Photo Photo

Albums

Streak	Vacation	Dates
Photos	2025	Night
(7)	(45)	(23)

[Home] [Chat] [Create] [Media] [Profile]

10.4.5 Creative Tools Page

Drawing Canvas:

← Collaborative Painting [Undo] [Redo] [Save]

Alex is drawing...

[Drawing Canvas]

Tools: [Brush] [Pen] [Marker] [Eraser]

Colors: [More...]

Size:
Small → Large

Scribble Interface:

Scribble to Alex

Draw with your finger or mouse...

[Clear] [Send]

10.4.6 Profile Page

User Profile:

← Profile

[Settings]

[Profile Picture]

You & Alex
Together for 342 days
Since January 20, 2025

[Alex Avatar] Alex Thompson
@alex_t
"My forever person"

[Your Avatar] You
@yourname
"Lucky to have you"

Relationship Stats

Messages Sent:	1,234
Photos Shared:	567
Current Streak:	7 days
Achievements:	24 / 50
Total Points:	1,250

Showcased Achievements

Week Champion	Creative Duo
Chatty Couple	Memory Makers

Single Friends (3/10) [Manage]

[Avatar] Sarah [Avatar] Mike [Avatar] Emma

Couple Circles (2) [Manage]

College Friends (6 couples)
Adventure Squad (4 couples)

[Home] [Chat] [Create] [Media] [Profile]

10.5 Animations & Transitions

10.5.1 Motion Principles

Animation Guidelines: - Purposeful - Every animation serves a function - Subtle - Dont distract from content - Fast - Keep under 300ms for most transitions - Natural - Use easing functions that feel organic - Delightful - Add personality to key moments

Easing Functions:

```
const easings = {  
  // Standard  
  easeInOut: 'cubic-bezier(0.4, 0, 0.2, 1)',  
  easeOut:   'cubic-bezier(0.0, 0, 0.2, 1)',  
  easeIn:    'cubic-bezier(0.4, 0, 1, 1)',  
  
  // Snappy  
  sharp:     'cubic-bezier(0.4, 0, 0.6, 1)',  
  
  // Bouncy  
  bounce:    'cubic-bezier(0.68, -0.55, 0.265, 1.55)',  
  
  // Smooth  
  smooth:    'cubic-bezier(0.25, 0.46, 0.45, 0.94)',  
};
```

10.5.2 Page Transitions

Route Transitions:

```
const pageTransitions = {  
  fadeIn: `  
    @keyframes fadeIn {  
      from {  
        opacity: 0;  
      }  
      to {  
        opacity: 1;  
      }  
    }`  
};
```

```

    }
  }
  animation: fadeIn 200ms ease-out;
},

slideInFromRight: `
  @keyframes slideInFromRight {
    from {
      transform: translateX(100%);
      opacity: 0;
    }
    to {
      transform: translateX(0);
      opacity: 1;
    }
  }
  animation: slideInFromRight 300ms ease-out;
},

slideUp: `
  @keyframes slideUp {
    from {
      transform: translateY(20px);
      opacity: 0;
    }
    to {
      transform: translateY(0);
      opacity: 1;
    }
  }
  animation: slideUp 250ms ease-out;
},

scaleIn: `
  @keyframes scaleIn {
    from {
      transform: scale(0.95);
      opacity: 0;
    }
    to {
      transform: scale(1);
      opacity: 1;
    }
  }
  animation: scaleIn 200ms ease-out;
},
};

```

10.5.3 Micro-interactions

Button Press:

```
.button {  
  transition: all 150ms ease-out;  
}  
  
.button:active {  
  transform: scale(0.95);  
}  
  
.button:hover {  
  transform: translateY(-2px);  
  box-shadow: 0 6px 16px rgba(244, 63, 94, 0.35);  
}
```

Message Send Animation:

```
@keyframes messageSend {  
  0% {  
    transform: scale(1);  
    opacity: 1;  
  }  
  50% {  
    transform: scale(1.1);  
  }  
  100% {  
    transform: translateX(100px) scale(0.8);  
    opacity: 0;  
  }  
}  
  
.message.sending {  
  animation: messageSend 400ms ease-out;  
}
```

Heart Animation (Like):

```
@keyframes heartBeat {  
  0%, 100% {  
    transform: scale(1);  
  }  
  25% {  
    transform: scale(1.3);  
  }  
  50% {  
    transform: scale(1.1);  
  }  
}
```

```

    }
}

.heart-icon.liked {
  animation: heartBeat 500ms ease-out;
  color: #F43F5E;
}

```

Achievement Unlock:

```

@keyframes achievementUnlock {
  0% {
    transform: translateY(100px) scale(0);
    opacity: 0;
  }
  50% {
    transform: translateY(-10px) scale(1.1);
  }
  75% {
    transform: translateY(5px) scale(0.95);
  }
  100% {
    transform: translateY(0) scale(1);
    opacity: 1;
  }
}

.achievement-toast {
  animation: achievementUnlock 600ms cubic-bezier(0.68, -0.55, 0.265, 1.55);
}

```

Streak Flame Animation:

```

@keyframes flameFlicker {
  0%, 100% {
    transform: scale(1) translateY(0);
  }
  50% {
    transform: scale(1.05) translateY(-2px);
  }
}

.streak-flame {
  animation: flameFlicker 2s ease-in-out infinite;
}

```

10.5.4 Loading States

Skeleton Loading:

```
@keyframes shimmer {
  0% {
    background-position: -1000px 0;
  }
  100% {
    background-position: 1000px 0;
  }
}

.skeleton {
  background: linear-gradient(
    90deg,
    #f0f0f0 0px,
    #e0e0e0 40px,
    #f0f0f0 80px
  );
  background-size: 1000px 100%;
  animation: shimmer 2s infinite;
}
```

Spinner:

```
@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid rgba(244, 63, 94, 0.2);
  border-top-color: #F43F5E;
  border-radius: 50%;
  animation: spin 800ms linear infinite;
}
```

10.6 Responsive Design

10.6.1 Breakpoints

```
const breakpoints = {
  xs: '0px',      // Mobile portrait
  sm: '640px',    // Mobile landscape
  md: '768px',    // Tablet portrait
```

```

lg: '1024px',    // Tablet landscape / Small desktop
xl: '1280px',    // Desktop
'2xl': '1536px', // Large desktop
};

// Media queries
const mediaQueries = {
  mobile:      '@media (max-width: 767px)',
  tablet:      '@media (min-width: 768px) and (max-width: 1023px)',
  desktop:     '@media (min-width: 1024px)',
  largeDesktop: '@media (min-width: 1536px)',

  // Orientation
  portrait:    '@media (orientation: portrait)',
  landscape:   '@media (orientation: landscape)',

  // Touch devices
  touch:       '@media (hover: none) and (pointer: coarse)',

  // Dark mode
  darkMode:    '@media (prefers-color-scheme: dark)',

  // Reduced motion
  reduceMotion: '@media (prefers-reduced-motion: reduce)',
};

```

10.6.2 Responsive Layouts

Grid System:

```

// Mobile-first approach
const ResponsiveGrid = `
  display: grid;
  gap: 16px;

  /* Mobile: 1 column */
  grid-template-columns: 1fr;

  /* Tablet: 2 columns */
  @media (min-width: 768px) {
    grid-template-columns: repeat(2, 1fr);
    gap: 24px;
  }

  /* Desktop: 3-4 columns */
  @media (min-width: 1024px) {
    grid-template-columns: repeat(3, 1fr);
  }

```

```

    gap: 32px;
  }

  @media (min-width: 1280px) {
    grid-template-columns: repeat(4, 1fr);
  }
};

```

Adaptive Typography:

```

/* Fluid typography */
h1 {
  font-size: clamp(2rem, 5vw, 3rem);
}

h2 {
  font-size: clamp(1.5rem, 4vw, 2.25rem);
}

body {
  font-size: clamp(1rem, 2vw, 1.125rem);
}

```

10.7 Accessibility

10.7.1 WCAG 2.1 AA Compliance

Color Contrast:

```

// Minimum contrast ratios
const contrastRatios = {
  normalText: 4.5, // 4.5:1 for normal text
  largeText: 3, // 3:1 for large text (18pt+)
  uiComponents: 3, // 3:1 for interactive elements
};

// Accessible color combinations
const accessibleColors = {
  // Text on white background
  onWhite: {
    primary: '#E11D48', // 4.52:1
    secondary: '#7E22CE', // 7.01:1
    neutral: '#44403C', // 10.5:1
  },

  // Text on dark background
  onDark: {
    primary: '#FB7185', // 4.53:1

```



```

        secondary: '#D8B4FE', // 8.12:1
        neutral:   '#E7E5E4', // 12.1:1
    },
};

```

Focus States:

```

/* Visible focus indicator */
*:focus-visible {
    outline: 3px solid #F43F5E;
    outline-offset: 2px;
    border-radius: 4px;
}

/* Button focus */
button:focus-visible {
    box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.2);
}

/* Input focus */
input:focus-visible,
textarea:focus-visible {
    border-color: #F43F5E;
    box-shadow: 0 0 0 4px rgba(244, 63, 94, 0.1);
}

```

Screen Reader Support:

```

// ARIA labels and roles
interface AriaProps {
    role?: string;
    'aria-label'? : string;
    'aria-labelledby'? : string;
    'aria-describedby'? : string;
    'aria-hidden'? : boolean;
    'aria-live'? : 'polite' | 'assertive' | 'off';
    'aria-atomic'? : boolean;
    'aria-current'? : boolean | 'page' | 'step';
}

// Example usage
<button
    aria-label="Send message"
    aria-describedby="message-helper"
>
    <SendIcon aria-hidden="true" />
</button>

<div
    role="status"

```

```

    aria-live="polite"
    aria-atomic="true"
  >
    Message sent successfully
</div>

Keyboard Navigation:

// Tab order and keyboard shortcuts
const keyboardShortcuts = {
  'Ctrl+K':      'Open command palette',
  'Ctrl+/' :     'Toggle sidebar',
  'Ctrl+Enter':  'Send message',
  'Esc':         'Close modal/dialog',
  'Tab':         'Navigate forward',
  'Shift+Tab':   'Navigate backward',
  '↑↓':         'Navigate list items',
  'Space':       'Select/toggle',
  'Enter':       'Activate/submit',
};

```

Chapter 10 Status: Complete

Summary: - Design Philosophy: Intimate, artistic, joyfully interactive, compassionately encouraging - Design Tokens: Complete color system (primary pink, secondary purple, accent orange), typography scale (Inter + Cal Sans), spacing (8px grid), shadows, border radius - Component Library: 40+ components including buttons (5 variants), inputs (text, textarea, select, checkbox, radio), cards (message bubbles, media cards), avatars (with status indicators), badges, modals, toasts - Page Specifications: 6 key screens with ASCII wireframes (login, registration, dashboard, chat, media gallery, creative tools, profile) - Animations: Page transitions, micro-interactions (button press, message send, heart animation, achievement unlock, streak flame), loading states (skeleton, spinner) - Responsive Design: Mobile-first breakpoints (640/768/1024/1280/1536px), adaptive grid layouts, fluid typography - Accessibility: WCAG 2.1 AA compliance, 4.5:1 contrast ratios, focus states, screen reader support, keyboard navigation

Next Chapter: Chapter 11: Business Model & Go-to-Market Strategy

Chapter 11: Business Model & Go-to-Market Strategy

Overview

LinkUp targets the Nepal market exclusively, focusing on young couples (18-35) who are tech-savvy and seeking intimate, private ways to connect. The business model combines freemium subscriptions with optional in-app purchases, tailored to Nepals purchasing power and payment ecosystem.

This chapter covers: - Market Analysis - Nepals couples app opportunity - Business Model - Freemium + premium tiers - Pricing Strategy - NPR-based pricing optimized for local market

- Go-to-Market Strategy - Launch and growth tactics - Marketing Channels - Acquisition and retention - Financial Projections - 5-year revenue forecast in NPR - Unit Economics - CAC, LTV, and profitability metrics

Target Market: - Primary: Urban couples in Kathmandu Valley (18-30 years) - Secondary: Couples in Pokhara, Biratnagar, other cities - Tertiary: Long-distance couples (Nepal abroad)

11.1 Market Analysis

11.1.1 Nepal Market Overview

Demographics:

```
const nepalMarketData = {
  population: {
    total: 30_500_000,
    urban: 6_100_000,           // 20% urbanization
    internetUsers: 21_000_000, // 69% penetration
    smartphoneUsers: 15_000_000, // 49% penetration
  },

  targetDemographic: {
    age18to35: 9_800_000,      // 32% of population
    urbanYoungAdults: 2_200_000,
    estimatedCouples: 800_000, // ~35-40% in relationships
    techSavvyCouples: 320_000, // 40% smartphone + internet
  },

  economicIndicators: {
    gdpPerCapita: 'NPR 152,000/year (~USD 1,140)',
    urbanIncomeAvg: 'NPR 35,000-50,000/month',
    disposableIncome: 'NPR 5,000-15,000/month',
    mobileDataPlan: 'NPR 500-1,500/month',
  },

  digitalPaymentAdoption: {
    eSewa: '6M+ users',
    khalti: '4M+ users',
    imePay: '3M+ users',
    bankTransfer: 'Widely used',
    cardPayment: 'Growing (~30% adoption)',
  },
};
```

Market Opportunity:

Total Addressable Market (TAM):

- Young couples in Nepal: ~800,000
- Tech-savvy segment: ~320,000 couples

- TAM Value: $320,000 \times \text{NPR } 3,000/\text{year} = \text{NPR } 960\text{M}/\text{year}$

Serviceable Addressable Market (SAM):

- Urban couples with smartphones: ~150,000
- SAM Value: $150,000 \times \text{NPR } 3,000/\text{year} = \text{NPR } 450\text{M}/\text{year}$

Serviceable Obtainable Market (SOM):

- Year 1 target: 2% market share = 3,000 couples
- Year 3 target: 10% market share = 15,000 couples
- Year 5 target: 20% market share = 30,000 couples

11.1.2 Competitive Landscape (Nepal Focus)

Current Players:

Platform	Type	Users (Nepal)	Strengths	Weaknesses
WhatsApp	General messaging	~15M	Universal, free, familiar	Not couple-specific, no privacy
Instagram	Social media	~5M	Visual sharing, stories	Public, not intimate
Snapchat	Messaging/Social	~800K	Private messaging	Not couple-focused
Between	Couples app	~5K	Couple-specific	Limited features, not localized
Couple (by Locket)	Couples app	~3K	Chat + calendar	Limited Nepal adoption
Telegram	Messaging	~3M	Secret chats, bots	Generic, not romantic

LinkUps Differentiation:

Couple-specific with creative tools (scribble, collaborative painting)
 Compassionate gamification (photo streaks, achievements)
 Social features (SingleFriend, Couple Circles)
 Nepal-focused (NPR pricing, local payment methods)
 Privacy-first by design
 Artistic, visually appealing interface
 Cross-platform (web, iOS, Android)
 Offline-first capabilities for unreliable internet

11.1.3 User Personas (Nepal Context)

Primary Persona 1: Aarav & Priya

Demographics:

- Age: 24 & 23
- Location: Kathmandu
- Occupation: IT professional & Teacher
- Income: NPR 50,000 + NPR 35,000/month
- Relationship: Dating for 1.5 years

Tech Profile:

- Devices: iPhone 12, Android flagship
- Apps: Instagram, WhatsApp, Netflix
- Internet: 4G unlimited plans
- Payment: eSewa, khalti

Pain Points:

- WhatsApp feels too public (family groups)
- No dedicated space for couple memories
- Want creative ways to express love
- Struggle to remember important dates

Goals:

- Private, intimate communication
- Track relationship milestones
- Fun, creative interactions
- Strengthen emotional connection

Willingness to Pay:

- NPR 500-800/month for premium features

Primary Persona 2: Rohan & Sneha (Long Distance)

Demographics:

- Age: 26 & 25
- Location: Kathmandu Pokhara
- Occupation: Both working professionals
- Income: NPR 45,000 + NPR 40,000/month
- Relationship: Engaged, long-distance

Tech Profile:

- Devices: Mid-range Android phones
- Apps: Viber, WhatsApp, Zoom
- Internet: 4G, occasional WiFi
- Payment: Bank transfer, khalti

Pain Points:

- Missing physical presence
- Time zone coordination (same but different cities)
- Expensive data for video calls
- Want daily connection rituals

Goals:

- Daily photo sharing (streaks)
- Virtual date experiences
- Feel closer despite distance
- Plan visits and future together

Willingness to Pay:

- NPR 400-600/month for unlimited features

Secondary Persona 3: Nischal & Anushka (College Students)

Demographics:

- Age: 21 & 20
- Location: Kathmandu (college students)
- Income: NPR 10,000-15,000/month (part-time + allowance)
- Relationship: Dating for 8 months

Tech Profile:

- Devices: Budget Android phones
- Apps: TikTok, Instagram, Snapchat
- Internet: Shared WiFi, limited data
- Payment: eSewa (prepaid)

Pain Points:

- Limited budget
- Want privacy from parents/friends
- Enjoy social media but need intimacy
- Creative expression important

Goals:

- Free or very affordable app
- Private space from public social media
- Fun, playful interactions
- Share memories securely

Willingness to Pay:

- NPR 200-300/month (if really valuable)
- Prefer free tier with ads or limitations

11.2 Business Model

11.2.1 Revenue Streams

Primary Revenue: 1. Subscription Plans (80% of revenue) - Monthly and annual premium subscriptions - Tiered pricing for different feature sets

2. In-App Purchases (15% of revenue)
 - Additional streak freezes

- Custom emoji packs
 - Special themes and designs
 - Extra storage for media
3. Couple Circles Premium (5% of revenue)
- Enhanced circle features for groups
 - Co-branded circle sponsorships (later stage)

Future Revenue (Post Year 2): - Virtual Gifts - Send digital gifts to partner - Printing Services - Physical photo books, calendars - Premium Content - Date ideas, relationship tips - API Access - For third-party integrations

11.2.2 Pricing Strategy (NPR)

Free Tier (Forever Free):

```
const freeTier = {
  name: 'LinkUp Free',
  price: 'NPR 0',
  features: [
    ' Unlimited messaging',
    ' Voice messages (5 min/day limit)',
    ' Video calls (30 min/day limit)',
    ' Photo sharing (50 photos/month)',
    ' Basic scribble',
    ' Photo streaks (with ads)',
    ' Up to 3 SingleFriends',
    ' Join 1 Couple Circle',
    ' 5 important dates',
    ' Basic achievements',
    '× No collaborative painting',
    '× No custom emojis',
    '× Limited media storage (1 GB)',
  ],

  limitations: {
    mediaStorage: '1 GB',
    singleFriends: 3,
    circles: 1,
    voiceLimit: '5 min/day',
    videoLimit: '30 min/day',
    photosPerMonth: 50,
    importantDates: 5,
    ads: true,
  },
};
```

Premium Tier (LinkUp Plus):

```

const premiumTier = {
  name: 'LinkUp Plus',
  priceMonthly: 'NPR 499/month',
  priceYearly: 'NPR 4,999/year (Save NPR 989 - 17% off)',
  effectiveMonthly: 'NPR 416/month',

  features: [
    ' Everything in Free',
    ' Unlimited voice & video calls',
    ' Unlimited photo sharing',
    ' Collaborative painting',
    ' Custom emoji creator (50 emojis)',
    ' Up to 10 SingleFriends',
    ' Join unlimited Couple Circles',
    ' Unlimited important dates',
    ' Advanced achievements',
    ' No ads',
    ' 5 GB media storage',
    ' HD video calls',
    ' Advanced scribble tools',
    ' Priority support',
  ],

  addOns: {
    extraStorage10GB: 'NPR 199/month',
    extraStorage50GB: 'NPR 499/month',
    customThemes: 'NPR 99 one-time',
    streakFreezePack5: 'NPR 99',
    emojiPack: 'NPR 149 one-time',
  },
};

```

Pro Tier (LinkUp Pro):

```

const proTier = {
  name: 'LinkUp Pro',
  priceMonthly: 'NPR 999/month',
  priceYearly: 'NPR 9,999/year (Save NPR 1,989 - 17% off)',
  effectiveMonthly: 'NPR 833/month',

  features: [
    ' Everything in Plus',
    ' Unlimited custom emojis',
    ' 50 GB media storage',
    ' 4K video calls',
    ' Advanced analytics (relationship stats)',
    ' Unlimited streak freezes',
    ' Custom app themes',
    ' Early access to new features',
  ],
};

```



```

    ' Relationship milestones tracker',
    ' Export all data anytime',
    ' Dedicated support',
    ' Create premium Couple Circles',
    ' Collaborative playlists (Spotify)',
  ],

  targetUser: 'Power users, long-distance couples, content creators',
};

```

Pricing Comparison:

Feature	Free	Plus	Pro
Price/month	NPR 0	NPR 499	NPR 999
Messaging			
Voice calls	5min/day	Unlimited	Unlimited
Video calls	30min/d	Unlimited HD	4K
Photo sharing	50/mo	Unlimited	Unlimited
Media storage	1 GB	5 GB	50 GB
Collaborative painting			
Custom emojis		50	Unlimited
SingleFriends	3	10	10
Couple Circles	1	Unlimited	Unlimited
Streak freezes	2	2	Unlimited
Ads	Yes	No	No
Analytics	Basic	Basic	Advanced
Support	Email	Priority	Dedicated

11.2.3 Pricing Rationale

Market Positioning:

```

const pricingContext = {
  competitorPricing: {
    netflix: 'NPR 450-1,100/month',
    spotify: 'NPR 489/month (student: NPR 244)',
    youtubeMusic: 'NPR 149/month',
    zoom: 'NPR 1,899/month (pro)',
    notion: 'NPR 533/month',
  },

  averageEntertainment: 'NPR 300-800/month',

  linkUpPositioning: {
    free: 'Competitive with WhatsApp/Telegram (free)',
  }
};

```

```

    plus: 'Lower than Netflix, comparable to Spotify',
    pro: 'Premium tier for power users',
  },

  valueProposition: {
    emotionalValue: 'Strengthen relationship',
    privacyValue: 'Safe, intimate space',
    creativeValue: 'Unique creative tools',
    socialValue: 'Connect with couple friends',
    nostalgicValue: 'Preserve memories forever',
  },
};

```

Conversion Strategy:

Free → Plus Conversion Triggers:

- Hit photo limit (50/month)
- Video call time limit reached
- Want collaborative painting
- Need more SingleFriends
- Annoyed by ads

Plus → Pro Conversion Triggers:

- Storage limit reached (5 GB)
- Want custom themes
- Need advanced analytics
- Long-distance couple (4K calls)
- Content creator (export data)

11.3 Go-to-Market Strategy

11.3.1 Launch Strategy (Phase 1: Months 1-3)

Pre-Launch (Month -2 to 0):

```

const preLaunchActivities = {
  week1to4: {
    buildLandingPage: 'Create waitlist on linkup.com.np',
    socialMediaSetup: 'Instagram, Facebook, TikTok accounts',
    contentCreation: 'Teaser videos, couple testimonials',
    influencerOutreach: 'Contact 20 Nepal micro-influencers',
    betaTesting: 'Recruit 50 couples for closed beta',
  },

  week5to8: {
    pressRelease: 'Nepal tech publications (Gadgetbyte, TechSansar)',
    communityBuilding: 'Reddit r/Nepal, Facebook groups',
    referralProgram: 'Beta users get free Plus for 3 months',
  },
};

```

```

    partnershipTalks: 'eSewa, Khalti integration discussions',
    finalPolish: 'Bug fixes, performance optimization',
  },

  waitlistGoal: '1,000 signups',
  betaUserTarget: '50 couples (100 users)',
  budgetAllocation: 'NPR 200,000',
};

Launch Week (Month 1):

const launchWeek = {
  day0: {
    activity: 'Soft launch to beta users',
    goal: 'Validate stability',
    target: '100 active users',
  },

  day1: {
    activity: 'Public launch announcement',
    channels: [
      'Social media blitz',
      'Press release to Nepal tech media',
      'Email to waitlist (1,000 people)',
      'Reddit r/Nepal post',
    ],
    goal: '500 signups',
  },

  day2to3: {
    activity: 'Influencer campaign',
    partnerships: '10 micro-influencers (10K-50K followers)',
    content: 'Unboxing/review videos',
    goal: '1,000 signups',
  },

  day4to7: {
    activity: 'Community engagement',
    tactics: [
      'Respond to all comments/messages',
      'Feature user stories on Instagram',
      'Run TikTok challenge (#LinkUpChallenge)',
      'Offer launch discount (50% off Plus for 3 months)',
    ],
    goal: '2,000 total signups',
  },

  launchPromo: {
    offer: 'First 500 couples get Plus free for 6 months',
  },
};

```

```

    code: 'LAUNCH2026',
    value: 'NPR 2,994 savings',
  },
};

```

11.3.2 Growth Strategy (Phase 2: Months 4-12)

Acquisition Channels:

```

const acquisitionChannels = {
  organic: {
    seo: {
      keywords: [
        'couples app Nepal',
        'private messaging for couples',
        'relationship app Kathmandu',
        'couple photo sharing',
      ],
      budget: 'NPR 20,000/month (content + SEO tools)',
      expectedROI: '200 signups/month',
    },

    contentMarketing: {
      blog: 'Relationship tips, date ideas in Nepal',
      youtube: 'Feature demos, couple vlogs',
      instagram: 'Daily tips, user stories',
      budget: 'NPR 30,000/month',
      expectedROI: '300 signups/month',
    },

    socialMedia: {
      platforms: ['Instagram', 'TikTok', 'Facebook'],
      strategy: 'User-generated content, challenges',
      budget: 'NPR 15,000/month (tools + ads)',
      expectedROI: '400 signups/month',
    },
  },

  paid: {
    facebookAds: {
      targeting: 'Nepal, 18-30, in relationship',
      budget: 'NPR 100,000/month',
      cpc: 'NPR 8-12',
      expectedClicks: '8,300/month',
      conversionRate: '5%',
      expectedSignups: '415/month',
      cac: 'NPR 241',
    },
  },
};

```

```

},

instagramAds: {
  format: 'Stories, Reels',
  budget: 'NPR 80,000/month',
  cpc: 'NPR 10-15',
  expectedClicks: '6,150/month',
  conversionRate: '4%',
  expectedSignups: '246/month',
  cac: 'NPR 325',
},

googleAds: {
  keywords: 'Couples app, relationship app',
  budget: 'NPR 50,000/month',
  cpc: 'NPR 15-25',
  expectedClicks: '2,500/month',
  conversionRate: '8%',
  expectedSignups: '200/month',
  cac: 'NPR 250',
},

tiktokAds: {
  format: 'In-feed videos',
  budget: 'NPR 60,000/month',
  cpm: 'NPR 300-500',
  expectedImpressions: '150,000/month',
  conversionRate: '0.5%',
  expectedSignups: '750/month',
  cac: 'NPR 80',
},

referral: {
  program: 'Invite friends, get 1 month Plus free',
  incentive: 'Both couples get reward',
  virality: 'k-factor 1.3 (each user refers 1.3)',
  budget: 'NPR 40,000/month (free subscriptions)',
  expectedSignups: '500/month',
  cac: 'NPR 80',
},

partnerships: {
  eSewa: 'Feature LinkUp in app recommendations',
  khalti: 'Co-marketing campaign',
  ncell: 'Data bundle partnerships',
  colleges: 'Campus ambassador program',
  budget: 'NPR 50,000/month',
}

```

```

    expectedSignups: '300/month',
  },
};

// Total Year 1 Acquisition
const year1Acquisition = {
  organic: '900 signups/month × 12 = 10,800',
  paid: '1,611 signups/month × 12 = 19,332',
  referral: '500 signups/month × 12 = 6,000',
  partnerships: '300 signups/month × 12 = 3,600',
  total: '39,732 signups (19,866 couples)',
  monthlyBudget: 'NPR 445,000',
  yearlyBudget: 'NPR 5,340,000',
  blendedCAC: 'NPR 269',
};

```

11.3.3 Retention Strategy

Engagement Tactics:

```

const retentionStrategy = {
  onboarding: {
    day1: 'Welcome tutorial, set up couple profile',
    day2: 'Send first message/photo prompt',
    day3: 'Introduce photo streak feature',
    day7: 'Unlock first achievement',
    day14: 'Invite to create first painting',
    day30: 'Premium trial offer',
    goal: '70% D30 retention',
  },

  habitFormation: {
    photoStreaks: 'Daily engagement driver',
    achievements: 'Weekly milestones to unlock',
    reminders: 'Important dates, anniversaries',
    notifications: 'Partner activity, streaks',
    goal: '3+ sessions/week per couple',
  },

  reengagement: {
    lapsedUsers: 'Email after 7 days inactive',
    winBackOffer: '50% off Plus for 1 month',
    personalizedContent: 'Missed milestones summary',
    goal: '30% reactivation rate',
  },

  communityBuilding: {

```

```

    coupleCircles: 'Encourage circle creation',
    userStories: 'Feature on social media',
    events: 'Virtual couple game nights',
    goal: 'Build belonging and FOMO',
  },
};

```

// Retention Metrics Targets

```

const retentionTargets = {
  d1: '80%',
  d7: '60%',
  d30: '70%',
  m6: '50%',
  m12: '40%',
};

```

11.4 Marketing Strategy

11.4.1 Brand Positioning

Brand Identity:

```

const brandIdentity = {
  tagline: 'Your Private Love Story',
  alternativeTaglines: [
    'Together, Privately',
    'Love, Creatively Shared',
    'Your Couple Universe',
  ],

  brandVoice: {
    tone: 'Warm, playful, intimate, supportive',
    language: 'Bilingual (English + Nepali)',
    style: 'Conversational, emoji-friendly',
    avoid: 'Corporate, preachy, intrusive',
  },

  messaging: {
    primary: 'Private space for couples to connect creatively',
    secondary: 'More than messaging - build your love story',
    emotional: 'Celebrate your unique relationship',
    rational: 'All your couple memories in one place',
  },

  visualIdentity: {
    colors: 'Warm pink, purple, orange gradient',
    style: 'Artistic, hand-drawn elements',
  }
};

```

```

    photography: 'Real Nepal couples, authentic moments',
    illustrations: 'Custom couple avatars',
  },
};

```

11.4.2 Marketing Campaigns

Campaign 1: Launch Campaign (#LinkUpNepal)

```

const launchCampaign = {
  name: '#LinkUpNepal - Your Love, Your Space',
  duration: 'Months 1-3',
  budget: 'NPR 800,000',

  tactics: {
    influencerMarketing: {
      microInfluencers: '20 influencers (10K-50K)',
      content: 'App review, couple challenges',
      budget: 'NPR 300,000',
      deliverables: '60 posts, 40 stories, 20 reels',
    },

    socialMediaAds: {
      platforms: 'Instagram, Facebook, TikTok',
      creatives: '10 video ads, 20 image ads',
      budget: 'NPR 250,000',
      targeting: 'Nepal, 18-35, in relationship',
    },

    contentMarketing: {
      blog: '15 articles on relationship tips',
      video: '10 YouTube tutorials',
      budget: 'NPR 100,000',
    },

    prLaunch: {
      outlets: 'TechSansar, Gadgetbyte, OnlineKhabar',
      format: 'Feature stories, founder interviews',
      budget: 'NPR 50,000',
    },

    launchEvent: {
      type: 'Virtual launch party',
      activities: 'Live demo, Q&A, giveaways',
      prizes: '50× 1-year Plus subscriptions',
      budget: 'NPR 100,000',
    },
  },
};

```



```

    },

    kpis: {
      reach: '500,000 people',
      signups: '5,000 couples',
      mediaImpressions: '2M+',
    },
  };

Campaign 2: Valentines Day 2026

const valentinesCampaign = {
  name: 'Love Elevated',
  timing: 'January 25 - February 20',
  budget: 'NPR 600,000',

  offers: {
    premium: '60% off Plus - NPR 199/month for 3 months',
    freeTrial: '14-day free trial of Pro',
    giveaway: 'Win romantic dinner for 50 couples',
  },

  content: {
    challenge: '#LinkUpLoveStories - Share your story',
    prizes: 'Best 10 couples get 1-year Pro free',
    ugc: 'Feature winning couples on social media',
  },

  partnerships: {
    restaurants: '10 Kathmandu restaurants (date vouchers)',
    florists: 'Flower delivery discounts',
    hotels: 'Staycation packages',
  },

  expectedResults: {
    signups: '3,000 couples',
    conversions: '500 paid subscriptions',
    revenue: 'NPR 300,000',
    roi: '50%',
  },
};

```

Campaign 3: Photo Streak Challenge

```

const streakCampaign = {
  name: '365 Days of Love',
  duration: 'Ongoing (Year 1)',
  budget: 'NPR 300,000/year',

  mechanics: {

```

```

challenge: 'Maintain photo streak for milestones',
milestones: ['7 days', '30 days', '100 days', '365 days'],
rewards: {
  7: 'Badge + social media feature',
  30: 'Free month of Plus',
  100: 'Exclusive emoji pack',
  365: '1-year Pro + physical photo book',
},
},

promotion: {
  leaderboard: 'Public streak leaderboard',
  communitySpotlight: 'Weekly couple features',
  socialProof: 'Share achievements on social',
},

expectedImpact: {
  engagement: '50% daily active users',
  retention: '+20% D30 retention',
  virality: 'k-factor 1.5',
},
};

```

11.5 Financial Projections (5-Year, NPR)

11.5.1 Revenue Projections

Year 1 (2026):

```

const year1Financials = {
  users: {
    signups: 19_866, // couples
    mau: 12_000,    // monthly active couples

    freeTier: 10_500, // 87.5%
    plusTier: 1_200,  // 10%
    proTier: 300,     // 2.5%
  },

  revenue: {
    subscriptions: {
      plus: {
        monthly: 600, // 50% of Plus users
        yearly: 600,   // 50% of Plus users
        monthlyRevenue: '600 × NPR 499 = NPR 299,400/month',
        yearlyRevenue: '600 × NPR 4,999 = NPR 2,999,400/year',
        totalMonthly: 'NPR 549,350/month',
      },
    },
  },
};

```

```

    totalYearly: 'NPR 6,592,200',
  },

  pro: {
    monthly: 150,      // 50% of Pro users
    yearly: 150,       // 50% of Pro users
    monthlyRevenue: '150 × NPR 999 = NPR 149,850/month',
    yearlyRevenue: '150 × NPR 9,999 = NPR 1,499,850/year',
    totalMonthly: 'NPR 274,838/month',
    totalYearly: 'NPR 3,298,050',
  },

  totalSubscription: 'NPR 9,890,250',
},

inAppPurchases: {
  storage: 'NPR 400,000',
  themes: 'NPR 300,000',
  freezes: 'NPR 200,000',
  emojis: 'NPR 150,000',
  total: 'NPR 1,050,000',
},

totalRevenue: 'NPR 10,940,250',
arr: 'NPR 10,940,250',
mrr: 'NPR 911,688',
},

costs: {
  development: {
    team: 'NPR 4,800,000 (4 developers × NPR 100K/month)',
    infrastructure: 'NPR 600,000 (AWS, CDN, databases)',
    tools: 'NPR 300,000 (licenses, SaaS)',
    total: 'NPR 5,700,000',
  },

  marketing: {
    paid: 'NPR 3,600,000 (NPR 300K/month)',
    organic: 'NPR 780,000 (content, tools)',
    influencers: 'NPR 600,000',
    events: 'NPR 360,000',
    total: 'NPR 5,340,000',
  },

  operations: {
    salaries: 'NPR 2,400,000 (PM, designer, support × 2)',
    office: 'NPR 480,000',
    legal: 'NPR 240,000',
  }
}

```

```

    misc: 'NPR 360,000',
    total: 'NPR 3,480,000',
  },

  totalCosts: 'NPR 14,520,000',
},

profitability: {
  grossProfit: 'NPR 5,240,250 (52% margin)',
  netProfit: '-NPR 3,579,750',
  burnRate: 'NPR 298,313/month',
  runway: '12 months (with NPR 3.5M seed funding)',
},
};

```

Year 2 (2027):

```

const year2Financials = {
  users: {
    signups: 45_000, // cumulative couples
    mau: 30_000,

    freeTier: 24_000, // 80%
    plusTier: 4_500,  // 15%
    proTier: 1_500,   // 5%
  },

  revenue: {
    subscriptions: {
      plus: 'NPR 24,724,500',
      pro: 'NPR 12,362,250',
      total: 'NPR 37,086,750',
    },
    inAppPurchases: 'NPR 3,150,000',
    totalRevenue: 'NPR 40,236,750',
    arr: 'NPR 40,236,750',
    growth: '268% YoY',
  },

  costs: {
    development: 'NPR 7,200,000 (scale team to 6)',
    infrastructure: 'NPR 1,440,000 (3x growth)',
    marketing: 'NPR 8,400,000 (aggressive growth)',
    operations: 'NPR 4,800,000',
    totalCosts: 'NPR 21,840,000',
  },

  profitability: {
    grossProfit: 'NPR 38,796,750 (96% margin)',

```

```

    netProfit: 'NPR 18,396,750 (46% net margin)',
    profitable: true,
  },
};

```

Year 3 (2028):

```

const year3Financials = {
  users: {
    signups: 90_000,
    mau: 65_000,

    freeTier: 45_000, // 70%
    plusTier: 13_500, // 20%
    proTier: 6_500,   // 10%
  },

  revenue: {
    subscriptions: {
      plus: 'NPR 74,173,500',
      pro: 'NPR 53,569,750',
      total: 'NPR 127,743,250',
    },
    inAppPurchases: 'NPR 9,000,000',
    circles: 'NPR 2,500,000',
    totalRevenue: 'NPR 139,243,250',
    growth: '246% YoY',
  },

  costs: {
    development: 'NPR 12,000,000',
    infrastructure: 'NPR 3,600,000',
    marketing: 'NPR 14,400,000',
    operations: 'NPR 7,200,000',
    totalCosts: 'NPR 37,200,000',
  },

  profitability: {
    grossProfit: 'NPR 135,643,250',
    netProfit: 'NPR 102,043,250 (73% net margin)',
  },
};

```

Year 4-5 Summary:

```

const year4to5 = {
  year4: {
    users: 140_000,
    mau: 105_000,
    revenue: 'NPR 252,000,000',
  },
};

```

```

    netProfit: 'NPR 189,000,000 (75% margin)',
  },

  year5: {
    users: 200_000,
    mau: 155_000,
    revenue: 'NPR 405,000,000',
    netProfit: 'NPR 324,000,000 (80% margin)',
  },
};

```

11.5.2 Unit Economics

Customer Acquisition Cost (CAC):

```

const cacAnalysis = {
  year1: {
    totalMarketingSpend: 'NPR 5,340,000',
    newCouples: 19_866,
    cac: 'NPR 269',
    breakdown: {
      organic: 'NPR 80 (referrals, SEO)',
      paid: 'NPR 250-325 (FB, IG, Google)',
      influencer: 'NPR 200',
      blended: 'NPR 269',
    },
  },
  year2: {
    cac: 'NPR 336', // Slightly higher as market saturates
  },
  year3to5: {
    cac: 'NPR 400-450', // Organic growth reduces blended CAC
  },
};

```

Lifetime Value (LTV):

```

const ltvAnalysis = {
  assumptions: {
    avgRevenuePerUser: {
      free: 'NPR 0/month',
      plus: 'NPR 450/month (mix of monthly/yearly)',
      pro: 'NPR 900/month',
    },
    avgLifetime: {
      free: '6 months',

```

```

    plus: '24 months',
    pro: '36 months',
  },
  churnRate: {
    free: '50%/year',
    plus: '25%/year',
    pro: '15%/year',
  },
},

calculations: {
  freeUserLTV: 'NPR 0',
  plusUserLTV: 'NPR 450 × 24 months = NPR 10,800',
  proUserLTV: 'NPR 900 × 36 months = NPR 32,400',

  blendedLTV: {
    year1: '(0.875 × 0) + (0.10 × 10,800) + (0.025 × 32,400) = NPR 1,890',
    year3: '(0.70 × 0) + (0.20 × 10,800) + (0.10 × 32,400) = NPR 5,400',
  },
},

ltvCacRatio: {
  year1: '1,890 / 269 = 7.0× (Excellent)',
  year3: '5,400 / 450 = 12.0× (Exceptional)',
  target: '3-5× (healthy SaaS)',
},

paybackPeriod: {
  plus: '269 / 450 = 0.6 months',
  pro: '269 / 900 = 0.3 months',
  target: '<12 months',
},
};

```

11.5.3 Key Metrics & Targets

North Star Metric:

```

const northStarMetric = {
  metric: 'Weekly Active Couples (WAC)',
  definition: 'Couples with 3+ sessions/week',
  rationale: 'Engagement → retention → monetization',

  targets: {
    year1: '8,000 WAC',
    year2: '20,000 WAC',
    year3: '45,000 WAC',
  }
}

```

```

    year5: '100,000 WAC',
  },
};

```

Secondary Metrics:

```

const secondaryMetrics = {
  growth: {
    mau: 'Monthly Active Users (couples)',
    signupGrowth: '% MoM growth in signups',
    k_factor: 'Viral coefficient (referrals)',
  },

  engagement: {
    dau_mau: 'Daily Active / Monthly Active ratio',
    sessionsPerWeek: 'Avg sessions per couple per week',
    streakParticipation: '% couples with active streaks',
    messagesPerDay: 'Avg messages per couple per day',
  },

  monetization: {
    conversionRate: 'Free → Paid %',
    arpu: 'Average Revenue Per User',
    arr: 'Annual Recurring Revenue',
    churn: 'Monthly churn rate',
  },

  targets: {
    year1: {
      mau: 12_000,
      dau_mau: '40%',
      conversionRate: '12.5%',
      arpu: 'NPR 76/month',
      churn: '8%/month',
    },

    year3: {
      mau: 65_000,
      dau_mau: '55%',
      conversionRate: '30%',
      arpu: 'NPR 178/month',
      churn: '5%/month',
    },
  },
};

```


11.6 Funding & Investment

11.6.1 Funding Requirements

Seed Round (Pre-launch to Year 1):

```
const seedRound = {
  amount: 'NPR 5,000,000 ( USD 37,500)',
  timing: 'Month -2 (before launch)',
  use: {
    development: 'NPR 2,000,000 (40%)',
    marketing: 'NPR 1,800,000 (36%)',
    operations: 'NPR 800,000 (16%)',
    reserve: 'NPR 400,000 (8%)',
  },
  runway: '12-15 months',
  milestones: [
    'Launch product (Month 1)',
    'Reach 5,000 couples (Month 3)',
    'Hit NPR 500K MRR (Month 9)',
    '15,000 MAU couples (Month 12)',
  ],
  equity: '15-20%',
  valuation: 'NPR 25M-30M pre-money',
};
```

Series A (Year 2):

```
const seriesA = {
  amount: 'NPR 20,000,000 ( USD 150,000)',
  timing: 'Month 15-18',
  use: {
    scaleTech: 'NPR 6,000,000 (30%)',
    aggressiveMarketing: 'NPR 10,000,000 (50%)',
    teamExpansion: 'NPR 3,000,000 (15%)',
    reserve: 'NPR 1,000,000 (5%)',
  },
  runway: '18-24 months',
  milestones: [
    'NPR 3M MRR',
    '50,000 MAU couples',
    '20% paid conversion',
    'Break even or profitable',
  ],
  equity: '20-25%',
  valuation: 'NPR 80M-100M pre-money',
};
```

11.6.2 Exit Strategy

Potential Exits:

```
const exitScenarios = {
  acquisition: {
    potentialAcquirers: [
      'Viber (Rakuten)',
      'Meta (WhatsApp owner)',
      'Snapchat',
      'Asian tech companies',
      'Nepal conglomerates (Chaudhary Group, IME Group)',
    ],
    timing: 'Year 4-5',
    valuation: 'NPR 500M - 1B (5-10× revenue)',
  },

  ipo: {
    market: 'Nepal Stock Exchange (NEPSE)',
    timing: 'Year 6-7',
    requirements: 'NPR 100M+ revenue, profitability',
    valuation: 'NPR 1.5B+',
  },

  sustainableGrowth: {
    strategy: 'Bootstrap after profitability (Year 2)',
    dividends: 'Distribute profits to founders/investors',
    longTerm: 'Build sustainable Nepal tech company',
  },
};
```

11.7 Risk Analysis & Mitigation

Key Risks:

```
const risks = {
  market: {
    risk: 'Low willingness to pay for apps in Nepal',
    likelihood: 'Medium',
    impact: 'High',
    mitigation: [
      'Strong free tier to build user base',
      'Low pricing (NPR 499 vs Netflix NPR 1,100)',
      'Clear value proposition',
      'Focus on emotional value',
    ],
  },
};
```

```

competition: {
  risk: 'WhatsApp/Telegram add couple features',
  likelihood: 'Low',
  impact: 'High',
  mitigation: [
    'Build network effects (circles, friends)',
    'Creative differentiation (painting, scribbles)',
    'First-mover advantage in Nepal',
    'Deep couple-specific features',
  ],
},

technical: {
  risk: 'Infrastructure costs exceed projections',
  likelihood: 'Medium',
  impact: 'Medium',
  mitigation: [
    'Optimize media storage (compression)',
    'CDN usage for Nepal region',
    'Gradual feature rollout',
    'Monitor unit economics closely',
  ],
},

regulatory: {
  risk: 'Data privacy regulations',
  likelihood: 'Low (Nepal)',
  impact: 'Medium',
  mitigation: [
    'GDPR-compliant from day 1',
    'Local data storage if required',
    'Privacy-first design',
    'Legal counsel on retainer',
  ],
},

userGrowth: {
  risk: 'Slower adoption than projected',
  likelihood: 'Medium',
  impact: 'High',
  mitigation: [
    'Aggressive referral program',
    'Pivot marketing channels based on data',
    'Extend runway with conservative burn',
    'Focus on retention over acquisition',
  ],
},

```

```

payment: {
  risk: 'Payment gateway failures/limitations',
  likelihood: 'Medium',
  impact: 'Medium',
  mitigation: [
    'Multiple payment options (eSewa, Khalti, cards)',
    'Bank transfer option',
    'Prepaid cards/vouchers',
    'Partner with telecom (mobile billing)',
  ],
},
};

```

Chapter 11 Status: Complete

Summary: - Market Analysis: Nepal-specific with 320,000 tech-savvy couples TAM, NPR 960M market value - Business Model: Freemium with 3 tiers (Free, Plus NPR 499/month, Pro NPR 999/month) - Pricing Strategy: Positioned lower than Netflix, comparable to Spotify, optimized for Nepal purchasing power - Go-to-Market: Phased launch strategy with influencers, paid ads (Facebook, Instagram, TikTok), referral program - Marketing: 3 major campaigns (Launch, Valentines, Streak Challenge) with NPR 1.7M Year 1 budget - Financial Projections: * Year 1: NPR 10.9M revenue, -NPR 3.6M net (investment phase) * Year 2: NPR 40.2M revenue, NPR 18.4M net profit (profitable!) * Year 3: NPR 139.2M revenue, NPR 102M net profit (73% margin) * Year 5: NPR 405M revenue, NPR 324M net profit (80% margin) - Unit Economics: CAC NPR 269, LTV NPR 1,890-5,400, LTV:CAC ratio 7-12X (excellent) - Funding: NPR 5M seed, NPR 20M Series A - Payment Integration: eSewa, Khalti, IME Pay, bank transfers

Next Chapter: Chapter 12: Development Roadmap, Security & Compliance

Chapter 12: Development Roadmap, Security & Compliance

Overview

This chapter provides a comprehensive roadmap for LinkUps development, launch, and scaling phases, alongside critical security, privacy, and compliance measures required for user trust and regulatory adherence.

Chapter contents: - Development Roadmap - Phased delivery timeline (MVP V1 V2) - Technology Stack Details - Platform and infrastructure decisions - Security Architecture - End-to-end encryption and data protection - Privacy Framework - GDPR compliance and data retention - Compliance Requirements - Nepal legal requirements and app store guidelines - Testing Strategy - Quality assurance, performance, and security testing - DevOps & Infrastructure - CI/CD, monitoring, and scaling - Launch Checklist - Pre-launch verification procedures

Development Philosophy: - Rapid iteration - Launch MVP in 3 months - Security-first architecture - Built-in protection from foundation - Privacy by design - Default privacy-preserving settings -

Scalable infrastructure - Architected for growth to millions of users - Continuous deployment - Regular update cadence

12.1 Development Roadmap

12.1.1 Phase 1: MVP Development (Months 1-3)

Objective: Launch core couples app with essential features

Month 1: Foundation

```
const month1Deliverables = {
  week1: {
    infrastructure: [
      'AWS account setup and configuration',
      'PostgreSQL database provisioning',
      'Redis cache setup',
      'S3 buckets for media storage',
      'CDN configuration (CloudFront)',
      'Domain and SSL certificates',
    ],

    backend: [
      'Monorepo setup (Turborepo)',
      'Express.js API scaffolding',
      'Authentication system (JWT)',
      'Database schema migration setup',
      'TypeScript configuration',
    ],

    frontend: [
      'Next.js web app initialization',
      'Design system foundation',
      'Routing structure',
      'State management (Zustand)',
    ],
  },

  week2: {
    authentication: [
      'User registration API',
      'Email verification flow',
      'Login/logout endpoints',
      'Password reset functionality',
      'JWT token management',
    ],

    userManagement: [
```

```

        'User profile CRUD',
        'Avatar upload',
        'Settings page',
    ],
},

week3: {
    coupleFeatures: [
        'Couple invitation system',
        'Accept/decline invitations',
        'Couple profile page',
        'Partner connection flow',
    ],

    messaging: [
        'Text messaging API',
        'Real-time messaging (Socket.io setup)',
        'Message history retrieval',
        'Message UI components',
    ],
},

week4: {
    mediaSharing: [
        'Photo upload to S3',
        'Image processing (thumbnails)',
        'Media gallery API',
        'Photo viewing UI',
    ],

    testing: [
        'Unit tests for auth',
        'Integration tests for messaging',
        'Manual QA of core flows',
    ],
},
};

```

Month 2: Core Features

```

const month2Deliverables = {
    week5: {
        photoStreaks: [
            'Streak logic implementation',
            'Daily photo upload flow',
            'Streak counter and history',
            'Freeze mechanism',
            'Streak notifications',
        ],
    },
};

```

```

},

week6: {
  achievements: [
    'Achievement definition system',
    'Achievement unlock logic',
    'User achievement tracking',
    'Achievement showcase UI',
    'Notification on unlock',
  ],

  importantDates: [
    'Date creation and management',
    'Reminder system',
    'Anniversary calculations',
    'Date celebration tracking',
  ],
},

week7: {
  creative: [
    'Basic scribble canvas',
    'Real-time scribble sync',
    'Save scribble as message',
    'Color and brush options',
  ],

  socialFeatures: [
    'SingleFriend invitation flow',
    'Friend list management',
    'Permission system',
  ],
},

week8: {
  notifications: [
    'Push notification setup (FCM, APNs)',
    'Notification preference management',
    'Email notifications',
    'In-app notifications UI',
  ],

  polish: [
    'UI/UX refinements',
    'Performance optimization',
    'Bug fixes from testing',
    'Mobile responsive design',
  ],
}

```

```
},  
};
```

Month 3: Mobile Apps & Launch Prep

```
const month3Deliverables = {  
  week9: {  
    ios: [  
      'SwiftUI app structure',  
      'API client integration',  
      'Authentication flow',  
      'Chat interface',  
      'Photo upload',  
    ],  
  },  
  
  week10: {  
    android: [  
      'Kotlin + Compose setup',  
      'API integration',  
      'Authentication',  
      'Messaging UI',  
      'Media handling',  
    ],  
  },  
  
  week11: {  
    polish: [  
      'Cross-platform feature parity',  
      'Push notifications on mobile',  
      'Performance optimization',  
      'Security audit',  
      'Accessibility review',  
    ],  
  
    infrastructure: [  
      'Production environment setup',  
      'Monitoring and logging',  
      'Backup systems',  
      'Load balancing',  
    ],  
  },  
  
  week12: {  
    launchPrep: [  
      'Beta testing with 50 couples',  
      'Bug fixes from beta',  
      'App store submissions',  
      'Landing page live',  
    ],  
  },  
};
```



```

        'Marketing materials ready',
        'Support system setup',
    ],
},
};

// MVP Feature Set
const mvpFeatures = [
    'User authentication and profiles',
    'Couple pairing and invitations',
    'Real-time text messaging',
    'Photo sharing and gallery',
    'Photo streaks with freezes',
    'Basic achievements',
    'Important dates and reminders',
    'Basic scribble tool',
    'SingleFriend (up to 3)',
    'Push notifications',
    'Web app (Next.js)',
    'iOS app (native)',
    'Android app (native)',
];

```

12.1.2 Phase 2: V1.0 - Enhanced Features (Months 4-6)

Objective: Add differentiation features and improve retention

```

const v1Roadmap = {
  month4: {
    collaborative: [
      'Collaborative painting canvas',
      'Real-time stroke synchronization',
      'Painting save and gallery',
      'Undo/redo functionality',
      'Multiple brush types',
    ],

    media: [
      'Video upload and playback',
      'Voice message recording',
      'Media albums creation',
      'Advanced search and filters',
      'Batch photo operations',
    ],
  },

  month5: {

```

```

    circles: [
      'Couple Circles creation',
      'Circle invitations',
      'Circle posts and comments',
      'Circle media sharing',
      'Circle events',
    ],

    entertainment: [
      'Watch party framework',
      'YouTube integration',
      'Basic playlist sharing',
    ],
  },

  month6: {
    customization: [
      'Custom emoji creator',
      'Theme customization',
      'Soundboard feature',
      'Custom notification sounds',
    ],

    analytics: [
      'Relationship statistics',
      'Activity insights',
      'Milestone tracker',
      'Export data feature',
    ],

    premium: [
      'Subscription payment flow (eSewa, Khalti)',
      'Premium feature gates',
      'Billing management',
      'Receipt generation',
    ],
  },
};

```

12.1.3 Phase 3: V2.0 - Scale & Optimize (Months 7-12)

Objective: Scale infrastructure, advanced features, monetization

```

const v2Roadmap = {
  quarter3: {
    performance: [
      'Database query optimization',

```

```

        'CDN optimization for Nepal',
        'Image compression improvements',
        'Lazy loading and code splitting',
        'Service worker for offline support',
    ],

    advanced: [
        'Video calling (WebRTC)',
        'Screen sharing',
        'Live streaming between couples',
        'Advanced painting tools',
        'AR filters (future)',
    ],

    social: [
        'Hall of Fame leaderboards',
        'Public couple profiles (optional)',
        'Couple challenges',
        'Community events',
    ],
},

quarter4: {
    monetization: [
        'In-app purchases (storage, themes)',
        'Virtual gifts',
        'Premium circles',
        'Sponsored content (careful approach)',
    ],

    integrations: [
        'Spotify full integration',
        'Netflix watch party',
        'Google Calendar sync',
        'Third-party API access',
    ],

    ai: [
        'AI-powered date suggestions',
        'Smart photo organization',
        'Sentiment analysis for messages',
        'Relationship health insights',
    ],
},
};

```

12.1.4 Release Cycle

Continuous Deployment:

```
const releaseCycle = {
  web: {
    frequency: 'Multiple times per day',
    process: 'Automated CI/CD',
    rollback: 'Instant rollback capability',
  },

  mobile: {
    ios: {
      frequency: 'Weekly to bi-weekly',
      process: 'TestFlight beta → App Store review',
      reviewTime: '24-48 hours',
    },

    android: {
      frequency: 'Weekly',
      process: 'Internal testing → Beta → Production',
      staggeredRollout: '10% → 50% → 100%',
    },
  },

  versionScheme: 'Semantic Versioning (MAJOR.MINOR.PATCH)',

  example: {
    mvp: 'v0.9.0 (beta)',
    launch: 'v1.0.0',
    features: 'v1.1.0, v1.2.0',
    hotfix: 'v1.1.1',
  },
};
```

12.2 Technology Stack (Final Decisions)

12.2.1 Complete Tech Stack

Frontend:

```
const frontendStack = {
  web: {
    framework: 'Next.js 14 (App Router)',
    language: 'TypeScript 5.3',
    styling: 'Tailwind CSS 3.4',
    stateManagement: 'Zustand',
    dataFetching: 'React Query (TanStack Query)',
  },
};
```

```

    forms: 'React Hook Form + Zod validation',
    animations: 'Framer Motion',
    charts: 'Recharts',
    dateTime: 'date-fns',
    icons: 'Lucide React',
  },

  ios: {
    language: 'Swift 5.9',
    ui: 'SwiftUI',
    architecture: 'MVVM',
    networking: 'URLSession + Combine',
    storage: 'Core Data + UserDefaults',
    images: 'Kingfisher',
    realtime: 'Socket.IO Swift Client',
  },

  android: {
    language: 'Kotlin 1.9',
    ui: 'Jetpack Compose',
    architecture: 'MVVM',
    di: 'Hilt',
    networking: 'Retrofit + OkHttp',
    storage: 'Room Database',
    images: 'Coil',
    realtime: 'Socket.IO Kotlin Client',
  },
};

```

Backend:

```

const backendStack = {
  api: {
    runtime: 'Node.js 20 LTS',
    framework: 'Express.js 4.18',
    language: 'TypeScript 5.3',
    validation: 'Zod',
    authentication: 'JWT (jsonwebtoken)',
    rateLimit: 'express-rate-limit',
  },

  realtime: {
    websocket: 'Socket.IO 4.6',
    pubsub: 'Redis Pub/Sub',
  },

  backgroundJobs: {
    queue: 'Bull (Redis-based)',
    scheduler: 'node-cron',
  },
};

```

```

    tasks: [
      'Send notifications',
      'Process media',
      'Calculate streaks',
      'Clean up expired data',
    ],
  },

  databases: {
    primary: 'PostgreSQL 16',
    cache: 'Redis 7',
    search: 'PostgreSQL Full-Text Search',
    fileStorage: 'AWS S3',
  },
};

Infrastructure:

const infrastructure = {
  hosting: {
    compute: 'AWS EC2 / ECS (containerized)',
    serverless: 'AWS Lambda (for background tasks)',
    database: 'AWS RDS PostgreSQL',
    cache: 'AWS ElastiCache (Redis)',
    storage: 'AWS S3 + CloudFront CDN',
  },

  devOps: {
    containerization: 'Docker',
    orchestration: 'Docker Compose (development)',
    ci_cd: 'GitHub Actions',
    monitoring: 'Datadog / AWS CloudWatch',
    logging: 'AWS CloudWatch Logs',
    errorTracking: 'Sentry',
    analytics: 'Mixpanel + Google Analytics 4',
  },

  domains: {
    api: 'api.linkup.com.np',
    web: 'app.linkup.com.np',
    cdn: 'cdn.linkup.com.np',
    ws: 'ws.linkup.com.np',
  },
};

```

12.3 Security Architecture

12.3.1 Authentication & Authorization

Multi-Layer Security:

```
const authSecurity = {
  passwordRequirements: {
    minLength: 8,
    requireUppercase: true,
    requireLowercase: true,
    requireNumber: true,
    requireSpecialChar: false, // For better Nepal UX
    preventCommon: true, // Check against common password list
  },

  passwordHashing: {
    algorithm: 'bcrypt',
    saltRounds: 12,
    example: 'bcrypt.hash(password, 12)',
  },

  jwtTokens: {
    accessToken: {
      expiry: '15 minutes',
      payload: ['userId', 'coupleId', 'email'],
      algorithm: 'HS256',
    },

    refreshToken: {
      expiry: '30 days',
      storage: 'HTTP-only secure cookie',
      rotation: true, // Rotate on each use
    },
  },

  sessionManagement: {
    maxActiveSessions: 5, // Per user
    deviceTracking: true,
    ipLogging: true,
    suspiciousActivityDetection: true,
  },

  mfa: {
    available: 'Email-based OTP (future: SMS, authenticator app)',
    enforcement: 'Optional but recommended',
  },
};
```

Rate Limiting:

```
const rateLimiting = {
  authentication: {
    login: '5 attempts per 15 minutes',
    register: '3 attempts per hour',
    passwordReset: '3 attempts per hour',
  },

  api: {
    general: '100 requests per minute',
    messaging: '30 messages per minute',
    mediaUpload: '20 uploads per hour',
    streakPhoto: '1 photo per day',
  },

  implementation: {
    library: 'express-rate-limit',
    storage: 'Redis',
    strategy: 'Sliding window',
  },
};
```

12.3.2 Data Encryption

End-to-End Encryption (Future Phase):

```
const encryptionStrategy = {
  inTransit: {
    protocol: 'TLS 1.3',
    certificates: 'Let's Encrypt (auto-renewed)',
    hsts: 'Enabled (max-age=31536000)',
    enforcement: 'All HTTP redirected to HTTPS',
  },

  atRest: {
    database: {
      encryption: 'AWS RDS encryption (AES-256)',
      backups: 'Encrypted snapshots',
    },

    media: {
      s3Encryption: 'AES-256 (SSE-S3)',
      keyManagement: 'AWS KMS',
    },

    sensitiveFields: {
```



```

        implementation: 'Application-level encryption',
        fields: ['email (hashed)', 'password (bcrypt)', 'tokens'],
        library: 'crypto (Node.js built-in)',
    },
},

messageEncryption: {
    phase1: 'Server-side TLS only',
    phase2: 'End-to-end encryption (Signal Protocol)',
    implementation: 'Future roadmap (Month 12+)',
},
};

Security Headers:

const securityHeaders = {
  helmet: {
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'", "'unsafe-inline'", 'cdn.linkup.com.np'],
        styleSrc: ["'self'", "'unsafe-inline'"],
        imgSrc: ["'self'", 'data:', 'cdn.linkup.com.np', 's3.amazonaws.com'],
        connectSrc: ["'self'", 'api.linkup.com.np', 'ws.linkup.com.np'],
      },
    },

    hsts: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true,
    },

    xssFilter: true,
    noSniff: true,
    frameguard: { action: 'deny' },
    referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
  },
};

```

12.3.3 Input Validation & Sanitization

Defense Against Attacks:

```

const inputValidation = {
  sqlInjection: {
    prevention: 'Parameterized queries (PostgreSQL prepared statements)',
    orm: 'Never use raw SQL with user input',
  },
};

```

```

    validation: 'Zod schema validation',
  },

  xss: {
    prevention: 'DOMPurify for HTML sanitization',
    encoding: 'Escape all user input before rendering',
    csp: 'Content Security Policy headers',
  },

  csrf: {
    prevention: 'CSRF tokens for state-changing operations',
    sameSite: 'Cookies with SameSite=Strict',
  },

  fileUpload: {
    validation: [
      'File type checking (magic bytes)',
      'File size limits (10MB photos, 100MB videos)',
      'Virus scanning (ClamAV)',
      'Strip EXIF metadata (privacy)',
    ],

    allowedTypes: {
      images: ['image/jpeg', 'image/png', 'image/webp', 'image/heic'],
      videos: ['video/mp4', 'video/quicktime'],
      audio: ['audio/mpeg', 'audio/mp4', 'audio/webm'],
    },
  },
};

```

12.4 Privacy Framework

12.4.1 Privacy by Design

Data Minimization:

```

const dataMinimization = {
  principles: [
    'Collect only necessary data',
    'Default privacy settings favor users',
    'Explicit consent for data sharing',
    'Easy data export and deletion',
  ],

  dataCollected: {
    required: [
      'Email (for account)',

```

```

    'Username (for identity)',
    'Password (hashed)',
    'Couple relationship data',
  ],

  optional: [
    'Birthday (for age verification, reminders)',
    'Avatar (for personalization)',
    'Bio (for profile)',
    'Location (for features, never shared)',
  ],

  notCollected: [
    'Real name (unless user provides)',
    'Phone number (unless MFA)',
    'Government ID',
    'Financial information (handled by payment gateways)',
  ],
},

dataSharing: {
  default: 'Private between couple only',
  singleFriends: 'Explicit permission-based',
  circles: 'Opt-in for each circle',
  public: 'Never (no public profiles)',
},
};

```

12.4.2 GDPR Compliance

User Rights Implementation:

```

const gdprCompliance = {
  rightToAccess: {
    feature: 'Export all data',
    format: 'JSON + ZIP file with media',
    timeline: 'Instant download',
    endpoint: 'GET /v1/users/me/export',
  },

  rightToErasure: {
    feature: 'Delete account',
    process: [
      '1. User confirms with password',
      '2. 30-day grace period (account suspended)',
      '3. Permanent deletion after 30 days',
      '4. Partner notified',
    ],
  },
};

```

```

    ],
    dataRemoved: [
        'User profile',
        'Messages (from couple)',
        'Media uploads',
        'Achievements',
        'All personal data',
    ],
    dataRetained: [
        'Anonymized analytics (no PII)',
        'Audit logs (legal requirement, 90 days)',
    ],
},

rightToRectification: {
    feature: 'Edit profile anytime',
    scope: 'All user-provided data',
},

rightToPortability: {
    format: 'Machine-readable JSON',
    included: 'All user data',
},

rightToObjection: {
    marketing: 'Opt-out of all marketing emails',
    analytics: 'Disable analytics tracking',
},

consent: {
    explicit: 'Checkbox for Terms & Privacy Policy',
    granular: 'Separate consent for optional features',
    withdrawable: 'Easy opt-out mechanisms',
},
};

```

Privacy Policy Requirements:

```

const privacyPolicy = {
    sections: [
        'Data we collect and why',
        'How we use your data',
        'Data sharing (third parties)',
        'Data retention periods',
        'Your rights (GDPR)',
        'Cookies and tracking',
        'Security measures',
        'Children\'s privacy (18+ only)',
        'Changes to policy',
    ],
};

```

```

    'Contact information',
  ],

  language: 'Simple, clear English (+ Nepali translation)',
  accessibility: 'Easy to find and read',
  updates: 'Notify users of material changes',
};

```

12.4.3 Data Retention

Retention Policies:

```

const dataRetention = {
  activeUsers: {
    messages: 'Forever (couple\'s choice)',
    media: 'Forever (with storage limits)',
    achievements: 'Forever',
    activityLogs: '90 days',
  },

  deletedAccounts: {
    gracePeriod: '30 days (recoverable)',
    afterGrace: 'Permanent deletion',
    exceptions: [
      'Anonymized analytics (no PII)',
      'Legal hold data (court orders)',
    ],
  },

  inactiveAccounts: {
    definition: 'No login for 2 years',
    action: 'Email warning → 30 days → account suspension',
    dataArchival: 'Move to cold storage',
  },

  mediaStorage: {
    activeTier: 'Hot storage (S3 Standard)',
    inactiveTier: 'After 1 year no access → Glacier',
    deletion: 'User-initiated only',
  },

  backups: {
    retention: '30 days for hot backups',
    archives: '1 year in cold storage',
    destruction: 'Secure deletion after retention',
  },
};

```

12.5 Compliance Requirements

12.5.1 Nepal Legal Requirements

Company Registration:

```
const nepalCompliance = {
  businessRegistration: {
    type: 'Private Limited Company',
    authority: 'Office of Company Registrar (OCR)',
    requirements: [
      'Minimum 2 directors (Nepal citizens or residents)',
      'Minimum capital: NPR 100,000',
      'Registered office in Nepal',
      'Company name approval',
    ],
  },

  taxCompliance: {
    pan: 'Permanent Account Number',
    vat: 'VAT registration if revenue > NPR 5M/year',
    incomeTax: '25% corporate tax rate',
    tds: 'Tax Deducted at Source for payments',
  },

  dataProtection: {
    law: 'Electronic Transaction Act 2063',
    requirements: [
      'User consent for data collection',
      'Secure data storage',
      'Data breach notification',
    ],
    note: 'Nepal doesn\'t have comprehensive GDPR-like law yet',
  },

  contentRegulation: {
    law: 'Electronic Transaction Act',
    prohibition: [
      'Illegal content',
      'Defamation',
      'Hate speech',
      'Obscene material',
    ],
    moderation: 'User-reported content review system',
  },

  paymentGateway: {
```

```

    regulation: 'Nepal Rastra Bank (NRB) guidelines',
    requirements: [
      'Use licensed payment gateways (eSewa, Khalti)',
      'No direct card processing without license',
      'Transaction records for 5 years',
    ],
  },
};

```

12.5.2 International Compliance

App Store Requirements:

```

const appStoreCompliance = {
  apple: {
    guidelines: 'App Store Review Guidelines',
    requirements: [
      'Privacy Policy (publicly accessible URL)',
      'Terms of Service',
      'Age rating: 17+ (due to user-generated content)',
      'In-app purchase compliance',
      'Data usage disclosure',
      'No hidden features',
    ],

    privacy: {
      appPrivacyDetails: 'Nutrition label disclosure',
      tracking: 'ATT (App Tracking Transparency) consent',
      dataTypes: [
        'Contact Info (email)',
        'User Content (messages, photos)',
        'Identifiers (user ID)',
        'Usage Data (analytics)',
      ],
    },
  },

  google: {
    guidelines: 'Google Play Developer Policy',
    requirements: [
      'Privacy Policy',
      'Data safety section',
      'Content rating (IARC)',
      'Restricted content compliance',
      'User-generated content moderation',
    ],
  },
};

```

```

permissions: {
  required: ['INTERNET', 'CAMERA', 'WRITE_EXTERNAL_STORAGE'],
  optional: ['LOCATION', 'RECORD_AUDIO', 'NOTIFICATIONS'],
  justification: 'Explain each permission in UI',
},
},
};

```

12.6 Testing Strategy

12.6.1 Testing Pyramid

Unit Tests:

```

const unitTesting = {
  framework: 'Jest + React Testing Library',
  coverage: '80% minimum',

  testAreas: [
    'Business logic functions',
    'Utility functions',
    'Data validation (Zod schemas)',
    'State management (Zustand stores)',
    'API route handlers',
  ],

  example: `
describe('StreakCalculation', () => {
  it('should increment streak on daily photo', () => {
    const result = calculateStreak(lastPhotoDate, today);
    expect(result.currentStreak).toBe(7);
  });

  it('should break streak if photo missed', () => {
    const result = calculateStreak(twoDaysAgo, today);
    expect(result.isBroken).toBe(true);
  });
});
`,
};

```

Integration Tests:

```

const integrationTesting = {
  framework: 'Supertest (API) + Playwright (E2E)',
  coverage: 'Critical user flows',

  apiTests: [

```



```

    'Authentication flow (register, login, logout)',
    'Couple pairing workflow',
    'Message sending and retrieval',
    'Media upload and processing',
    'Streak photo submission',
    'Achievement unlock',
  ],

```

example: `

```

describe('POST /v1/auth/register', () => {
  it('should create new user account', async () => {
    const response = await request(app)
      .post('/v1/auth/register')
      .send({
        email: 'test@linkup.com.np',
        password: 'SecurePass123',
        username: 'testuser',
        displayName: 'Test User',
      });

    expect(response.status).toBe(201);
    expect(response.body.user.email).toBe('test@linkup.com.np');
    expect(response.body.tokens.accessToken).toBeDefined();
  });
});
`,
};

```

End-to-End Tests:

```

const e2eTesting = {
  framework: 'Playwright',
  browsers: ['Chromium', 'Firefox', 'WebKit'],

  criticalFlows: [
    'User registration → couple pairing → first message',
    'Photo upload → streak increment → achievement unlock',
    'Create painting → real-time collaboration',
    'Invite friend → accept → view couple content',
    'Purchase premium → unlock features',
  ],
},

```

example: `

```

test('Complete user onboarding flow', async ({ page }) => {
  // Register
  await page.goto('https://app.linkup.com.np/register');
  await page.fill('[name="email"]', 'user1@test.com');
  await page.fill('[name="password"]', 'SecurePass123');
  await page.click('button[type="submit"]');

```

```

    // Verify email (mock)
    await page.goto('/verify?token=mock_token');

    // Send couple invitation
    await page.goto('/couple/invite');
    await page.fill('[name="partnerEmail"]', 'user2@test.com');
    await page.click('button:has-text("Send Invitation")');

    // Assert invitation sent
    await expect(page.locator('.success-message')).toBeVisible();
  });
},
};

```

12.6.2 Performance Testing

Load Testing:

```

const loadTesting = {
  tool: 'k6 (Grafana)',

  scenarios: {
    baseline: {
      vus: 100, // Virtual users
      duration: '5m',
      target: 'Normal operation',
    },

    stress: {
      vus: 1000,
      duration: '10m',
      target: 'Peak traffic (launch day)',
    },

    spike: {
      vus: 5000,
      duration: '1m',
      target: 'Viral moment',
    },
  },

  metrics: {
    responseTime: {
      p50: '< 200ms',
      p95: '< 500ms',
      p99: '< 1000ms',
    },
  },
};

```

```

    },

    throughput: '1000 requests/second',
    errorRate: '< 0.1%',
  },

  example: `
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  vus: 100,
  duration: '5m',
};

export default function() {
  let response = http.get('https://api.linkup.com.np/v1/messages');

  check(response, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });

  sleep(1);
}
`,
};

```

Mobile Performance:

```

const mobilePerformance = {
  metrics: {
    appStartTime: '< 2 seconds (cold start)',
    frameRate: '60 FPS (UI interactions)',
    memoryUsage: '< 150 MB (idle)',
    batteryImpact: 'Low (< 5%/hour active use)',
  },

  optimization: [
    'Image lazy loading',
    'Pagination for large lists',
    'Debounce real-time events',
    'Background task optimization',
    'Reduce network calls',
  ],

  tools: {
    ios: 'Xcode Instruments',
    android: 'Android Profiler',
  }
}

```

```
  },  
};
```

12.6.3 Security Testing

Penetration Testing:

```
const securityTesting = {  
  frequency: 'Quarterly',  
  scope: 'Full application (web + mobile + API)',  
  
  owasp_top10: [  
    'Broken Access Control',  
    'Cryptographic Failures',  
    'Injection',  
    'Insecure Design',  
    'Security Misconfiguration',  
    'Vulnerable Components',  
    'Authentication Failures',  
    'Data Integrity Failures',  
    'Logging & Monitoring Failures',  
    'SSRF (Server-Side Request Forgery)',  
  ],  
  
  tools: {  
    automated: [  
      'OWASP ZAP',  
      'Burp Suite',  
      'npm audit (dependencies)',  
      'Snyk (vulnerability scanning)',  
    ],  
  
    manual: 'Security researcher (bounty program)',  
  },  
  
  bugBounty: {  
    platform: 'HackerOne or private program',  
    scope: 'Web app, mobile apps, API',  
    rewards: 'NPR 5,000 - 100,000 based on severity',  
  },  
};
```

12.7 DevOps & Infrastructure

12.7.1 CI/CD Pipeline

Continuous Integration:

```
# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run type check
        run: npm run type-check

      - name: Run unit tests
        run: npm run test:unit

      - name: Run integration tests
        run: npm run test:integration

      - name: Upload coverage
        uses: codecov/codecov-action@v3

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v3

- name: Build Docker image
  run: docker build -t linkup-api .

- name: Push to ECR
  run: |
    aws ecr get-login-password | docker login --username AWS --password-stdin
    docker push linkup-api:latest

deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Deploy to production
      run: |
        aws ecs update-service --cluster linkup-prod --service api --force-new-deployment

```

12.7.2 Monitoring & Observability

Monitoring Stack:

```

const monitoring = {
  applicationMonitoring: {
    tool: 'Datadog / New Relic',
    metrics: [
      'Request rate (RPM)',
      'Error rate (%)',
      'Response time (p50, p95, p99)',
      'Database query time',
      'WebSocket connections',
    ],
  },
  infrastructureMonitoring: {
    tool: 'AWS CloudWatch',
    metrics: [
      'CPU utilization',
      'Memory usage',
      'Disk I/O',
      'Network throughput',
      'Database connections',
    ],
  },
  logging: {

```

```

    tool: 'AWS CloudWatch Logs',
    structure: 'Structured JSON logs',
    retention: '30 days',

    levels: {
      error: 'Immediate alert',
      warn: 'Review daily',
      info: 'Normal operation',
      debug: 'Development only',
    },
  },

  errorTracking: {
    tool: 'Sentry',
    features: [
      'Real-time error alerts',
      'Stack traces',
      'User context',
      'Breadcrumbs',
      'Performance monitoring',
    ],
  },

  alerting: {
    critical: 'Slack + PagerDuty',
    high: 'Slack',
    medium: 'Email',
    low: 'Dashboard only',
  },
},

uptime: {
  tool: 'UptimeRobot / Pingdom',
  checks: [
    'Web app homepage',
    'API health endpoint',
    'WebSocket connection',
  ],
  frequency: '1 minute',
  notifications: 'Slack + SMS',
},
};

```

Alerting Rules:

```

const alerts = {
  critical: {
    errorRate: 'If > 5% for 5 minutes',
    responseTime: 'If p99 > 3s for 5 minutes',
    downtime: 'If any service down',
  },
};

```

```

    databaseConnections: 'If > 90% pool used',
  },

  warning: {
    errorRate: 'If > 2% for 10 minutes',
    responseTime: 'If p95 > 1s for 10 minutes',
    cpuUsage: 'If > 80% for 15 minutes',
    memoryUsage: 'If > 85% for 15 minutes',
  },

  oncall: {
    rotation: 'Weekly rotation',
    escalation: 'Primary → Secondary → Manager',
    response: '< 15 minutes',
  },
};

```

12.7.3 Scaling Strategy

Horizontal Scaling:

```

const scalingStrategy = {
  webServers: {
    initial: '2 instances (t3.medium)',
    scaling: 'Auto-scaling based on CPU',
    min: 2,
    max: 20,
    targetCPU: '70%',
  },

  apiServers: {
    initial: '3 instances (t3.large)',
    scaling: 'Auto-scaling based on request rate',
    min: 3,
    max: 50,
    targetRPM: '1000 RPM per instance',
  },

  database: {
    primary: 'RDS PostgreSQL (db.t3.large)',
    readReplicas: '2 replicas for read-heavy queries',
    scaling: 'Vertical scaling as needed',
    max: 'db.r6g.2xlarge',
  },

  cache: {
    redis: 'ElastiCache (cache.t3.medium)',
  }
};

```



```

    replication: 'Multi-AZ replication',
    scaling: 'Shard for horizontal scaling',
  },

  mediaStorage: {
    s3: 'Unlimited (pay per use)',
    cdn: 'CloudFront with edge caching',
    optimization: 'Image compression, lazy loading',
  },

  websocket: {
    initial: '2 dedicated instances',
    scaling: 'Based on concurrent connections',
    loadBalancing: 'Sticky sessions (socket.io)',
  },
};

// Cost Projections
const infrastructureCosts = {
  year1: {
    compute: 'NPR 25,000/month',
    database: 'NPR 15,000/month',
    storage: 'NPR 10,000/month',
    cdn: 'NPR 8,000/month',
    monitoring: 'NPR 5,000/month',
    total: 'NPR 63,000/month = NPR 756,000/year',
  },

  year3: {
    total: 'NPR 300,000/month (5× scaling)',
  },
};

```

12.8 Launch Checklist

12.8.1 Pre-Launch Verification

Technical Checklist:

```

const technicalChecklist = {
  infrastructure: [
    'Production environment configured',
    'Database migrations tested',
    'Backups automated and tested',
    'CDN configured and tested',
    'SSL certificates installed',
    'DNS records configured',
  ],
};

```

```

    'Load balancer configured',
    'Auto-scaling rules set',
  ],

  security: [
    'Security headers configured',
    'Rate limiting enabled',
    'CORS configured correctly',
    'Input validation on all endpoints',
    'File upload restrictions enforced',
    'Secrets moved to environment variables',
    'Dependency vulnerabilities fixed',
    'Penetration testing completed',
  ],

  monitoring: [
    'Error tracking (Sentry) configured',
    'Analytics (Mixpanel) integrated',
    'Logging infrastructure ready',
    'Uptime monitoring active',
    'Alert rules configured',
    'On-call rotation scheduled',
  ],

  testing: [
    'Unit test coverage > 80%',
    'Integration tests passing',
    'E2E tests for critical flows',
    'Load testing completed',
    'Mobile app performance validated',
    'Cross-browser testing done',
    'Accessibility audit passed',
  ],

  compliance: [
    'Privacy Policy published',
    'Terms of Service published',
    'Cookie consent implemented',
    'GDPR compliance verified',
    'Data export feature working',
    'Account deletion flow tested',
  ],
};

```

Business Checklist:

```

const businessChecklist = {
  legal: [
    'Company registered in Nepal',

```

```

    'PAN obtained',
    'Bank account opened',
    'Payment gateway contracts signed',
    'Insurance obtained',
  ],

  marketing: [
    'Landing page live',
    'Social media accounts active',
    'Press release ready',
    'Influencer partnerships confirmed',
    'Launch event planned',
    'Email templates created',
  ],

  support: [
    'Help center articles written',
    'FAQ page published',
    'Support email configured',
    'Support team trained',
    'Escalation process defined',
  ],

  appStores: [
    'iOS app submitted to App Store',
    'Android app submitted to Play Store',
    'App store listings optimized',
    'Screenshots and videos prepared',
    'App descriptions written (English + Nepali)',
  ],
};

```

12.8.2 Launch Day Plan

Timeline:

```

const launchDay = {
  t_minus_24h: {
    time: '24 hours before',
    tasks: [
      'Final smoke tests on production',
      'Verify all monitoring alerts working',
      'Brief team on launch procedures',
      'Prepare rollback plan',
      'Schedule social media posts',
    ],
  },
};

```

```

t_minus_1h: {
  time: '1 hour before',
  tasks: [
    'All team members on standby',
    'Final production checks',
    'Monitoring dashboards open',
    'Communication channels ready (Slack)',
  ],
},

t_zero: {
  time: 'Launch moment',
  tasks: [
    'Flip feature flag to enable public access',
    'Send launch email to waitlist',
    'Post on social media',
    'Activate paid ads',
    'Monitor real-time metrics',
  ],
},

t_plus_1h: {
  time: 'First hour',
  tasks: [
    'Monitor error rates closely',
    'Respond to user feedback',
    'Check server performance',
    'Engage with early users on social media',
  ],
},

t_plus_24h: {
  time: 'First day',
  tasks: [
    'Review metrics dashboard',
    'Compile bug reports',
    'Plan immediate fixes',
    'Send thank you to beta users',
    'Publish launch recap on social media',
  ],
},

weekOne: {
  tasks: [
    'Daily metrics review',
    'User feedback analysis',
    'Bug fix releases',
  ],
}

```

```

        'Continue marketing push',
        'Prepare Week 1 report',
    ],
},
};

```

12.9 Post-Launch Operations

12.9.1 Incident Response

Incident Management:

```

const incidentResponse = {
  severity: {
    critical: {
      definition: 'App down or major feature broken',
      responseTime: '< 15 minutes',
      resolution: '< 2 hours',
      notification: 'All users via status page',
    },

    high: {
      definition: 'Feature degradation affecting many users',
      responseTime: '< 1 hour',
      resolution: '< 4 hours',
      notification: 'Affected users',
    },

    medium: {
      definition: 'Minor feature issues',
      responseTime: '< 4 hours',
      resolution: '< 24 hours',
      notification: 'Optional',
    },
  },

  process: [
    '1. Detect (monitoring alerts)',
    '2. Triage (assess severity)',
    '3. Communicate (status page, users)',
    '4. Investigate (logs, metrics)',
    '5. Fix (code change or config)',
    '6. Deploy (hotfix release)',
    '7. Verify (monitoring)',
    '8. Post-mortem (within 48 hours)',
  ],
};

```

```

postMortem: {
  template: [
    'What happened?',
    'Root cause analysis',
    'Timeline of events',
    'Impact assessment',
    'What went well?',
    'What could be improved?',
    'Action items',
  ],

  distribution: 'All team + stakeholders',
  blameFree: true,
},
};

```

12.9.2 Feature Flagging

Progressive Rollout:

```

const featureFlags = {
  tool: 'LaunchDarkly / Custom implementation',

  useCases: [
    'New feature testing (5% → 25% → 50% → 100%)',
    'A/B testing',
    'Kill switch for problematic features',
    'Different experiences for free vs paid',
  ],

  example: {
    collaborativePainting: {
      enabled: true,
      rollout: {
        percentage: 25,
        userSegment: 'premium_users',
      },
      killSwitch: true,
    },
  },

  implementation: `
    const featureFlags = useFeatureFlags();

    if (featureFlags.collaborativePainting.enabled) {
      return <CollaborativePaintingCanvas />;
    }
  `
};

```

```
        return <ComingSoonMessage />;
    },
};
```

Chapter 12 Status: Complete

LinkUp PRA Document Summary

Document Scope: 19,900+ lines of comprehensive technical documentation

Chapters Completed: 1. Executive Summary & Vision - Project overview, personas, competitive analysis 2. Core Communication Features - Messaging, voice/video, WebRTC specifications 3. Creative & Interactive Features - Scribble, painting, emojis, soundboard 4. Social & Relationship Features - SingleFriend, Circles, Hall of Fame 5. Entertainment & Shared Experiences - Watch parties, streaming, playlists 6. Notifications & Alerts System - Multi-platform push, reminders, global search 7. Technical Architecture - Monorepo, Next.js/Swift/Kotlin, microservices 8. Database Design & Data Schemas - PostgreSQL, Redis, MongoDB schemas 9. API Specifications - 100+ REST endpoints, WebSocket events 10. UI/UX Design System - Design tokens, components, pages, animations 11. Business Model - Nepal market (NPR pricing), 5-year financials 12. Development Roadmap - 3-month MVP, security, compliance, launch

Key Specifications: - Target Market: Nepal couples (320,000 TAM, NPR 960M market) - Tech Stack: Next.js 14, Swift/SwiftUI, Kotlin/Compose, PostgreSQL, Redis - Pricing: Free tier, Plus (NPR 499/month), Pro (NPR 999/month) - Revenue Projection: Year 1: NPR 10.9M, Year 5: NPR 405M (80% margin) - Development Timeline: 3-month MVP, launch preparation, profitability target Year 2 - Security: TLS 1.3, bcrypt, GDPR-compliant, future E2E encryption - Scale Target: Support 200,000 couples by Year 5

Document Use Cases: Agentic code generation, development team reference, investor presentations

-
Chap-
ters
can
be
writ-
ten
inde-
pen-
dently
based
on
prior-
ity -
This
docu-
ment
serves
as
the
sin-
gle
source
of
truth
for
LinkUp
de-
vel-
op-
ment
-
Suit-
able
for
agen-
tic
code
gen-
era-
tion
and
de-
vel-
oper
refer-
ence

Document prepared for: LinkUp Development Team
Prepared by: Product Requirements Analysis
Last Updated: January 20, 2026